

로봇운영체제 ROS 2 특강

2025.06.25

발표자 소개: 표윤석

- 現 로보티즈 부사장 <Physical AI 로봇 플랫폼, 자율주행 로봇 플랫폼, 로봇운영체제 플랫폼 개발 총괄>
- 現 국가인공지능전략위원회 자문위원
- 前 ROS 2 TSC (Technical Steering Committee, 기술운영위원회)
- 前 일본학술진흥회(JSPS) 연구원
- 前 한국과학기술연구원(KIST) 연구원
- 前 광운대학교 로봇게임단 로봡 창단, 주장
- 前 한국로봇산업진흥원 산업인력양성 재직자 교육 강사
(2016년~2019년 16시간 ROS 강의 20회 진행, 총 320시간)
- “ROS 2로 시작하는 로봇 프로그래밍” 등 10권의 서적 출판
- ROS 개발 경험 (2011~2026, ROS 1 Diamondback 버전부터 ROS 2까지 16년차)
- ROS Standard Platform Robot: [TurtleBot3](#) 개발
- [The 10 most influential people in ROS in 2019](#)
- [Top 10 ROS Based Robotics Companies in 2019](#)
- [ROS Amazing Contributor](#) (23 people worldwide)
- 로보틱스 커뮤니티 오로카, 로열모 운영진
- 규슈대학교 정보지능공학 석사, 박사
- 광운대학교 전자공학과 학사
- 대통령상 수상 (International Robot Contest 2008)
- 산업통상부 장관 표창 (2025 M.AX 얼라이언스 공헌)
- 강영국 기술상 수상 (2025 제어로봇시스템학회)
- 산업통상자원부 장관 표창 (2019 로봇산업 발전 유공자)
- Finalist of Best Service Robotics Paper Award (ICRA 2017)
- 산업자원부 장관상 2회 수상 (2015 RUF, 2005 ROBOTPIAD)
- 일본기계학회 교육상 수상 (2019 JSME)
- 일본 제15회 로보원 우승(2009 ROBO-ONE)



3 로봇 운영체제 ROS 2 특강 목차 (기초)

- ROS 2 개발 환경 구축
- ROS 2 중요 컨셉
- ROS 1과 2의 차이점을 통해 알아보는 ROS 2의 특징
- ROS 2와 DDS (Data Distribution Service)
- ROS 2 패키지 설치와 노드 실행
- ROS 2 노드와 메시지 통신
- ROS 2 토픽 (Topic), 서비스 (Service), 액션 (Action), 파라미터 (parameter)
- ROS 2 토픽, 서비스, 액션 정리 및 비교
- ROS 2의 도구 (CLI, GUI)
- ROS 2의 물리량 표준 단위
- ROS 2의 좌표 표현
- ROS 2의 QoS (Quality of Service)
- ROS 2의 파일 시스템
- ROS 2의 빌드 시스템과 빌드 툴
- ROS 2의 패키지 파일 (환경 설정, 빌드 설정)

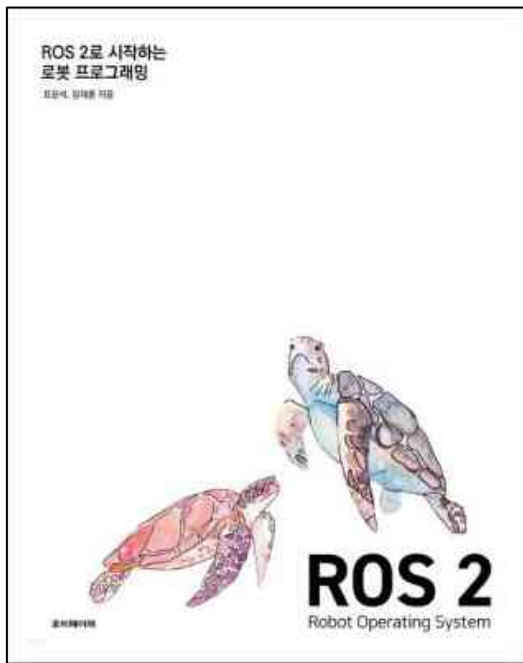
4

로봇 운영체제 ROS 2 특강 목차 (실습)

- ROS 2 프로그래밍 규칙
- ROS 2 프로그래밍 기초
- ROS 2 토픽, 서비스, 액션 인터페이스
- ROS 2 패키지 설계
- ROS 2 토픽 프로그래밍 실습
- ROS 2 서비스 프로그래밍 실습
- ROS 2 액션 프로그래밍 실습
- ROS 2 파라미터 프로그래밍 실습
- ROS 2 실행 인자 프로그래밍 실습
- ROS 2 런치 프로그래밍 실습
- ROS 2 + Gazebo 시뮬레이션 실습 (SLAM, Navigation, Manipulation)
 - Mobile Robot: TurtleBot3 (SLAM / Navigation)
 - Mobile Manipulator: TurtleBot3 + OpenManipulator-X (Manipulation)
- ROS 2 Tips
- ROS 2 추천 패키지
- ROS 2 도입을 위한 다음 단계는?
- Q&A

5 강의 자료

- 로봇 운영체제 ROS 2 강의 전체 자료
 - ROS 2 관련 총 56개 강의 글 (온라인) [\[링크\]](#)
 - ROS 2로 시작하는 로봇 프로그래밍 (책) [\[링크\]](#)
 - 본 강의에 사용된 예제 코드 [\[링크\]](#)
- 본 강의 자료
 - 로봇 개발 내용을 담고 있지 않습니다.
 - 로봇 개발을 위한 프로그래밍 강의입니다.
 - 그림 위주의 발표 자료는 깔끔하지만 혼자 다시 복습하기 어렵죠.
 - 이에 추후 혼자 다시 **복습**할 수 있도록 **텍스트를 많이 넣었어요.**
 - **중요한 부분**은 굵은 글씨로 기재했구요.
 - 스스로 찾아보면 좋은 자료들은 **밑줄의 링크** 형태로 추가했어요.
 - 강의 자체는 중요한 부분을 발췌하여 설명할게요.
 - 만약 보충 자료가 필요하시다면 [ROS 2 Documentation](#)을 추천드려요
- 추천 강의
 - 민형기 님의 PinkLAB [R2R\(Rush to ROS\)](#) 강의
 - 김수영 님의 Road Balance [ROS 2 BASICS](#) 강의



6 강의 자료 라이선스 및 업데이트

- 본 강의 자료에 포함된 일부 이미지 및 문서의 경우 해당 원본 콘텐츠의 개별 라이선스를 따르며 이를 각 장에 표시하고 있습니다. 그 이외의 직접 작성한 모든 콘텐츠는 [CC BY-NC 4.0 라이선스](#)를 따릅니다. 이는 본 자료를 비영리목적 으로 사용해야하며 저작자 표시를 반드시 해주어야 한다는 것을 의미합니다. 즉, 적절한 출처와 해당 라이선스 링크를 표시하고, 변경이 있는 경우 공지해야 합니다.
- 본 강의 자료는 ROS 2 보급과 오픈소스 커뮤니티 활성화를 위하여 수업, 세미나, 강연, 강의 등의 비영리목적 으로 사용할 경우 수정 권한을 포함한 원본 강의 자료를 제공해 드릴 수 있습니다. 또한 특수한 목적일 경우, 저작자와 협의하여 저작자 표시를 하지 않아도 됩니다. 이를 원할 경우에는 하기의 연락처 중 편하신 방법으로 연락을 주세요.
- 본 강의 자료는 안내 없이 수시로 업데이트 될 수 있습니다. 본 강의 자료를 받아 둔지 오래 되었다면 새롭게 받아 사용하기를 추천합니다. 항상 최신의 정보와 좀 더 개선된 자료를 공유할 수 있도록 노력하겠습니다. 본 강의 자료에 오타나 잘못된 설명 및 명령어 등이 있다면 문서 코멘트 추가를 통해 알려주시거나, 아래 연락처로 연락주세요. 메일도 좋고, DM도 좋습니다. 여러분의 피드백은 좀 더 좋은 자료 공유하는데에 큰 도움이 됩니다. 감사합니다.
- 연락처
 - pyo@robotis.com
 - www.facebook.com/yoonseok.pyo
 - <https://www.linkedin.com/in/yoonseokpyo/>

ROS 2 특강 발표자료
(강의 자료)



<https://m.site.naver.com/1ptXy>

ROS 2 강좌 목차
(네이버 카페)



5만명

<https://m.site.naver.com/1ptNR>

오로카 ROS 유저모임
(카카오톡 단톡방)



1,200명

<https://open.kakao.com/o/gAeOb2nc>

8

본 강의에 앞서서...

● 맞춤형 강의를 위해!

- 직업이 어떻게 되세요?
- 로봇 업계에 계시나요?
- 직장이라면 몇 년차 직장인 이실까요?
- 학생이라면 몇 학년 이세요?
- 전공은 어떻게 되시나요?
- ROS 사용해 보신 적 있으세요?
- ROS 1 vs ROS 2 어떤것을 사용하고 계세요?
- 리눅스 경험은 어떻게 되시나요?
- C++, Python, Java, JS, Rust ... 어떤 언어를 사용하세요?
- 왜 ROS를 시작하시려고 하시는지 여쭙봐도 될까요?
- 8시간 강의인데 우리 같이 집중해서, 하루에 책 1권 끝낸다고 생각하고 달려봐요!

ROS = Tool

(로봇 개발을 위한)

10

본 강의에 앞서서... ROS로 이런걸 만들어요.



11

본 강의에 앞서서 ... ROS로 이런걸 만들어요.

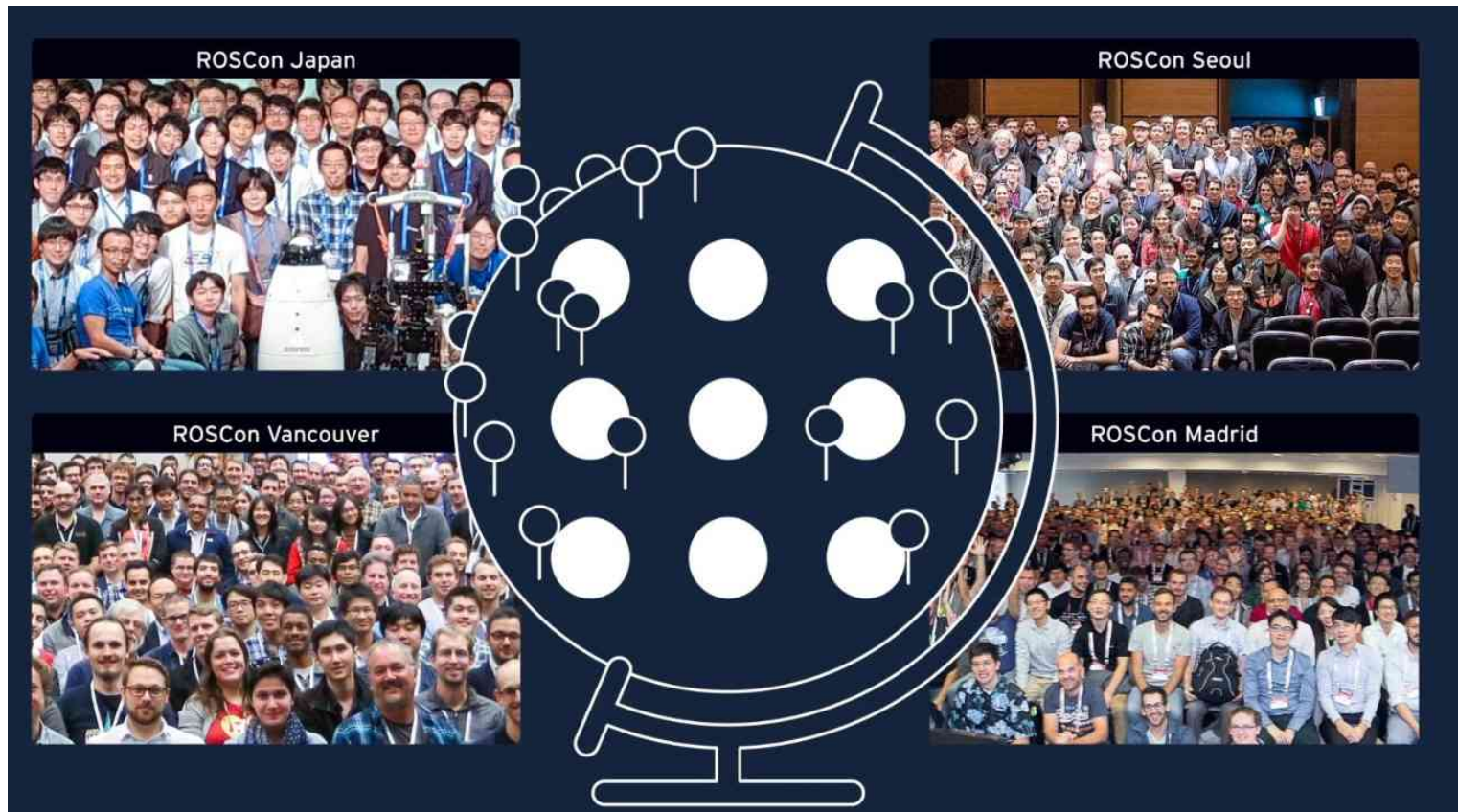


ROS

ROS - Robot Operating System

The Robot Operating System (ROS) is a set of **software libraries** and **tools** that help you build robot applications. From **drivers** to **state-of-the-art algorithms**, and with powerful **developer tools**, ROS has what you need for your next robotics project. And it's all **open source**.





15 Why ROS 2?



Why ROS 2?



ROS 2 (Robot Operating System 2) is an open source software development kit for robotics applications. The purpose of ROS 2 is to offer a standard software platform to developers across industries that will carry them from research and prototyping through to deployment and production. ROS 2 builds on the success of ROS 1, which is used today in myriad robotics applications around the world.

» Shorten time to market

ROS 2 provides the robotics tools, libraries, and capabilities that you need to develop your applications, allowing you to spend your time on the work that is important for your business. Because it is open source, you have the flexibility to decide where and how to use ROS 2, as well as the freedom to customize it for your needs.

» Designed for production

Drawing on a decade of experience in establishing ROS 1 as the de facto global standard for robotics R&D, ROS 2 was built from the ground up to be industry-grade and used in production, including high reliability and safety critical systems. Design choices, development practices, and project governance for ROS 2 are based on requirements from industry stakeholders.

» Multi-platform

ROS 2 is supported and tested on Linux, Windows, and macOS, allowing seamless development and deployment of on-robot autonomy, back-end management, and user interfaces. The tiered support model allows for ports to new platforms, such as real-time and embedded OSs, to be introduced and promoted as they gain interest and investment.

» Multi-domain

Like ROS 1 before it, ROS 2 is ready for use across a wide array of robotics applications, from indoor to outdoor, home to automotive, underwater to space, and consumer to industrial.



www.openrobotics.org www.ros2.org



Why ROS 2?



» No vendor lock-in

ROS 2 is built on an abstraction layer that insulates the robotics libraries and applications from the communication technologies. Below the abstraction are multiple implementations of the communications code, including both open source and proprietary solutions. Above the abstraction, core libraries and user applications are portable.

» Built on open standards

The default communications method in ROS 2 uses industry standards like IDL, DDS, and DDS-1 RTPS, which are already widely deployed in a variety of industrial applications, from factories to aerospace.

» Permissive open source license

ROS 2 code is licensed under Apache 2.0 License, with ported ROS 1 code under the 3-clause (or "new") BSD License. Both licenses allow permissive use of the software, without implications on the user's intellectual property.

» Global community

Over 10+ years the ROS project has produced a vast ecosystem of software for robotics by nurturing a global community of hundreds of thousands of developers and users who contribute to and improve that software. ROS 2 is developed by and for that community, who will be its stewards into the future.

» Industry support

As demonstrated by the membership of the ROS 2 Technical Steering Committee, industry support for ROS 2 is strong. Companies large and small from around the world are committing their resources to making open source contributions to ROS 2, in addition to developing products on top.

» Interoperability with ROS 1

ROS 2 includes a bridge to ROS 1 that handles bidirectional communication between the two systems. If you have an existing ROS 1 application, you can start experimenting with ROS 2 via the bridge and port your application incrementally according to your requirements and available resources.



www.openrobotics.org www.ros2.org

- Brochure: Why ROS 2?
 - Open Robotics의 전 CEO인 Brian Gerkey가 2019년 2월에 초안을 작성
 - ROS 2 TSC 멤버 공동 작성
 - ROSCon2014에서 발표
 - 2015년 08월 31일 ROS 2 Alpha 공개
 - 2016년 12월 19일 ROS 2 Beta 공개
 - 2017년 12월 08일 ROS 2 공개
1. Shorten time to market
 2. Designed for production
 3. Multi-platform
 4. Multi-domain
 5. No vendor lock-in
 6. Built on open standards
 7. Permissive open source license
 8. Global community
 9. Industry support
 10. Interoperability with ROS 1

16 ROS의 8가지 중요 컨셉

(1) 글로벌 커뮤니티

ROS 커뮤니티는 10년 이상 ROS 프로젝트는 소프트웨어에 기여하고 개선하는 수십만 명의 개발자와 사용자로 구성된 글로벌 커뮤니티를 육성함으로써 로봇 공학을 위한 방대한 **소프트웨어 에코 시스템**을 만들어 왔다. **ROS 2는 커뮤니티를 위해, 커뮤니티에 의해 개발되었다.**

(2) 검증된 사용 사례

ROS는 **로봇 산업 전반에 걸쳐 사용**되고 있다. **로봇 공학 교육을 위한 표준**이다. 단일 프로젝트부터 여러 기관의 협업 및 대규모 경연 대회에 이르기까지 대부분의 로봇 공학 연구의 기반이 되었다. 그리고 오늘날 전 세계에서 생산 중인 로봇의 내부에도 이 기술이 적용되고 있다. 자율 이동 로봇(AMR)에서만 ROS는 수십억 달러의 가치를 창출하는 데 기여했다.

(3) 시장 출시 시간 단축

ROS는 로봇 응용 프로그램을 개발하는 데 필요한 도구, 라이브러리 및 기능을 제공하므로 **본연의 중요한 로봇 개발 작업에 더 많은 시간을 할애할 수 있다는 것**이 가장 큰 장점이다. 특정 로봇 개발을 위해 처음부터 프레임워크를 만들고 통신 방법을 선정하고 디버깅 툴과 시각화 툴을 다시 만드는 비효율적인 방법에서 벗어날 수 있게 한다. 더불어 ROS는 상용 소프트웨어가 아닌 ROS 커뮤니티에서 개발해오고 있는 오픈소스이기 때문에 ROS를 사용할 부분과 사용 방법을 유연하게 결정할 수 있을 뿐만 아니라 필요에 따라 자유롭게 수정할 수 있다.

17 ROS의 8가지 중요 컨셉

(4) 다중 도메인

ROS는 실내에서 실외, 가정에서 자동차, 수중에서 우주, 소비자에서 산업에 이르기까지 **다양한 로봇 공학 애플리케이션** 에서 사용할 수 있다.

(5) 멀티 플랫폼

ROS 2는 **Linux, Windows, macOS**에서 개발, 지원, 테스트 진행하고 있기에 자율성, 백엔드 관리 및 사용자 인터페이스의 원활한 개발 및 배포가 가능하다. 계층형 지원 모델을 사용하고 있기에 실시간 운영체제 (Real-time OS) 및 임베디드 OS(Embedded OS)와 같은 새로운 플랫폼으로의 포팅에 대한 관심과 투자를 통해 도입 및 홍보도 할 수 있다.

(6) 100% 오픈 소스

ROS 2 코드는 **Apache 2.0 라이선스** 를 기본 라이선스로 사용하여 지적 재산권에 영향을 주지 않으면서 자유 재량으로 넓은 범위로 사용 가능하다.

18 ROS의 8가지 중요 컨셉

(7) 상업 친화성

사용자 커뮤니티의 ROS에 대한 오픈소스 기여를 진심으로 권장하지만, 이를 요구하지는 않는다. 오픈 소스 라이선스에 따라 ROS를 배포하며, Apache 2.0이 기본 라이선스이다. **ROS를 수정하고, 자신 또는 다른 사람의 비공개 소프트웨어와 혼합하고, 그 결과물을 여러분의 독점 제품에 배포할 수 있으며, 그 사실을 알릴 필요도 없다.** 물론 항상 사용자들이 ROS를 어떻게 사용하고 있는지 알고 싶기에 커뮤니티에 이를 알려주길 바란다.

(8) 업계 지원

ROS 2 기술 운영위원회 (ROS 2 Technical Steering Committee)의 멤버쉽에서 알 수 있듯이 ROS 2에 대한 업계 지원은 강력하다. 전 세계의 크고 작은 회사는 제품을 개발할 뿐만 아니라 ROS 2에 오픈소스 기여를 하기 위해 자원을 투입하고 있다.

- [ROS Package Documentation](#)
- [Robotics Stack Exchange](#)
- [ROS Discourse Forums](#)
- [Open Robotics Discord Server](#)
- [ROS Index](#)
- [Issue Trackers](#)





로봇 문제에 대한 솔루션을 제공하는 ROS

이름과는 달리 ROS는 사실 운영체제가 아니다. 그보다는 로봇 애플리케이션을 구축하는 데 필요한 빌딩 블록을 제공하는 **SDK(소프트웨어 개발 키트)**이다. 수업 프로젝트, 과학 실험, 연구 프로토타입, 최종 제품 등 어떤 애플리케이션이든 ROS를 사용하면 목표를 더 빨리 달성할 수 있다.

그리고, 모두 **오픈 소스** 이다.

Plumbing

ROS의 핵심은 흔히 "**미들웨어**" 또는 "**배관**"이라고 불리는 **메시지 전달 시스템**을 제공한다. 커뮤니케이션은 새로운 로봇 애플리케이션이나 하드웨어와 상호 작용하는 모든 소프트웨어 시스템을 구현할 때 가장 먼저 해결해야 할 문제 중 하나이다. ROS에 내장되어 있고 충분한 테스트를 거친 메시징 시스템은 **퍼블리시/서브스크라이브** 패턴을 통해 분산 노드 간의 통신 세부 사항을 관리함으로써 시간을 절약해 준다. 이러한 접근 방식은 결함 격리, 우려 사항 분리, 명확한 인터페이스 등 소프트웨어 개발의 어려운 부분들을 해결할 수 있게 돕는다. ROS를 사용하면 유지 관리, 기여, 재사용이 더 쉬운 시스템을 만들 수 있다.

그 과정에서 방대한 커뮤니티 경험을 활용하여 **표준 ROS 메시지 형식**을 만들 수 있으며, 이러한 표준은 LIDAR 및 카메라부터 현지화 알고리즘 및 사용자 인터페이스에 이르기까지 **모든 것과 상호 작용**하는 데 사용된다.

Tools

로봇 애플리케이션을 구축하는 일은 쉽지 않다. 센서와 액추에이터를 통해 물리적 세계와 비동기적으로 상호 작용해야 한다는 점과 소프트웨어 개발의 모든 어려움이 결합되어 있다. 애플리케이션을 효율적으로 구축하려면 우수한 개발자 도구가 필요하다. ROS에는 **실행, 상태, 디버깅, 시각화, 플로팅, 로깅, 재생 등 다양한 개발 도구가 포함**되어 있다. 이러한 도구는 개발 팀의 진행 속도를 높여주며, 제품 출시 시 함께 제공될 수 있다.

Capabilities

ROS 에코시스템은 로봇 소프트웨어의 보고이다. GPS용 디바이스 드라이버, 4족 보행 로봇을 위한 보행 및 균형 컨트롤러, 모바일 로봇을 위한 매핑 시스템 등 어떤 것이 필요한 ROS는 여러분을 위한 모든 것을 제공하고 있다. **드라이버부터 알고리즘, 사용자 인터페이스에 이르기까지 ROS는 애플리케이션에 집중할 수 있는 빌딩 블록을 제공한다.**

ROS 프로젝트의 목표는 당연히 여겨지는 것에 대한 기준을 지속적으로 높여 **로봇 애플리케이션 구축에 대한 진입 장벽을 낮추는 것이다.** 유용 하거나 재미있고 흥미로운 로봇에 대한 좋은 아이디어가 있는 사람이라면 누구나 기본 하드웨어와 소프트웨어에 대한 모든 것을 이해하지 않고도 그 아이디어를 현실화할 수 있어야 한다.

Community

ROS 커뮤니티는 규모가 크고 다양하며 글로벌하다. **학생과 취미 활동가부터 다국적 기업, 정부 기관에 이르기까지 다양한 사람과 조직이 ROS 프로젝트를 계속 이어가고 있다.**

이 프로젝트의 커뮤니티 허브이자 중립적인 관리자는 이 웹사이트와 같은 공유 온라인 서비스를 호스팅하고, 사용자가 설치하는 바이너리 패키지를 포함한 배포 릴리스를 생성 및 관리하며, ROS 내 대부분의 핵심 소프트웨어를 개발 및 유지 관리하는 **Open Robotics**이다. 오픈 로보틱스는 ROS와 관련된 엔지니어링 서비스도 제공하고 있다.

ROS 2 개발 환경 구축

- OS
 - Ubuntu 22.04 (Jammy Jellyfish) 또는 Linux Mint 21.3 (Virginia)
- ROS
 - ROS 2 Humble
- 에디터
 - Visual Studio Code
- 설치 매뉴얼
 - [ROS 2 Installation \(Humble\)](#)
- 환경 설정 및 간단 튜토리얼
 - [ROS 2 Configuring environment](#)
 - [ROS 2 Turtlesim Tutorial](#)



- OS
 - Ubuntu 24.04 (Noble Numbat) 또는 Linux Mint 22.1 (Xia)
- ROS
 - ROS 2 Jazzy
- 에디터
 - Visual Studio Code
- 설치 매뉴얼
 - [ROS 2 Installation \(Jazzy\)](#)
- 환경 설정 및 간단 튜토리얼
 - [ROS 2 Configuring environment](#)
 - [ROS 2 Turtlesim Tutorial](#)



지역 설정

```
$ locale
$ sudo apt update && sudo apt install locales
$ sudo locale-gen en_US en_US.UTF-8
$ sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
$ export LANG=en_US.UTF-8
$ locale
```

소스 설정

```
$ sudo apt install software-properties-common
$ sudo add-apt-repository universe
$ sudo apt update && sudo apt install curl -y
$ sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee
/etc/apt/sources.list.d/ros2.list > /dev/null
```

ROS 설치

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install ros-humble-desktop
$ sudo apt install ros-dev-tools
```

```
$ printenv | grep -i ROS
ROS_VERSION=2
ROS_PYTHON_VERSION=3
ROS_DISTRO=humble
...
```

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_py listener
[listener] [INFO] [1724745119.174348942]: I heard: [Hello World: 1]
[listener] [INFO] [1724745120.142077417]: I heard: [Hello World: 2]
[listener] [INFO] [1724745121.142486640]: I heard: [Hello World: 3]
```

```
$ source /opt/ros/humble/setup.bash
$ ros2 run demo_nodes_cpp talker
[talker] [INFO] [1724745119.139744855]: Publishing: 'Hello World: 1'
[talker] [INFO] [1724745120.139659405]: Publishing: 'Hello World: 2'
[talker] [INFO] [1724745121.139723166]: Publishing: 'Hello World: 3'
```

```
$ mkdir -p ~/robot_ws/src
$ cd ~/robot_ws/
$ colcon build --symlink-install
Summary: 0 packages finished [0.41s]
$ ls
build install log src
```

```
$ nano ~/.bashrc
```

```
source /opt/ros/humble/setup.bash
source ~/robot_ws/install/local_setup.bash
```



~/.bashrc 파일 하단에 추가

```
$ source ~/.bashrc
```

```
$ nano ~/.bashrc
```

```
source /opt/ros/humble/setup.bash
source ~/robot_ws/install/local_setup.bash
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
source /usr/share/vcstool-completion/vcs.bash
source /usr/share/colcon_cd/function/colcon_cd.sh
```

```
export _colcon_cd_root=~/.robot_ws
export ROS_DOMAIN_ID=1
export ROS_LOCALHOST_ONLY=1
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
# export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{severity}] {time}
[{name}]: {message} ({function_name}() at {file_name}:{line_number})'
export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{name}] [{severity}]
[{time}]: {message}'
export RCUTILS_COLORIZED_OUTPUT=1
export RCUTILS_LOGGING_USE_STDOUT=0
export RCUTILS_LOGGING_BUFFERED_STREAM=1
```

```
alias nr='nano ~/.bashrc'
alias sr='source ~/.bashrc'
alias cw='cd ~/robot_ws'
alias cs='cd ~/robot_ws/src'
alias dw='cd ~/robot_ws && rm -rf build/ install/ log/'
alias cb='cd ~/robot_ws && colcon build --symlink-install
--continue-on-error'
alias cbs='colcon build --symlink-install'
alias cbp='colcon build --symlink-install --packages-select'
alias cbu='colcon build --symlink-install --packages-up-to'
alias ct='cd ~/robot_ws && mkdir test_result && colcon test
--test-result-base ./test_result/ '
alias ctp='colcon test --packages-select'
alias ctr='colcon test-result'
alias rt='ros2 topic list'
alias re='ros2 topic echo'
alias rn='ros2 node list'
alias killgazebo='killall -9 gazebo & killall -9 gzserver & killall -9 gzclient'
alias roslint='ament_uncrustify && \ ament_cpplint && \ ament_cppcheck
&& \ ament_flake8 && \ ament_pep257 && \ ament_lint_cmake && \
ament_xmllint && \ ament_copyright'
```

```
$ source ~/.bashrc
```

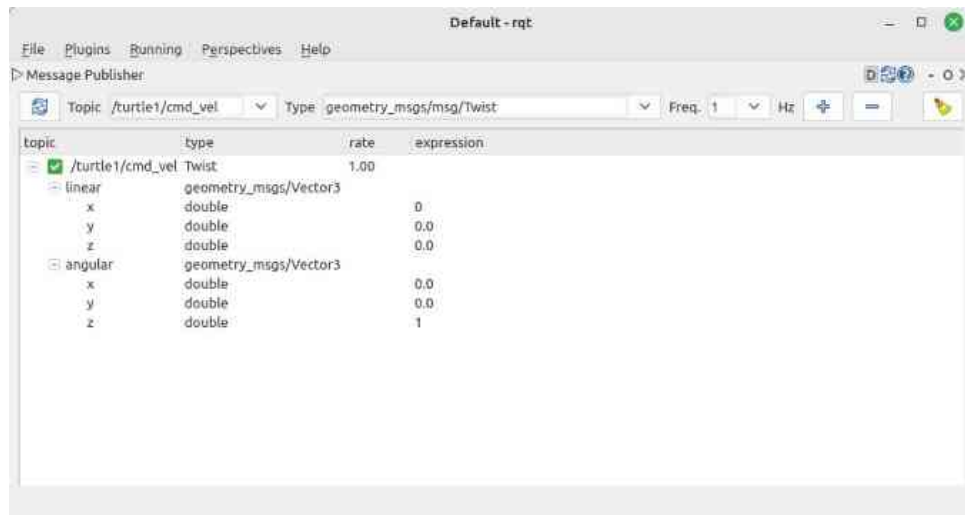
32 Turtlesim

```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtle_teleop_key
```

```
$ ros2 topic echo /turtle1/cmd_vel
```

```
$ rqt
```



ROS 1과 2의 차이점을 통해 알아보는 ROS 2의 특징

- 2025.05.23 - ROS 2 Kilted Kaiju
- **2024.05.23 - ROS 2 Jazzy Jalisco (LTS, 5 years support)**
- 2023.05.23 - ROS 2 Iron Irwini
- **2022.05.23 - ROS 2 Humble Hawksbill (LTS, 5 years support)**
- 2021.05.23 - ROS 2 Galactic Geochelone
- 2020.06.05 - ROS 2 Foxy Fitzroy (LTS, 3 years support)
- 2019.11.22 - ROS 2 Eloquent Elusor
- 2019.05.31 - ROS 2 Dashing Diademata (First LTS, 2 years support)
- 2018.12.14 - ROS 2 Crystal Clemmys
- 2018.07.02 - ROS 2 Bouncy Bolson
- **2017.12.08 - ROS 2 Ardent Apalone (1st version)**

- 2017.09.13 - ROS 2 Beta3 (code name R2B3)
- 2017.07.05 - ROS 2 Beta2 (code name R2B2)
- **2016.12.19 - ROS 2 Beta1 (code name Asphalt)**

- 2016.10.04 - ROS 2 Alpha8 (code name Hook.and.Loop)
- 2016.07.14 - ROS 2 Alpha7 (code name Glue Gun)
- 2016.06.02 - ROS 2 Alpha6 (code name Fastener)
- 2016.04.06 - ROS 2 Alpha5 (code name Epoxy)
- 2016.02.17 - ROS 2 Alpha4 (code name Duct tape)
- 2015.12.18 - ROS 2 Alpha3 (code name Cement)
- 2015.11.03 - ROS 2 Alpha2 (code name Baling wire)
- **2015.08.31 - ROS 2 Alpha1 (code name Anchor)**



- **ROS Noetic Ninjemys**
- Released May, 2020
- LTS, supported until May, 2025



ROS 2 Jazzy Jalisco



2020년 마지막 릴리즈 후, 2025년 EOL된 ROS 1?

[ROS 1 Noetic Ninjemys goes End-of-Life 2025-05-31](#)

매년 새로운 버전을 릴리즈하고 있는 새로운 ROS 2?

여러분의 선택은?

ROS 1은 2007년에 개발이 시작되어 지금은 대학, 연구 기관, 산업계, 로봇 자작의 취미활동까지 폭넓게 이용되고 있다. 원래 **ROS 1은 Willow Garage사가 개인 서비스 로봇인 PR2개발에 필요한 미들웨어 형태의 로봇 개발 프레임워크를 다양한 개발 툴과 함께 오픈 소스로 공개 한 것으로 시작**하였다. 따라서 개발 환경으로는 PR2의 초기 컨셉을 그대로 이어받아 다음과 같은 **제한 사항**이 있었다.

- 단일 로봇
- 워크스테이션급 컴퓨터
- Linux 환경
- 실시간 제어 지원하지 않음
- 안정된 네트워크 환경이 요구됨
- 주로 대학이나 연구소와 같은 아카데미 연구 용도

37 ROS 1과 2의 차이점

오늘날 요구되는 로봇 개발 환경과는 큰 차이가 있다. 예를 들어, 최근의 ROS 1은 종래 가장 많이 이용되고 있던 **학술분야뿐만 아니라, 제조 로봇, 농업 로봇, 드론, 소셜 로봇과 같은 상용 로봇 등으로 이용**되고 있다. 극단적인 예로 NASA가 국제 우주 정거장에서 사용한 Robonaut에는 ROS 1가 채용되고 있지만, 거기에서는 실시간 제어가 요구되었고 이를 위해서 ROS 1을 수정하여 사용하였다. 이러한 **새로운 로봇 개발 환경 및 요구되는 기능**을 정리하면 다음과 같다.

- 복수대의 로봇
- 임베디드 시스템에서의 ROS 사용
- 실시간 제어
- 불안정한 네트워크 환경에서도 동작 할 수 있는 유연함
- 멀티 플랫폼 (Linux, Windows, macOS)
- 최신 기술 지원 (Zeroconf, Protocol Buffers, ZeroMQ, WebSockets, DDS 등)
- 상업용 제품 지원

ROS 1에서 이러한 새롭게 요구되는 기능을 제공하려면 대규모 API의 변경이 필요하다. 그러나 기존의 ROS 1과의 호환성을 유지하면서 수 많은 새로운 기능을 추가하는 것은 쉽지 않았다. 또한, 기존의 ROS 1을 문제 없이 이용하고 있는 사용자에게는 큰 API의 변경은 바람직하지 않다.

그래서 ROS의 차세대 기능을 도입한 버전을 ROS 2라고, ROS 1에서 분리하여 개발하게 된 것이다. 기존의 ROS 1 사용자는 필요하다면 그대로 ROS 1을 이용할 수 있다. 한편, 새로운 기능이 필요한 사용자는 ROS 2를 선택하면 된다. 또한, ROS 1과 ROS 2 사이에서 서로 메시지 통신이 가능한 브리지 프로그램 (ros1_bridge) 제공되므로 두 버전 모두를 함께 사용하는 것도 가능하다.

Features	ROS 1	ROS 2
Platforms	Linux, macOS	Linux, macOS, Windows
Real-time	external frameworks like OROCOS	real-time nodes when using a proper RTOS with carefully written user code
Security	SROS	SROS 2, DDS-Security, Robotic Systems Threat Model
Communication	XMLRPC + TCPROS	DDS (RTPS)
Middleware interface	-	rmw
Node manager (discovery)	ROS Master	No, use DDS's dynamic discovery
Languages	C++03, Python 2.7	C++14 (C++17), Python 3.5+
Client library	roscpp, rospy, rosjava, rosnodesjs, and more	rcldcpp, rclpy, rcljava, rcljs, and more
Build system	roscpp -- catkin (CMake)	ament (CMake), Python setuptools (Full support)
Build tools	catkin_make, catkin_tools	colcon
Build options	-	Multiple workspace, No non-isolated build, No devel space
Version control system	rosws -- wstool, rosinstall (*.roinstall)	vcstool (*.repos)
Life cycle	-	node life cycle
Multiple nodes	one node in a process	multiple nodes in a process
Threading model	single-threaded execution or multi-threaded execution	custom executors
Messages (topic, service, action)	*.msg, *.srv, *.action	*.msg, *.srv, *.action, *.idl
Command Line Interface	roscpp, roslaunch, ros topic ...	ros2 run, ros2 launch, ros2 topic ...
roslaunch	XML	Python, XML, YAML
Graph API	remapping at startup time only	remapping at runtime
Embedded Systems	roscpp, mROS	microROS, XEL Network, rosZardulino, Renesas DDS-XRCE(Micro-XRCE-DDS), AWS ARCLM

(1) Platforms

ROS 2부터는 3대 운영체제인 **Linux, Windows, macOS**를 모두 지원한다. 이는 바이너리 파일로 설치가 가능하다는 의미로 Windows 사용자가 많은 한국의 경우에는 반가운 소식으로 받아들이는 분들이 많을 것 같다. ROS 2 Jazzy Jalisco 기준으로 보았을 때 Linux는 Ubuntu Noble (24.04), Windows는 Windows 10 버전, macOS는 Mojave (10.14)버전을 지원하고 있다.

Linux의 경우에는 Linux 배포판중 일반 사용자가 가장 많은 Canonical의 Ubuntu 진영에서 [ROS 2 TSC](#)로 가입되어 있어서 관련된 내용을 많이 볼 수 있다. 리눅스 이용자라면 Canonical에서 연재 중인 아래 참고 자료 [Ubuntu Robotics](#), [Ubuntu Robotics Blogs](#)의 내용을 참고하면 ROS 2 사용에 도움이 될 듯싶다. 그리고 Linux, macOS는 ROS 1 부터 지원하고 있었는데 이번 ROS 2에서는 Microsoft가 ROS 2 TSC로 들어오고 Windows용 패키지 및 테스트, [Visual Studio Code Extension for ROS](#) 까지 준비하는 등 굉장한 노력을 기울여 Windows 사용자들도 ROS 2를 쉽게 사용할 수 있게 되었다. Windows 사용자라면 [Win ROS Landing Page](#), [WSL 2\(Windows Subsystem For Linux 2\)](#)의 Windows 관련 문서를 참고하기를 추천한다.

41 ROS 2의 20가지 특징 2. Real-time

(2) Real-time

ROS 2는 [Real-time](#)을 지원한다. 단, 선별된 하드웨어 사용, 리얼타임 운영체제 사용, DDS의 [RTPS\(Real-time Publish-Subscribe Protocol\)](#)와 같은 통신 프로토콜을 사용, 매우 잘 짜여진 리얼타임 코드 사용을 전제로 실시간성을 지원하고 있다. 이 부분은 ROSCon2019의 [Pre-conference Workshops](#)으로 진행되었던 `Doing real-time with ROS 2: Capabilities and challenges`에서 자세히 설명되었는데 이 부분의 내용은 이 강좌에서 설명하기에는 내용이 많아 상당히 길어질 것 같으므로 추후 해당 내용 및 더 자세히 알아보려면 해당 [워크샵의 페이지](#)를 참고하도록 하자.

(3) Security

ROS 1에서는 항상 보안이 문제였다. 노드를 관리하는 [ROS master](#)의 하나의 IP와 포트만 노출되면 모든 시스템을 죽일 수 있었으며, 보안 입장에서 [TCPROS](#)는 뽕 뚫린 큰 구멍에 가까웠다. 하지만 ROS 1은 이러한 부족한 부분을 매우기 보다는 로봇 개발에 사용되는 다양한 하드웨어와 소프트웨어를 평가하고 작동하는 유연성에 더 무게를 두었고 이러한 유연성은 보안에 대한 기회비용보다 더 중요하게 여겼다. 즉, 보안 이슈는 개발 우선 순위에서 뒤져 있었다. 당연히 ROS 초창기에는 이 선택은 옳았다.

하지만 시간이 흘러 이러한 취약점은 **상용 로봇에 ROS를 도입할 수 없게 만드는 첫번째 이유이자 가장 큰 걸림돌**이 되었다. 이에 ROS 2에서는 디자인 설계부터 이 부분을 명확히 짚고 넘어갔다. 우선, TCP 기반의 통신은 [OMG\(Object Management Group\)](#)에서 산업용으로 사용 중인 [DDS\(Data Distribution Service\)](#)를 도입하였고, 자연스럽게 [DDS-Security](#) 이라는 DDS 보안 사양을 ROS에 적용하여 보안에 대한 이슈를 통신단부터 해결하였다. 또한 ROS 커뮤니티에서는 [SROS 2\(Secure Robot Operating System 2\)](#)라는 툴을 개발하였고 보안 관련 RCL 서포트 및 보안관련 프로그래밍에 익숙지 않은 로보틱스 개발자를 위해 보안을 위한 툴킷을 만들어 배포하고 있다. 이 부분에 대한 더 자세한 내용은 ROS 2 디자인 문서([DDS-Security integration](#), [Access Control Policies](#), [Security Enclaves](#), [Robotic Systems Threat Model](#))의 `Security` 관련 부분을 참고하길 바라며 지난 ROSCon에서의 워크샵에서 진행한 [`Is your robot secure? ROS 1 & ROS 2 Security Workshop`](#)의 문서도 참고하면 도움이 될 것 같다. 우리는 추후 강좌를 통해 직접 보안 설정 및 프로그램하는 실습을 진행해보기로 하자.

43 ROS 2의 20가지 특징 4. Communication

4. Communication

ROS 1에서는 자체 개발한 TCPROS와 같은 통신 라이브러리를 사용하고 있던 반면, ROS 2은 리얼타임 퍼블리시와 서브스크라이브 프로토콜인 **RTPS(Real Time Publish Subscribe)**를 지원하는 통신 미들웨어 **DDS**를 사용하고있다. DDS는 **OMG(Object Management Group)**에 의해 **표준화**가 진행되고 있으며, 상업적인 용도에도 적합하다는 평가가 지배적이다. DDS에서는 **IDL(Interface Description Language)**를 사용하여 메시지 정의 및 직렬화를 더 쉽게, 더 포괄적으로 다룰 수 있다. 또한 통신 프로토콜로는 RTPS를 채용하여 실시간 데이터 전송을 보장하고 임베디드 시스템에도 사용할 수 있다. **DDS는 노드 간의 자동 감지 기능을 지원**하고 있어서 기존 ROS 1에서 각 노드들의 정보를 관리하였던 ROS마스터가 없어도 여러 DDS 프로그램 간에 통신 할 수 있다. 또한 노드 간의 통신을 조정하는 **QoS(Quality of Service)** 매개 변수를 설정할 수 있어서 TCP 처럼 데이터 손실을 방지함으로써 신뢰도를 높이거나, UDP 처럼 통신 속도를 최우선하여 사용할 수도 있다. 이러한 다양한 기능을 갖춘 DDS를 이용하여 ROS 1의 퍼블리시, 서브스크라이브 형 메세지 전달은 물론, 실시간 데이터 전송, 불안정한 네트워크에 대한 대응, 보안 강화 등이 강화되었다. DDS의 채용은 ROS 1에서 ROS 2로 바뀌면서 가장 큰 변화점 이다.

44 ROS 2의 20가지 특징 5. Middleware interface

5. Middleware interface

앞서 설명한 DDS는 다양한 기업에서 통신 미들웨어 형태로 제공하고 있다. 그 벤더로는 10 곳이 있는데 이 중 ROS 2를 지원하는 업체는 ADLink, Eclipse Foundation, Eprosima, Gurum Network, RTI로 총 5 곳이다. DDS 제품명으로는 ~~ADLINK의~~ **OpenSplice**, Eclipse Foundation의 **Cyclone DDS**, Eprosima의 **Fast DDS**, Gurum Network의 **Gurum DDS**, RTI의 **Connex DDS**가 있다. 참고로 이 중 Gurum Network는 유일하게 대한민국 기업으로 DDS를 순수 국산 기술로 개발하여 상용화에 성공한 기업이다.

ROS 2에서는 이러한 벤더들의 미들웨어를 사용자가 원하는 사용 목적에 맞게 선택하여 사용할 수 있도록 ROS Middleware(RMW) 형태로 지원하고 있다. 이는 각 벤더들의 미들웨어마다 API가 약간씩 달라도 ROS 2 유저들은 이를 생각하지 않고 통일된 코드로 쉽게 바꿔서 사용할 수 있도록 것으로, RMW는 여러 DDS 구현을 지원하기 위하여 API의 추상화 인터페이스로 지원하고 있다.

6. Node manager (discovery)

ROS 1에서의 필수 실행 프로그램으로는 roscore가 있다. 이를 실행시키면 ROS Master, ROS Parameter Server, rosout logging node가 실행 되었다. 특히 ROS Master는 ROS 시스템의 노드들의 이름 지정 및 등록 서비스를 제공하였고, 각 노드에서 퍼블리시 또는 서브스크라이브하는 메시지를 찾아서 연결할 수 있도록 정보를 제공해 주었다. 즉, 각각 독립되어 실행되는 노드들의 정보를 관리하여 서로 연결해야 하는 노드들에게 상대방 노드의 정보를 건네주어 연결할 수 있게해 주는 매우 중요한 중매 역할을 수행했었다. 이 때문에 ROS 1에서는 노드 사이의 연결을 위해 네임 서비스를 마스터에서 실행했었어야 했고, 이 ROS Master가 연결이 끊기거나 죽는 경우 모든 시스템이 마비되는 단점이 있었다.

ROS 2에서는 roscore가 없어지고 3가지 프로그램이 각각 독립 수행으로 바뀌었다. 특히, **ROS Master의 경우 완전히 삭제**되었는데 이는 DDS를 사용함에 따라 노드를 DDS의 Participant 개념으로 취급하게 되었으며, Dynamic Discovery 기능을 이용하여 **DDS 미들웨어**를 통해 직접 검색하여 노드를 연결 할 수 있게 되었다. 이제 ROS 2에서는 roscore는 Bye~ Bye~ 다.

7. Languages

ROS 2의 프로그램 언어로는 ROS 1과 마찬가지로 **다양한 프로그래밍 언어를 지원**할 예정이다. 아직까지는 **C++, Python**이 주력 언어라고 볼 수 있는데 이 주력 언어도 아래와 같이 큰 변화가 있었다. ROS 2 배포판마다 조금씩 다르기는 하지만 Jazzy를 기준으로 보았을 때, **C++은 C++17, 파이썬은 Python 3.8이 기본 요구 사항**으로 되어있다. 이다. 같은 언어를 쓰더라도 ROS 1을 사용했을 때에는 뭔가 올드한 느낌이었고 최신 언어들이 제공하는 기능들을 못쓰고 그림에 떡이었는데 이제는 최신의 아름다운 언어를 쓰는 기분이다. 이는 사용해보면 알게 될 것이다. 물론 새로운걸 배운다는 것은 어쩔 수 없는 엔지니어의宿命이다.

- ROS 1: C++03, Python 2.7
- ROS 2: C++17, Python 3.8

8. Build system

ROS 2에서는 새로운 빌드 시스템인 **ament**를 사용한다. **ament**는 ROS 1에서 사용되는 빌드 시스템인 **catkin**의 업그레이드 버전이다. ROS 1의 **catkin**이 CMake만을 지원했던 반면, **ament**는 CMake를 사용하지 않는 Python 패키지 관리도 가능하다. 즉, ROS 2에 와서는 Python 패키지는 비로서 처음으로 완전 독립을 이루게 되었는데 ROS 1에서 Python 코드가 있는 패키지는 **setup.py** 파일이 CMake 내에서 사용자 정의 로직으로 처리되었다. 하지만 ROS 2에서 Python 패키지는 **setup.py** 파일의 모든 기능을 순수 Python 모듈과 동등한 수준으로 개발할 수 있게 되었다. 마지막으로 TMI일 수 있으나 **catkin**과 **ament**는 이음동의어로 버드나무의 화수를 의미하며 ROS 1의 개발 주체인 Willow Garage 뒷 마당에 있던 버드나무 화수를 보고 지었다고 한다. 즉, 뭔가 심오한 뜻이 있는 것은 아니다.

- ROS 1: rosbuilt → catkin (CMake)
- ROS 2: ament (CMake), Python setuptools (Full support)

9. Build tools

ROS 1의 경우 여러 가지 다른 도구, 즉 `catkin_make`, `catkin_make_isolated` 및 `catkin_tools`가 지원되었다. ROS 2에서는 알파, 베타, 그리고 Ardent 릴리스까지 빌드 도구로 [ament_tools](#)이 이용되었고 지금에 와서는 `colcon`을 추천하고 있다.

[colcon](#)은 ROS 2 패키지를 작성, 테스트, 빌드 등 ROS 2 기반의 프로그램할 때 빼놓을 수 없는 툴로 작업흐름을 향상시키는 CLI 타입의 명령어 도구이다. 사용 방법은 ``colcon test``와 ``colcon build``와 같이 터미널창에서 수행하게 되며 다양한 옵션을 사용할 수 있다. 이는 추후 이어지는 강좌들에서 더 자세히 다루도록 하겠다.

49 ROS 2의 20가지 특징 10. Build options

10. Build options

ROS 2에서는 빌드 관련 내용들이 모두 변경되면서 빌드 옵션에도 새로운 변화가 생겼다. 그중 사용하면서 가장 좋았던 3가지를 꼽자면 아래와 같다.

우선 **`Multiple workspace`** 이다.

이는 ROS 1에서는 **`catkin\_ws`**와 같이 특정 워크스페이스를 확보하고 하나의 워크스페이스에서 모든 작업을 다 했는데 ROS 2에서는 복수의 독립된 워크스페이스를 사용할 수 있어서 작업 목적 및 패키지 종류별로 관리할 수 있게 되었다.

둘째는 **`No non-isolated build`** 이다.

ROS 1에서는 하나의 CMake 파일로 여러 개의 패키지를 동시에 빌드 할 수 있었다. 이렇게 하면 빌드 속도가 빨라지지만 모든 패키지의 종속성에 신경을 많이 써야 하고 빌드 순서가 매우 중요하게 된다. 또한 모든 패키지가 동일 네임스페이스 사용하게 되므로 이름에서 충돌이 발생할 수 있었다. ROS 2에서는 이전 빌드 시스템인 catkin에서 일부 기능으로 사용되었던 **`catkin\_make\_isolated`** 형태와 같은 격리 빌드만을 지원함으로써 모든 패키지를 별도로 빌드하게 되었다. 이 기능 변화를 통해 설치용 폴더를 분리하거나 병합할 수 있게 되었다.

10. Build options

셋째는 **`No devel space`**이다.

catkin은 패키지를 빌드 한 후 **devel** 이라는 폴더에 코드를 저장한다. 이 폴더는 패키지를 설치할 필요 없이 패키지를 사용할 수 있는 환경을 제공한다. 이를 통해 파일 복사를 피하면서 사용자는 파이썬 코드를 편집하고 즉시 코드 실행할 수 있었다. 단 이러한 기능은 매우 편리한 기능이지만 패키지를 관리하는 측면에서 복잡성을 크게 증가시켰다.

이에 ROS 2에서는 패키지를 빌드 한 후 설치해야 패키지를 사용할 수 있도록 바뀌었다. 단 쉬운 사용성도 고려하여 colcon 사용 시에 **`colcon build --symlink-install`** 와 같은 옵션을 사용하여 심벌릭 링크 설치의 선택적 기능을 사용하여 동일한 이점을 제공하고 있다.

11. Version control system

ROS는 수 많은 소스 코드 공여자로부터 만들어가는 코드의 집합이기 때문에 개인은 물론 소속도 정말 다양하고 각 코드들의 리포지토리도 제각각이다. 예를 들어 어느 패키지는 GitHub를 이용하고 어떤 것은 Bitbucket를 이용한다. 그리고 사용하는 버전 관리 시스템 (Version Control System, VCS)도 Git, Mercurial, Subversion, Bazaar 등 다양하다.

- ROS 1: rosws → wstool, rosininstall (*.rosinstall)
- ROS 2: vcstool (*.repos)

ROS 커뮤니티에서는 이러한 다양한 리포지토리와 혼재된 버전 관리 시스템을 사용하더라도 ROS를 사용함에 있어서 불편함이 없도록 통합적인 툴이 필요했다. ROS 1에서는 처음에 rosws이라는 툴에서 wstool을 이용하였다가 최근 ROS 2에서는 vcstool으로 통합하였다.

11. Version control system

현재 ROS 1에서도 **vcstool**를 사용하고 있는 상황이다. **vcstool**은 여러 리포지토리 작업을 보다 쉽게 관리할 수 있도록 설계된 버전 관리 시스템(VCS) 툴이다. 이 툴은 ROS 2를 소스코드로부터 설치해본 사람이라면 자신도 모르게 사용했을 것이다. 아래의 명령어 2줄을 살펴보자. 우선 **wget**을 통하여 **ros2.repos**라는 파일을 받게 되는데 이 파일에는 **vcs** 타입은 무엇이고, 리포지토리 주소는 어떻게 되며, 설치해야하는 브랜치는 어떤 것인지가 명시된 파일이다. 이러한 정보가 기재된 *.repos 파일을 이용하여 다양한 리포지토리, 다양한 **vcs**를 지원하며 패키지들을 관리할 수 있도록 하는 것을 의미한다. 특히 ROS 2에서는 기존 **vcs** 툴을 통합하여 **vcstool**이라는 이름으로 제공되어 사용에 매우 편리하게 되었다. 자세한 사용법은 [README](#) 파일을 참고하도록 하자.

```
wget https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos
vcs import src < ros2.repos
```

12. Client library

ROS 기반의 프로그래밍을 작성한다는 것은 [ROS Middleware Interface](#)에서 유저 코드 영역(user land)을 다룬다는 것으로 그 밑에는 ROS 클라이언트 라이브러리 (ROS Client Library)이 있고, 이 클라이언트 라이브러리는 앞서 설명한 미들웨어(middleware interface)를 사용하고 있다는 것을 알고 있어야 한다. 여기서 유저는 개발 목적에 따라 C/C++, Python, Java, Node.js 등을 사용할 것이다. ROS에서는 초창기 부터 이러한 멀티 프로그래밍 언어를 지원하고 있는데 ROS 1에서는 roscpp, rospy, roslisp 등 각 프로그래밍 언어에 대해 [클라이언트 라이브러리 \(Client Library\)](#)를 제공했다. 한편, ROS 2에서는 ROS 클라이언트 라이브러리를 **RCL(ROS Client Library)**이라는 이름으로 제공한다. 그리고 프로그래밍 언어별로 [rclcpp](#), [rclcpp](#), [rclpy](#), [rcljava](#), [rclobjc](#), [rclada](#), [rclgo](#), [rclnodejs](#) 등으로 제공된다. 또한 ROS 2는 앞서 설명한바와 같이 C이면 C99, C++ 이라면 C++ 14/17, Python라면 Python 3 (3.5+) 등 최신 기술 사양에 대응하고 있다. 각 C++/Python ROS Client Library API는 [rclcpp](#), [rclpy](#)을 미리 봐둔다면 도움일 될 것이다.

13. Life cycle

로봇 개발에 있어서 로봇의 현재 상태를 파악하고 현재 상태에서 다른 상태로 변경되는 상태전이 제어는 수십년간 로봇공학에서도 주요 연구 주제로 다루었던 중요한 부분 중에 하나이다. 특히 태스크 수행 측면에서 현재의 상태 파악과 전이는 멀티 태스크 수행에서 빠질 수 없는 중요한 부분일 것이고 복수의 로봇 복수의 복합 태스크, 서비스 수행과 같은 상위 레벨의 프로그램일수록 더 중요하게 다루어지는 부분이다. ROS 1에서는 이러한 기능을 구현하기 위해서는 [SMACH](#)과 같은 상태 전이를 관리하는 독립적인 패키지를 사용했어야 했고 클라이언트 라이브러리에서는 상태 관리하는 부분이 없었기에 사용자가 임의로 클라이언트 라이브러리 부분까지 수정하여 사용했어야 했다.

ROS 2에서는 이러한 니즈를 반영하여 패키지의 각 노드들의 현재 상태를 모니터링하고 상태를 제어 가능한 [lifecycle](#)을 클라이언트 라이브러리에 포함시켰으며 이를 통해 ROS 시스템 상태를 보다 효과적으로 제어 할 수 있게 되었다. 이를 이용하게 되면 기존 ROS 1에서는 할 수 없었던 노드의 상태를 모니터링하고 상태를 전이시키거나 노드를 상태에 따라 재시작하거나 교체할 수도 있게 된다.

14. Multiple nodes

ROS 1의 초기에는 **하나의 프로세스에서 여러 노드를 실행** 할 수 없었다. 하지만 이러한 요구는 지속적으로 제기되었고 하나의 프로세스에서 여러 노드를 작성하기 위해 **nodelet**라는 새로운 기능이 ROS 1에 추가되었다. 이는 하드웨어 리소스가 제한적이거나 노드 간에 수 많은 메시지를 보내야 할 때 유용하게 사용되었다.

ROS 2에서 **nodelet**이 사용되지는 않고 **RCL**에 포함되어 있다. 이름은 **컴포넌트 (components)**라고 부르며 ROS 2에서는 이 컴포넌트를 사용하여 동일한 실행 파일에서 복수의 노드를 수행할 수 있게 되었다. 이를 사용하게 되면 노드의 실행 파일 수준은 더 세분화 시킬 수 있으며 프로세스 내 통신 **IPC(intra-process communication)** 기능을 이용하여 ROS 2의 통신 오버 헤드를 제거 할 수 있어서 더 효율적인 ROS 2 응용 프로그램을 작성 가능하다.

15. Threading model

ROS 1에서 개발자는 단일 스레드 실행 또는 다중 스레드 실행 중 하나만 선택할 수 있었다. ROS 2에서는 더 세분화 된 실행 모델(executor)을 C++과 Python에서 사용할 수 있으며 사용자가 정의한 실행기도 제공되는 RCL API를 이용하여 쉽게 구현할 수 있다. **Single Threaded Executor, Multi Threaded Executor**는 각 클라이언트 라이브러리마다 구현 방식이 다르기에 관련 설명은 [rclcpp executors](#), [rclpy executors](#) 문서를 참고하도록 하자.

57 ROS 2의 20가지 특징 16. Messages (topic, service, action)

16. Messages (topic, service, action)

ROS 2에서도 기존 ROS 1의 메시지([Messages](#))와 마찬가지로 단일 데이터 구조를 메시지라고 정의하며 정해진 또는 사용자가 정의한 메시지를 사용할 수 있으며, 각 패키지 이름과 마찬가지로 이름과 각 지정된 형식으로 메시지를 고유하게 식별할 수 있다. 사용처도 기존과 마찬가지로 Topic, Service, Action 등에서 사용하며 기존과 비슷한 형태([Interface definition](#), [interface file](#))로 사용 가능하다.

여기에 ROS 2에서는 OMG(Object Management Group)에서 정의된 [IDL\(Interface Description Language\)](#)을 사용하여 메시지 정의 및 직렬화를 더 쉽게, 더 포괄적으로 다룰 수 있게 되었다. IDL을 이용하게 되면 기존 ROS 메시지 컨셉과 마찬가지로 다양한 프로그래밍 언어로 작성된 메시지를 사용할 수 있다. 이전에 CORBA (일명, 코바)를 써본 사람들은 IDL이 친숙할 것이다. ROS 2에서는 기존 **msg**, **srv**, **action** 파일 이외에도 IDL을 지원한다. 그리고 ROS 2가 DDS를 채용하게 되면서 기존 메시지들과 DDS 규칙을 맞추는 작업이 진행되었다. ROS 인터페이스 유형과 DDS IDL 유형 간의 매핑은 [Mapping between ROS interface types and DDS IDL types](#) 자료를 참고하면 좋을 듯 싶으며 전체적으로 다듬어진 정리표는 [Interfaces](#) 글을 참고하도록 하자.

그리고, ROS 2에서는 DDS를 사용하면서 메시지를 이용한 Topic, Service, Action 등의 컨셉은 변하지 않으나 사용 방법은 상당히 많이 바뀌었다. 이 부분에 대한 설명은 이어지는 강좌를 통해 예제와 함께 하나 하나 알아가보자.

17. Command Line Interface

대부분의 CLI 타입의 명령어 사용법은 기존 ROS 1과 매우 비슷해서 약간의 이름 변경과 일부 옵션 사용법만 익힌다면 사용시 큰 차이는 없다. 자주 사용되는 명령어를 예를 들어 보자면 아래와 같은 차이 정도이다. ROS 1 명령어에 비해 명령어가 약간 길어진듯 보이긴 하지만 자주 쓰는 명령어는 "alias rt='ros2 topic list'"와 같이 설정하여 사용하면 되기에 큰 무리는 없다. 더욱이 ROS 2의 CLI 형태의 명령어는 ROS 2 TSC 멤버이자 Ubuntu 개발 업체인 Canonical이 담당하고 있어서 더욱 믿음이 간다. 자세한 설명은 [ROS 2 Command Line Interface](#)를 참고하기 바람에 실습을 해보고 싶다면 [발표 자료](#)를 참고하면 좋다.

- ROS 1: 'rostopic list'
- ROS 2: 'ros2 topic list'

18. roslaunch

ROS의 실행 시스템은 대표적으로 `run`과 `launch`가 있는데 `run`은 단일 프로그램 실행, `launch`는 사용자 지정 프로그램 실행을 수행한다. 사용면에서는 `run`에 비해 다양한 설정을 할 수 있는 `launch` 사용이 월등히 사용 빈도가 높다. `launch`는 사용자가 실행하고자하는 프로그램의 각종 설정을 기술하고 기술된 설정에 맞추어 각종 프로그램을 실행하도록 도와준다. 사용자가 지정하는 설정에는 실행할 프로그램, 실행할 위치, 전달할 인수 등 시스템 전체의 구성 요소를 쉽게 재사용 할 수 있도록 하고있다. 그 목적과 컨셉은 ROS 2에서도 크게 다르지 않다.

launch의 [ROS 1과 ROS 2의 차이점](#)을 살펴보면 다양한 파일 사용이다. ROS 1에서는 `roslaunch` 파일이 특정 `XML` 형식을 사용해었다. 이 형식을 이용해도 다양한 설정을 추가하여 프로그램을 실행 시킬 수 있어서 매우 편했는데, ROS 2에서는 `XML`, `YAML` 형식 이외에도 `Python`이 새롭게 채용되어 조건문 및 Python 모듈을 추가로 사용하여 보다 복잡한 논리와 기능을 사용할 수 있게 되었다. 어떤 방식으로 사용하는지에 대한 추가 설명은 [튜토리얼](#)을 참고하도록 하자.

19. Graph API

ROS는 메타패키지, 패키지, 노드 그리고 노드간의 데이터 교환을 위한 토픽 등으로 구성되어 있다. 이 때의 각 노드와 토픽, 메시지 등이 고유의 이름을 가지고 있고, 매핑이 이루어져 각 노드와 노드간의 토픽, 메시지의 관계를 그래프화 시킬 수 있도록 되어있다. 이러한 그래프 구조를 시각화하는 툴인 rqt_graph를 제공하고 있어서 현재 네트워크 상의 각 구성요소의 연결성을 시각적으로 확인할 수 있다.

61 ROS 2의 20가지 특징 20. Embedded Systems

20. Embedded Systems

로봇 개발에 있어서 실시간성을 담보 받으며 모터 및 센싱을 제어하는 부분은 매우 중요하게 다루어져 왔다. 이 강좌 초반에서 언급했듯이 ROS 커뮤니티에서도 이를 위해 ROS 2에서는 선별된 하드웨어 사용, 리얼타임 운영체제 사용, DDS의 RTPS(Real-time Publish-Subscribe Protocol)와 같은 통신 프로토콜을 사용, 매우 잘 짜여진 리얼타임 코드 사용을 전제로 실시간성을 지원하고 있다. 하지만 실시간성이라는 것은 상위 소프트웨어에서 다루기에는 제약이 많다. 오히려 Embedded Systems 안에서 해결하는게 더 적합하다고 보고 있다.

이에 ROS 2 개발 초기부터 Embedded Systems에 대한 관심이 높았는데 초기 개발 컨셉은 ROS의 창시자인 Morgan Quigley가 ROSCon2015에서 ‘ROS 2 on “small” embedded systems’이라는 이름으로 [발표한 자료](#) 및 [영상](#)를 보면 도움이 될 것이다. ROS 1에서도 Embedded Systems를 지원하지 않는 것은 아니었다. 단, 매우 기초적인 수단이라고 볼 수 있는 임베디드 보드와 메시지를 주고 받을 때 시리얼([rosserial](#))로 통하여 통신하였다. **ROS 2에서는 한발 더 나아가 기존 시리얼 통신, 블루투스 및 와이파이 통신을 지원하거나 RTOS (Real-Time Operating System)를 사용하고 기존 DDS 대신 eXtremely Resource Constrained Environments (DDS-XRCE)를 사용 하는 등 임베디드 보드에서 직접 ROS 프로그래밍을 하여 하드웨어 펌웨어로 구현된 노드를 실행할 수도 있다.** 이 방법론에는 여러 가지가 있을 수 있는데 현재 ARM 사를 포함한 다양한 MCU 제조 업체에서 이를 지원하기 위하여 다양한 방법론을 내놓고 있는 상태이고, eProsima, BOSCH, ROBOTIS, FIWARE, Amazon, Renesas 등에서 다음 참고 자료와 같이 다양한 임베디드 지원 방법에 대해 개발, 공개하고 있다

20. Embedded Systems

임베디드 환경에서 DDS 및 ROS 메시지 통신 등에 관심 있는 사람은 아래 참고 자료를 참고하도록 하자.

- ROS 1: roserial, mROS
- ROS 2: micro-ROS, XEL Network, ros2arduino, Renesas, DDS-XRCE(Micro-XRCE-DDS), AWS ARCLM

[참고 자료]

- <https://micro-ros.github.io/>
- <http://xelnetwork.robotis.com/>
- <https://github.com/ROBOTIS-GIT/ros2arduino>
- <https://www.renesas.com/us/en/solutions/key-technology/robot/robot-operating-system.html>
- <https://micro-xrce-dds.docs.eprosima.com/en/latest/>

ROS 2와 DDS (Data Distribution Service)

64 ROS의 메시지 통신

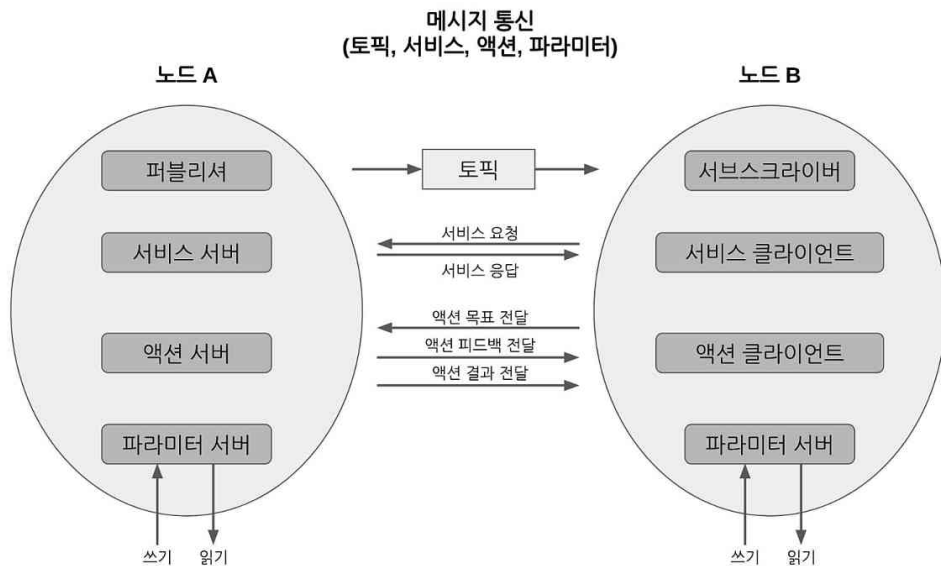
로봇 운영체제 ROS에서 중요시 여기는 몇 가지 용어 정의 및 메시지, 메시지 통신에 대해 먼저 알아보도록 하자. 특히, **메시지 통신**은 ROS 프로그래밍에 있어서 **ROS 1과 2의 공통된 중요한 핵심 개념** 이기에 ROS 프로그래밍에 들어가기 전에 꼭 이해하고 넘어가야 할 부분이다.

ROS에서는 프로그램의 재사용성을 극대화하기 위하여 최소 단위의 실행 가능한 프로세스라고 정의하는 **노드 (node)** 단위의 프로그램을 작성하게 된다. 이는 하나의 실행 가능한 프로그램으로 생각하면 된다. 그리고 하나 이상의 노드 또는 노드 실행을 위한 정보 등을 묶어 놓은 것을 **패키지 (package)**라고 하며, 패키지의 묶음을 **메타패키지 (metapackage)**라 하여 따로 분리한다.

여기서 제일 중요한 것은 실제 실행 프로그램인 노드인데 앞서 이야기한 것과 마찬가지로 ROS에서는 최소한의 실행 단위로 프로그램을 나누어 프로그래밍하기 때문에 노드는 각각 별개의 프로그램이라고 이해하면 된다. 이에 수많은 노드들이 연동되는 ROS 시스템을 위해서는 **노드와 노드 사이에 입력과 출력 데이터**를 서로 주고받게 설계해야만 한다.

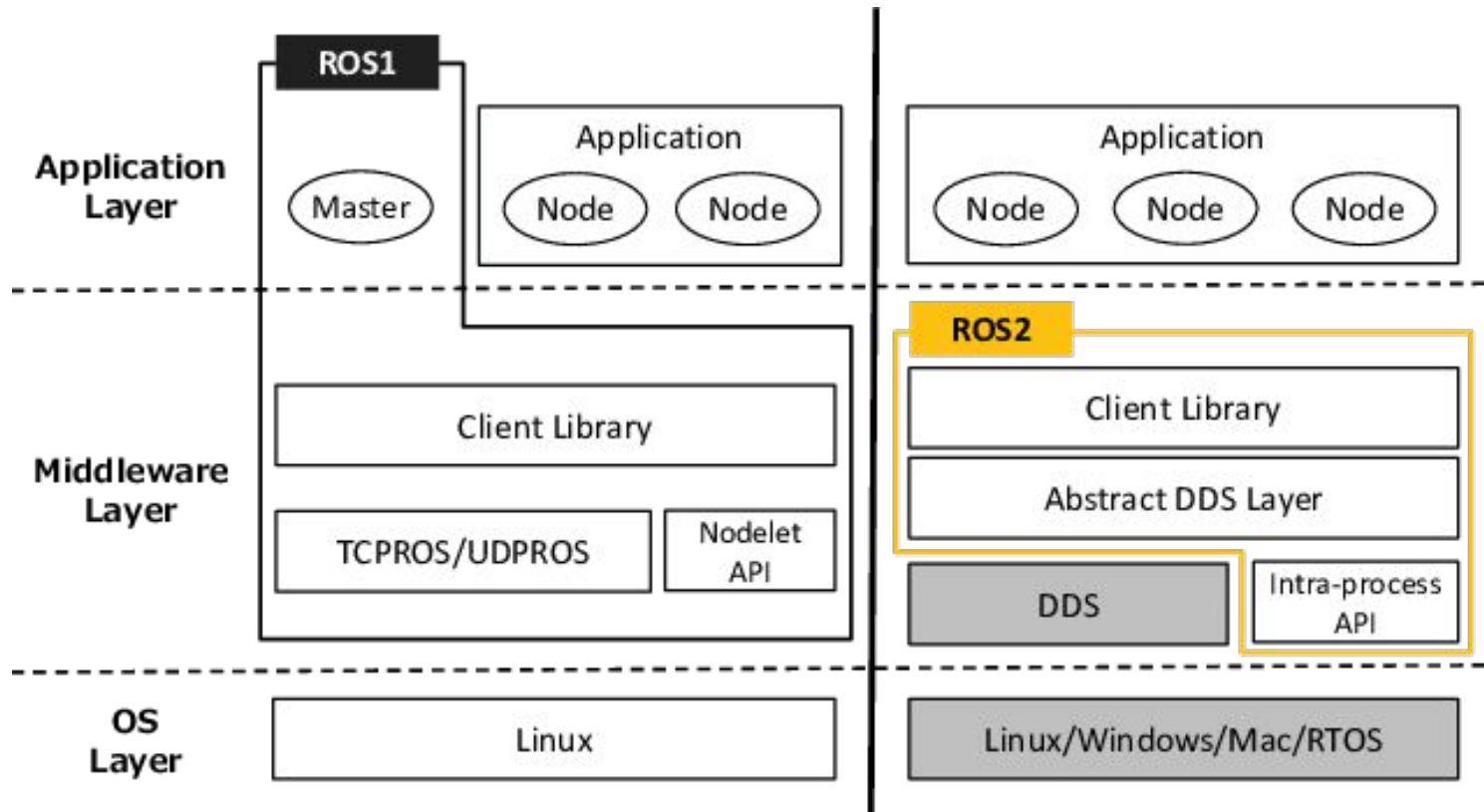
65 ROS의 메시지 통신

여기서 주고받는 데이터를 ROS에서는 **메시지 (message)**라고 하고 주고받는 방식을 메시지 통신이라고 한다. 여기서 데이터에 해당되는 메시지(message)는 integer, floating point, boolean, string 와 같은 변수 형태이며 메시지 안에 메시지를 품고 있는 간단한 데이터 구조 및 메시지들의 배열과 같은 구조도 사용할 수 있다. 그리고 메시지를 주고받는 통신 방법에 따라 **토픽 (topic)**, **서비스 (service)**, **액션 (action)**, **파라미터 (parameter)**로 구분된다.

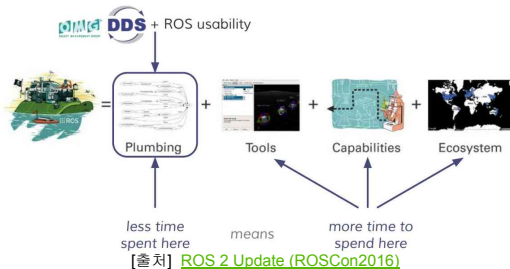


ROS에서 사용되는 메시지 통신 방법으로는 **토픽(topic)**, **서비스(service)**, **액션(action)**, **파라미터(parameter)**가 있다. 각 메시지 통신 방법의 목적과 사용 방법은 다르기는 하지만 **토픽의 발간(publish)과 구독(subscribe)의 개념**을 응용하고 있다. 이 데이터를 보내고 받는 발간, 구독 개념은 ROS 1은 물론 ROS 2에서도 매우 중요한 개념으로 변함이 없는데 이 기술에 사용된 통신 라이브러리는 ROS 1, 2에서 조금씩 다르다.

ROS 1에서는 자체 개발한 TCPROS와 같은 통신 라이브러리를 사용하고 있던 반면, ROS 2에서는 OMG(Object Management Group)에 의해 표준화된 **DDS(Data Distribution Service)**의 리얼타임 퍼블리시와 서브스크라이브 프로토콜인 **DDSI-RTPS(Real Time Publish Subscribe)**를 사용하고 있다. ROS 2 개발 초기에는 기존 TCPROS를 개선하거나 ZeroMQ, Protocol Buffers 및 Zeroconf 등을 이용하여 미들웨어처럼 사용하는 방법도 제안되었으나 무엇보다 산업용 시장을 위해 표준 방식 사용을 중요하게 여겼고, ROS 1때와 같이 자체적으로 만들기 보다는 산업용 표준을 만들고 생태계를 꾸려가고 있었던 DDS를 통신 미들웨어로써 사용하기로 하였다. DDS 도입에 따라 다음 그림과 같이 ROS의 레이어아웃은 크게 바뀌게 되었다. 처음에는 DDS 채용에 따른 장점과 단점에 대한 팽팽한 줄다리기 토론으로 걱정의 목소리도 높였지만 지금에 와서는 **ROS 2에서의 DDS 도입은 상업적인 용도로 ROS를 사용할 수 있게 발판을 만들었다는 것에 가장 큰 역할을 했다는 평가가 지배적**이다.



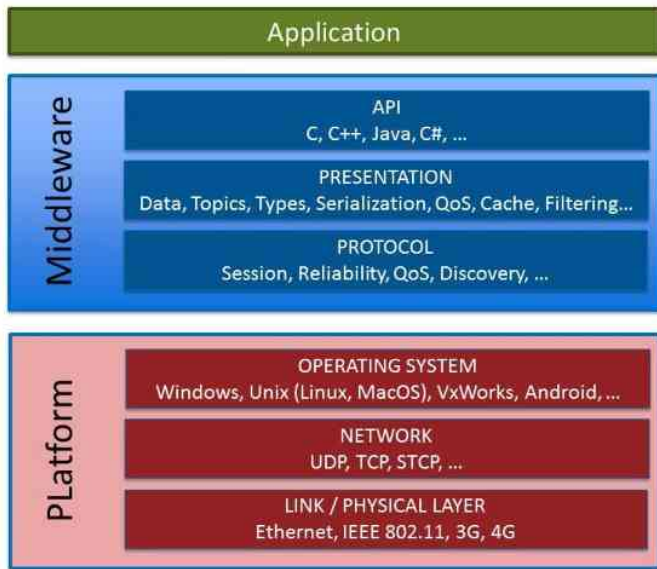
DDS 도입으로 기존 메시지 형태 이외에도 OMG의 CORBA 시절부터 사용되던 IDL(Interface Description Language,)를 사용하여 메시지 정의 및 직렬화를 더 쉽게, 더 포괄적으로 다룰 수 있게 되었다. 또한 DDS의 중요 컨셉인 DCPS(data-centric publish-subscribe), DLRL(data local reconstruction layer)의 내용을 담아 재정한 통신 프로토콜로인 **DDSI-RTPS**를 채용하여 실시간 데이터 전송을 보장하고 임베디드 시스템에도 사용할 수 있게 되었다. DDS의 사용으로 노드 간의 동적 검색 기능을 지원하고 있어서 기존 ROS 1에서 각 노드들의 정보를 관리하였던 ROS Master가 없어도 여러 DDS 프로그램 간에 통신할 수 있다. 또한 노드 간의 데이터 통신을 세부적으로 조정하는 QoS(Quality of Service)를 매개 변수 형태로 설정할 수 있어서 TCP처럼 데이터 손실을 방지함으로써 신뢰도를 높이거나, UDP처럼 통신 속도를 최우선시하여 사용할 수도 있다. 그리고 산업용으로 사용되는 미들웨어인 만큼 DDS-Security 도입으로 보안 측면에서도 큰 혜택을 얻을 수 있었다. 이러한 다양한 기능을 갖춘 DDS를 이용하여 ROS 1의 퍼블리시, 서브스크라이브형 메시지 전달은 물론, 실시간 데이터 전송, 불안정한 네트워크에 대한 대응, 보안 등이 강화되었다. **DDS의 채용은 ROS 1에서 ROS 2로 바뀌면서 가장 큰 변화점이자 다음 그림과 같이 개발자 및 사용자로 하여금 통신 미들웨어에 대한 개발 및 이용 부담을 줄여 진짜로 집중해야 할 부분에 더 많은 시간을 쏟을 수 있게 되었다.**



자~ 이제 기본 배경이 되는 썰을 풀었으니 본격적으로 DDS에 대해 알아보자. 처음 DDS를 ROS 2에 도입하자는 이야기가 나왔을 때, DDS라는 단어 자체를 처음 들어봤기에 너무 어려웠다. 결론부터 말하자면 DDS는 데이터 분산 시스템이라는 용어로 OMG에서 표준을 정하고자 만든 트레이드 마크(TM)였다. 그냥 용어이고 그 실체는 데이터 통신을 위한 미들웨어이다.

DDS가 ROS 2의 미들웨어로 사용하는 만큼 그 자체에 대해 너무 자세히 알 필요는 없을 듯싶고 ROS 프로그래밍에 필요한 개념만 알고 넘어가면 될 듯싶다. 우선 정의부터 알아보자. **DDS**는 **Data Distribution Service**, 즉 데이터 분산 서비스의 약자이다.

DDS는 데이터 분산 시스템이라는 개념을 나타내는 단어이고 실제로는 데이터를 중심으로 연결성을 갖는 미들웨어의 프로토콜 (DDSI-RTPS)과 같은 DDS 사양을 만족하는 미들웨어 API가 그 실체이다. 이 미들웨어는 ISO 7 계층 레이어에서 호스트 계층(Host layers)에 해당되는 4~7 계층에 해당되고 ROS 2에서는 위에서 언급한 다음 그림과 같이 운영 체제와 사용자 애플리케이션 사이에 있는 소프트웨어 계층으로 이를 통해 시스템의 다양한 구성 요소를 보다 쉽게 통신하고 데이터를 공유할 수 있게 된다.



DDS의 특징은 다양하겠지만 DDS를 ROS 2의 미들웨어로 사용해보면서 느낀 장점은 아래와 같이 10가지이다. 여기서는 이 10가지에 대해 하나씩 정리해보고 각 기능들은 이어지는 강좌에서 실습을 통해 더 자세히 알아보자.

1. **Industry Standards**
2. **OS Independent**
3. **Language Independent**
4. **Transport on UDP/IP**
5. **Data Centricity**
6. **Dynamic Discovery**
7. **Scalable Architecture**
8. **Interoperability**
9. **Quality of Service (QoS)**
10. **Security**

1. 산업 표준

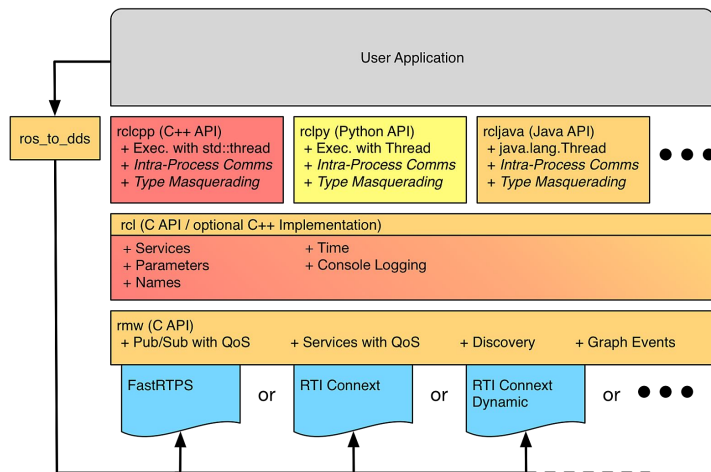
DDS는 분산 객체에 대한 기술 표준을 제정하기 위해 1989년에 설립된 비영리 단체인 OMG(Object Management Group, 객체 관리 그룹)가 관리하고 있는 만큼 산업 표준으로 자리 잡고 있다. 지금까지 OMG가 진행하여 ISO 승인된 표준으로는 UML, SysML, CORBA 등이 있다. 2001년에 시작된 DDS 표준화 작업도 잘 진행되어 지금에 와서는 OpenFMB, Adaptive AUTOSAR, MD PnP, GVA, NGVA, ROS 2와 같은 시스템들에서 DDS를 사용하며 산업 표준의 기반이 되고 있다. ROS 1에서의 TCPROS는 독자적인 미들웨어라는 성격이 짙었는데 ROS 2에 와서는 DDS 사용으로 더 넓은 범위로 사용 가능하게 되었으며 **산업 표준을 지키고 있는 만큼 로봇 운영체제 ROS가 IoT, 자동차, 국방, 항공, 우주 분야로 넓혀갈 수 있는 발판이 마련 되었다고** 생각한다.

2. 운영체제 독립

DDS는 **Linux, Windows, macOS, Android, VxWorks** 등 다양한 운영체제를 지원하고 있기에 사용자가 사용하던 운영체제를 변경할 필요가 없다. 멀티 운영체제 지원을 컨셉으로 하고 있는 **ROS 2**에도 매우 적합하다고 볼 수 있다.

3. 언어 독립

DDS는 미들웨어이기에 그 상위 레벨이라고 볼 수 있는 사용자 코드 레벨에서는 DDS 사용을 위해 기존에 사용하던 프로그래밍 언어를 바꿀 필요가 없다. ROS 2에서도 이 특징을 충분히 살려 하기 그림과 같이 DDS를 **RMW(ROS middleware)**으로 디자인되었으며 벤더 별로 각 RMW가 제작되었으며, 그 위에 사용자 코드를 위해 **rclcpp, rcl, rclpy, rcljava, rclobjc, rclada, rclgo, rclnodejs** 같이 다양한 언어를 지원하는 **ROS 클라이언트 라이브러리 (ROS Client Library)**를 제작하여 멀티 프로그래밍 언어를 지원하고 있다.



* Intra-Process Comms and Type Masquerading could be implemented in the client library, but may not currently exist.

[출처] https://github.com/ros2/ros_core_documentation

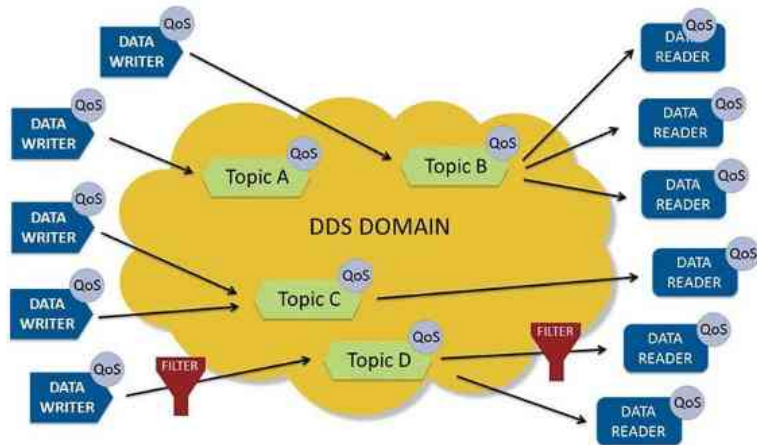
4. UDP 기반의 전송 방식

DDS 벤더 별로 **DDS Interoperability Wire Protocol (DDSI-RTPS)**의 구현 방식에 따라 상이할 수 있으나 일반적으로 **UDP 기반의 신뢰성 있는 멀티캐스트 (reliable multicast)**를 구현하여 시스템이 최신 네트워킹 인프라의 이점을 효율적으로 활용할 수 있도록 돕고 있다. UDP 기반이라는 것이 ROS 1에서의 TCPROS가 TCP 기반이었던 것에 비해 매우 큰 변화인데 UDP의 멀티캐스트(multicast)는 브로드캐스트(broadcast)처럼 여러 목적지로 동시에 데이터를 보낼 수 있지만, 불특정 목적지가 아닌 특정된 도메인 그룹에 대해서만 데이터를 전송하게 된다. 참고로 ROS 2에서는 `'ROS_DOMAIN_ID'`라는 환경 변수로 도메인을 설정하게 된다. 이 멀티캐스트의 방식 도입으로 ROS 2에서는 전역 공간이라 불리는 DDS Global Space이라는 공간에 있는 토픽들에 대해 구독 및 발행을 할 수 있게 된다. Best effort 개념인 UDP는 reliable을 보장하는 TCP에 비해 장단점이 있는데 이 또한 후에 설명하는 QoS(Quality of Service)를 통해 보완 및 해결되었다.

* 참고로 일부 RMW 기능에는 TCP 기반으로 구현되는 경우도 있다.

5. 데이터 중심적 기능

다양한 미들웨어가 있겠지만 그중 DDS를 사용하면서 제일 많이 듣는 말 중에 하나는 'Data Centric'이라는 것이다. 우리말로로는 데이터 중심적이라는 것인데 실제로 DDS를 사용하다보면 이 말이 이해가 된다. DDS 사양에도 **DCPS(data-centric publish-subscribe)**이라는 개념이 나오는데 이는 적절한 수신자에게 적절한 정보를 효율적으로 전달하는 것을 목표로 하는 **발간 및 구독 방식**이라는 것이다. **DDS의 미들웨어를 사용자 입장에서 본다면 어떤 데이터인지, 이 데이터가 어떤 형식인지, 이 데이터를 어떻게 보낼 것인지, 이 데이터를 어떻게 안전하게 보낼 것인지에 대한 기능이 DDS 미들웨어에 녹여있기 때문이다.**



6. 동적 검색

DDS는 **동적 검색(Dynamic Discovery)**을 제공한다. 즉, 응용 프로그램은 DDS의 동적 검색을 통하여 어떤 토픽이 지정 도메인 영역에 있으며 어떤 노드가 이를 발신하고 수신하는지 알 수 있게 된다. 이는 ROS 프로그래밍할 때 데이터를 주고받을 노드들의 IP 주소 및 포트를 미리 입력하거나 따로 구성하지 않아도 되며 사용하는 시스템 아키텍처의 차이점을 고려할 필요가 없기 때문에 모든 운영 체제 또는 하드웨어 플랫폼에서 매우 쉽게 작업할 수 있다.

ROS 1에서는 ROS Master에서 ROS 시스템의 노드들의 이름 지정 및 등록 서비스를 제공하였고, 각 노드에서 퍼블리시 또는 서브스크라이브하는 메시지를 찾아서 연결할 수 있도록 정보를 제공해 주었다. 즉, 각각 독립되어 실행되는 노드들의 정보를 관리하여 서로 연결해야 하는 노드들에게 상대방 노드의 정보를 건네주어 연결할 수 있게 해 주는 매우 중요한 중매 역할을 수행했었다. 이 때문에 ROS 1에서는 노드 사이의 연결을 위해 네임 서비스를 마스터에서 실행했었어야 했고, 이 ROS Master가 연결이 끊기거나 죽는 경우 모든 시스템이 마비되는 단점이 있었다.

ROS 2에서는 ROS Master가 없어지고 DDS의 동적 검색 기능을 사용함에 따라 노드를 DDS의 Participant 개념으로 취급하게 되었으며, 동적 검색 기능을 이용하여 DDS 미들웨어를 통해 직접 검색하여 노드를 연결할 수 있게 되었다.

7. 확장 가능한 아키텍처

OMG의 DDS 아키텍처는 IoT 디바이스와 같은 소형 디바이스부터 인프라, 국방, 항공, 우주 산업과 같은 초대형 시스템으로까지 확장할 수 있도록 설계되었다. 그렇다고 사용하기 복잡한 것도 아니다. DDS의 **Participant 형태의 노드는 확장 가능한 형태로 제공**되어 사용할 수 있으며 단일 표준 통신 계층에서 많은 복잡성을 흡수하여 **분산 시스템 개발을 더욱 단순화 시켜 편의성을 높였다.**

특히 ROS와 같이 최소 실행 가능한 노드 단위로 나누어 수백, 수천 개의 노드를 관리해야 하는 시스템에서는 이 부분이 강점으로 보이며 한대의 로봇이 아닌 **복수의 로봇, 주변 인프라와 다양한 IT 기술, 데이터베이스, 클라우드로 연결 및 확장**해야 하는 ROS 시스템에 매우 적합한 기능이다.

79 DDS의 특징 8. 상호 운용성

8. 상호 운용성

ROS 2에서 통신 미들웨어로 사용하고 있는 DDS는 상호 운용성을 지원하고 있다. 즉, DDS의 표준 사양을 지키고 있는 벤더 제품을 사용한다면 A라는 회사의 제품을 사용하였다가도 B라는 회사 제품으로 변경이 가능하고, A 제품과 B 제품을 혼용하여 서로 다른 제품의 DDS 제품을 사용하더라도 A 제품과 B 제품간의 상호 통신도 지원한다는 것이다. 현재 DDS 벤더로는 10 곳이 있는데 이 중 ROS 2를 지원하는 업체는 **ADLink**, Eclipse Foundation, Eprosima, Gurum Network, RTI로 총 5 곳이며 DDS 제품명으로는 **ADLINK**의 **OpenSplice**, Eclipse Foundation의 **Cyclone DDS**, Eprosima의 **Fast DDS**, Gurum Network의 **Gurum DDS**, RTI의 **Connex DDS**가 있다. 이 중 Fast DDS와 Cyclone DDS는 오픈 소스를 지향하고 있기에 자유롭게 사용 가능하며 기술 지원을 개별적으로 받기 원한다면 상용 제품인 Connex DDS, Gurum DDS를 사용하면 된다.



80 DDS의 특징 9. 서비스 품질 (QoS)

9. 서비스 품질 (QoS)

ROS 2에서는 DDS 도입으로 데이터의 송수신 관련 설정을 목적에 맞추어 유저가 직접 설정할 수 있게 되었다. 노드 간의 DDS 통신 옵션을 설정하는 **QoS(Quality of Service)**가 그것인데 퍼블리셔 및 서브스크라이브 등을 선언하고 사용할 때 매개 변수처럼 QoS를 사용할 수 있다.

DDS 사양상 설정 가능한 QoS 항목은 22가지인데 ROS 2에서는 현재 TCP처럼 데이터 손실을 방지함으로써 신뢰도를 우선시하거나 (**reliable**), UDP처럼 통신 속도를 최우선시하여 사용(**best effort**)할 수 있게 하는 신뢰성(**reliability**) 기능이 대표적으로 사용되고 있다. 그 이외에 또한 통신 상태에 따라 정해진 사이즈만큼의 데이터를 보관하는 **History** 기능, 데이터를 수신하는 서브스크라이버가 생성되기 전의 데이터를 사용할지 폐기할지에 대한 설정인 **Durability** 기능, 정해진 주기 안에 데이터가 발신 및 수신되지 않을 경우 이벤트 함수를 실행시키는 **Deadline** 기능, 정해진 주기 안에서 수신되는 데이터만 유효 판정하고 그렇지 않은 데이터는 삭제하는 **Lifespan** 기능, 정해진 주기 안에서 노드 혹은 토픽의 생사 확인하는 **Liveliness**도 설정할 수 있다. 이러한 다양한 QoS 설정을 통해 DDS는 적시성, 트래픽 우선순위, 안정성 및 리소스 사용과 같은 데이터를 주고받는 모든 측면을 사용자가 제어할 수 있게 되었고 특정 상황, 예를 들어 매우 빠른 속도 데이터를 주고받거나 매우 역동적이고 까다롭고 예측할 수 없는 통신환경에서 데이터 송/수신에 다양한 옵션 설정으로 이를 달성하거나 장애를 극복할 수 있게 되었다.

* 이 QoS 관련 부분은 실제 코딩에서 어떻게 설정하느냐가 매우 중요하기에 **reliability, History, Durability, Deadline, Lifespan, Liveliness**에 대한 스터디가 필요하다.

81 DDS의 특징 10. 보안

10. 보안

ROS 1의 가장 큰 구멍이었던 보안 부분은 ROS 2 개발에서 DDS으로 해결되었다. DDS의 사양에는 **DDS-Security**이라는 DDS 보안 사양을 ROS에 적용하여 보안에 대한 이슈를 통신단부터 해결하였다. 또한 ROS 커뮤니티에서는 **SROS 2(Secure Robot Operating System 2)**라는 툴을 개발하였고 보안 관련 RCL 서포트 및 보안 관련 프로그래밍에 익숙지 않은 로보틱스 개발자를 위해 보안을 위한 툴킷을 만들어 배포하고 있다. 이 부분에 대한 설명도 추후 이어지는 강좌에서 실습을 통해 더 자세히 알아보기로 하자.

82 DDS 사용 예시: 기본적인 퍼블리셔 노드와 서브스크라이브 노드 실행

```
$ ros2 run demo_nodes_cpp listener
```

```
[INFO]: I heard: [Hello World: 1]
```

```
[INFO]: I heard: [Hello World: 2]
```

```
[INFO]: I heard: [Hello World: 3]
```

```
[INFO]: I heard: [Hello World: 4]
```

```
[INFO]: I heard: [Hello World: 5]
```

```
$ ros2 run demo_nodes_cpp talker
```

```
[INFO]: Publishing: 'Hello World: 1'
```

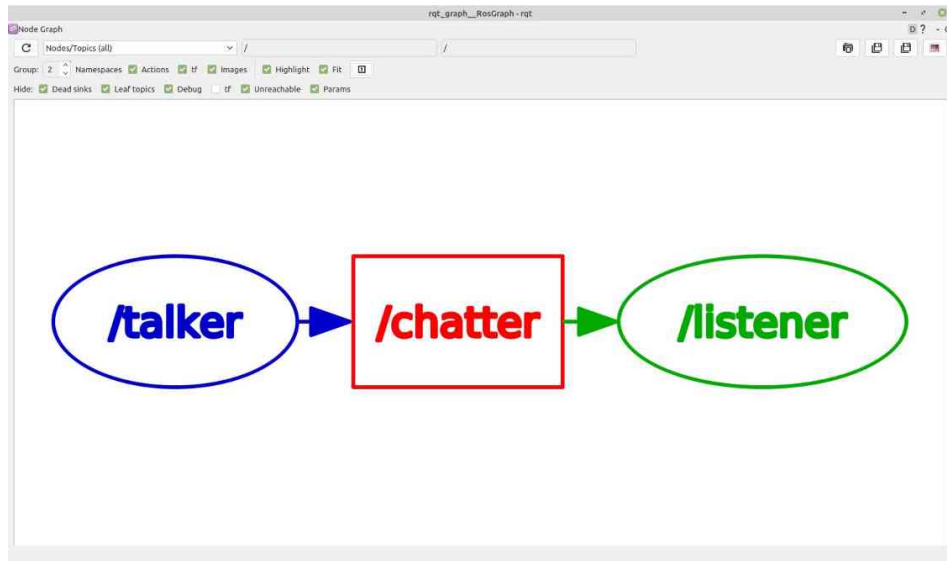
```
[INFO]: Publishing: 'Hello World: 2'
```

```
[INFO]: Publishing: 'Hello World: 3'
```

```
[INFO]: Publishing: 'Hello World: 4'
```

```
[INFO]: Publishing: 'Hello World: 5'
```

```
$ rqt_graph
```



DDS 사용 예시: RMW 변경 방법

- RMW_IMPLEMENTATION 변수로 RMW 변경 가능
 - rmw_cyclonedds_cpp
 - rmw_fastrtps_cpp
 - rmw_connext_cpp
 - rmw_gurumdds_cpp

```
$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
$ ros2 run demo_nodes_cpp listener
[INFO]: I heard: [Hello World: 1]
[INFO]: I heard: [Hello World: 2]
[INFO]: I heard: [Hello World: 3]
(생략)
```

```
$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
$ ros2 run demo_nodes_cpp talker
[INFO]: Publishing: 'Hello World: 1'
[INFO]: Publishing: 'Hello World: 2'
[INFO]: Publishing: 'Hello World: 3'
(생략)
```

```
$ export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
$ ros2 run demo_nodes_cpp listener
[INFO]: I heard: [Hello World: 1]
[INFO]: I heard: [Hello World: 2]
[INFO]: I heard: [Hello World: 3]
(생략)
```

```
$ export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
$ ros2 run demo_nodes_cpp talker
[INFO]: Publishing: 'Hello World: 1'
[INFO]: Publishing: 'Hello World: 2'
[INFO]: Publishing: 'Hello World: 3'
(생략)
```

```
$ export ROS_DOMAIN_ID=11
$ ros2 run demo_nodes_cpp talker
[INFO]: Publishing: 'Hello World: 1'
[INFO]: Publishing: 'Hello World: 2'
[INFO]: Publishing: 'Hello World: 3'
(생략)
```

```
$ export ROS_DOMAIN_ID=12
$ ros2 run demo_nodes_cpp listener
(서로 다른 도메인을 사용하고 있기에 아무런 반응이 없을 것이다.)
```

```
$ export ROS_DOMAIN_ID=11
$ ros2 run demo_nodes_cpp listener
[INFO]: I heard: [Hello World: 13]
[INFO]: I heard: [Hello World: 14]
[INFO]: I heard: [Hello World: 15]
(생략)
```

86 DDS 사용 예시: QoS 테스트 (Reliability = RELIABLE vs BEST_EFFORT)

```
$ sudo tc qdisc add dev lo root netem loss 10%
```

```
$ ros2 run demo_nodes_cpp talker  
[INFO]: Publishing: 'Hello World: 1'  
[INFO]: Publishing: 'Hello World: 2'  
[INFO]: Publishing: 'Hello World: 3'  
[INFO]: Publishing: 'Hello World: 4'  
[INFO]: Publishing: 'Hello World: 5'  
[INFO]: Publishing: 'Hello World: 6'  
[INFO]: Publishing: 'Hello World: 7'  
[INFO]: Publishing: 'Hello World: 8'  
[INFO]: Publishing: 'Hello World: 9'  
[INFO]: Publishing: 'Hello World: 10'  
[INFO]: Publishing: 'Hello World: 11'  
[INFO]: Publishing: 'Hello World: 12'  
[INFO]: Publishing: 'Hello World: 13'  
[INFO]: Publishing: 'Hello World: 14'  
[INFO]: Publishing: 'Hello World: 15'
```

```
$ ros2 run demo_nodes_cpp listener  
[INFO]: I heard: [Hello World: 1]  
[INFO]: I heard: [Hello World: 2]  
[INFO]: I heard: [Hello World: 3]  
[INFO]: I heard: [Hello World: 4]  
[INFO]: I heard: [Hello World: 5]  
[INFO]: I heard: [Hello World: 6]  
[INFO]: I heard: [Hello World: 7]  
[INFO]: I heard: [Hello World: 8]  
[INFO]: I heard: [Hello World: 9]  
[INFO]: I heard: [Hello World: 10]  
[INFO]: I heard: [Hello World: 11]  
[INFO]: I heard: [Hello World: 12]  
[INFO]: I heard: [Hello World: 13]  
[INFO]: I heard: [Hello World: 14]  
[INFO]: I heard: [Hello World: 15]
```

87 DDS 사용 예시: QoS 테스트 (Reliability = RELIABLE vs BEST_EFFORT)

```
$ ros2 run demo_nodes_cpp talker
[INFO]: Publishing: 'Hello World: 1'
[INFO]: Publishing: 'Hello World: 2'
[INFO]: Publishing: 'Hello World: 3'
[INFO]: Publishing: 'Hello World: 4'
[INFO]: Publishing: 'Hello World: 5'
[INFO]: Publishing: 'Hello World: 6'
[INFO]: Publishing: 'Hello World: 7'
[INFO]: Publishing: 'Hello World: 8'
[INFO]: Publishing: 'Hello World: 9'
[INFO]: Publishing: 'Hello World: 10'
[INFO]: Publishing: 'Hello World: 11'
[INFO]: Publishing: 'Hello World: 12'
[INFO]: Publishing: 'Hello World: 13'
[INFO]: Publishing: 'Hello World: 14'
[INFO]: Publishing: 'Hello World: 15'
```

```
$ ros2 run demo_nodes_cpp listener_best_effort
[INFO]: I heard: [Hello World: 1]
[INFO]: I heard: [Hello World: 3]
[INFO]: I heard: [Hello World: 4]
[INFO]: I heard: [Hello World: 5]
[INFO]: I heard: [Hello World: 6]
[INFO]: I heard: [Hello World: 7]
[INFO]: I heard: [Hello World: 8]
[INFO]: I heard: [Hello World: 10]
[INFO]: I heard: [Hello World: 11]
[INFO]: I heard: [Hello World: 12]
[INFO]: I heard: [Hello World: 13]
[INFO]: I heard: [Hello World: 14]
[INFO]: I heard: [Hello World: 15]
```

```
$ sudo tc qdisc delete dev lo root netem loss 10%
```

ROS 2 패키지 설치와 노드 실행

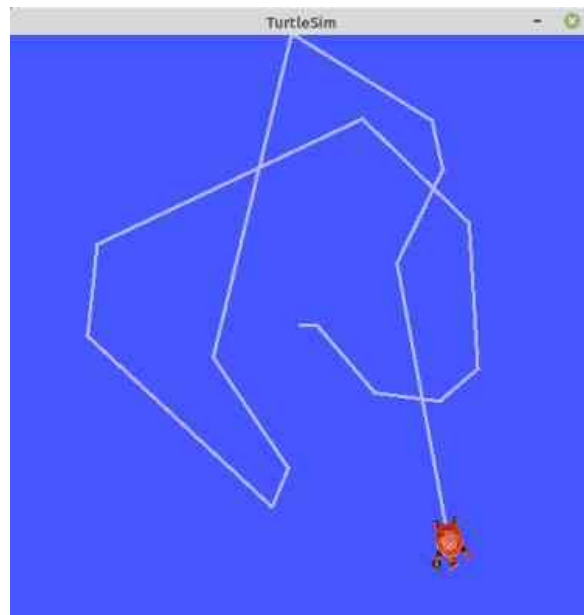
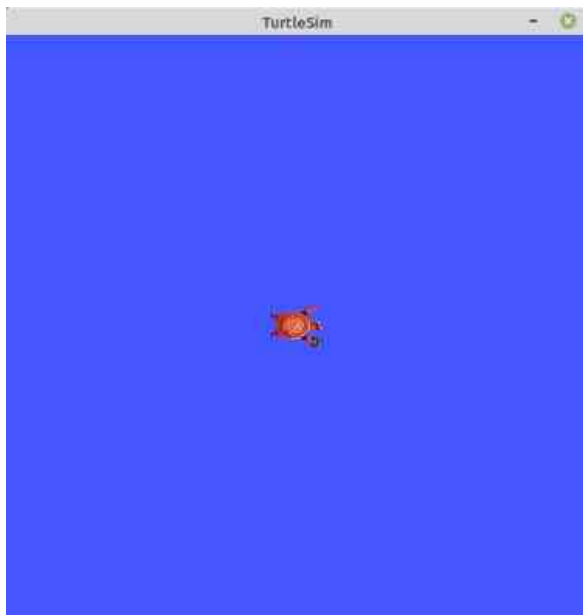

```
$ sudo apt update  
$ sudo apt install ros-humble-turtlesim
```

```
$ ros2 pkg list  
ackermann_msgs  
ackermann_steering_controller  
action_msgs  
action_tutorials_cpp  
action_tutorials_interfaces  
action_tutorials_py  
actionlib_msgs  
(생략)
```

```
$ ros2 pkg executables turtlesim  
turtlesim draw_square  
turtlesim mimic  
turtlesim turtle_teleop_key  
turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtle_teleop_key
```



```
$ ros2 node list
```

```
/turtlesim
```

```
/teleop_turtle
```

```
$ ros2 topic list
```

```
/parameter_events
```

```
/rosout
```

```
/turtle1/cmd_vel
```

```
/turtle1/color_sensor
```

```
/turtle1/pose
```

```
$ ros2 action list
```

```
/turtle1/rotate_absolute
```

```
$ ros2 service list
```

```
/clear
```

```
/kill
```

```
/reset
```

```
/spawn
```

```
/teleop_turtle/describe_parameters
```

```
/teleop_turtle/get_parameter_types
```

```
/teleop_turtle/get_parameters
```

```
/teleop_turtle/list_parameters
```

```
/teleop_turtle/set_parameters
```

```
/teleop_turtle/set_parameters_atomically
```

```
/turtle1/set_pen
```

```
/turtle1/teleport_absolute
```

```
/turtle1/teleport_relative
```

```
/turtlesim/describe_parameters
```

```
/turtlesim/get_parameter_types
```

```
/turtlesim/get_parameters
```

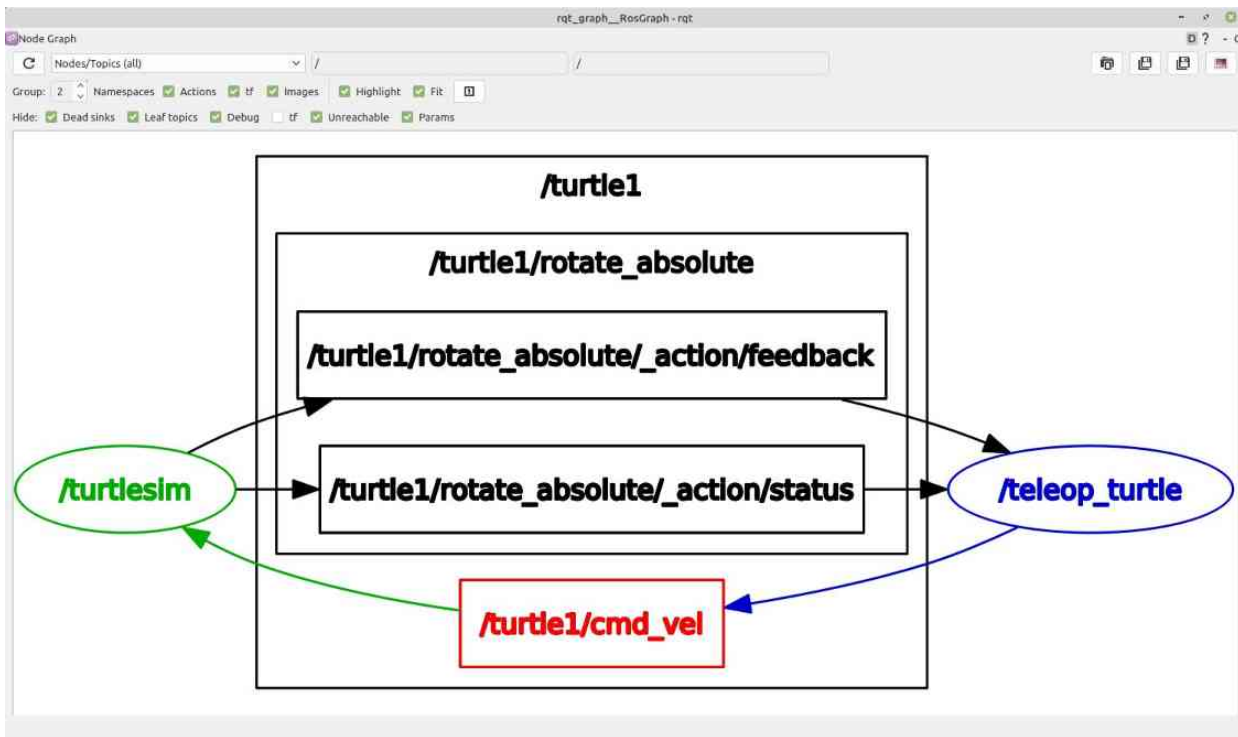
```
/turtlesim/list_parameters
```

```
/turtlesim/set_parameters
```

```
/turtlesim/set_parameters_atomically
```

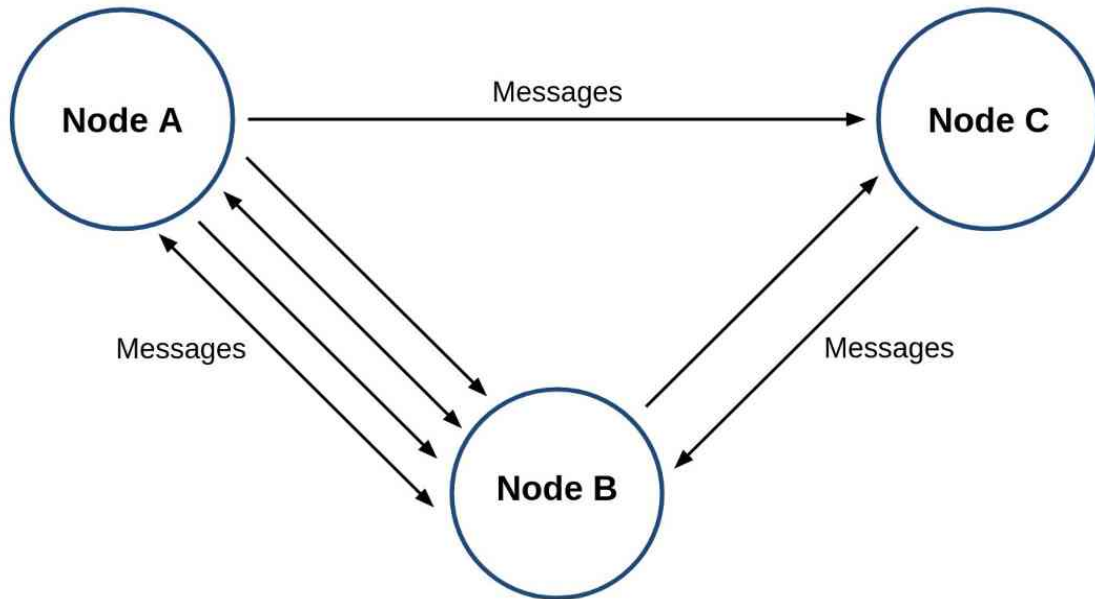
92 rqt_graph 로 보는 노드와 토픽의 그래프 뷰

\$ rqt_graph



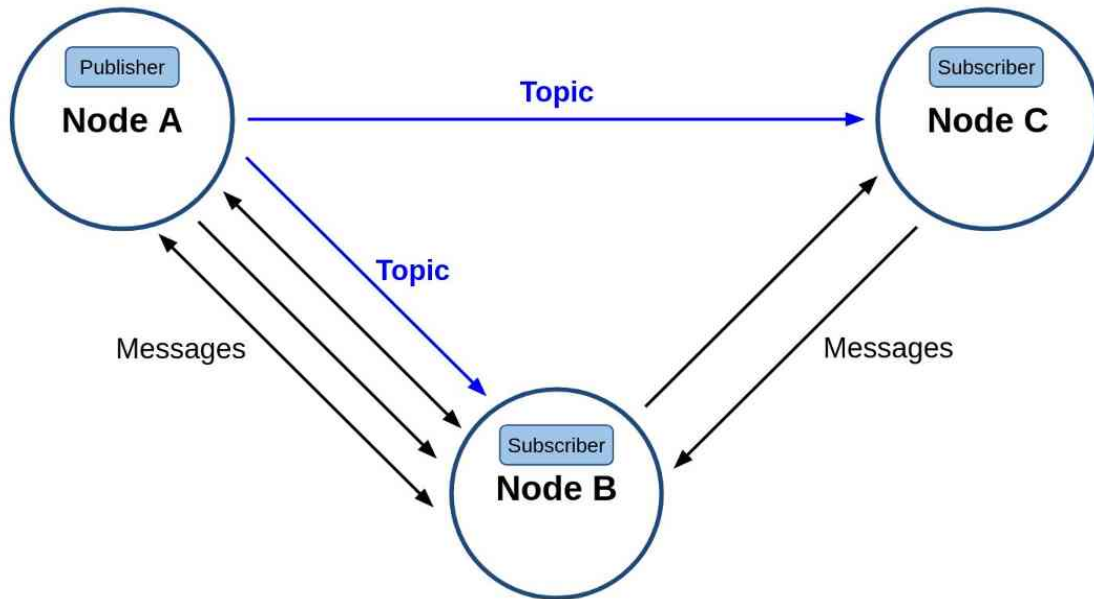
ROS 2 노드와 메시지 통신

노드(node)는 아래 그림처럼 Node A, Node B, Node C라는 노드가 있을 때 각각의 노드들은 서로 유기적으로 Message로 연결되어 사용된다. 지금은 단순히 3개의 노드만 표시하였지만 수행하고자 하는 태스크가 많아질수록 메시지로 연결되는 노드가 늘어나며 시스템이 확장할 수 있게된다.



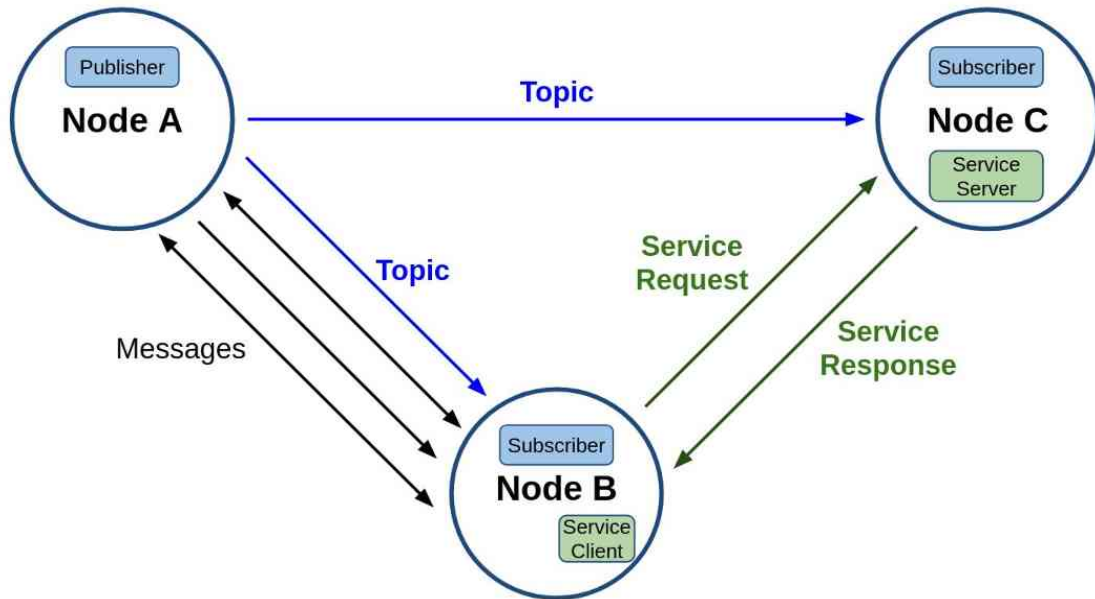
노드(node)와 메시지 통신(message communication)

토픽(topic)은 아래 그림의 `Node A - Node B`, `Node A - Node C`처럼 비동기식 단방향 메시지 송수신 방식으로 **msg** 메시지 형태의 메시지를 발간하는 **Publisher**와 메시지를 구독하는 **Subscriber** 간의 통신이라고 볼 수 있다. 이는 **1:N**, **N:1**, **N:N** 통신도 가능하며 ROS 메시지 통신에서 가장 널리 사용되는 통신 방법이다.



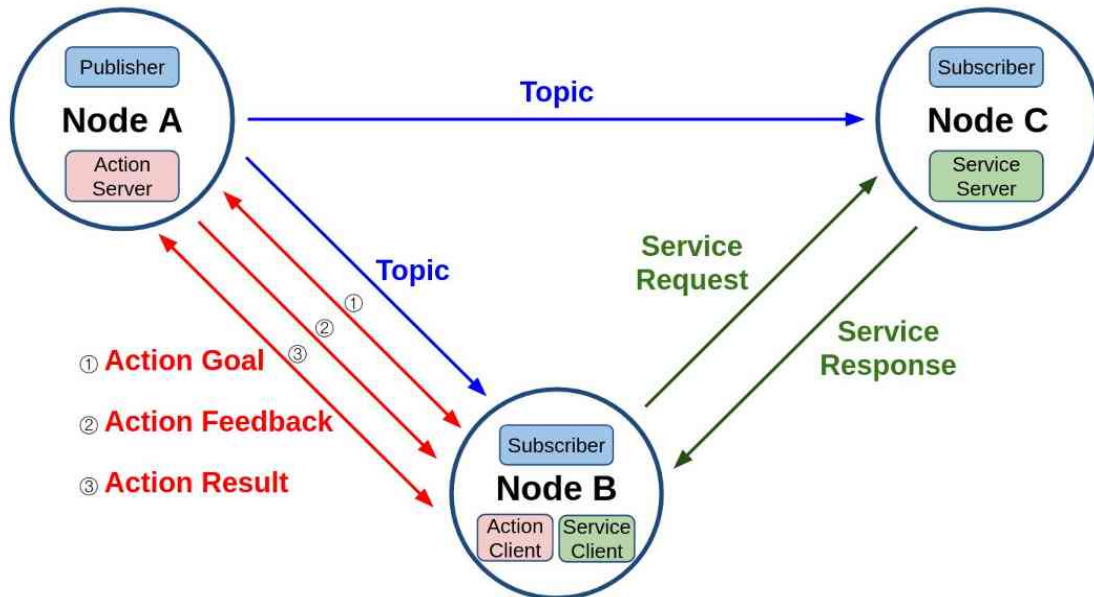
노드(node)와 메시지 통신(message communication)

서비스 (Service)는 아래 그림의 `Node B - Node C`처럼 동기식 양방향 메시지 송수신 방식으로 서비스의 요청 (Request)을 하는 쪽을 **Service client**라고 하며 서비스의 응답 (Response)을 하는 쪽을 **Service server**라고 한다. 결국 서비스는 특정 요청을 하는 클라이언트 단과 요청받은 일을 수행 후에 결과 값을 전달하는 서버 단과의 통신이라고 볼 수 있다. 서비스 요청 및 응답(Request/Response) 또한 위에서 언급한 msg 메시지의 변형으로 **srv** 메시지 라고 한다.



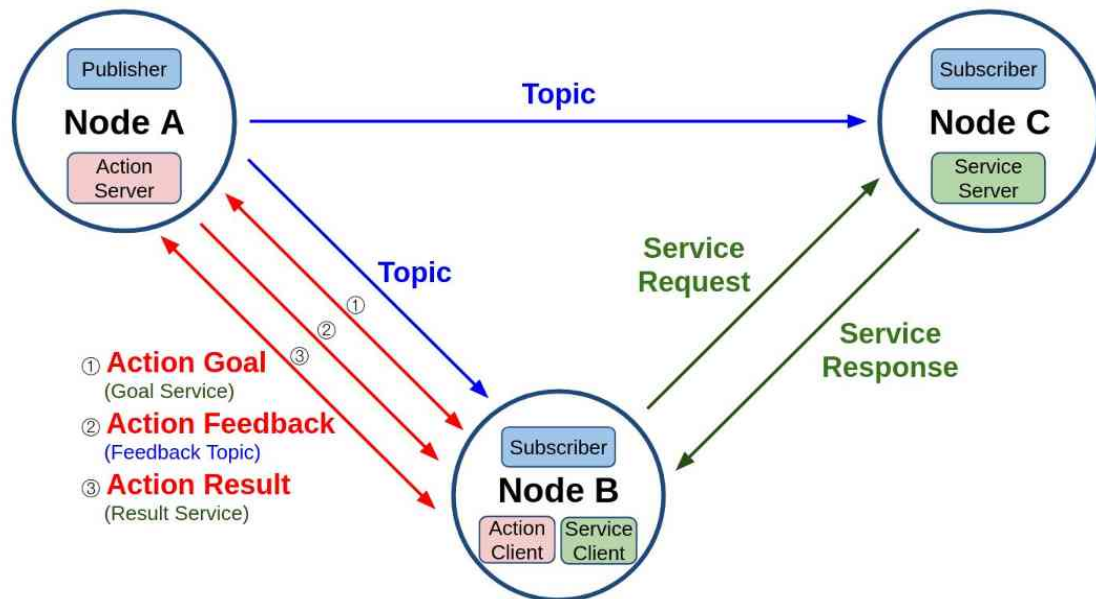
노드(node)와 메시지 통신(message communication)

액션 (Action)은 아래 그림의 `Node A - Node B`처럼 비동기식 + 동기식 양방향 메시지 송수신 방식으로 액션 목표 **Goal**를 지정하는 **Action client**과 액션 목표를 받아 특정 태스크를 수행하면서 중간 결과 값에 해당되는 액션 피드백 (**Feedback**)과 최종 결과 값에 해당되는 액션 결과(**Result**)를 전송하는 **Action server** 간의 통신이라고 볼 수 있다.



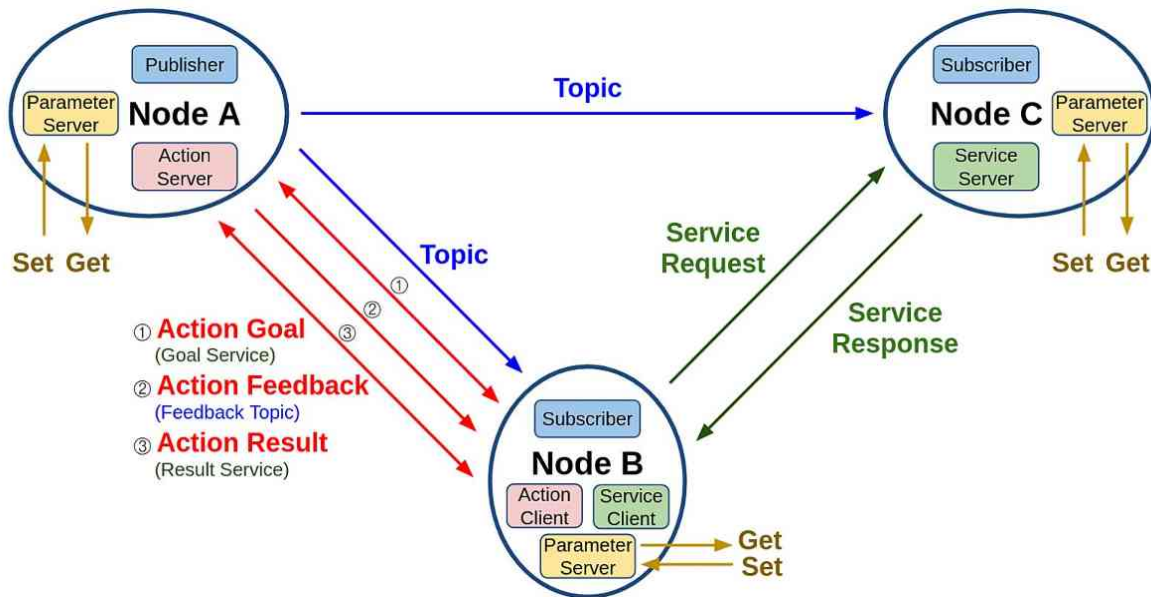
노드(node)와 메시지 통신(message communication)

액션의 구현 방식을 더 자세히 살펴보면 아래 그림과 같이 **토픽(topic)**과 서비스(service)의 **혼합**이라고 볼 수 있는데 액션 목표 및 액션 결과를 전달하는 방식은 서비스와 같으며 액션 피드백은 토픽과 같은 메시지 전송 방식이다. 액션 목표/피드백/결과(Goal/Feedback/Result) 메시지 또한 위에서 언급한 msg 메시지의 변형으로 **action 메시지**라고 한다.



노드(node)와 메시지 통신(message communication)

파라미터 (Parameter)는 아래 그림의 각 노드에 파라미터 관련 **Parameter server**를 실행시켜 **외부의 Parameter client** 간의 **통신으로 파라미터를 변경하는 것으로 서비스와 동일**하다고 볼 수 있다. 단 노드 내 매개변수 또는 글로벌 매개변수를 서비스 메시지 통신 방법을 사용하여 노드 내부 또는 외부에서 쉽게 지정(**Set**) 하거나 변경할 수 있고, 쉽게 가져(**Get**)와서 사용할 수 있게 하는 점에서 목적이 다르다고 볼 수 있다.

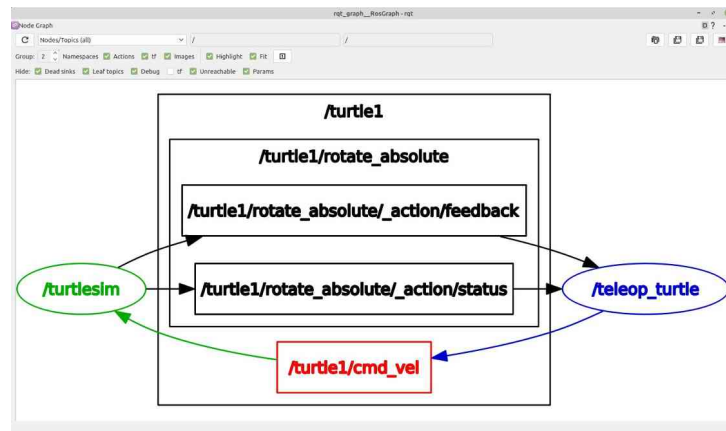
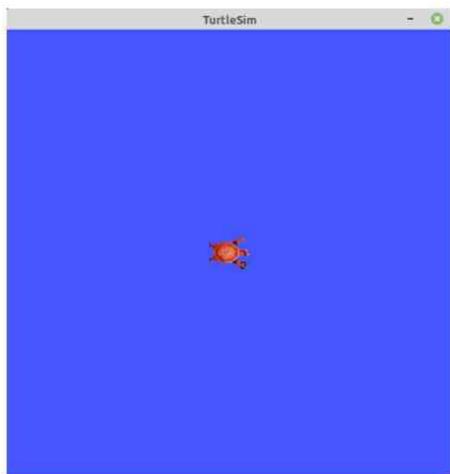


100 메시지 통신 테스트

```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtle_teleop_key
```

```
$ rqt_graph
```

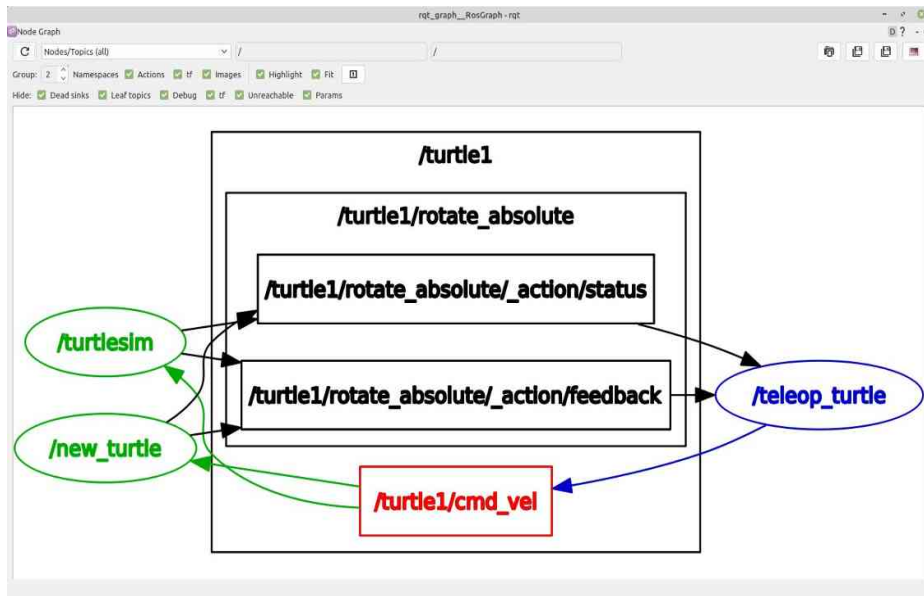


101 노드 목록 (ros2 node list)

```
$ ros2 node list
/rqt_gui_py_node_28168
/teleop_turtle
/turtlesim
```

```
$ ros2 run turtlesim turtlesim_node __node:=new_turtle
```

```
$ ros2 node list
/rqt_gui_py_node_29017
/teleop_turtle
/new_turtle
/turtlesim
```



```
$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

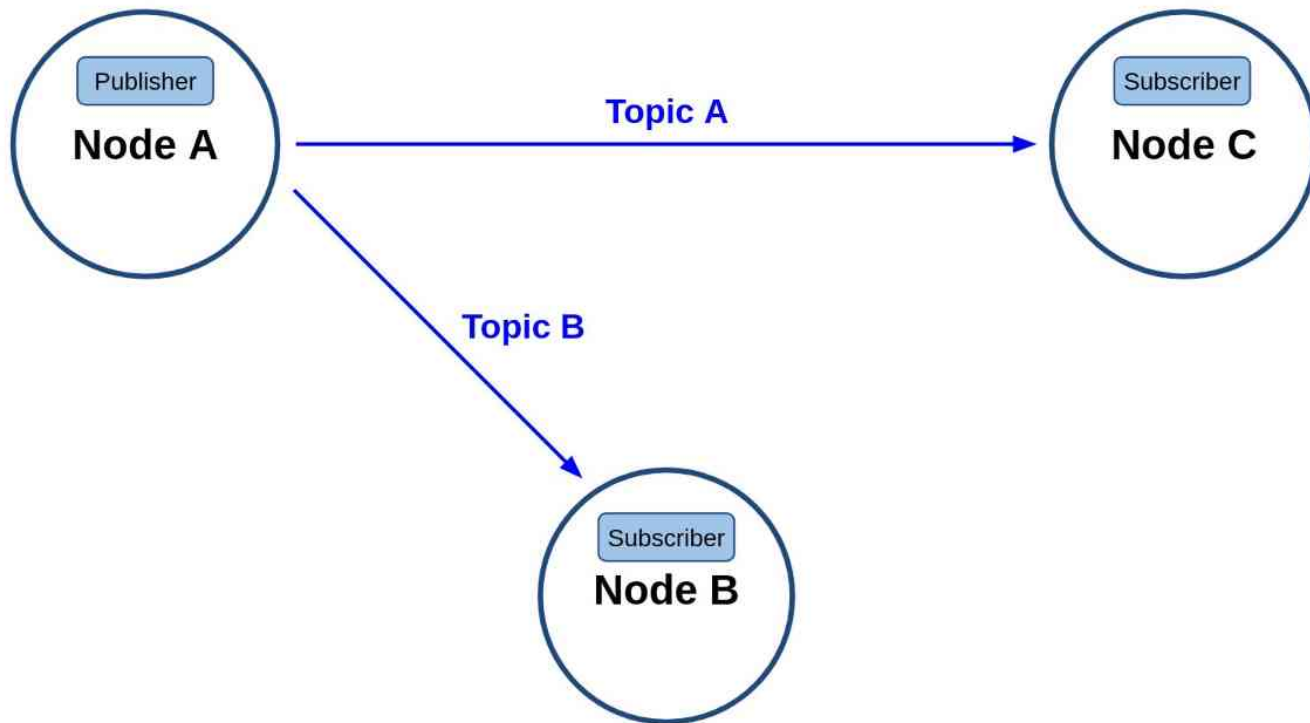
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

```
$ ros2 node info /teleop_turtle
/teleop_turtle
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Service Servers:
  /teleop_turtle/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /teleop_turtle/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /teleop_turtle/get_parameters: rcl_interfaces/srv/GetParameters
  /teleop_turtle/list_parameters: rcl_interfaces/srv/ListParameters
  /teleop_turtle/set_parameters: rcl_interfaces/srv/SetParameters
  /teleop_turtle/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:

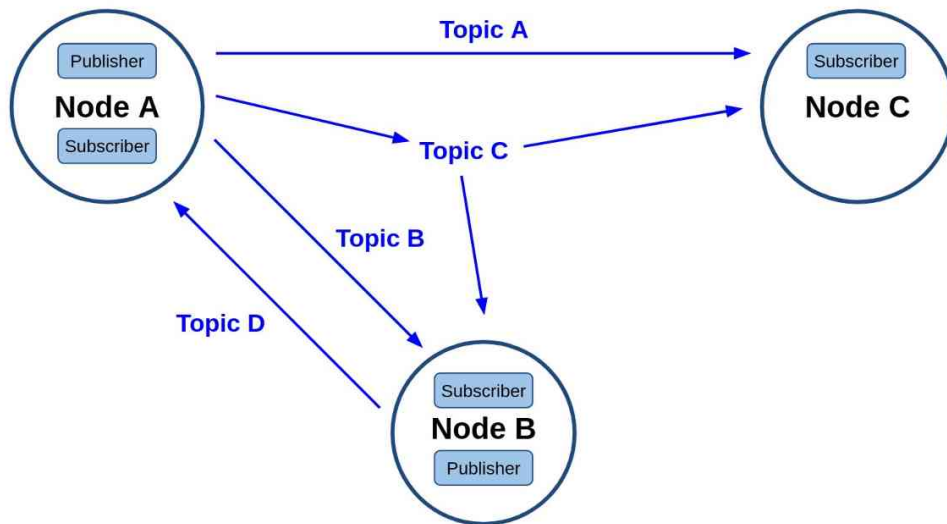
Action Clients:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
```

ROS 2 토픽 (Topic)



106 토픽 (Topic)

‘Node A’처럼 하나의 이상의 토픽을 발행할 수 있을 뿐만이 아니라 ‘Publisher’ 기능과 동시에 토픽(예: Topic D)을 구독하는 ‘Subscriber’ 역할도 동시에 수행할 수 있다. 원한다면 자신이 발행한 토픽을 셀프 구독할 수 있게 구성할 수도 있다. 이처럼 토픽 기능은 목적에 따라 다양한 방법으로 사용할 수 있는데 이러한 유연성으로 다양한 곳에 사용중에 있다. 경험상 ROS 프로그래밍시에 70% 이상이 토픽으로 사용될 정도로 통신 방식 중에 가장 기본이 되며 가장 널리쓰이는 방법이다. 기본 특징으로 비동기성과 연속성을 가지기에 센서 값 전송 및 항시 정보를 주고 받아야하는 부분에 주로 사용된다.



```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 node info /turtlesim
```

```
/turtlesim
```

```
Subscribers:
```

```
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
```

```
(생략)
```

```
Publishers:
```

```
  /turtle1/color_sensor: turtlesim/msg/Color
```

```
  /turtle1/pose: turtlesim/msg/Pose
```

```
(생략)
```

```
Services:
```

```
  /clear: std_srvs/srv/Empty
```

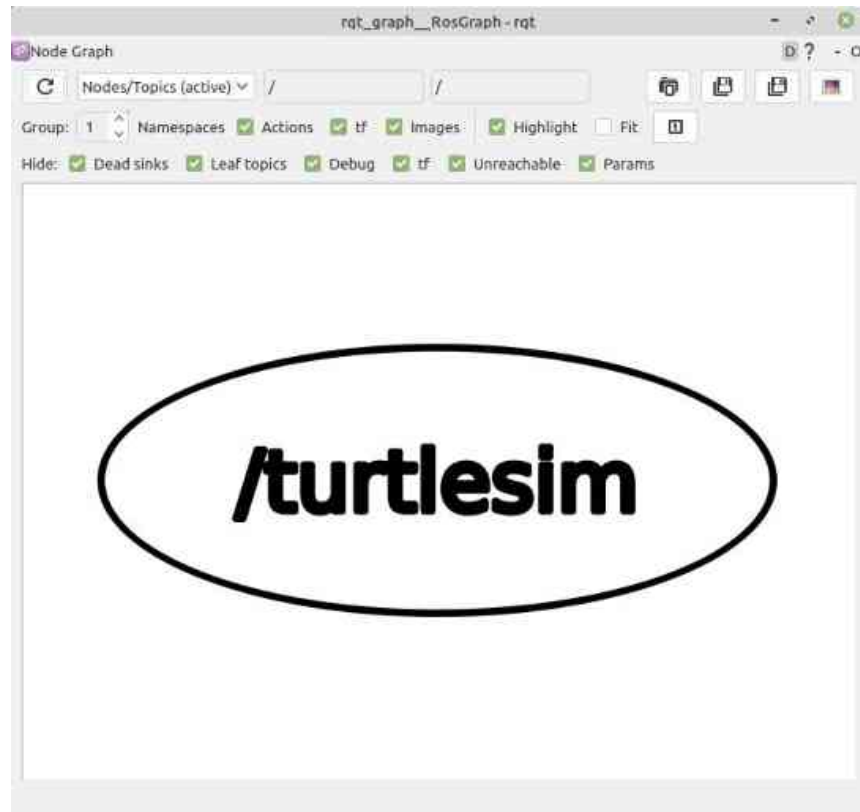
```
  /kill: turtlesim/srv/Kill
```

```
(생략)
```

108 토픽 목록 확인 (ros2 topic list)

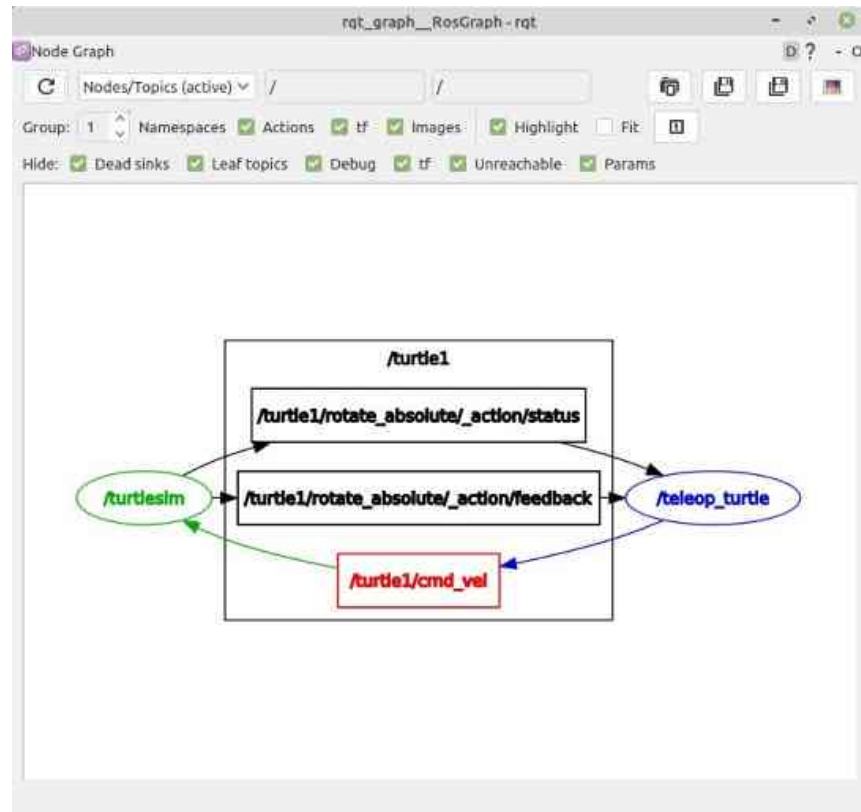
```
$ ros2 topic list -t  
/parameter_events [rcl_interfaces/msg/ParameterEvent]  
/rosout [rcl_interfaces/msg/Log]  
/turtle1/cmd_vel [geometry_msgs/msg/Twist]  
/turtle1/color_sensor [turtlesim/msg/Color]  
/turtle1/pose [turtlesim/msg/Pose]
```

```
$ rqt_graph
```

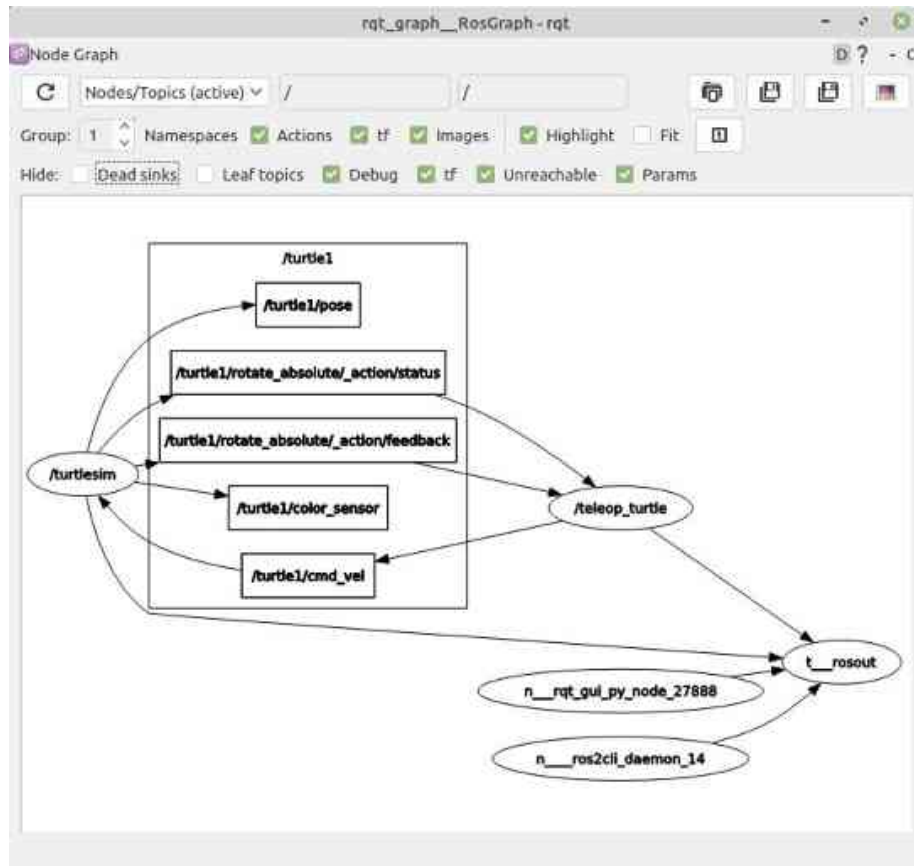


109 토픽 목록 확인 (rqt)

```
$ ros2 run turtlesim turtle_teleop_key
```



110 토픽 목록 확인 (rqt)



111

토픽 정보 확인 (ros2 topic info)

```
$ ros2 topic info /turtle1/cmd_vel  
Type: geometry_msgs/msg/Twist  
Publisher count: 1  
Subscriber count: 1
```

112 토픽 내용 확인 (ros2 topic echo)

```
$ ros2 topic echo /turtle1/cmd_vel
```

```
linear:
```

```
x: 1.0
```

```
y: 0.0
```

```
z: 0.0
```

```
angular:
```

```
x: 0.0
```

```
y: 0.0
```

```
z: 0.0
```


113 토픽 대역폭 확인 (ros2 topic bw)

```
$ ros2 topic bw /turtle1/cmd_vel  
Subscribed to [/turtle1/cmd_vel]  
average: 1.74KB/s  
mean: 0.05KB min: 0.05KB max: 0.05KB window: 100  
(생략)
```

114 토픽 주기 확인 (ros2 topic hz)

```
$ ros2 topic hz /turtle1/cmd_vel  
average rate: 33.212  
min: 0.029s max: 0.089s std dev: 0.00126s window: 2483  
(생략)
```

115 토픽 지연 시간 확인 (ros2 topic delay)

```
$ ros2 topic delay /TOPIC_NAME  
average delay: xxx.xxx  
min: xxx.xxxx max: xxx.xxxx std dev: xxx.xxxx window: 10
```

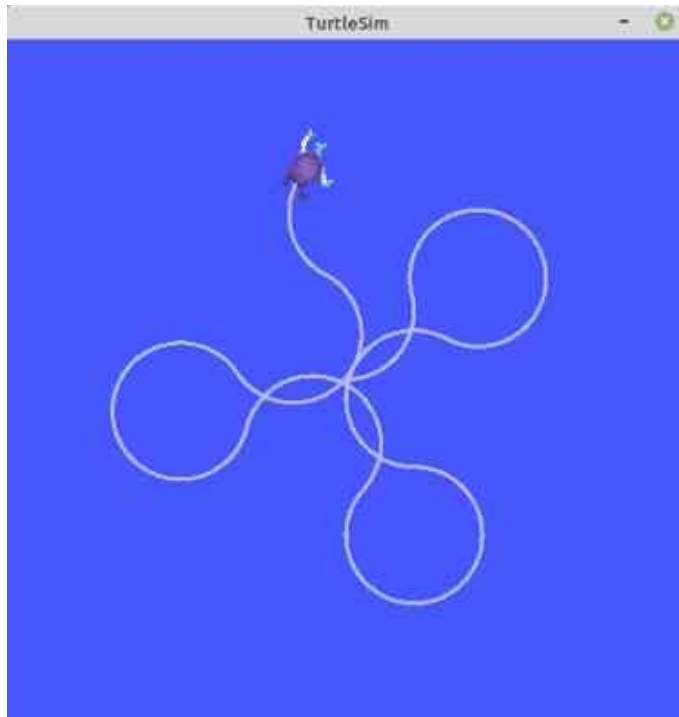
116 토픽 발행 (ros2 topic pub)

```
$ ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```



117 토픽 발행 (ros2 topic pub)

```
$ ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```



118 bag 기록 (ros2 bag record)

```
$ ros2 bag record /turtle1/cmd_vel  
[INFO]: Opened database 'rosbag2_2024_06_20-20_21_39'.  
[INFO]: Listening for topics...  
[INFO]: Subscribed to topic '/turtle1/cmd_vel'
```

119 bag 정보 (ros2 bag info)

```
$ ros2 bag info rosbag2_2020_09_04-08_31_06/
```

```
Files:          rosbag2_2020_09_04-08_31_06.db3
```

```
Bag size:       84.4 KiB
```

```
Storage id:     sqlite3
```

```
Duration:       31.602s
```

```
Start:          Sep 4 2020 08:31:09.952 (1599175869.952)
```

```
End:            Sep 4 2020 08:31:41.554 (1599175901.554)
```

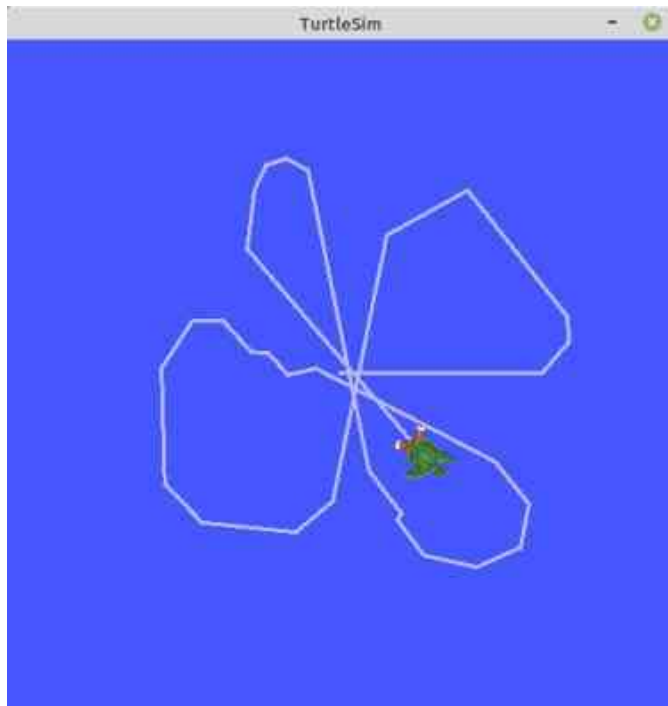
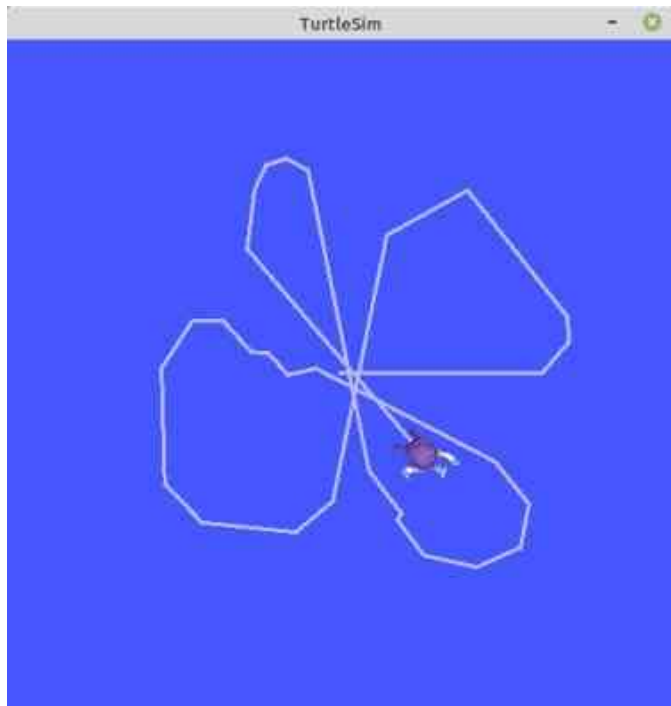
```
Messages:       355
```

```
Topic information: Topic: /turtle1/cmd_vel | Type: geometry_msgs/msg/Twist | Count: 355 | Serialization
```

```
Format:         cdr
```

120 bag 재생 (ros2 bag play)

```
$ ros2 bag play rosbag2_2020_09_04-08_31_06/  
[INFO]: Opened database 'rosbag2_2020_09_04-08_31_06/'.
```



121 ROS 인터페이스 (interface) 이란?

ROS의 노드 간에 데이터를 주고받을 때에는 토픽, 서비스, 액션이 사용되는데 이 때 사용되는 데이터의 형태를 **ROS 인터페이스 (interface)** 이라고 한다. ROS 인터페이스에는 ROS 2에 새롭게 추가된 IDL(interface definition language)과 ROS 1부터 ROS 2까지 널리 사용 중인 **msg, srv, action** 이 있다. 토픽, 서비스, 액션은 각각 msg, srv, action interface를 사용하고 있으며 정수, 부동 소수점, 불리언과 같은 단순 자료형을 기본으로 하여 메시지 안에 메시지를 품고 있는 간단한 데이터 구조 및 메시지들이 나열된 배열과 같은 구조도 사용할 수 있다.

[단순 자료형]

- 예) 정수(integer), 부동 소수점(floating point), 불(boolean)
- https://github.com/ros2/common_interfaces/tree/humble/std_msgs

[메시지 안에 메시지를 품고 있는 간단한 데이터 구조]

- 예) geometry_msgs/msg/Twist의 `Vector3 linear`
- https://github.com/ros2/common_interfaces/blob/humble/geometry_msgs/msg/Twist.msg

[메시지들이 나열된 배열과 같은 구조]

- 예) sensor_msgs/msg/LaserScan의 `float32[] ranges`
- https://github.com/ros2/common_interfaces/blob/humble/sensor_msgs/msg/LaserScan.msg

122 메시지 인터페이스 (message interface, msg)



[teleop_turtle]

Publisher
메시지 발행



[/turtle1/cmd_vel]

geometry_msgs/msg/Twist
메시지



[turtlesim]

Subscriber
메시지 구독

[geometry_msgs/msg/Twist]

Vector3 linear
Vector3 angular

[geometry_msgs/msg/Vector3]

float64 x
float64 y
float64 z

[geometry_msgs/msg/Vector3]

float64 x
float64 y
float64 z

123 메시지 인터페이스 (message interface, msg)

토픽은 고유의 인터페이스를 가지고 있는데 이를 **메시지 인터페이스**라 부르며, 파일로는 **msg 파일**을 가르킨다.

예를 들어, 위 예제에서 `/turtle1/cmd_vel` 토픽은 `geometry_msgs/msgs/Twist` 형태이다. 이름이 좀 긴데 풀어서 설명하면 기하학 관련 메시지를 모아둔 `geometry_msgs` 패키지의 `msgs` 분류의 `Twist` 데이터 형태라는 것이다.

`Twist` 데이터 형태를 자세히 보면 `Vector3 linear`과 `Vector3 angular`이라고 되어 있다. 이는 메시지 안에 메시지를 품고 있는 것으로 `Vector3` 형태에 `linear`이라는 이름의 메시지와 `Vector3` 형태에 `angular`이라는 이름의 메시지, 즉 2개의 메시지가 있다는 것이며 `Vector3`는 다시 `float64` 형태에 `x`, `y`, `z` 값이 존재한다.

다시 말해 `geometry_msgs/msgs/Twist` 메시지 형태는 `float64` 자료형의 `linear.x`, `linear.y`, `linear.z`, `angular.x`, `angular.y`, `angular.z` 라는 이름의 메시지인 것이다. 이를 통해 병진 속도 3개, 회전 속도 3개를 표현할 수 있게 된다.

124 ros2 interface 명령어

```
$ ros2 interface show geometry_msgs/msg/Twist
```

```
Vector3 linear
```

```
Vector3 angular
```

```
$ ros2 interface show geometry_msgs/msg/Vector3
```

```
float64 x
```

```
float64 y
```

```
float64 z
```

```
$ ros2 interface list
```

```
Messages:
```

```
  action_msgs/msg/GoalInfo  
  action_msgs/msg/GoalStatus  
  action_msgs/msg/GoalStatusArray
```

```
(생략)
```

```
Services:
```

```
  action_msgs/srv/CancelGoal  
  composition_interfaces/srv/ListNodes
```

```
(생략)
```

```
Actions:
```

```
  action_tutorials_interfaces/action/Fibonacci  
  example_interfaces/action/Fibonacci
```

```
(생략)
```

126 ros2 interface 명령어

```
$ ros2 interface packages
action_msgs
action_tutorials_interfaces
actionlib_msgs
builtin_interfaces
(생략)
```

```
$ ros2 interface package turtlesim  
turtlesim/srv/Spawn  
turtlesim/msg/Color  
turtlesim/msg/Pose  
turtlesim/srv/TeleportAbsolute  
turtlesim/srv/Kill  
turtlesim/action/RotateAbsolute  
turtlesim/srv/SetPen  
turtlesim/srv/TeleportRelative
```

128 ros2 interface 명령어

```
$ ros2 interface proto geometry_msgs/msg/Twist
```

```
"linear:
```

```
  x: 0.0
```

```
  y: 0.0
```

```
  z: 0.0
```

```
angular:
```

```
  x: 0.0
```

```
  y: 0.0
```

```
  z: 0.0
```

```
"
```

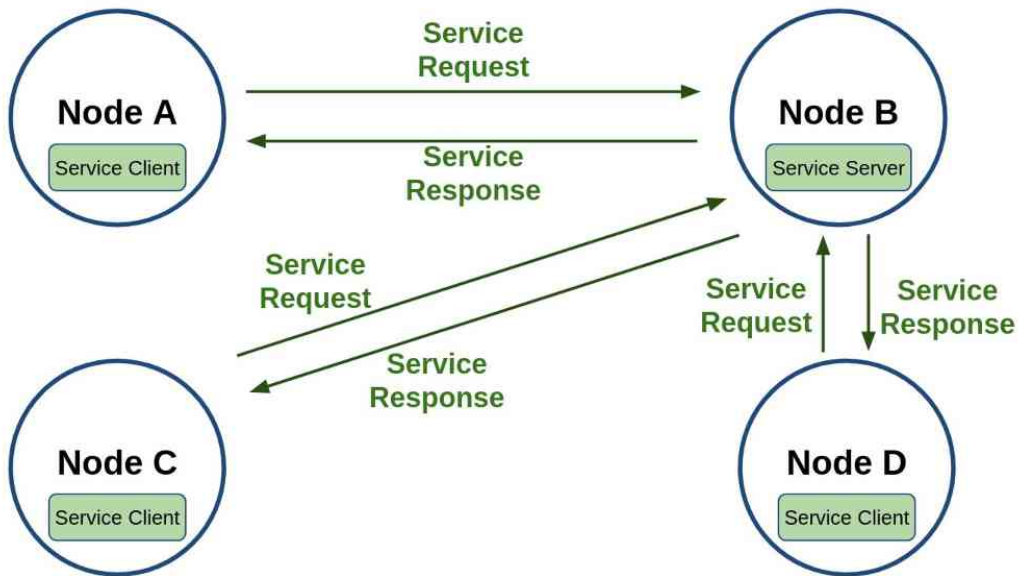

ROS 2 서비스 (Service)

130 서비스 (Service)



131 서비스 (Service)

서비스는 다음 그림과 같이 동일 서비스에 대해 복수의 클라이언트를 가질 수 있도록 설계되었다. 단, 서비스 응답은 서비스 요청이 있었던 서비스 클라이언트에 대해서만 응답을 하는 형태로 그림의 구성에서 예를 들자면 Node C의 Service Client가 Node B의 Service Server에게 서비스 요청을 하였다면 Node B의 Service Server는 요청받은 서비스를 수행한 후 Node C의 Service Client에게만 서비스 응답을 하게 된다.



```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 service list
```

```
/clear
```

```
/kill
```

```
/reset
```

```
/spawn
```

```
/turtle1/set_pen
```

```
/turtle1/teleport_absolute
```

```
/turtle1/teleport_relative
```

```
/turtlesim/describe_parameters
```

```
/turtlesim/get_parameter_types
```

```
/turtlesim/get_parameters
```

```
/turtlesim/list_parameters
```

```
/turtlesim/set_parameters
```

```
/turtlesim/set_parameters_atomically
```

133 서비스 형태 확인 (ros2 service type)

```
$ ros2 service type /clear  
std_srvs/srv/Empty
```

```
$ ros2 service type /kill  
turtlesim/srv/Kill
```

```
$ ros2 service type /spawn  
turtlesim/srv/Spawn
```

```
$ ros2 service list -t  
/clear [std_srvs/srv/Empty]  
/kill [turtlesim/srv/Kill]  
/reset [std_srvs/srv/Empty]  
/spawn [turtlesim/srv/Spawn]  
/turtle1/set_pen [turtlesim/srv/SetPen]  
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]  
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]  
(생략)
```

134 서비스 찾기 (ros2 service find)

```
$ ros2 service find std_srvs/srv/Empty
```

```
/clear
```

```
/reset
```

```
$ ros2 service find turtlesim/srv/Kill
```

```
/kill
```

135 서비스 요청 (ros2 service call)

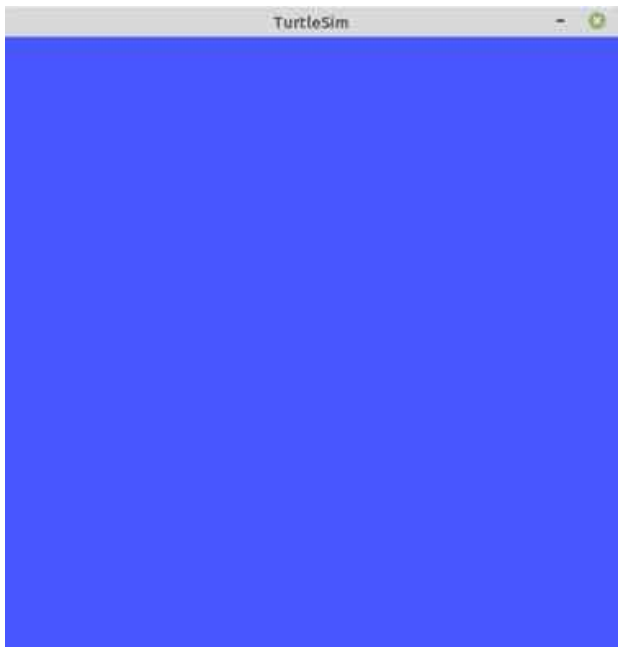
```
$ ros2 run turtlesim turtle_teleop_key
```

```
$ ros2 service call /clear std_srvs/srv/Empty  
requester: making request: std_srvs.srv.Empty_Request()  
response:  
std_srvs.srv.Empty_Response()
```



136 서비스 요청 (ros2 service call)

```
$ ros2 service call /kill turtlesim/srv/Kill "name: 'turtle1'"
requester: making request: turtlesim.srv.Kill_Request(name='turtle1')
response:
turtlesim.srv.Kill_Response()
```

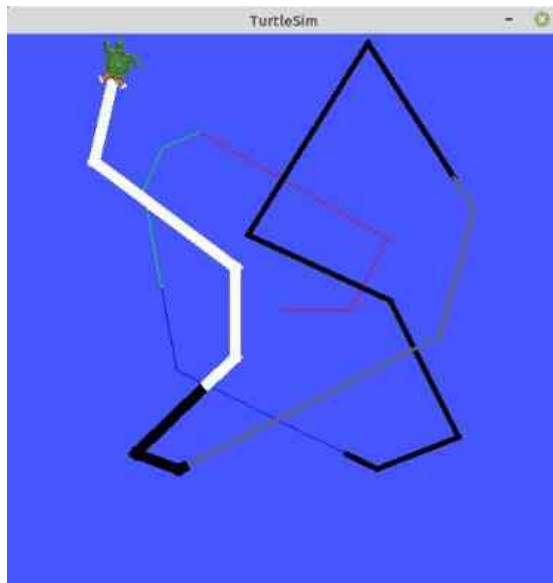


```
$ ros2 service call /reset std_srvs/srv/Empty
requester: making request: std_srvs.srv.Empty_Request()
response:
std_srvs.srv.Empty_Response()
```



137 서비스 요청 (ros2 service call)

```
$ ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 255, b: 255, width: 10}"  
requester: making request: turtlesim.srv.SetPen_Request(r=255, g=255, b=255, width=10, off=0)  
response:  
turtlesim.srv.SetPen_Response()
```



138 서비스 요청 (ros2 service call)

```
$ ros2 service call /kill turtlesim/srv/Kill "name: 'turtle1'"
requester: making request: turtlesim.srv.Kill_Request(name='turtle1')
response:
turtlesim.srv.Kill_Response()
```

```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.5, y: 9, theta: 1.57, name: 'leonardo'}"
requester: making request: turtlesim.srv.Spawn_Request(x=5.5, y=9.0, theta=1.57, name='leonardo')
response:
turtlesim.srv.Spawn_Response(name='leonardo')
```

```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.5, y: 7, theta: 1.57, name: 'raffaelo'}"
requester: making request: turtlesim.srv.Spawn_Request(x=5.5, y=7.0, theta=1.57, name='raffaelo')
response:
turtlesim.srv.Spawn_Response(name='raffaelo')
```

```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.5, y: 5, theta: 1.57, name: 'michelangelo'}"
requester: making request: turtlesim.srv.Spawn_Request(x=5.5, y=5.0, theta=1.57, name='michelangelo')
response:
turtlesim.srv.Spawn_Response(name='michelangelo')
```

```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 5.5, y: 3, theta: 1.57, name: 'donatello'}"
requester: making request: turtlesim.srv.Spawn_Request(x=5.5, y=3.0, theta=1.57, name='donatello')
response:
turtlesim.srv.Spawn_Response(name='donatello')
```

139 서비스 요청 (ros2 service call)



```
$ ros2 topic list
/donatello/cmd_vel
/donatello/color_sensor
/donatello/pose
/leonardo/cmd_vel
/leonardo/color_sensor
/leonardo/pose
/michelangelo/cmd_vel
/michelangelo/color_sensor
/michelangelo/pose
/new_turtle/cmd_vel
/new_turtle/pose
/parameter_events
/raffaello/cmd_vel
/raffaello/color_sensor
/raffaello/pose
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
/turtle2/cmd_vel
/turtle2/pose
/turtle3/cmd_vel
/turtle3/color_sensor
/turtle4/cmd_vel
/turtle4/color_sensor
/turtle4/pose
```

140 서비스 인터페이스 (service interface, srv)

서비스 또한 토픽과 마찬가지로 별도의 인터페이스를 가지고 있는데 이를 **서비스 인터페이스**라 부르며, 파일로는 **srv 파일**을 가르킨다. 서비스 인터페이스는 메시지 인터페이스의 확장형이라고 볼 수 있는데 위에서 서비스 요청때 실습으로 사용하였던 `/spawn` 서비스를 예를 들어 설명하겠다.

`/spawn` 서비스에 사용된 `Spawn.srv` 인터페이스를 알아보기 위해서는 원본 파일을 참고해도 되고, `'ros2 interface show'` 명령어를 이용하여 확인할 수 있다. 이 명령어를 이용하면 아래의 결과 값과 같이 `'turtlesim/srv/Spawn.srv'`은 `float32` 형태의 `x, y, theta` 그리고 `string` 형태로 `name` 이라고 두개의 데이터가 있음을 알 수 있다. 여기서 특이한 것은 `'---'` 이다. 이는 구분자라 부른다. 서비스 인터페이스는 메시지 인터페이스와는 달리 **서비스 요청 및 응답 (Request/Response) 형태**로 구분되는데 **요청 (Request)과 응답 (Response)을 나누어 사용하기 위해서** `'---'`를 **구분자로 사용**하게 된다. 즉 `x, y, theta, name`은 서비스 요청에 해당되고 서비스 클라이언트 단에서 서비스 서버단에 전송하는 값이 된다. 그리고 서비스 서버단은 지정된 서비스를 수행하고 `name` 데이터를 서비스 클라이언트단에 전송하게 된다.

```
$ ros2 interface show turtlesim/srv/Spawn.srv
float32 x
float32 y
float32 theta
string name
---
string name
```

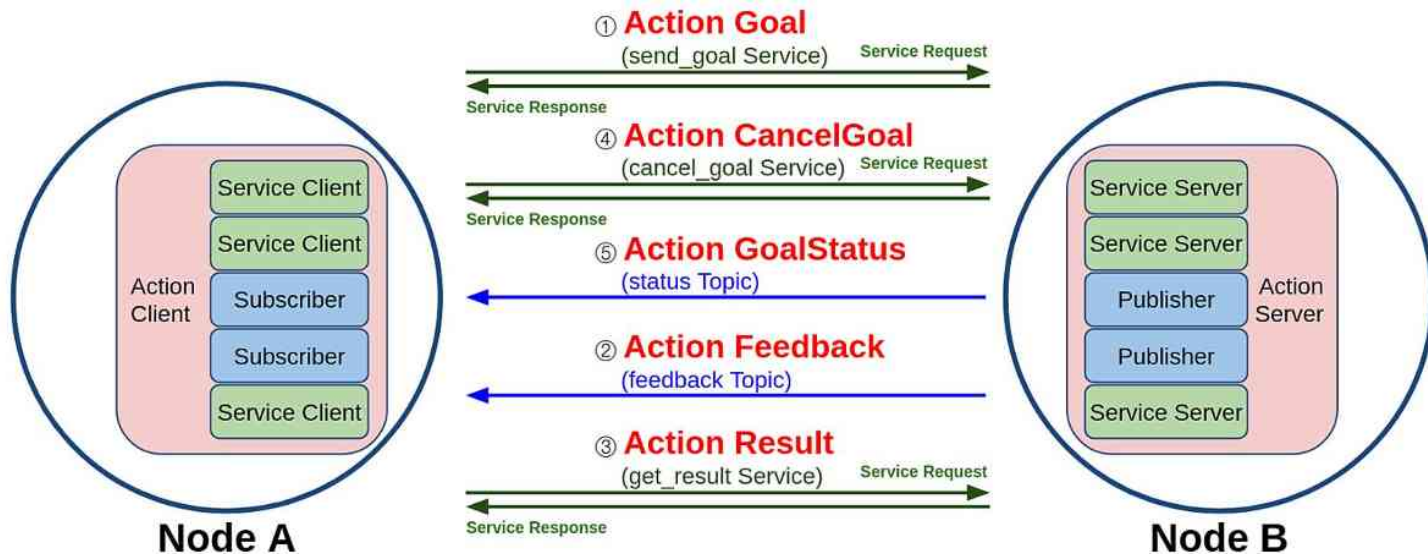
ROS 2 액션 (Action)



143 액션 (Action)

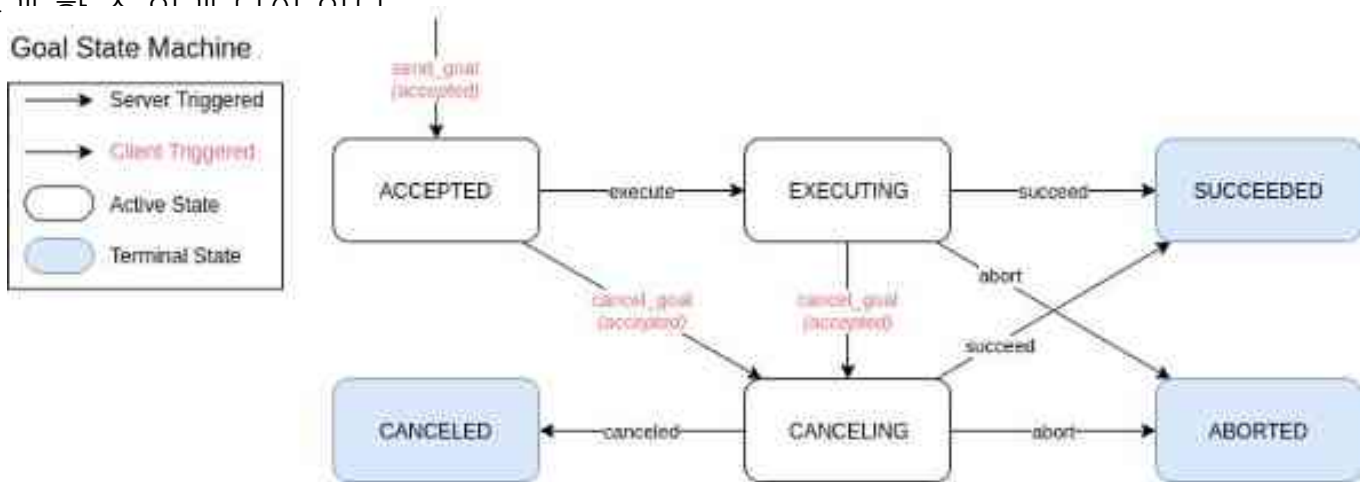
액션의 구현 방식을 더 자세히 살펴보면 다음 그림과 같이 토픽과 서비스의 혼합이라고 볼 수 있는데 ROS 1이 토픽만을 사용하였다면 ROS 2에서는 액션 목표, 액션 결과, 액션 피드백은 토픽과 서비스가 혼합되어 있다.

즉, 다음 그림과 같이 **Action Client**는 **Service Client 3개**와 **Topic Subscriber 2개**로 구성되어있으며, **Action Server**는 **Service Server 3개**와 **Topic Publisher 2개**로 구성된다. 액션 목표/피드백/결과(goal/feedback/result) 데이터는



144 액션 (Action)

ROS 1에서의 액션은 목표, 피드백, 결과 값을 토픽으로만 주고 받았는데 **ROS 2에서는 토픽과 서비스 방식을 혼합하여 사용**하였다. 그 이유로 토픽으로만 액션을 구성하였을 때 토픽의 특징인 비동기식 방식을 사용하게 되어 ROS 2 액션에서 새롭게 선보이는 **목표 전달 (send_goal), 목표 취소 (cancel_goal), 결과 받기 (get_result)**를 동기식인 서비스를 사용하기 위해서이다. 이런 비동기 방식을 이용하다보면 원하는 타이밍에 적절한 액션을 수행하기 어려운데 이를 원활히 구현하기 위하여 **목표 상태 (goal_state)**라는 것이 ROS 2에서 새롭게 선보였다. 목표 상태는 **목표 값을 전달 한 후의 상태 머신을 구동하여 액션의 프로세스를 쫓는 것**이다. 여기서 말하는 상태머신은 **Goal State Machine**으로 다음 그림과 같이 액션 목표 전달 이후의 액션의 상태 값을 액션 클라이언트에게 전달할 수 있어서 비동기, 동기 방식이 혼재된 액션의 처리를 원활하게 할 수 있게 되어 있다.




```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtle_teleop_key
```

```
Reading from keyboard
```

```
-----
```

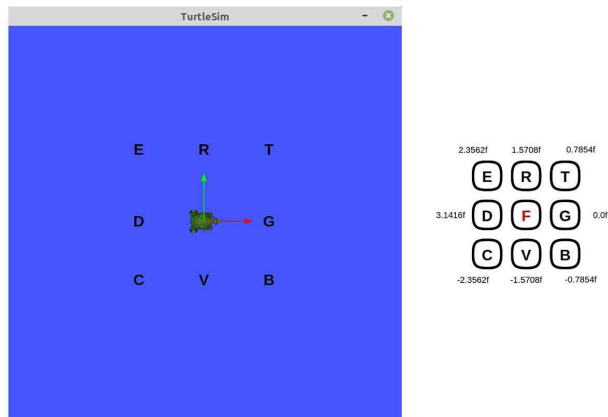
```
Use arrow keys to move the turtle.
```

```
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
```

```
'Q' to quit.
```

146 액션 서버 및 클라이언트

지금까지는 `teleop_turtle` 노드가 실행된 터미널 창에서 $\leftarrow \uparrow \downarrow \rightarrow$ 키와 같이 화살표 키를 눌러 `turtlesim`의 거북이를 움직여봤는데 이번에는 `G, B, V, C, D, E, R, T` 키를 사용해보겠다. 이 키는 각 거북이들의 `rotate_absolute` 액션을 수행함에 있어서 액션의 목표 값을 전달하는 목적으로 사용된다. `F` 키를 중심으로 주위의 8개의 버튼을 사용하는 것으로 다음 그림과 같이 각 버튼은 거북이를 절대 각도로 회전하도록 목표 값이 설정되어 있다. 그리고 `F` 키를 누르면 전달한 목표 값을 취소하여 동작을 바로 멈추게 된다. 여기서 `G` 키는 시계 방향 3시를 가리키는 `theta` 값인 `0.0` 값에 해당되고 `rotate_absolute` 액션의 기준 각도가 된다. 다른 키는 위치별로 `0.7854 radian` 값씩 정회전 방향(반시계 방향)으로 각 각도 값이 할당되어 있다. 예를 들어 `R` 키를 누르면 `1.5708 radian` 목표 값이 전달되어 거북이는 12기 방향으로 회전하게 된다. 각 키에 할당된 라디안 값은 코드를 확인하면 자세히 살펴볼 수 있다.



147 액션 서버 및 클라이언트

액션 목표는 도중에 취소할 수도 있는데 `turtlesim_node` 에도 그 상황을 터미널 창에 표시해준다. 예를 들어 액션 목표의 취소 없이 목표 `theta` 값에 도달하면 아래와 같이 표시된다.

```
[INFO]: Rotation goal completed successfully
```

하지만 액션 목표 `theta` 값에 도달하기 전에 `turtle_teleop_key`가 실행된 터미널 창에서 F 키를 눌러 액션 목표를 취소하게 되면 `turtlesim_node`이 실행된 터미널 창에 아래와 같이 목표가 취소 되었음을 알리면서 거북이는 그 자리에서 멈추게된다.

```
[INFO]: Rotation goal canceled
```

148 노드 정보 (ros2 node info)

```
$ ros2 node info /turtlesim
/turtlesim
(생략)
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

```
$ ros2 node info /teleop_turtle
/teleop_turtle
(생략)
Action Servers:

Action Clients:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
```

149 액션 목록 (ros2 action list -t)

```
$ ros2 action list -t  
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

```
$ ros2 action info /turtle1/rotate_absolute
```

```
Action: /turtle1/rotate_absolute
```

```
Action clients: 1
```

```
  /teleop_turtle
```

```
Action servers: 1
```

```
  /turtlesim
```

```
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.5708}"
```

```
Waiting for an action server to become available...
```

```
Sending goal:
```

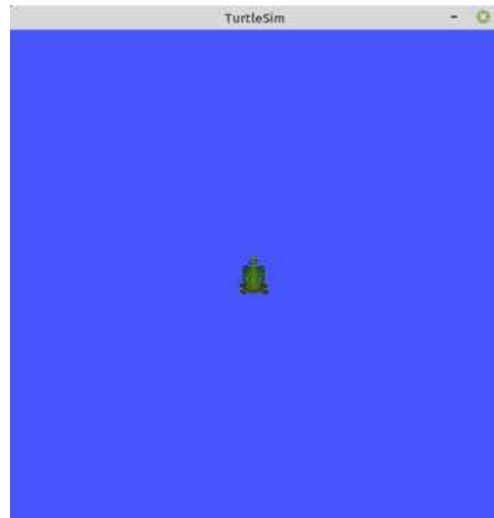
```
  theta: 1.5708
```

```
Goal accepted with ID: b991078e96324fc994752b01bc896f49
```

```
Result:
```

```
  delta: -1.5520002841949463
```

```
Goal finished with status: SUCCEEDED
```



```
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: -1.5708}" --feedback
Waiting for an action server to become available...
Sending goal:
  theta: -1.5708

Goal accepted with ID:
ad7dc695c07c499782d3ad024fa0f3d2
Feedback:
  remaining: -3.127622127532959

Feedback:
  remaining: -3.111621856689453

Feedback:
  remaining: -3.0956220626831055

(생략)

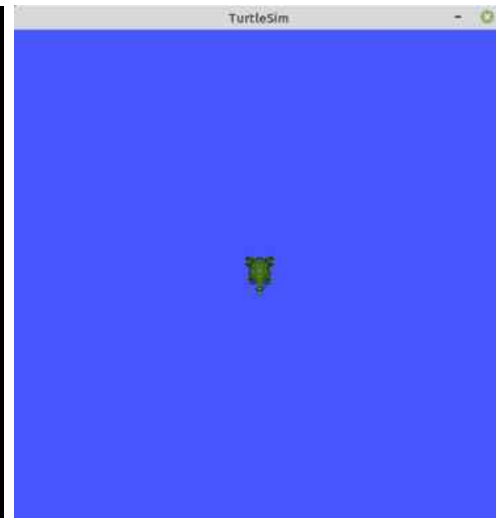
Feedback:
  remaining: -0.03962135314941406

Feedback:
  remaining: -0.023621320724487305

Feedback:
  remaining: -0.007621288299560547

Result:
  delta: 3.1040005683898926

Goal finished with status: SUCCEEDED
```



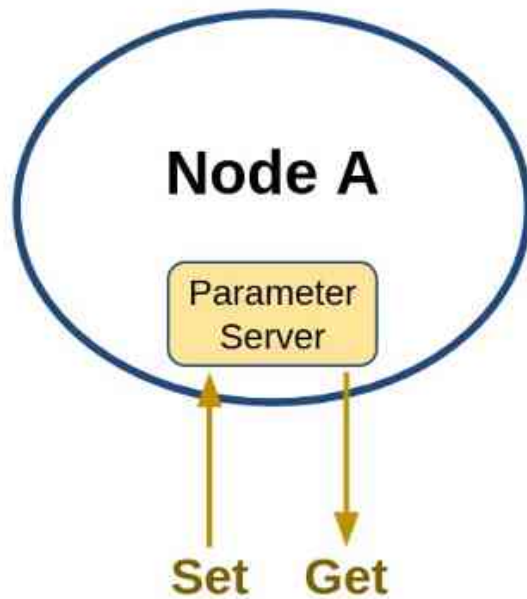
153 액션 인터페이스 (action interface, action)

여기서 다룬 액션 또한 토픽, 서비스와 마찬가지로 별도의 인터페이스를 가지고 있는데 이를 **액션 인터페이스**라 부르며, 파일로는 **action 파일**을 가르킨다. 액션 인터페이스는 메시지 및 서비스 인터페이스의 확장형이라고 볼 수 있는데 위에서 액션 목표를 전달할 때 실습으로 사용하였던 `/turtle1/rotate_absolute` 서비스를 예를 들어 설명하겠다.

`/turtle1/rotate_absolute` 액션에 사용된 `RotateAbsolute.action` 인터페이스를 알아보기 위해서는 원본 파일을 참고해도 되고, `'ros2 interface show'` 명령어를 이용하여 확인할 수 있다. 이 명령어를 이용하면 아래의 결과 값과 같이 `'turtlesim/action/RotateAbsolute.action'`은 float32 형태의 `theta`, `delta`, `remaining` 라는 세개의 데이터가 있음을 알 수 있다. 여기서 서비스와 마찬가지로 `'---'`이라는 구분자를 사용하여 **액션 목표(goal)**, **액션 결과(result)**, **액션 피드백(feedback)**으로 나누어 사용하게 된다. 즉 `theta`는 액션 목표, `delta`는 액션 결과, `remaining`는 액션 피드백에 해당된다. 참고로 각 데이터는 각도의 SI 단위인 라디안(radian)을 사용한다.

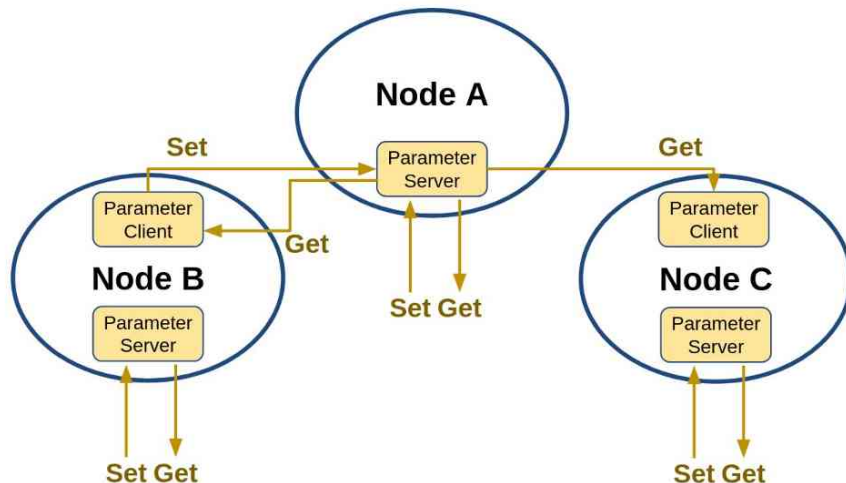
```
$ ros2 interface show turtlesim/action/RotateAbsolute.action
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

ROS 2 파라미터 (parameter)



156 파라미터 (parameter)

파라미터 관련 기능은 **RCL(ROS Client Libraries)**의 기본 기능으로 다음 그림과 같이 모든 노드가 자신만의 **Parameter server**를 가지고 있고, 그림과 같이 각 노드는 **Parameter client**도 포함시킬 수 있어서 자기 자신의 파라미터 및 다른 노드의 파라미터를 읽고 쓸 수 있게 된다. 이를 활용하면 각 노드의 다양한 매개변수를 글로벌 매개변수처럼 사용할 수 있게 되어 추가 프로그래밍이나 컴파일 없이 능동적으로 변화 가능한 프로세스를 만들 수 있게 된다. 그리고 각 파라미터는 **yaml** 파일 형태의 파라미터 설정 파일을 만들어 초기 파라미터 값 설정 및 노드 실행시에 파라미터 설정 파일을 불러와서 사용할 수 있기에 **ROS 2** 프로그래밍에 매우 유용하게 사용할 수 있다.



```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 run turtlesim turtle_teleop_key
```

```
$ ros2 param list
```

```
/teleop_turtle:
```

```
scale_angular
```

```
scale_linear
```

```
use_sim_time
```

```
/turtlesim:
```

```
background_b
```

```
background_g
```

```
background_r
```

```
use_sim_time
```

158 파라미터 내용 확인 (ros2 param describe)

```
$ ros2 param describe /turtlesim background_b  
Parameter name: background_b  
Type: integer  
Description: Blue channel of the background color  
Constraints:  
  Min value: 0  
  Max value: 255  
  Step: 1
```

159 파라미터 읽기 (ros2 param get)

```
$ ros2 param get /turtlesim background_r
```

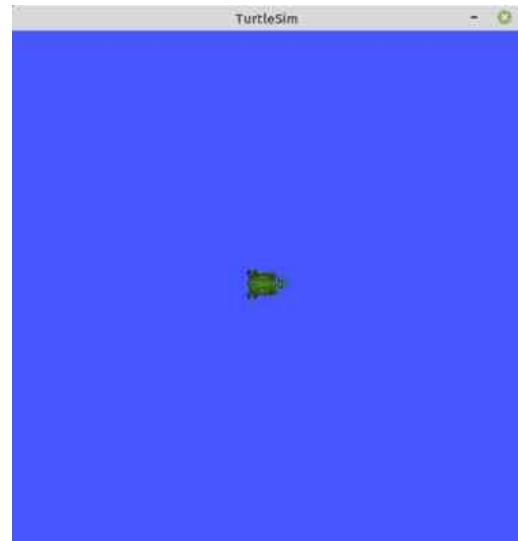
```
Integer value is: 69
```

```
$ ros2 param get /turtlesim background_g
```

```
Integer value is: 86
```

```
$ ros2 param get /turtlesim background_b
```

```
Integer value is: 255
```



160 파라미터 쓰기 (ros2 param set)

```
$ ros2 param set /turtlesim background_r 148
```

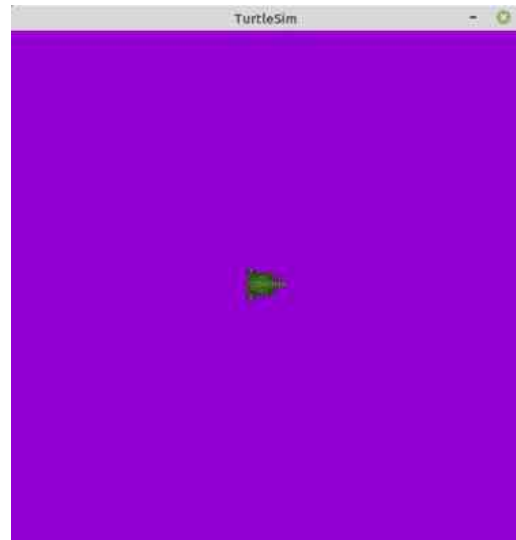
```
Set parameter successful
```

```
$ ros2 param set /turtlesim background_g 0
```

```
Set parameter successful
```

```
$ ros2 param set /turtlesim background_b 211
```

```
Set parameter successful
```



161 파라미터 저장 (ros2 param dump) 및 재사용

위 내용에서 파라미터 쓰기 (ros2 param set)를 사용하면 기본 파라미터의 값을 바꾸어 사용할 수 있다는 것을 알았다. 이 명령어를 이용하면 파라미터를 변경할 수 있지만 turtlesim 노드를 종료하였다가 다시 시작하면 모든 파라미터는 초기 값으로 다시 설정된다. 이에 필요하다면 현재 파라미터를 저장하고 다시 불러와야 하는데 이 때에는 파라미터 저장 (ros2 param dump) 명령어를 이용하면 된다. 파라미터 저장 (ros2 param dump)는 아래와 같이 'ros2 param dump' 명령어에 노드 이름을 적어주면 현재 폴더에 해당 노드 이름으로 설정 파일이 yaml 형태로 저장된다. 예를 들어 아래 예제에서는 'turtlesim.yaml' 파일로 저장되었다.

```
$ ros2 param dump /turtlesim
Saving to: ./turtlesim.yaml
```

```
$ ros2 run turtlesim turtlesim_node --ros-args --params-file
./turtlesim.yaml
```

```
$ cat ./turtlesim.yaml
turtlesim:
  ros__parameters:
    background_b: 211
    background_g: 0
    background_r: 148
    use_sim_time: false
```

162 파라미터 삭제 (ros2 param delete)

```
$ ros2 param delete /turtlesim background_b  
Deleted parameter successfully
```

```
$ ros2 param list /turtlesim  
background_g  
background_r  
use_sim_time
```

ROS 2 토픽, 서비스, 액션 정리 및 비교

164 토픽, 서비스, 액션 비교

지금까지 강좌에서 다루었던 토픽, 서비스, 액션은 ROS의 중요 컨셉이자 앞으로 강좌에서 다룰 ROS 프로그래밍에 있어서 매우 중요한 부분이기때 다시 한번 비교를 해보도록 하겠다. 여기서 비교한 연속성, 방향성, 동기성, 다자간 연결, 노드 역할, 동작 트리거, 인터페이스를 각 토픽, 서비스, 액션의 서로 다른 특징이라고 볼 수 있고 노드간의 데이터 전송에 있어서 특성에 맞게 선택하여 ROS 프로그래밍을 하게 된다.

	토픽 (topic)	서비스 (service)	액션 (action)
연속성	연속성	일회성	복합 (토픽+서비스)
방향성	단방향	양방향	양방향
동기성	비동기	동기	동기 + 비동기
다자간 연결	1:1, 1:N, N:1, N:N (publisher:subscriber)	1:1 (server:client)	1:1 (server:client)
노드 역할	발행자 (publisher) 구독자 (subscriber)	서버 (server) 클라이언트 (client)	서버 (server) 클라이언트 (client)
동작 트리거	발행자	클라이언트	클라이언트
인터페이스	msg 인터페이스	srv 인터페이스	action 인터페이스
CLI 명령어	ros2 topic ros2 interface	ros2 service ros2 interface	ros2 action ros2 interface
사용 예	센서 데이터, 로봇 상태, 로봇 좌표, 로봇 속도 명령 등	LED 제어, 모터 토크 On/Off, IK/FK 계산, 이동 경로 계산 등	목적지로 이동, 물건 파지, 복합 태스크 등

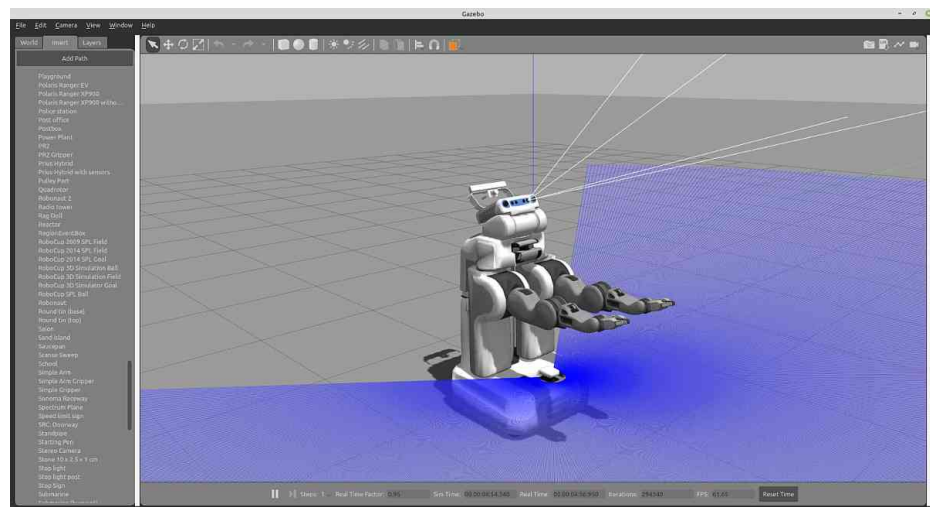
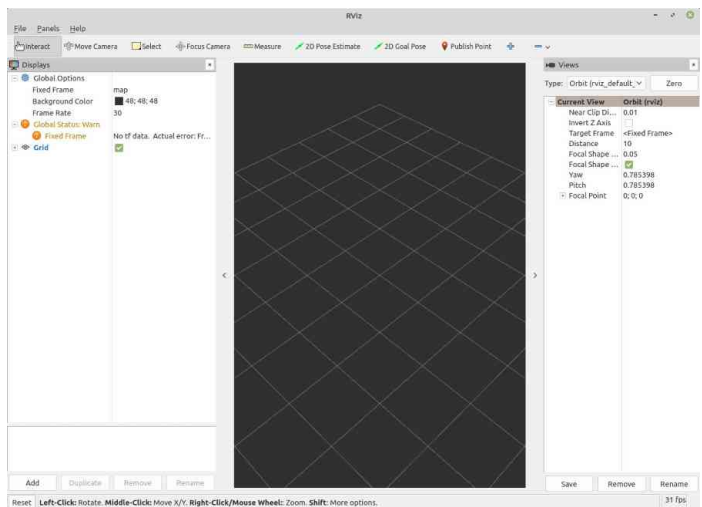
165 토픽, 서비스, 액션 비교

아래의 표는 토픽, 서비스, 액션에서 사용되는 `msg`, `srv`, `action` 인터페이스를 비교한 내용이다. 모든 인터페이스는 `msg` 인터페이스의 확장형이라고 생각하면 되고 `---` 구분자가 0, 1, 2개로 몇개를 넣어 데이터를 구분했냐의 차이이다.

	msg 인터페이스	srv 인터페이스	action 인터페이스
확장자	*.msg	*.srv	*.action
데이터	토픽 데이터 (data)	서비스 요청 (request) --- 서비스 응답 (response)	액션 목표 (goal) --- 액션 결과 (result) --- 액션 피드백 (feedback)
형식	fieldtype1 fieldname1 fieldtype2 fieldname2 fieldtype3 fieldname3	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4 --- fieldtype5 fieldname5 fieldtype6 fieldname6
사용 예	[geometry_msgs/msg/Twist]	[turtlesim/srv/Spawn.srv]	[turtlesim/action/RotateAbsolute.action]
	Vector3 linear Vector3 angular	float32 x float32 y float32 theta string name --- string name	float32 theta --- float32 delta --- float32 remaining

ROS 2 도구

- CLI 형태의 Command-Line Tools
- GUI 형태의 RQt
- 3차원 시각화툴인 RViz
- 3차원 시뮬레이터 Gazebo



- CLI

- CLI 기반 Command-Line Tools
- 명령어 기반의 툴로 로봇 액세스 및 거의 모든 ROS 기능을 다룬다.
- 개발환경 및 빌드, 테스트 툴 (colcon)
- 데이터를 기록, 재생, 관리하는 툴 (ros2bag)
- colcon, ros2bag 외 20여 가지
 - action
 - bag
 - component
 - daemon
 - doctor
 - extension_points
 - extensions
 - interface
 - launch
 - lifecycle
 - multicast
 - node
 - param
 - pkg
 - run
 - security
 - service
 - topic
 - wtf


```
$ ros2 [verbs] [sub-verbs] [options] [arguments]
```

```
$ ros2 [tab] [tab] [tab] [tab] ...
```

```
$ ros2 -h
```

```
$ ros2 node -h
```

```
$ ros2 node info -h
```

ros2cli + [verbs]	[arguments]	기능
ros2 run	<package> <executable>	특정 패키지의 특정 노드 실행 (1개의 노드) * executable에 따라 복수 노드도 실행 가능
ros2 launch	<package> <launch-file>	특정 패키지의 특정 런치 파일 실행 (0개~복수개의 노드)

```
$ ros2 run turtlesim turtlesim_node
```

```
$ ros2 launch demo_nodes_cpp talker_listener.launch.py
```

ros2cli + [verbs]	[sub-verbs]	기능
ros2 pkg	create executables list prefix xml	새로운 ROS 2 패키지 생성 지정 패키지의 실행 파일 목록 출력 사용 가능한 패키지 목록 출력 지정 패키지의 저장 위치 출력 지정 패키지의 패키지 정보 파일(xml) 출력
ros2 node	info list	실행 중인 노드 중 지정한 노드의 정보 출력 실행 중인 모든 노드의 목록 출력
ros2 topic	bw delay echo find hz info list pub type	지정 토픽의 대역폭 측정 지정 토픽의 지연시간 측정 지정 토픽의 데이터 출력 지정 타입을 사용하는 토픽 이름 출력 지정 토픽의 주기 측정 지정 토픽의 정보 출력 사용 가능한 토픽 목록 출력 지정 토픽의 토픽 발행 지정 토픽의 토픽 타입 출력
ros2 service	call find list type	지정 서비스의 서비스 요청 전달 지정 서비스 타입의 서비스 출력 사용 가능한 서비스 목록 출력 지정 서비스의 타입 출력
ros2 action	info list send_goal	지정 액션의 정보 출력 사용 가능한 액션 목록 출력 지정 액션의 액션 목표 전송

ros2cli + [verbs]	[sub-verbs]	기능
ros2 interface	list package packages proto show	사용 가능한 모든 인터페이스 목록 출력 특정 패키지에서 사용 가능한 인터페이스 목록 출력 인터페이스 패키지들의 목록 출력 지정 패키지의 프로토타입 출력 지정 인터페이스의 데이터 형태 출력
ros2 param	delete describe dump get list set	지정 파라미터 삭제 지정 파라미터 정보 출력 지정 파라미터 저장 지정 파라미터 읽기 사용 가능한 파라미터 목록 출력 지정 파라미터 쓰기
ros2 bag	info play record	저장된 rosbag 정보 출력 rosbag 기록 rosbag 재생

```
$ ros2 pkg executables turtlesim
```

```
$ ros2 node info /turtlesim
```

```
$ ros2 topic echo /turtle1/cmd_vel
```

```
$ ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 255, b: 255, width: 10}"
```

```
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.5708}"
```

```
$ ros2 interface show turtlesim/srv/Spawn.srv
```

```
$ ros2 param set /turtlesim background_r 148
```

```
$ ros2 bag record /turtle1/cmd_vel
```

ros2cli + [verbs]	[sub-verbs] (options)	기능
ros2 extensions	(-a) (-v)	ros2cli의 extension 목록 출력
ros2 extension_points	(-a) (-v)	ros2cli의 extension point 목록 출력
ros2 daemon	start status stop	daemon 시작 daemon 상태 보기 daemon 정지
ros2 multicast	receive send	multicast 수신 multicast 전송
ros2 doctor	hello (-r) (-rf) (-iw)	ROS 설정 및 네트워크, 패키지 버전, rmw 미들웨어 등과 같은 잠재적 문제를 확인하는 도구
ros2 wtf	hello (-r) (-rf) (-iw)	doctor와 동일함 (ros2 doctor의 alias) (WTF: Where's The Fire)
ros2 lifecycle	get list nodes set	라이프사이클 정보 출력 지정 노드의 사용 가능한 상태전이 목록 출력 라이프사이클을 사용하는 노드 목록 출력 라이프사이클 상태 전환 트리거

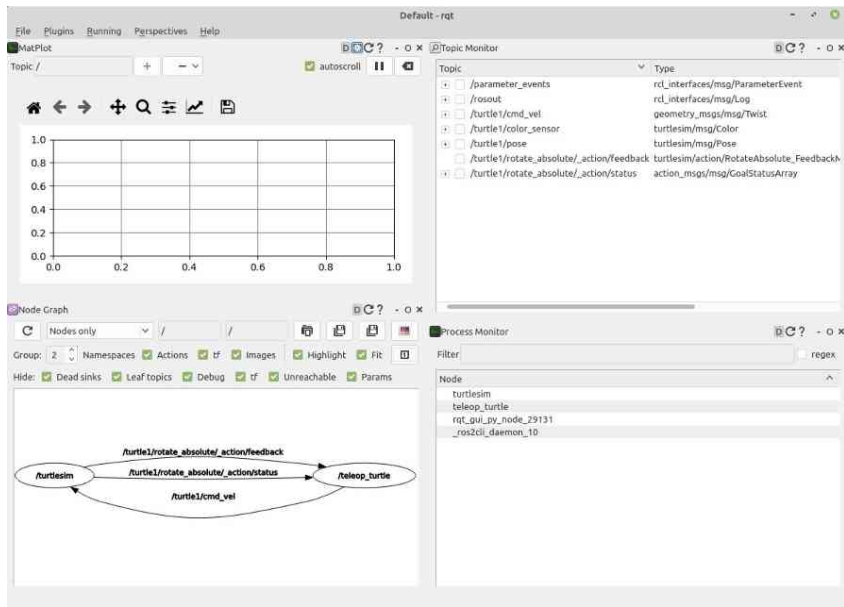
ros2cli + [verbs]	[sub-verbs] (options)	기능
ros2 component	list load standalone types unload	실행 중인 컨테이너와 컴포넌트 목록 출력 지정 컨테이너 노드의 특정 컴포넌트 실행 표준 컨테이너 노드로 특정 컴포넌트 실행 사용 가능한 컴포넌트들의 목록 출력 지정 컴포넌트의 실행 중지
ros2 security	create_key create_keystore create_permission generate_artifacts generate_policy list_keys	보안키 생성 보안키 저장소 생성 보안 허가 파일 생성 보안 정책 파일을 이용하여 보안키 및 보안 허가 파일 생성 보안 정책 파일(policy.xml) 생성 보안키 목록 출력

```
$ ros2 extensions -a  
$ ros2 extension_points -a  
$ ros2 daemon start  
$ ros2 multicast send  
$ ros2 doctor -r  
$ ros2 wtf -r  
$ ros2 lifecycle list lc_talker  
$ ros2 component load /ComponentManager composition composition::Talker  
$ ros2 security create_key demo_keys /talker_listener/listener
```


- RQt

- 그래픽 인터페이스 개발을 위한 Qt 기반 프레임워크 제공
- 노드와 그들 사이의 연결 정보 표시(rqt_graph)
- 속도, 전압 또는 시간이 지남에 따라 변화하는 데이터를 플로팅(rqt_plot)
- rqt_graph, rqt_plot 외 30여 가지

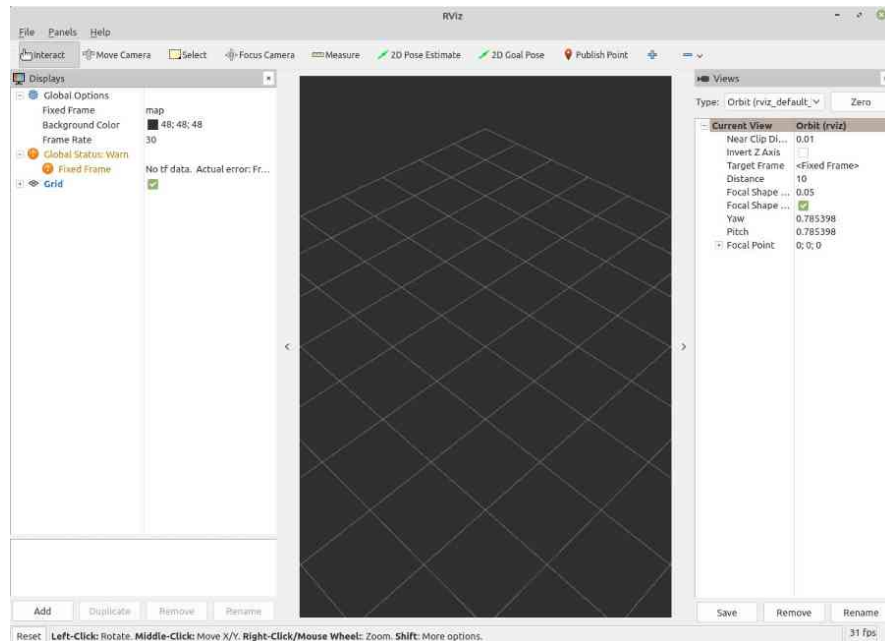
```
$ rqt  
  
$ rqt_bag  
  
$ rqt_graph
```



178 ROS 2 도구: 3차원 시각화툴인 RViz

- RViz
 - 3차원 시각화툴
 - 레이저, 카메라 등의 센서 데이터를 시각화
 - 로봇 외형과 계획된 동작을 표현

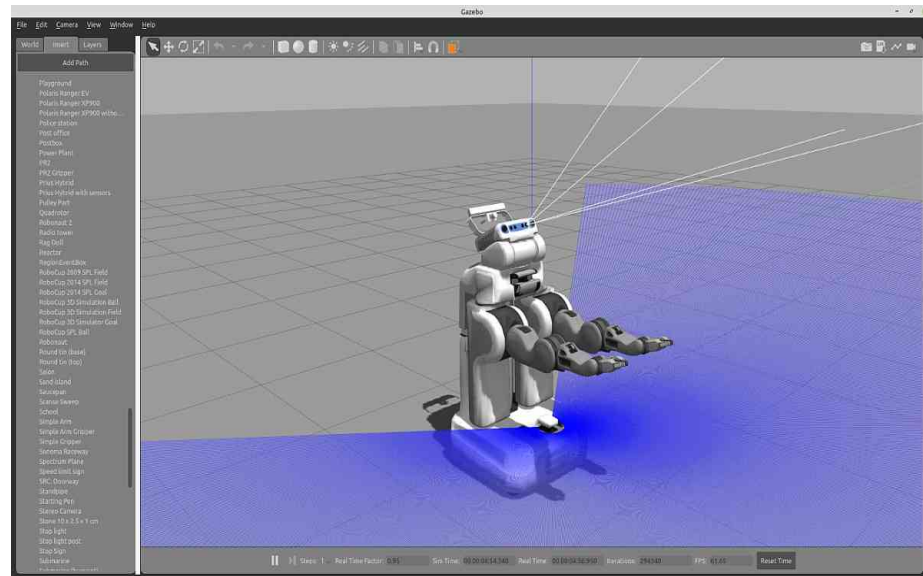
```
$ rviz2
```



179 ROS 2 도구: 3차원 시뮬레이터 Gazebo

- Gazebo
 - 3차원 시뮬레이터
 - 물리 엔진을 탑재, 로봇, 센서, 환경 모델 등을 지원
 - 타 시뮬레이터 대비 ROS와의 높은 호환성

\$ gazebo



ROS의 종합 GUI 툴 RQt

181 RQt란?

RQt는 플러그인 형태로 다양한 도구와 인터페이스를 구현할 수 있는 그래픽 사용자 인터페이스(GUI, Graphical User Interface) 프레임 워크이며 다양한 목적의 GUI 툴을 모아둔 ROS의 종합 GUI 툴박스이다.

Topics - Message Publisher - Message Type Browser - Topic Monitor	Introspection - Node Graph - Process Monitor Configuration - Dynamic Reconfigure Logging - Console Miscellaneous Tools - Python Console - Shell	Visualization - Image View - Plot Robot Tools - Diagnostics Viewer - Robot Steering
--	---	---

The screenshot displays the RQt (Robot Query Tool) interface, which is used for monitoring and controlling ROS (Robot Operating System) topics and parameters. The interface is divided into several panels:

- Topic Monitor:** This panel shows a list of topics being monitored. The columns include Topic, Type, Bandwidth, Hz, and Value. The topics listed are:
 - `angular` (Type: `geometry_msgs/Vector3`)
 - `linear` (Type: `geometry_msgs/Vector3`)
 - `/turtle1/color_sensor` (Type: `turtlesim/msg/Color`)
 - `/turtle1/pose` (Type: `turtlesim/msg/Pose`)
 - `/turtle1/rotate_absolute/_action/feedback` (Type: `turtlesim/action/RotateAbsolute_FeedbackMessage`)
 - `/turtle1/rotate_absolute/_action/status` (Type: `action_msgs/msg/GoalStatusArray`)
- Parameter Reconfigure:** This panel allows for the configuration of parameters for the `/turtlesim` node. The parameters shown are:
 - `background_b` (Value: 0, Range: 0 to 255)
 - `background_g` (Value: 0, Range: 0 to 255)
 - `background_r` (Value: 0, Range: 0 to 255)
 - `use_sim_time` (Boolean: ☐)
- Console:** This panel displays a list of messages received from the `turtlesim` node. The messages are:
 - #594: "Oh no! I hit the wal..." (Severity: Warn, Node: turtlesim, Stamp: 07:25:25.183...)
 - #593: "Oh no! I hit the wal..." (Severity: Warn, Node: turtlesim, Stamp: 07:25:25.167...)

The interface also includes a **Robot Steering** panel on the right, which shows a slider for controlling the turtle's velocity (0.00 m/s) and a **Console** panel at the bottom right for displaying messages.

- RQt 설치

```
$ sudo apt install ros-humble-rqt*
```

- RQt 실행

- **첫번째 실행 방법**은 RQt를 실행하여 메뉴에서 원하는 플러그인을 골라서 실행하는 것이다. 간단히 설명하면 터미널 창에서 `rqt`로 RQt를 실행하면 하나의 GUI 툴이 실행되는데 `Plugins` 메뉴에서 원하는 플러그인을 선택하면 해당 플러그가 화면에 표시될 것이다.
 - 예) `Plugins` -> `Topics` -> `Topic Monitor`
- **두번째 실행 방법**은 `ros2 run` 명령어를 이용하여 각 rqt 관련 패키지들의 노드들을 하나씩 실행시키는 방법이다. RQt 플러그인이 상당히 많아서 자주 쓰는 한두개의 플러그인을 사용할 때 이용하고 alias로 설정하여 쓸 때 사용된다.
 - 예) \$ ros2 run rqt_msg rqt_msg
- **세번째 실행 방법**은 단축 명령어를 이용하는 것이다. 아래와 같이 미리 지정된 명령어가 있는데 이전 강좌들에서 소개되었던 방법이다. 단 많지는 않다. 비슷한 단축 명령어가 필요하다면 2번 실행 방법을 alias 단축하여 만들어 두면 동일한 효과를 가져올 수 있다.
 - 예) \$ rqt_graph

1. 액션 (Actions)

- Action Type Browser: Action 타입의 데이터 구조를 확인

2. 구성 (Configuration)

- Dynamic Reconfigure: 노드들에서 제공하는 파라미터 값 확인 및 변경
- Launch: roslaunch 의 GUI 버전

3. 내성 (Introspection)

- Node Graph: 실행 중인 노드들의 관계 및 토픽을 확인 가능한 그래프 뷰
- Package Graph: 노드의 의존 관계를 표시하는 그래프 뷰
- Process Monitor: 실행 중인 노드들의 CPU, 메모리 사용률, 스레드 수 등을 확인

4. 로깅 (Logging)

- Bag: ROS 데이터 로깅
- Console: 노드들에서 발생하는 경고(Warning), 에러(Error) 등의 메시지를 확인
- Logger Level: ROS의 Debug, Info, Warn, Error, Fatal 로거 정보를 선택하여 표시

5. 다양한 툴 (Miscellaneous Tools)

- Python Console: 파이썬 콘솔 화면
- Shell: 셸(shell)을 구동
- Web: 웹 브라우저를 구동

6. 로봇 (Robot)

- 사용하는 로봇에 따라 계기판(dashboard) 등의 플러그인을 이곳에 추가

7. 로봇 툴 (Robot Tools)

- Controller Manager: 컨트롤러 관리에 필요한 플러그인
- Diagnostic Viewer: 로봇 디바이스의 경고 및 에러 확인
- Moveit! Monitor: 로봇 매니플레이터 툴인 Moveit! 데이터 확인
- Robot Steering: 로봇에게 병진 속도와 회전 속도를 토픽으로 발행하는 GUI 툴
- Runtime Monitor: 실시간으로 노드들에서 발생하는 에러 및 경고를 확인

8. 서비스 (Services)

- Service Caller: 실행 중인 서비스 서버에 접속하여 서비스를 요청
- Service Type Browser: 서비스 타입의 데이터 구조를 확인

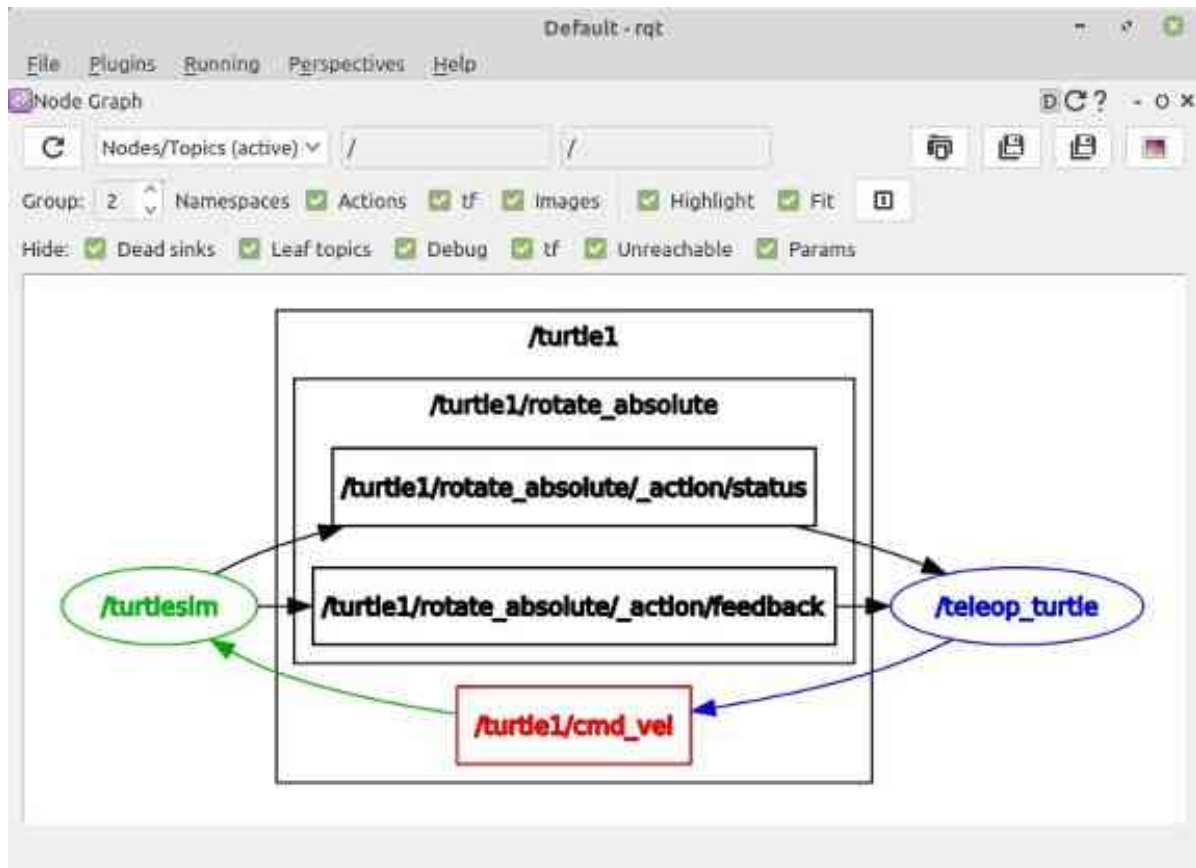
9. 토픽 (Topics)

- Message Publisher: 메시지 발행
- Message Type Browser: 메시지 타입의 데이터 구조 확인
- Topic Monitor: 토픽 목록 확인 및 사용자가 선택한 토픽의 정보를 확인

10. 시각화 (Visualization)

- Image View: 카메라의 영상 데이터를 확인
- Navigation Viewer: 로봇 내비게이션의 위치 및 목표지점 확인
- Plot: 2차원 데이터 플롯 GUI 플러그인, 2차원 데이터의 도식화
- Pose View: 현재 TF의 위치 및 모델의 위치 표시
- RViz: 3차원 시각화 툴인 RViz 플러그인
- TF Tree: tf 관계를 트리로 나타내는 그래프 뷰

185 RQt 사용 예시 : Node Graph

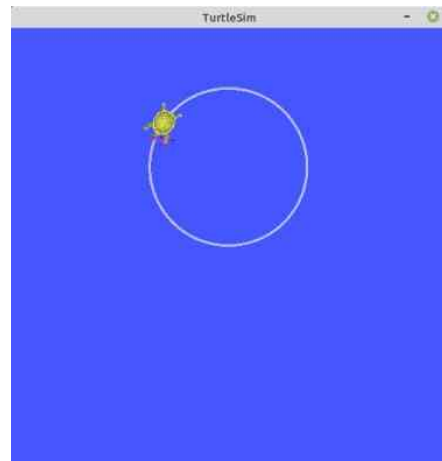
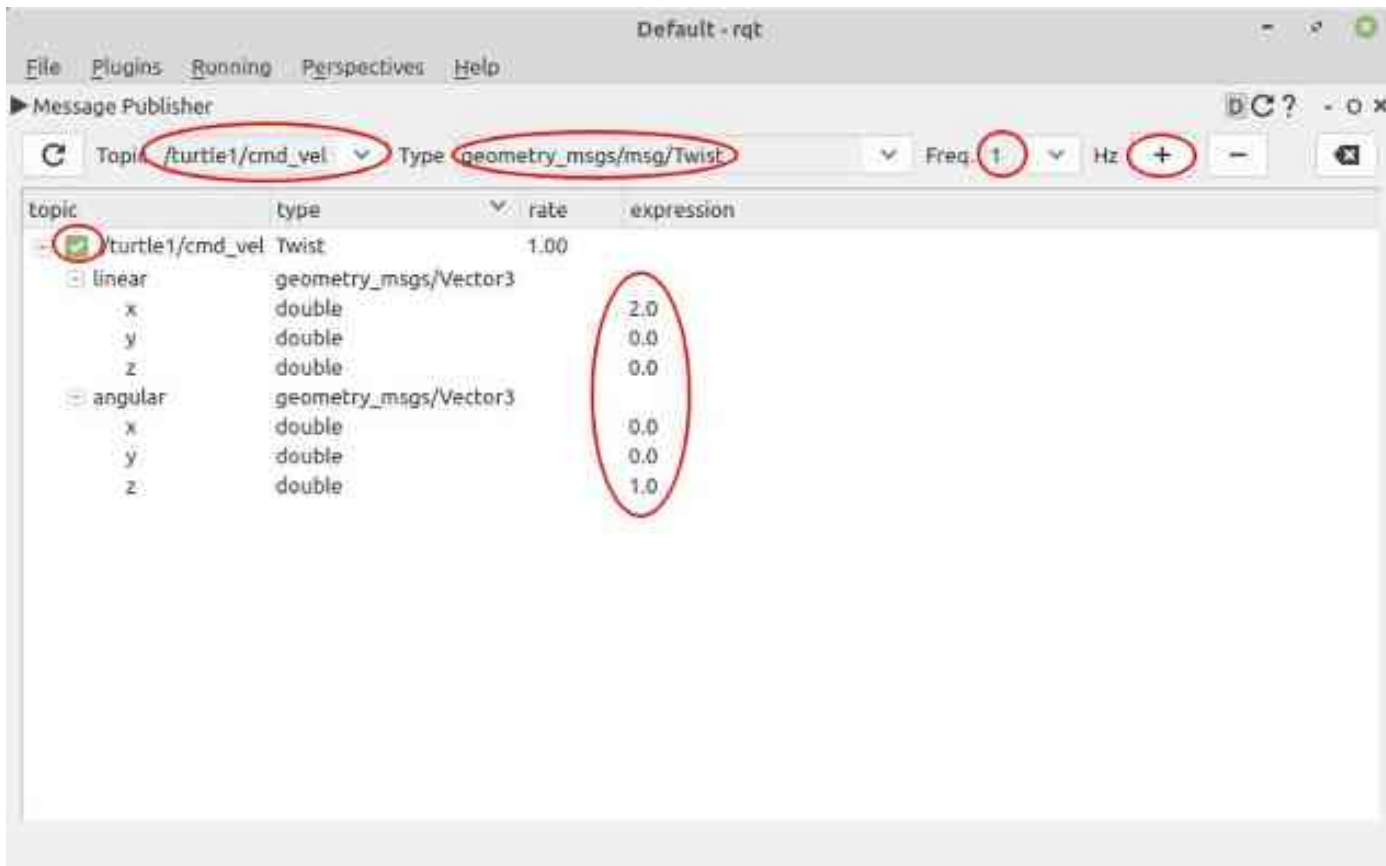


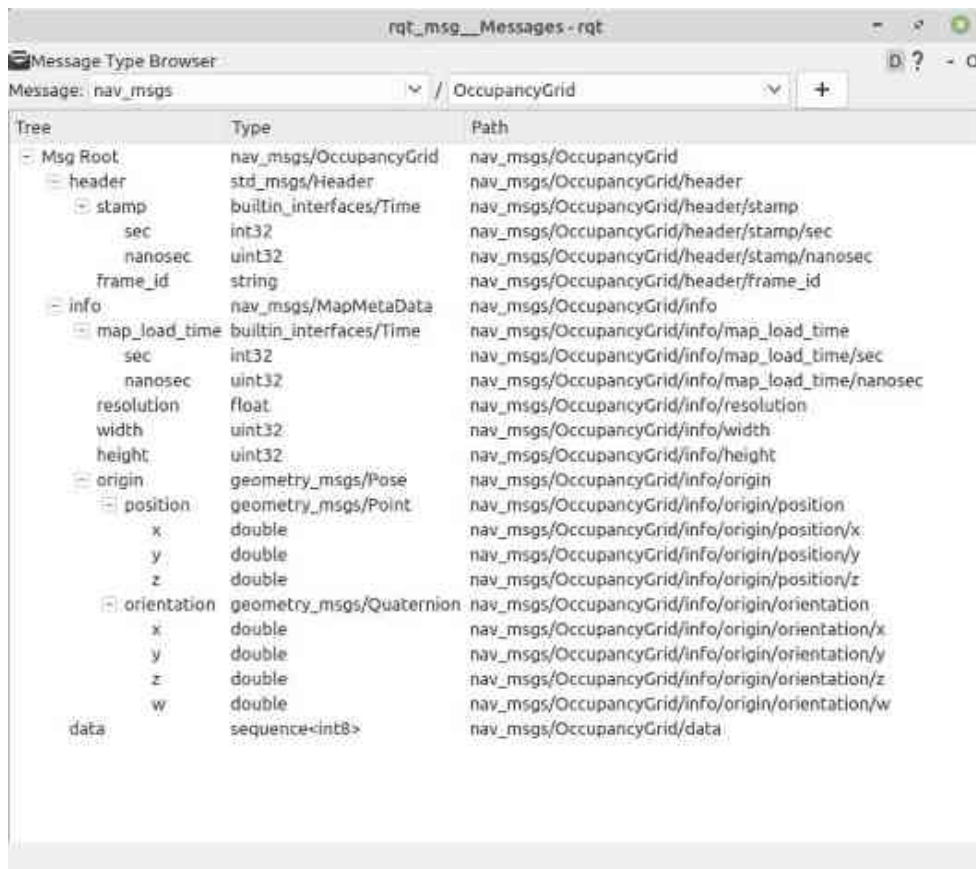
186 RQt 사용 예시 : Topic Monitor



187

RQt 사용 예시 : Message Publisher





189 RQt 사용 예시: Service Caller

Default - rqt

File Plugins Running Perspectives Help

Service Caller

Service: /turtle1/set_pen

Call

Request

Topic	Type	Expression
/turtle1/set_pen	turtlesim/srv/SetPen	
r	uint8	128
g	uint8	0
b	uint8	128
width	uint8	10
off	uint8	0

Response

Field	Type	Value
/	turtlesim/srv/SetPen.Response	

Default - rqt

File Plugins Running Perspectives Help

Service Caller

Service: /turtle1/set_pen

Call

Request

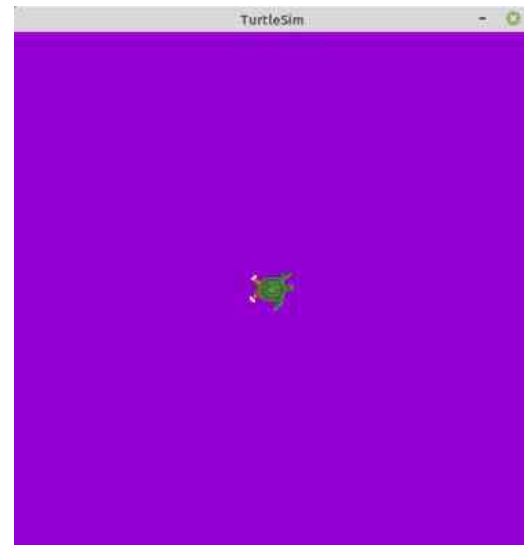
Topic	Type	Expression
/turtle1/set_pen	turtlesim/srv/SetPen	
r	uint8	128
g	uint8	0
b	uint8	128
width	uint8	10
off	uint8	0

Response

Field	Type	Value
/	turtlesim/srv/SetPen.Response	

TurtleSim

190 RQt 사용 예시 : Parameter Reconfigure

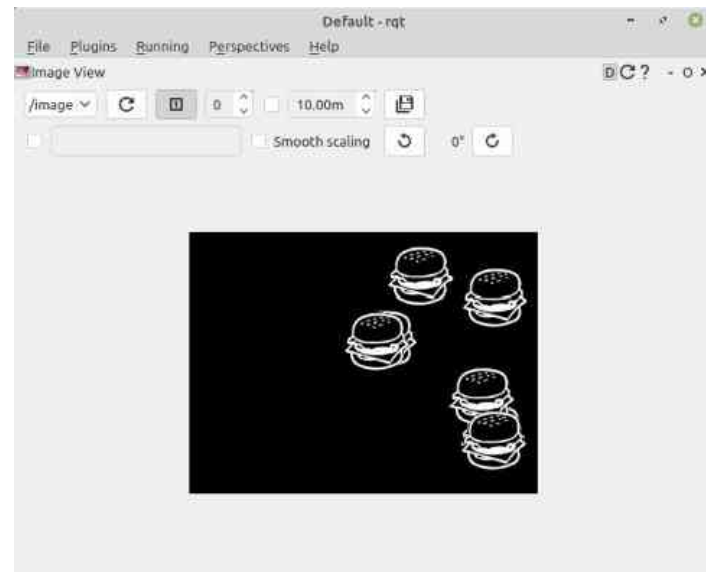
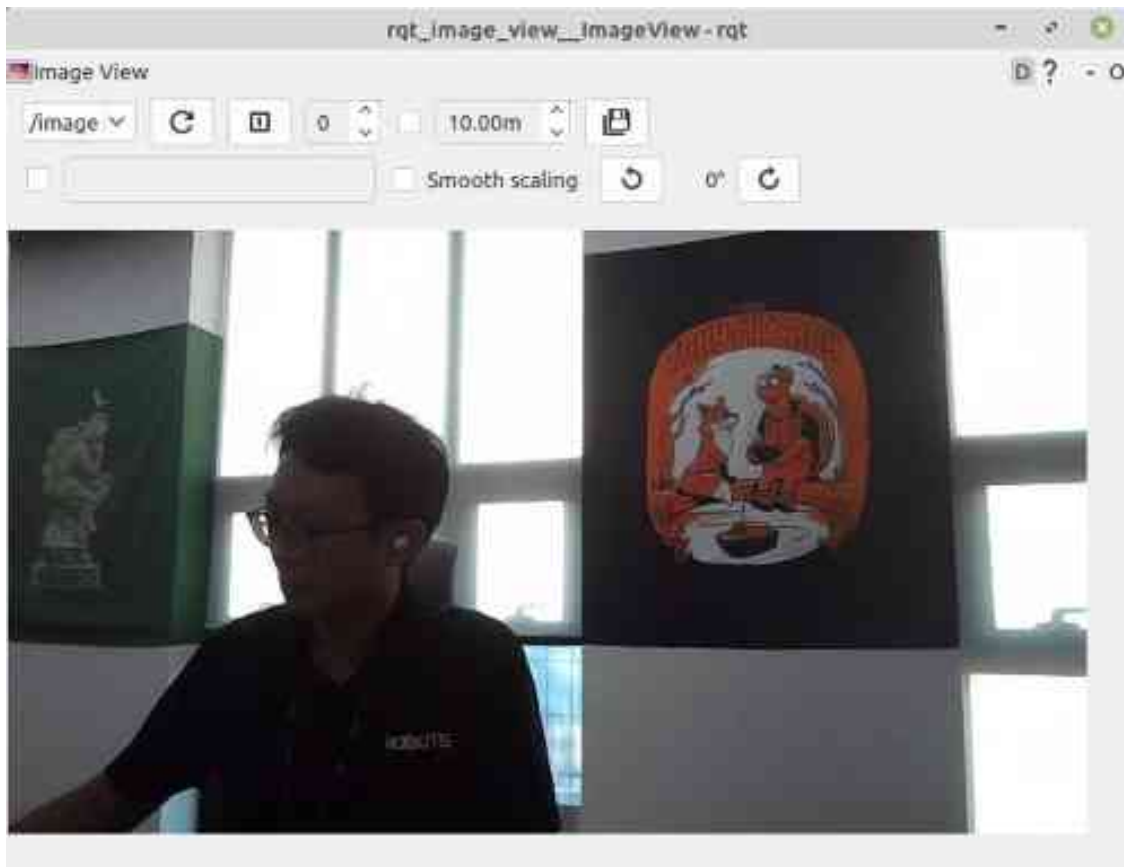


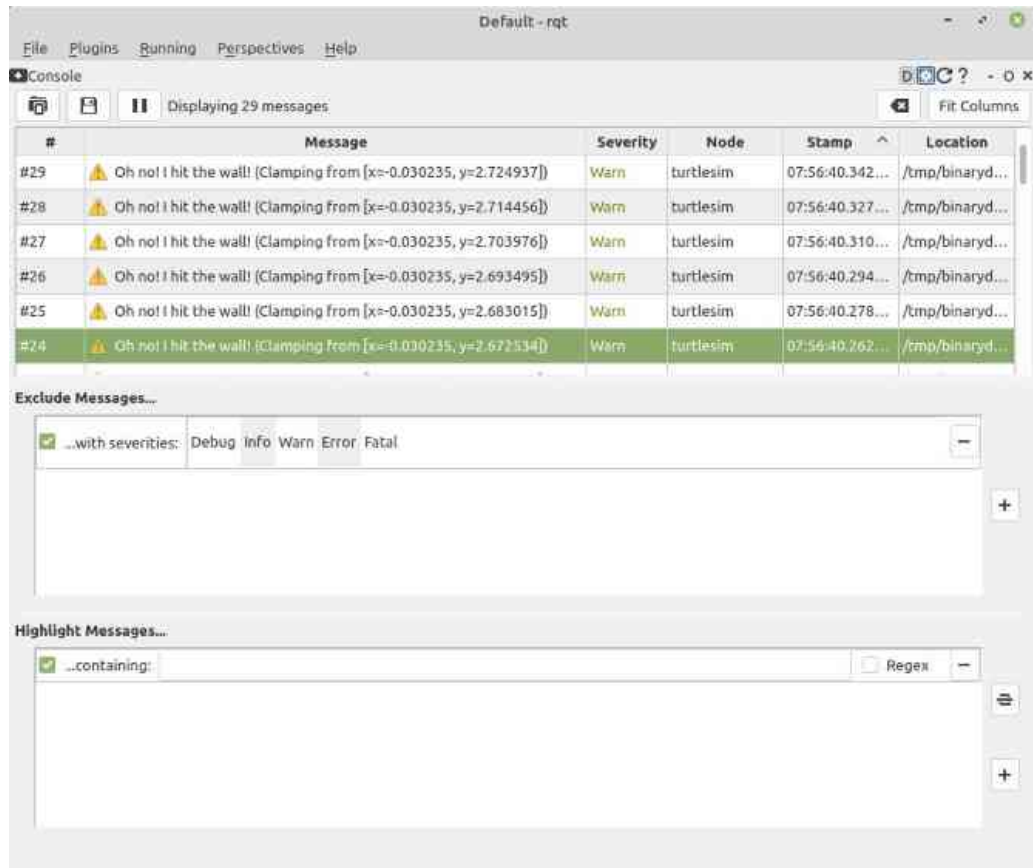
(System message might be shown here when necessary)

191 RQt 사용 예시: Plot



192 RQt 사용 예시: Image View





194 Re-inventing

한가지 더 언급하고 싶은 포인트가 있다면 Re-inventing에 대한 것이다. ROS에서의 재사용성과 코드 공유에 대한 철학은 매우 중요하여 ROS의 기본 컨셉 깊숙이 반영되어 있고 로봇 연구/개발에 있어서 Re-inventing을 줄이는 역할을 하고 있다. 예를 들어 ROS에서의 노드처럼 프로세스의 목적에 따라 프로그램을 잘게 쪼개서 다른 프로젝트에서 쉽게 다시 사용하게 만들거나 ROS 도구 설명할 때 언급된 ROS CLI 툴(ros2cli), GUI 툴(Rqt), 시각화 툴(RViz), 3차원 시뮬레이터(Gazebo)와 같은 툴처럼 로봇 개발에서 많이 사용되는 툴들을 플러그인과 개방성 중심으로 설계/개발하여 공개하고 있다. 이러한 ROS의 기본 철학은 오른쪽 그림처럼 로보틱스에서 지금껏 하던 연구/개발 방식을 벗어나게 해주었다. 자신의 연구/개발 부분에서 소모적으로 개선없이 Re-inventing되고 있는 부분이 없는지 점검하고 더 고민해보자.



[출처] Re-Inventing the Wheel from Willow Garage / PHD Comics

ROS 2의 물리량 표준 단위

196 ROS 2의 표준 단위의 필요성

ROS 2의 msg, service, action 인터페이스는 노드 간에 데이터를 주고받는데 사용된다. 그 내용에는 어떤 `이름`으로 어떠한 `자료형`으로 데이터를 보내느냐가 기술되어 있다.

예를 들어 geometry_msgs/msg/Twist 메시지 형태는 float64 자료형의 linear.x, linear.y, linear.z, angular.x, angular.y, angular.z 값을 사용하고 있다. 이는 병진 속도 3개, 회전 속도 3개를 의미하는데 **주고 받는 단위가 틀린다면 어떻게 될까?** 개발자는 m/s와 rad/s로 정의하여 개발하였는데 사용자가 cm/s, degree/s로 생각하여 사용하면 어떻게 될까? 아마 원하는 결과물을 얻을 수 없을 수 없는 것은 물론이고 자칫 큰 사고로 이어질 수도 있다.

이처럼 단위의 불일치는 개발자와 사용자 간의 불편을 겪게 하고 치명적인 소프트웨어 버그로 이어질 수도 있다. 단위의 경우 변환 프로그램을 추가로 넣는 방법도 있긴 하지만 이렇게 하면 데이터 변환으로 인해 불필요한 계산이 발생한다. 이에 ROS 커뮤니티에서는 ROS 개발 초기(2010년)부터 이러한 **단위 불일치로 인한 문제를 줄이기 위해 표준 단위에 관한 규칙**을 세웠다.

- <https://www.ros.org/reps/rep-0103.html>

197 ROS 2의 표준 단위

물리량	단위 (SI unit)	물리량	단위 (SI derived unit)
length (길이)	meter (m)	angle (평면각)	radian (rad)
mass (질량)	kilogram (kg)	frequency (주파수)	hertz (Hz)
time (시간)	second (s)	force (힘)	newton (N)
current (전류)	ampere (A)	power (일률)	watt (W)
		voltage (전압)	volt (V)
		temperature (온도)	celsius (°C)
		magnetism (자기장)	tesla (T)

ROS 2의 좌표 표현

199 ROS 2의 좌표 표현 통일의 필요성

ROS 2의 물리량 표준 단위가 ROS 프로그래밍에서의 단위 불일치를 사전에 막기위한 규칙이었다면 이번 강좌에서 다룰 내용은 **서로 다른 좌표 표현 방식을 사용할 경우 발생하는 좌표계 (coordinate system) 불일치**를 사전에 막기위한 규칙이다.

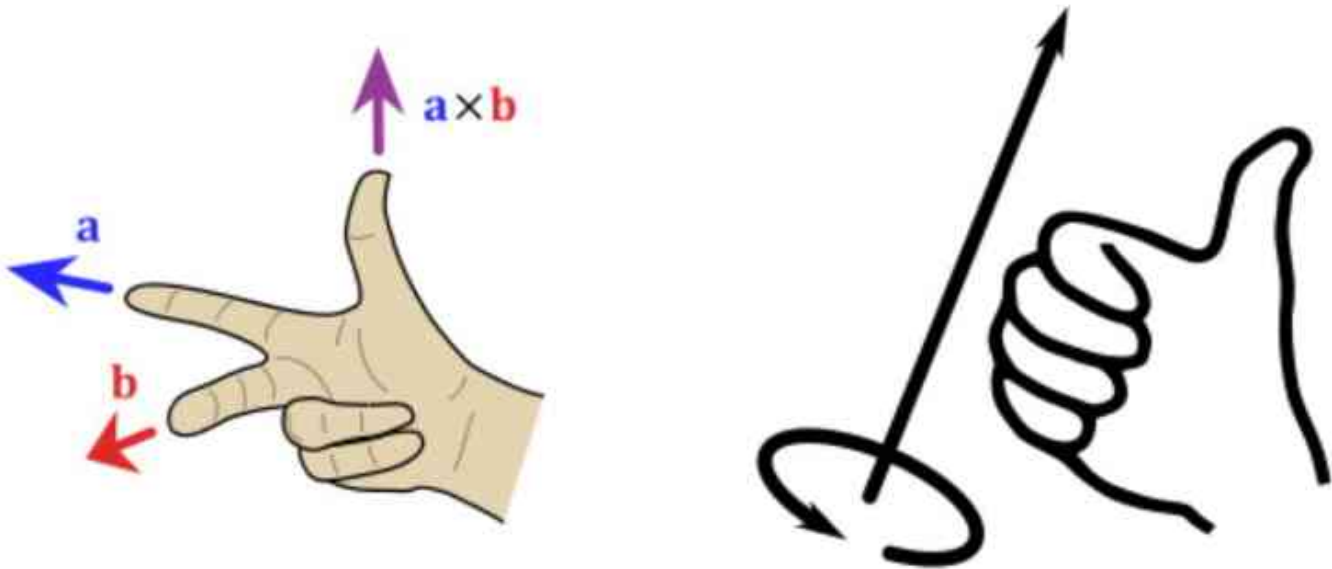
예를 들어 로봇의 센서로서 널리 사용되는 카메라의 경우, 컴퓨터 비전 분야에서 널리 사용되는 **z forward, x right, y down**를 기본 좌표계로 사용하고 있는데 로봇은 기본적으로 **x forward, y left, z up**를 기본 좌표계로 사용한다. 이러한 상황에서 한쪽 좌표계를 기준으로 맞추어 지지 않는다면 두 좌표계를 사용함에 따른 논리적 문제에 빠지게 된다. 이 문제 이외에도 로봇에서 사용하는 LiDAR 및 IMU, Torque 센서들도 제조사별로 서로 다른 좌표계를 사용할 수도 있으며 좌표 변환 API가 있더라도 기본 출력을 무엇인가를 미리 알아볼 필요가 있다. 예를 들어 IMU가 NED 타입(**x north, y east, z down, relative to magnetic north**) 이나 ENU 타입(**x-east, y-north, z-up, relative to magnetic north**)이냐를 잘 살펴봐야하고 각 센서의 값을 사용하기에 전에 각 센서들의 좌표를 로봇 좌표에 맞추어 적절한 좌표 변환이 필요하다.

앞선 강좌에서 단위의 불일치가 개발자와 사용자 간의 불편을 겪게하고 치명적인 소프트웨어 버그로 이어질 수도 있다고 하였는데 좌표 표현 또한 마찬가지로 통일된 좌표 표현을 사용하지 않으면 불편을 겪게 하고 소프트웨어 버그로 유발시킬 수 있다. 이에 ROS 커뮤니티에서는 ROS 개발 초기(2010년)부터 이러한 문제를 줄이기 위해 표준 단위에 대한 규칙과 함께 **좌표 표현에 규칙**을 세워 ROS 프로그래밍에 대한 가이드를 제공하고 있다.

200 ROS 2 좌표 표현의 기본 규칙

ROS 커뮤니티에서는 모든 좌표계를 삼차원 벡터 표기관습을 이해하기 위한 일반적인 기억법인 오른손 법칙에 따라 표현한다. 기본적으로 회전 축의 경우에는 다음 그림과 같이 검지, 중지, 엄지를 축을 사용하며 오른손의 손가락을 감는 방향이 정회전 + 방향이다.

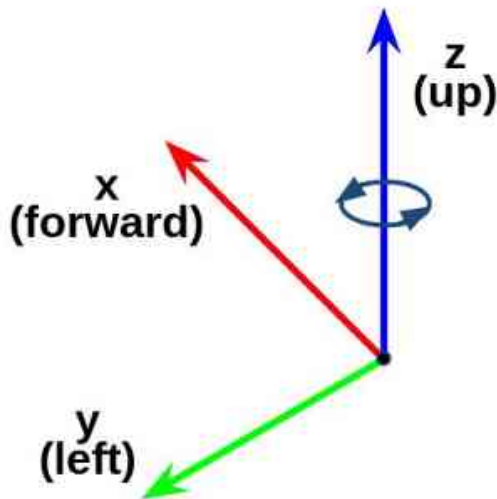
예를 들어, 표준 단위와 함께 설명하자면 회전 각은 라디안(radian)단위를 이용하고 있고, 로봇이 제자리에서 시계 12시 방향에서 9시 방향으로 회전하였다면 로봇은 정방향 +1.5708 rad 만큼 회전하였다고 이야기한다



201 ROS 2 좌표 표현의 축 방향 (Axis Orientation) 규칙

1) 기본 3축

ROS 커뮤니티에서는 다음 그림과 같이 축 방향 (Axis Orientation)으로 **x forward**, **y left**, **z up** 을 사용한다. 시각화 툴 RViz나 3차원 시뮬레이터 Gazebo에서 이러한 기본 3축의 표현에 있어서 헷갈리지 않도록 RGB의 원색으로 표현하는데 순서대로 Red는 x 축, Green은 y축, Blue는 z축을 의미한다.



202 ROS 2 좌표 표현의 축 방향 (Axis Orientation) 규칙

2) ENU 좌표

지리적 위치 (geographic locations)의 단거리 데카르트 표현의 경우 ENU(east north up) 규칙을 사용한다. 실내 로봇에서는 잘 다루지 않는 좌표이긴 하지만 비교적 큰 맵을 다루는 드론, 실외 자율주행 로봇에서 사용하는 좌표라고 생각하면 된다.

3) 접미사 프레임 사용 (Suffix Frames)

위에서 언급한 x forward, y left, z up의 기본 3축 및 ENU 좌표에서 벗어나는 경우 접미사 프레임을 사용하여 기본 좌표계와 구별하여 사용한다. 자주 사용되는 접미사로 프레임으로는 `\_optical` 접미사와 `\_ned` 접미사가 있으며 필요시 좌표 변환을 통해 사용하고 있다.

3-1) `\_optical` 접미사

컴퓨터 비전 분야의 경우, 카메라 좌표계로 많이 사용되는 z forward, x right, y down를 사용하게 되는데 이럴 경우에는 카메라 센서의 메시지에 `\_optical` 접미사를 붙여 구분한다. 이때에는 z forward, x right, y down의 카메라 좌표계와 x forward, y left, z up의 로봇 좌표계 간의 TF(transform)가 필요하다.

3-2) `\_ned` 접미사

실외에서 동작하는 시스템의 경우, 사용하는 센서 및 지도에 따라 ENU가 아닌 NED (north east down) 좌표계를 사용해야 할 때가 있다. 이때에는 `\_ned` 접미사를 붙여 구분한다.

1) 쿼터니언 (quaternion)

- 간결한 표현방식으로 가장 널리 사용됨 (x, y, z, w)
- 특이점 없음 (No singularities)

2) 회전 매트릭스 (rotation matrix)

- 특이점 없음 (No singularities)

3) 고정축 roll, pitch, yaw (fixed axis roll, pitch, yaw about X, Y, Z axes respectively)

- 각속도에 사용

4) 오일러 각도 yaw, pitch, roll (euler angles yaw, pitch, and roll about Z, Y, X axes respectively)

- 전역 좌표계에서 회전이 발생하기 때문에 한 축의 회전이 다른 축의 회전과 겹치는 문제(일명 짐벌락)로 인해 사용을 권장하지 않는다.

ROS 2의 QoS (Quality of Service)

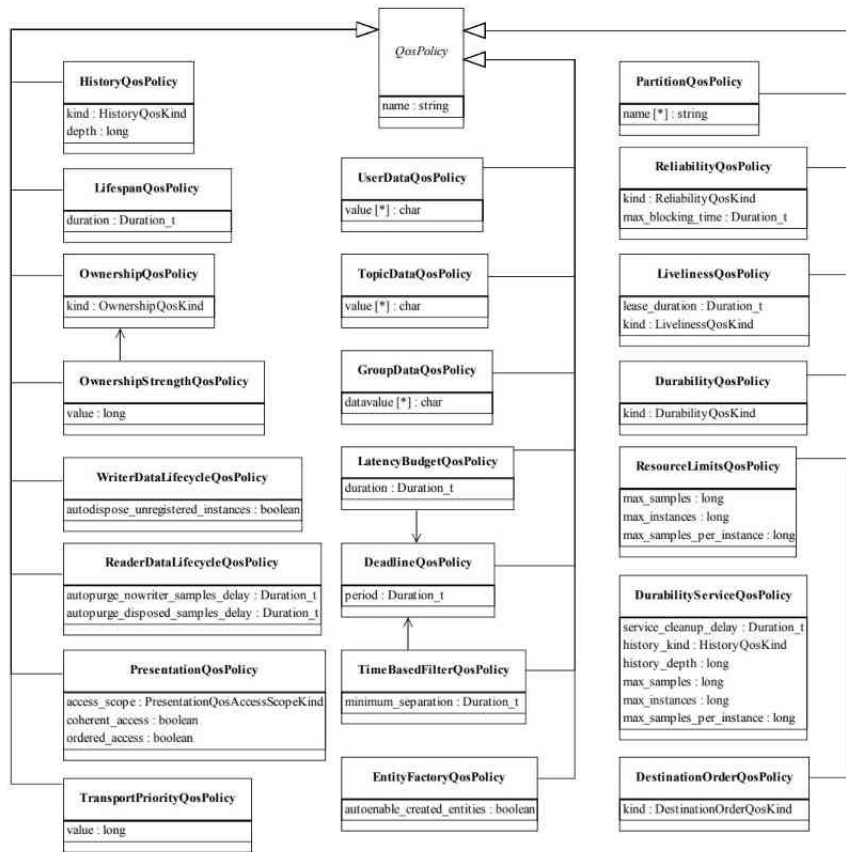
205 ROS 2의 좌표 표현 통일의 필요성

본격적으로 **DDS의 서비스 품질 (QoS, Quality of Service)**에 대해 알아보자. QoS라는 용어가 조금 낯설게 느껴져 어려울 수 있을텐데 쉽게 말해서 "**데이터 통신의 옵션**"이라고 생각하면 된다. TCPROS라는 자체 프로토콜을 사용했던 기존 ROS 1과는 달리 ROS 2에서는 TCP처럼 신뢰성을 중시여기는 통신 방식과 UDP처럼 통신 속도에 포커스를 맞춘 통신 방식을 선택적으로 사용할 수 있다. 이를 위해 ROS 2에서는 이 DDS의 QoS를 도입하였고 퍼블리셔나 서브스크라이브 등을 선언할 때 QoS를 매개 변수형태로 지정할 수 있어서 원하는 데이터 통신의 옵션 설정을 유저가 직접할 수 있게 되어 있다. 이 QoS로 바꿀 수 있는 것은 데이터 전송시의 실시간성 설정과 관련된 부분과 대역폭에 영향을 미치는 옵션, 데이터의 지속성과 관련한 옵션, 중복성에 관련한 옵션 등이 있다. 이 옵션을 어떤 상황에서 어떻게 설정하냐가 중요한데 이는 이어지는 설명에서 하나씩 알아가도록 하겠다.

206 QoS의 종류

현재의 DDS 사양서에서 설정 가능한 QoS 항목으로는 오른쪽 그림과 같이 22가지인데 ROS 2에서는 TCP처럼 데이터 손실을 방지함으로써 신뢰도를 우선시하거나 (reliable), UDP처럼 통신 속도를 최우선시하여 사용 (best effort)할 수 있게 하는 **Reliability** 기능이 대표적으로 사용되고 있다. 그 이외에 통신 상태에 따라 정해진 사이즈만큼의 데이터를 보관하는 **History** 기능, 데이터를 수신하는 서브스크라이버가 생성되기 전의 데이터를 사용할지 폐기할지에 대한 설정인 **Durability** 기능, 정해진 주기 안에 데이터가 발신 및 수신되지 않을 경우 이벤트 함수를 실행시키는 **Deadline** 기능, 정해진 주기 안에서 수신되는 데이터만 유효 판정하고 그렇지 않은 데이터는 삭제하는 **Lifespan** 기능, 정해진 주기 안에서 노드 혹은 토픽의 생사 확인하는 **Liveliness**도 옵션으로 설정할 수 있다.

- <https://www.omg.org/spec/DDS>



(1) Values

History	데이터를 몇 개나 보관할지를 결정하는 QoS 옵션
KEEP_LAST	정해진 메시지 큐 사이즈 만큼의 데이터를 보관 * depth : 메시지 큐의 사이즈 (KEEP_LAST 설정일 경우에만 유효)
KEEP_ALL	모든 데이터를 보관 (메시지 큐의 사이즈는 DDS 벤더마다 다름)

(2) RxO (requested by offered)

: 해당 사항 없음

(3) Examples

```
[RCLCPP]
rclcpp::QoS(rclcpp::KeepLast(10)).reliable().durability_volatile();
```

```
[RCLPY]
qos_profile = QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=10)
```

208 Reliability

(1) Values

Reliability	데이터 전송에 있어 속도를 우선시 하는지 신뢰성을 우선시 하는지를 결정하는 QoS 옵션
BEST_EFFORT	데이터 송신에 집중. 전송 속도를 중시하며 네트워크 상태에 따라 유실이 발생할 수 있음
RELIABLE	데이터 수신에 집중. 신뢰성을 중시하며 유실이 발생하면 재전송을 통해 수신을 보장함

(2) RxO (requested by offered)

Pub(↓) / Sub (→)	BEST_EFFORT	RELIABLE
BEST_EFFORT	BEST_EFFORT	불가
RELIABLE	BEST_EFFORT	RELIABLE

(3) Examples

[RCLCPP]

```
rclcpp::QoS(rclcpp::KeepAll).best_effort().transient_local();
```

[RCLPY]

```
qos_profile = QoSProfile(reliability=QoSReliabilityPolicy.BEST_EFFORT)
```


209 Durability

(1) Values

Durability	데이터를 수신하는 서브스크라이버가 생성되기 전의 데이터를 사용할지 폐기할지에 대한 QoS 옵션
TRANSIENT_LOCAL	Subscription이 생성되기 전의 데이터도 보관 (Publisher에만 적용 가능)
VOLATILE	Subscription이 생성되기 전의 데이터는 무효

(2) RxO (requested by offered)

Pub(↓) / Sub (→)	TRANSIENT_LOCAL	VOLATILE
TRANSIENT_LOCAL	TRANSIENT_LOCAL	VOLATILE
VOLATILE	불가	VOLATILE

(3) Examples

```
[RCLCPP]
rclcpp::QoS(rclcpp::KeepAll).best_effort().transient_local()
```

```
[RCLPY]
qos_profile = QoSProfile(durability=QoSDurabilityPolicy.TRANSIENT_LOCAL)
```

210

Deadline

(1) Values

Deadline	정해진 주기 안에 데이터가 발신 및 수신되지 않을 경우 EventCallback를 실행시키는 QoS 옵션
deadline_duration	Deadline을 확인하는 주기

(2) RxO (requested by offered)

Pub(↓) / Sub (→)	1000ms	2000ms
1000ms	가능	가능
2000ms	불가	가능

(3) Examples

```
[RCLCPP]
rclcpp::QoS(10).deadline(100ms);
```

```
[RCLPY]
qos_profile = QoSProfile(depth=10, deadline=Duration(0.1))
```

(1) Values

Lifespan	정해진 주기 안에서 수신되는 데이터만 유효 판정하고 그렇지 않은 데이터는 삭제하는 QoS 옵션
lifespan_duration	Lifespan을 확인하는 주기

(2) RxO (requested by offered)

: 해당 사항 없음

(3) Examples

[RCLCPP]

```
rclcpp::QoS(10).reliable().transient_local().lifespan(10ms);
```

[RCLPY]

```
qos_profile = QoSProfile(lifespan=Duration(0.01))
```

212 Liveliness

(1) Values

Liveliness	정해진 주기 안에서 노드 혹은 토픽의 생사 확인하는 QoS 옵션
liveliness	자동 또는 매뉴얼로 확인할지를 지정하는 옵션, 하기 3가지 중 선택 * AUTOMATIC, MANUAL_BY_NODE, MANUAL_BY_TOPIC 중 선택
lease_duration	Liveliness을 확인하는 주기

(2) RxO (requested by offered)

Pub(↓) / Sub (→)	AUTOMATIC	MANUAL_BY_NODE	MANUAL_BY_TOPIC
AUTOMATIC	가능	불가	불가
MANUAL_BY_NODE	가능	가능	불가
MANUAL_BY_TOPIC	가능	가능	가능

(3) Examples

```
[RCLCPP]
rclcpp::QoS qos_profile(10);
qos_profile.liveliness(RMW_QOS_POLICY_LIVELINESS_AUTOMATIC).liveliness_lease_duration(1000ms);
```

```
[RCLPY]
qos_profile = QoSProfile(liveliness=AUTOMATIC, liveliness_lease_duration=Duration(1.0))
```

213 RMW QoS Profile

ROS 2의 RMW에서 QoS 설정을 쉽게 사용할 수 있도록 가장 많이 사용하는 QoS 설정을 세트로 표현해둔 것이 있는데 이를 RMW QoS Profile이라고 한다. 그 목적에 따라 Default, Sensor Data, Service, Action Status, Parameters, Parameter Events와 같이 6가지로 구분하며 Reliability, History, Depth(History Depth), Durability 를 설정하게 된다.

	Default	Sensor Data	Service	Action Status	Parameters	Parameter Events
Reliability	RELIABLE	BEST_EFFORT	RELIABLE	RELIABLE	RELIABLE	RELIABLE
History	KEEP_LAST	KEEP_LAST	KEEP_LAST	KEEP_LAST	KEEP_LAST	KEEP_LAST
Depth (History Depth)	10	5	10	1	1,000	1,000
Durability	VOLATILE	VOLATILE	VOLATILE	TRANSIENT LOCAL	VOLATILE	VOLATILE

214 User QoS Profile

이외에도 유저가 직접 설정하여 새로운 프로파일을 만들어 사용할 수 있는데 이를 위해서는 아래와 같이 QoS 모듈들을 임포트하고, 그 뒤 실제 코드에서 `QoSProfile`을 선언하며 **reliability, history, durability, depth**를 아래와 같이 원하는 옵션으로 커스텀하게 설정할 수 있다. 그 뒤 아래와 같이 퍼블리셔를 선언하는 `create\_publisher` 함수를 사용할 때 `rmw\_qos\_profile` 대신에 사용자가 정의한 유저 QoS 프로파일 매개변수로 사용하면 된다.

개인적으로는 후자로 소개한 유저 QoS 프로파일 사용 방법이 자유롭게 원하는 형태로 사용할 수 있어서 실제 개발할 때 더 많이 사용하는 편이다.

```
from rclpy.qos import QoSDurabilityPolicy
from rclpy.qos import QoSHistoryPolicy
from rclpy.qos import QoSProfile
from rclpy.qos import QoSReliabilityPolicy
```

```
QOS_RKL10V = QoSProfile(
    reliability=QoSReliabilityPolicy.RELIABLE,
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=10,
    durability=QoSDurabilityPolicy.VOLATILE)
```

```
self.sensor_publisher = self.create_publisher(Int8MultiArray, 'sensor', QOS_RKL10V)
```

ROS 2의 파일 시스템

216 패키지 와 메타패키지

ROS 2의 파일시스템을 설명하기 위하여 기본적인 폴더 및 파일 구성과 설치 폴더, 사용자 패키지에 대해 알아보도록 하자. ROS 2에서 소프트웨어 구성을 위한 기본 단위가 패키지 (package)로써 ROS의 응용프로그램은 패키지 단위로 개발되고 관리된다. 패키지는 ROS의 최소 단위의 실행 프로세서인 노드 (node)를 하나 이상 포함하거나 다른 노드를 실행하기 위한 런치 (launch)와 같은 실행 및 설정 파일들을 포함하게 된다.

2024년 8월 기준으로 REP-2005으로 등록된 ROS 2 Common Packages문서를 기준으로 보았을 때 480여개가 기본 패키지이며 ROS Index에 기술된 정보에 의하면 총 1882개의 패키지가 존재한다. `sudo apt install ros-humble-\*` 명령어로 설치 가능한 패키지를 기준으로 했을 때에는 2,600 여개의 패키지가 있다.

이러한 패키지는 메타패키지 (metapackage)라는 공통된 목적을 지닌 패키지들을 모아둔 패키지들의 집합 단위로 관리되기도 한다. 예를 들어 Navigation2 메타패키지는 nav2_amcl, nav2_bt_navigator, nav2_costmap_2d, nav2_core, nav2_dwb_controller 등 20여 개의 패키지로 구성되어 있다. 각 패키지는 package.xml이라는 파일을 포함하고 있는데 이는 패키지의 정보를 담은 XML 파일로서 패키지의 이름, 저작자, 라이선스, 의존성 패키지 등을 기술하고 있다. 또한 ROS 2의 빌드 시스템인 `ament`는 기본적으로 CMake를 이용하고 있어서 패키지 폴더에 CMakeLists.txt라는 파일에 빌드 설정을 기술하고 있다. 이외에 노드의 소스 코드 및 노드 간의 메시지 통신을 위한 msg, srv, action 인터페이스 파일 등으로 구성되어 있다.

217 바이너리 설치와 소스 코드 설치

ROS 패키지 설치는 바이너리 형태로 제공되어 별도의 빌드과정 없이 바로 실행하는 방법과 해당 패키지의 소스 코드를 직접 다운로드 한 후 사용자가 빌드하여 사용하는 방법이 있다. 이는 패키지 사용 목적에 따라 달리 사용된다. 만약, 해당 패키지를 수정하여 사용하고자 할 경우나 소스 코드 내용을 확인할 필요가 있다면 후자의 설치 방법을 이용하면 된다. 다음은 `teleop_twist_joy` 패키지를 예로 들어 두 설치 방법의 차이를 기술한다.

바이너리 설치

```
$ sudo apt install ros-humble-teleop-twist-joy
```

소스 코드 설치

```
$ cd ~/robot_ws/src  
$ git clone https://github.com/ros2/teleop_twist_joy.git  
$ cd ~/robot_ws/  
$ colcon build --symlink-install --packages-select teleop_twist_joy
```

218 기본 설치 폴더

ROS 2는 '/opt/ros/[버전이름]' 폴더에 설치된다. 예를 들어 ROS 2 Humble Hawksbill 버전을 설치했다면 ROS가 설치된 경로는 다음과 같다.

- 기본 설치 폴더 경로
 - /opt/ros/humble
- 세부 내용
 - /bin 실행 가능한 바이너리 파일
 - /cmake 빌드 설정 파일
 - /include 헤더 파일
 - /lib 라이브러리 파일
 - /opt 기타 의존 패키지
 - /share 패키지의 빌드, 환경 설정 파일
 - local_setup.* 환경 설정 파일
 - setup.* 환경 설정 파일

219 사용자 작업 폴더

사용자 작업 폴더는 사용자가 원하는 곳에 생성할 수 있으나 이 강좌에서는 원활한 진행을 위하여 리눅스 사용자 폴더인 '~/robot_ws/'를 사용하도록 하자. 즉 '/home/사용자이름/robot_ws'를 사용할 것이다. 예를 들어, 사용자 이름이 'oroce'이고 workspace 폴더 이름은 'robot_ws'라고 설정하였다면 경로는 다음과 같다.

- 사용자 작업 폴더 경로
 - /home/oroce/robot_ws/
- 세부 내용
 - /build 빌드 설정 파일용 폴더
 - /install msg, srv 헤더 파일과 사용자 패키지 라이브러리, 실행 파일용 폴더
 - /log 빌드 로깅 파일용 폴더
 - /src 사용자 패키지용 폴더

- /src C/C++ 코드용 폴더
 - /include C/C++ 헤더파일 용 폴더 (각 패키지 이름별 폴더로 패키지별 헤더를 구분함)
 - /param 파라미터 파일용 폴더
 - /launch roslaunch에 사용되는 launch 파일용 폴더
 - /패키지_이름_폴더 Python 코드용 폴더
 - /test 테스트 코드 및 테스트 데이터용 폴더
 - /msg 메시지 파일용 폴더
 - /srv 서비스 파일용 폴더
 - /action 액션 파일용 폴더
 - /doc 문서용 폴더
-
- package.xml 패키지 설정 파일 ([REP-0140](#), [REP-0149](#) 참고)
 - CMakeLists.txt C/C++ 빌드 설정 파일
 - setup.py 파이썬 코드 환경 설정 파일
 - README 사용자 문서, [github](#) 리포지토리의 메인에 표시된다.
 - CONTRIBUTING 해당 패키지 개발에 공헌하는 방법을 기술하는 파일
 - LICENSE 이 패키지의 라이선스를 기술하는 파일
 - CHANGELOG.rst 이 패키지의 버전별 변경 사항 모음 파일 ([REP-0132](#) 참고)

ROS 2의 빌드 시스템과 빌드 툴

- ROS 1 (ROS Fuerte 까 지): rosbuilt (CMake)
- ROS 1 (ROS Groovy 이 후): catkin (CMake)
- ROS 2
 - **ament**
 - **ament_cmake** (CMake)
 - **ament_python**: Python setuptools (Full support)

- **catkin_make** : catkin_make는 ROS 1 빌드 시스템을 포함하는 ROS 패키지 catkin에서 제공하던 기본 툴로 ROS Fuerte 버전 이후 rosbuilt의 대체 툴로서 오랜 기간 사용되어온 ROS 1의 대표 빌드 툴이다.
- **catkin_make_isolated**: catkin_make과 마찬가지로 ROS 1 빌드 시스템을 포함하는 ROS 패키지 catkin에서 제공하던 기본 툴로 하나의 CMake으로 복수의 패키지를 빌드 할 수 있었으며 격리 빌드를 지원함으로써 모든 패키지를 별도로 빌드하게 되었다. 이 기능 변화를 통해 설치용 폴더를 분리하거나 병합할 수 있게 되었다.
- **catkin_tools**: catkin_make, catkin_make_isolated의 독립 사용에 불편함을 해결하고 Python으로 구성된 패키지도 관리할 수 있게 해주는 툴로 catkin_make의 부족한 부분을 제공하였다.
- **ament_tools**: ROS 2의 ament_cmake 및 ament_python, 순수 CMake 패키지를 모두 지원하는 툴로 catkin_make, catkin_make_isolated, catkin_tools 모두의 기능을 사용할 수 있으며 ROS 2 Bouncy 버전 이전까지 사용되었다.
- **colcon**: ROS 1과 ROS 2 모두를 지원하기 위하여 통합된 빌드 툴로서 소개되었으며 ROS 2 Bouncy 이후 ROS 2의 기본 빌드 툴로 사용 중에 있다.

224 패키지 생성

```
$ cd ~/robot_ws/src
```

```
$ ros2 pkg create my_first_ros_rclcpp_pkg --build-type ament_cmake --dependencies rclcpp std_msgs
```

```
$ ros2 pkg create my_first_ros_rclpy_pkg --build-type ament_python --dependencies rclpy std_msgs
```

(my_first_ros_rclcpp_pkg)

```
.
├── include
│   └── my_first_ros_rclcpp_pkg
├── src
├── CMakeLists.txt
└── package.xml
```

3 directories, 2 files

(my_first_ros_rclpy_pkg)

```
.
├── my_first_ros_rclpy_pkg
│   └── __init__.py
├── resource
│   └── my_first_ros_rclpy_pkg
├── test
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
├── package.xml
├── setup.cfg
└── setup.py
```

3 directories, 8 files

225 빌드

1) 전체 패키지를 빌드할 때

```
$ cd ~/robot_ws && colcon build --symlink-install
```

2) 해당 패키지만 빌드할 때

```
$ cd ~/robot_ws && colcon build --symlink-install --packages-select [패키지 이름]
```

226 빌드 시스템에 필요한 부가 기능

1) vcstool (버전 컨트롤 시스템 툴)

```
$ wget https://raw.githubusercontent.com/ros2/ros2/humble/ros2.repos
$ vcs import src < ros2.repos
```

2) rosdep (의존성 관리 툴)

```
$ sudo rosdep init
$ rosdep update
$ rosdep install --from-paths src -y --ignore-src
```

3) bloom (바이너리 패키지 관리 툴)

```
$ bloom-release --ros-distro humble --track humble awesome_pkg
```

패키지 파일 (환경 설정, 빌드 설정)

228 패키지 파일 (환경 설정, 빌드 설정)

이번 강좌에서 설명할 패키지 파일 사용 방법은 **ROS 2 프로그래밍 전에 알아둬야할 필수 사전 정보**라고 할 수 있겠다.

- 패키지 설정 파일 package.xml
- 빌드 설정 파일 CMakeLists.txt
- 파이썬 패키지 설정 파일 setup.py
- 파이썬 패키지 환경 설정 파일 setup.cfg
- RQt 플러그인 설정 파일 plugin.xml
- 패키지 변경로그 파일 CHANGELOG.rst
- 라이선스 파일 LICENSE
- 패키지 설명 파일 README.md

229 패키지 설정 파일 (package.xml)

패키지 설정 파일은 ROS 패키지의 필수 구성 요소로서 패키지의 정보를 기술하는 파일이다. 기술하는 내용으로는 패키지 이름, 저작자, 라이선스, 의존성 패키지 등이 있으며 XML 형식으로 기술하고 파일명은 'package.xml'을 사용한다. 사용되는 빌드 툴, 의존성 패키지들이 모두 기술되기에 빌드 및 패키지 설치, 사용에 있어서 매우 중요한 파일이라고 말할 수 있다. 이에 모든 ROS 패키지의 필수 파일로 각 패키지당 무조건 1개의 패키지 설정 파일 (package.xml)을 포함하고 있다.

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_first_ros_rclcpp_pkg</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="pyo@robotis.com">pyo</maintainer>
  <license>TODO: License declaration</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>rclcpp</depend>
  <depend>std_msgs</depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

(my_first_ros_rclcpp_pkg)

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_first_ros_rclpy_pkg</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="pyo@robotis.com">pyo</maintainer>
  <license>TODO: License declaration</license>

  <depend>rclpy</depend>
  <depend>std_msgs</depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

(my_first_ros_rclpy_pkg)

패키지 설정 파일 (package.xml)

<?xml>	문서 문법을 정의하는 문구로 아래의 내용은 xml 버전 1.0을 따르고 있다는 것을 알린다.
<package>	이 구문부터 맨 끝의 </package>까지가 ROS 패키지 설정 부분이다. 세부 사항으로 format="3" 이라고 패키지 설정 파일의 버전을 기재한다. ROS 2는 3를 사용하면 된다.
<name>	패키지의 이름이다. 패키지를 생성할 때 입력한 패키지 이름이 사용된다. 다른 옵션도 마찬가지로이지만 이는 사용자가 원할 때 언제든지 변경할 수 있다.
<version>	패키지의 버전이다. 자유롭게 지정할 수 있는데 나중에 패키지를 바이너리 패키지로 공개한다면 버전 관리에 사용되므로 신중할 필요가 있다.
<description>	패키지의 간단한 설명이다. 보통 2~3 문장으로 기술한다.
<maintainer>	패키지 관리자의 이름과 이메일 주소를 기재한다.
<license>	라이선스를 기재한다. Apache 2.0, BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3, Proprietary 등을 기재하면 된다.
<url>	패키지를 설명하는 웹 페이지 또는 버그 관리, 소스 코드 저장소 등의 주소를 기재한다. 이 종류에 따라 type에 website, bugtracker, repository를 대입하면 된다.
<author>	패키지 개발에 참여한 개발자의 이름과 이메일 주소를 적는다. 복수의 개발자가 참여한 경우에는 바로 다음 줄에 <author> 태그를 이용하여 추가로 넣어주면 된다.
<buildtool_depend>	빌드 툴의 의존성을 기술한다.
<build_depend>	패키지를 빌드할 때 필요한 의존 패키지 이름을 적는다.
<exec_depend>	패키지를 실행할 때 필요한 의존 패키지 이름을 적는다.
<test_depend>	패키지를 테스트할 때 필요한 의존 패키지 이름을 적는다.
<export>	위에서 명시하지 않은 확장 태그명을 사용할 때 쓰인다. 빌드 타입을 적는 <build_type>, RViz 플러그인에 사용되는 <rviz>, RQt 플러그인에 사용되는 <rqt_gui>, deprecated되는 패키지일 경우 유저에게 알릴 수 있는 <deprecated> 태그 등이 있다.

231 빌드 설정 파일 (CMakeLists.txt)

ROS 2의 빌드 시스템인 ament에서는 **C++ 프로그래밍 언어**를 사용한 패키지나 RQt Plugin의 경우 CMake(Cross Platform Make)를 이용하고 있고 패키지 폴더의 **`CMakeLists.txt`**라는 파일에 **빌드 환경을 기술**하여 사용하고 있다.

이 빌드 설정 파일에 실행 파일 생성, 의존성 패키지 우선 빌드, 링크 생성 등을 설정하게 되어 있다. ROS에서 CMake를 이용하는 이유는 ROS 패키지를 멀티 플랫폼에서 빌드할 수 있게 하기 위함이다. Make가 유닉스 계열만 지원하는 것과 달리, CMake는 유닉스 계열인 리눅스, BSD, OS X뿐만 아니라 윈도우즈 계열도 지원하기 때문이다.

또한 CMakeLists.txt은 Visual Studio, Eclipse, Qt Creator 등 다양한 IDE에서 기본으로 지원하여 쉽게 사용할 수 있다.

빌드 설정 파일 (CMakeLists.txt)

```
cmake_minimum_required(VERSION 3.5)
project(my_first_ros_rclcpp_pkg)

# Default to C99
if(NOT CMAKE_C_STANDARD)
  set(CMAKE_C_STANDARD 99)
endif()

# Default to C++14
if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# find dependencies
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  # the following line skips the linter which checks for copyrights
  # uncomment the line when a copyright and license is not present in all source files
  #set(ament_cmake_copyright_FOUND TRUE)
  # the following line skips cpplint (only works in a git repo)
  # uncomment the line when this package is not in a git repo
  #set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```


파이썬 패키지 설정 파일 (setup.py)

name	패키지의 이름
version	패키지의 버전
packages	의존하는 패키지, 하나씩 나열해도 되지만 <code>find_packages()</code> 를 기입해주면 자동으로 의존하는 패키지를 찾아준다.
data_files	이 패키지에서 사용되는 파일들을 기입하여 함께 배포한다. `ROS`에서는 주로 <code>resource</code> 폴더 내에 있는 <code>ament_index</code> 를 위한 패키지의 이름의 빈 파일이나 <code>package.xml</code> , <code>*.launch.py</code> , <code>*.yaml</code> 등을 기입한다.
install_requires	의존하는 패키지, 이 패키지를 <code>pip</code> 을 통해 설치할 때 이곳에 기술된 패키지들을 함께 설치하게 된다. `ROS`에서는 <code>pip</code> 로 설치하지 않기에 <code>setuptools</code> , <code>launch</code> 만을 기입해준다.
tests_require	테스트에 필요한 패키지, `ROS`에서는 <code>pytest</code> 를 사용한다.
zip_safe	설치시 zip 파일로 아카이브할지 여부를 설정한다.
author author_email maintainer maintainer_email	저작자, 관리자의 이름과 이메일을 기입한다.
keywords	이 패키지의 키워드, Python Package Index (PyPI) 배포시 검색하여 이 패키지를 찾을 수 있도록 한다.
classifiers	PyPI에 등록될 메타 데이터 설정으로 <code>PyPI</code> 페이지의 좌측 <code>Meta</code> 란에서 확인 가능하다.
description	패키지 설명을 기입한다.
license	라이선스 종류를 기입한다.
entry_points	플랫폼 별로 콘솔 스크립트를 설치하도록 콘솔 스크립트 이름과 호출 함수를 기입한다.

파이썬 패키지 설정 파일 (setup.py)

```
from setuptools import setup

package_name = 'my_first_ros_rclpy_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='pyo',
    maintainer_email='pyo@robotis.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        ],
    },
)
```

235 파이썬 패키지 환경 설정 파일 (setup.cfg)

이 파일은 순수한 `ROS 2 Python` 패키지에서만 사용하는 **배포를 위한 구성 파일**이다. 파일명으로는 `setup.cfg`를 사용한다. `setup.py`파일의 `setup` 함수에서 설정하지 못하는 기타 옵션을 이 구성 파일 (setup.cfg)을 사용하여 정의할 수 있다. `ROS 2 Python` 패키지에서는 이 파일에 [develop]와 [install] 옵션을 설정하여 스크립트의 저장 위치를 설정한다. 이 파일의 설정 값으로는 ROS 파일 시스템을 고려하여 설정하게 되는데 그 내용은 매우 간단하여 다음과 같이 해당 패키지의 이름만 변경하여 사용하면 된다.

```
[develop]
script-dir=$base/lib/my_first_ros_rclpy_pkg
[install]
install-scripts=$base/lib/my_first_ros_rclpy_pkg
```

236 RQt 플러그인 설정 파일 (plugin.xml)

RQT 플러그인으로 패키지를 작성할 때의 필수 구성 요소로 XML 태그로 각 속성을 기술하는 파일이다. 파일명으로는 `plugin.xml`을 사용하며 파일이름에서 알 수 있듯이 XML 태그를 사용한다. 기본적인 내용은 아래와 같으며 각 태그의 내용만 조금씩 변경하여 사용하면 된다. 자세한 XML 태그 속성에 대해서는 [Attributes of library element in plugin](#) 부분을 참고하자.

```
<library path="src">
  <class name="Examples" type="examples_rqt.examples.Examples"
base_class_type="rqt_gui_py::Plugin">
    <description>
      A plugin visualizing messages and services values
    </description>
    <qtgui>
      <group>
        <label>Visualization</label>
        <icon type="theme">folder</icon>
        <statustip>Plugins related to visualization</statustip>
      </group>
      <label>Viewer</label>
      <icon type="theme">utilities-system-monitor</icon>
      <statustip>A plugin visualizing messages and services values</statustip>
    </qtgui>
  </class>
</library>
```

‘CHANGELOG.rst’ 파일은 **패키지의 업데이트 내역을 기술하는 파일**이다. 이 파일은 `reStructuredText(rst)` 파일로서 다양한 기능을 제공하지만 `ROS`에서는 아래와 같이 간단한 문법만을 사용하고 있다. 상단의 `example_rqt_package` 부분에 패키지명을 적고, `0.0.1 (2024-08-01)`처럼 버전과 업데이트된 날짜 밑에 구분자(`---`)를 넣고 변경된 코드 내용 및 작업자를 기술하면 된다. 이 변경로그파일은 패키지에 필수적으로 포함시켜야하는 파일은 아니지만 적절히 사용한다면 개발 이력을 추적할때에도 도움이 될 것이다. 그리고 개발된 패키지를 바이너리 패키지로 공개하는 절차를 수행한다면 이 파일은 필수로 포함시켜야하며 이를 통해 사용자들에게 변경 사항을 버전별로 공지할 수 있게 된다. 자세한 내용은 [REP-0132](#) 문서를 참고하자.

```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
Changelog for package example_rqt_package
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

0.0.2 (2024-08-28)
-----
* Added new indicators to view message
* Contributors: Pyo

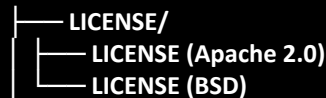
0.0.1 (2024-08-01)
-----
* Added example_rqt_package as rqt plugin for visualizing messages and
services
* Contributors: Pyo

```

238 라이선스 파일 (LICENSE)

‘LICENSE’ 파일은 패키지의 코드에 사용된 **라이선스를 기술하는 파일**이다. 단일 라이선스의 경우에는 파일명은 ‘LICENSE’으로 확장자를 붙이지 않는다. 복수 라이선스의 경우 하기와 같이 ‘LICENSE’ 폴더를 생성한 후 서로 다른 파일명으로 구분한다.

‘ROS 2’의 기본 라이선스인 ‘Apache 2.0’의 예제는 ROS 2 RCL(ROS Client Libraries) 패키지의 [LICENSE 파일](#)을 참고하자. 그리고 오픈 소스 라이선스에는 ‘Apache 2.0’, ‘BSD’, ‘MIT’, ‘Boost Software License’, ‘GPLv2’, ‘GPLv3’, ‘LGPLv2.1’, ‘LGPLv3’ 등이 있다. 더 자세한 내용은 Open Source Initiative 및 Choose a License, OLIS 내용을 참고하자. 참고로 오픈소스 이외에 ‘Proprietary License’는 ‘사유 소프트웨어’를 의미하며 ‘독점 소프트웨어’ 또는 ‘클로즈드 소스 소프트웨어 (Closed Source Software)’, ‘비자유 소프트웨어 (Non-free software)’라고도 부른다. ‘Proprietary License’의 경우 ‘코드’ 및 ‘package.xml’에만 명시하고 별도의 ‘LICENSE’ 파일은 추가하지 않아도 된다.



```
├── LICENSE/
│   ├── LICENSE (Apache 2.0)
│   └── LICENSE (BSD)
```

239 패키지 설명 파일 (README.md)

`README.md`은 패키지의 부가 설명을 기술하는 파일로 ROS 패키지의 필수 포함 파일은 아니지만 패키지를 사용하는 사용자를 배려하는 문서라고 생각하면 된다. 파일명으로는 `README.md`을 사용한다. 이 파일은 `Markdown(md)` 파일로서 **Markdown** 문법을 따른다.

자세한 문법은 하기 Markdown을 참고하자. 예제로는 터틀봇3의 README 파일을 참고하자. 주로 개발 환경, 의존성 패키지, 설치 방법, 사용 방법들을 기재하고 더 자세한 내용을 다룬 매뉴얼이 있으면 이를 링크로 달아주는게 일반적이다. 패키지 설명 파일에 대한 특별한 가이드는 없고 사용자로 하여금 쉽게 관련자료를 찾아 사용할 수 있도록 기술하면 된다.

- [Basic writing and formatting syntax for Markdown](#)
- <https://github.com/ROBOTIS-GIT/turtlebot3/blob/master/README.md>

240 실전이 중요!

오늘 강좌에서는 ROS 2의 패키지 설정 파일 `package.xml`, 빌드 설정 파일 `CMakeLists.txt`, 파이썬 패키지 설정 파일 `setup.py`, 파이썬 패키지 환경 설정 파일 `setup.cfg`, RQt 플러그인 설정 파일 `plugin.xml`, 패키지 변경로그 파일 `CHANGELOG.rst`, 라이선스 파일 `LICENSE`, 패키지 설명 파일 `README.md`에 대해 자세히 알아보았다.

이 강좌를 통해 ROS 패키지에서 필요한 기본 파일들을 살펴보았는데 각 파일의 기술되는 내용은 케바케 (case by case)의 경우가 많아 모든 내용을 다 설명하기는 쉽지 않다. 그래서 **각 패키지 파일들의 실전 연습이 중요한데 이는 이어지는 실습 강좌에서 하나씩 케바케를 익혀보도록 하겠다.**

ROS 프로그래밍 규칙 (코드 스타일)

242 코드 스타일 가이드

오픈소스 코드는 해당 커뮤니티의 공동의 결과물로 협업이 기반이 된다. 협업 프로그래밍 작업 시에는 일관된 규칙을 만들고 이를 준수하며 프로그래밍하고 혹시 모를 실수에 대비하여 자동화 툴로 자가 검토를 실시하고 공개 후에도 여러 동료들로부터 코드 리뷰도 받게 된다. 이러한 **코드 스타일 가이드 준수**는 처음에는 귀찮고 버거울 수 있으나 소스 코드 작업 시 빈번히 생기는 개발자의 부가적인 선택을 줄여주고, 다른 협업 개발자 및 이용자의 코드 이해도를 높이며 상호 간의 코드 리뷰를 쉽게 할 수 있으며, 프로그래밍 언어의 특정 기능으로 인해 생길 수 있는 오류와 다양한 이슈를 피할 수 있게 해준다.

로봇운영체제 ROS도 ROS 커뮤니티의 공동의 결과물로 협업이 기반이 된다. ROS 커뮤니티도 이를 위해 **ROS 2 Developer Guide**와 **ROS Enhancement Proposals (REPs)**과 같은 가이드와 규칙도 만들고 **ROS 2 Code style**과 같이 일관된 코드 스타일을 지키기 위하여 사용되는 각 언어에 대해 스타일 가이드라인을 세우고 구성원들의 합의하에 이를 따르고 있다. 그렇다고 독단적인 패키지 레이아웃이나 문서 레이아웃이 아닌 오픈소스 커뮤니티 등에서 가장 많이 사용되고 있는 인기 있는 스타일을 바탕으로 자체 가이드라인을 만들어 사용하고 있다. 또한 일관된 코드 스타일을 관리하기 위하여 개발 툴을 제공하고 있으며 개발자가 커뮤니티의 프로그래밍 지침을 준수하는지 확인할 수 있게 하고 있다.

이 강좌에서는 이 중에서 코드 스타일 중심으로 설명하도록 하겠다. 코드 스타일 가이드는 ROS가 지원하는 프로그래밍 언어만큼 다양한데 오늘은 그중에서 기본적인 네이밍 규칙과 가장 많이 사용되는 C++과 Python 코드 스타일을 알아보도록 하자.

243 기본 이름 규칙

하기와 같이 3종류의 네이밍을 기본으로 하며 파일 이름은 모두 소문자로 **`snake\_case`** 이름 규칙을 사용한다. 가독성을 해치는 축약어는 가능한 사용하지 않으며 확장자명은 모두 소문자로 표기한다.

단, **`ROS`** 인터페이스 류의 파일은 **/msg** 및 **/srv** 또는 **/action** 에 폴더에 위치시키며 인터페이스 파일명은 **`CamelCased`** 규칙을 따른다. 그 이유는 *.msg 및 *.srv 또는 *.action는 *.h(pp) 변환 후 인터페이스 타입으로 구조체 및 타입으로 사용되기 때문이다.

- **CamelCased**
- **snake_case**
- **ALL_CAPITALS**

그 이외에 특정 목적에 의해 만들어지는 하기 파일 이름은 예외적으로 대소문자 규칙을 따르지 않고 하기과 같이 고유의 이름을 사용한다.

- package.xml
- CMakeLists.txt
- README.md
- LICENSE
- CHANGELOG.rst
- .gitignore
- .travis.yml
- *.repos

244 C++ Style

ROS 2 Developer Guide 및 **ROS 2 Code style**에서 다루고 있는 C++ 코드 스타일은 오픈소스 커뮤니티에서 가장 널리 사용 중인 **Google C++ Style Guide**를 사용하고 있으며 ROS의 특성에 따라 일부를 수정하여 사용하고 있다. 하기에 기재되지 않는 부분은 **REPs** 및 Google C++ Style Guide 문서를 참고하도록 하자.

(1) 기본 규칙

- [C++14 Standard](#)를 준수한다.

(2) 라인 길이

- 최대 100 문자

(3) 이름 규칙 (Naming)

- CamelCased : 타입, 클래스, 구조체, 열거형
- snake_case : 파일, 패키지, 인터페이스, 네임스페이스, 변수, 함수, 메소드
- ALL_CAPITALS : 상수, 매크로

- 소스 파일은 `.cpp` 확장자를 사용한다.
- 헤더 파일은 `.hpp` 확장자를 사용한다.
- 전역변수(global variable)는 사용이 피치 못한 경우에는 `g_` 접두어를 붙인다.
- 클래스 멤버 변수(class member variable)는 마지막에 밑줄(`_`)을 붙인다.

245 C++ Style

(4) 공백 문자 대 탭 (Spaces vs. Tabs)

- 기본 들여쓰기(indent)는 공백 문자(space) `2개`를 사용한다. (탭(tab)문자 사용 금지)
- `Class`의 `public:`, `protected:`, `private:`은 들여쓰기를 사용하지 않는다.

(5) 괄호 (Brace)

- 모든 if, else, do, while, for 구문에 괄호를 사용한다.
- 괄호 및 공백 사용은 아래 예제를 참고하자.

(6) 주석 (Comments)

- 문서 주석에는 `/\*\* \*/`을 사용한다.
- 구현 주석에는 `//`을 사용한다.

(7) 린터 (Linters)

- C++ 코드 스타일의 자동 오류 검출을 위하여 [ament_cpplint](#), [ament_uncrustify](#)를 사용하고 정적 코드 분석이 필요한 경우 [ament_cppcheck](#)를 사용하자.

(8) 기타

- Boost 라이브러리의 사용은 가능한 피하고 어쩔 수 없을 경우에만 사용한다.
- 포인터 구문은 `char \* c;`처럼 사용한다. (`char\* c;` 이나 `char \*c;` 처럼 사용하지 않는다.)
- 중첩 템플릿은 `set<list<string>>`처럼 사용한다. (`set<list<string>>` 또는 `set< list<string> >` 처럼 사용하지 않는다.)

246 Python Style

ROS 2 Developer Guide 및 ROS 2 Code style에서 다루고 있는 Python 코드 스타일은 Python Enhancement Proposals (PEPs)의 PEP 8를 준수한다.

(1) 기본 규칙

- Python 3 (Python 3.5 이상)를 사용한다.

(2) 라인 길이

- 최대 100 문자

(3) 이름 규칙 (Naming)

- CamelCased : 타입, 클래스
- snake_case : 파일, 패키지, 인터페이스, 모듈, 변수, 함수, 메소드
- ALL_CAPITALS : 상수

(4) 공백 문자 대 탭 (Spaces vs. Tabs)

- 기본 들여쓰기(indent)는 공백 문자(space) `4개`를 사용한다. (탭(tab)문자 사용 금지)
- `Hanging indent`(문장 중간에 들여쓰기를 사용하는 형식)의 사용 방법은 아래 예제를 참고하자.
- 괄호 및 공백 사용은 아래 예제를 참고하자.

(5) 괄호 (Brace)

- 괄호는 계산식 및 배열 인덱스로 사용하며, 자료형에 따라 적절한 괄호(대괄호 `[ ]`, 중괄호 `{ }`, 소괄호 `( )`)를 사용한다.

(6) 주석 (Comments)

- 문서 주석에는 `"""`을 사용하며 Docstring Conventions을 기술한 [PEP 257](#)을 준수한다.
- 구현 주석에는 `#`을 사용한다.

(7) 린터 (Linters)

- Python 코드 스타일의 자동 오류 검출을 위하여 [ament_flake8](#)를 사용하자.

(8) 기타

- 모든 문자는 큰 따옴표(`""`, double quotes)가 아닌 작은 따옴표(`'`, single quotes)를 사용하여 표현한다.

* 다른 언어

- C 언어는 C99 Standard를 준수하며 [PEP-7](#)를 참고하자.
- Javascript 언어는 [airbnb Javascript Style guide](#)를 참고하자.

ROS 프로그래밍 기초 (Python)

249 ROS의 Hello World, rclpy 버전

프로그래밍 언어를 배울 때 처음에 등장하는 Hello World는 화면에 Hello World라는 문구를 출력하는 것으로 시작한다. ROS의 Hello World 또한 다르지 않지만 출력보다는 **메시지 전송에 더 초점을 둔다**. 오늘은 Python 언어로 ROS 2의 가장 간단한 구조의 토픽(topic) 퍼블리셔 (**publisher**)와 서브스크라이버 (**subscriber**)를 작성하고 동작시켜 보겠다.

```
$ ros2 pkg create [패키지이름] --build-type [빌드 타입] --dependencies [의존패키지1] [의존패키지n]
```

```
$ cd ~/robot_ws/src/
```

```
$ ros2 pkg create my_first_ros_rclpy_pkg --build-type ament_python --dependencies rclpy std_msgs
```

```
.
├── my_first_ros_rclpy_pkg
│   ├── __init__.py
│   └── resource
│       └── my_first_ros_rclpy_pkg
├── test
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py
├── package.xml
├── setup.cfg
└── setup.py
```

3 directories, 8 files

250 패키지 설정 파일 (package.xml)

```
$ nano ~/robot_ws/src/my_first_ros_rclpy_pkg/package.xml
```

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema">
<package format="3">
  <name>my_first_ros_rclpy_pkg</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="pyo@robotis.com">pyo</maintainer>
  <license>TODO: License declaration</license>

  <depend>rclpy</depend>
  <depend>std_msgs</depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

251

파이썬 패키지 설정 파일 (setup.py)

```
$ nano ~/robot_ws/src/my_first_ros_rclpy_pkg/setup.py
```

```
from setuptools import find_packages, setup

package_name = 'my_first_ros_rclpy_pkg'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='pyo',
    maintainer_email='pyo@robotis.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'helloworld_publisher = my_first_ros_rclpy_pkg.helloworld_publisher:main',
            'helloworld_subscriber = my_first_ros_rclpy_pkg.helloworld_subscriber:main',
        ],
    },
)
```

```
$ nano ~/robot_ws/src/my_first_ros_rclpy_pkg/my_first_ros_rclpy_pkg/helloworld_publisher.py
```

```
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile
from std_msgs.msg import String

class HelloworldPublisher(Node):

    def __init__(self):
        super().__init__('helloworld_publisher')
        qos_profile = QoSProfile(depth=10)
        self.helloworld_publisher = self.create_publisher(String, 'helloworld',
qos_profile)
        self.timer = self.create_timer(1, self.publish_helloworld_msg)
        self.count = 0

    def publish_helloworld_msg(self):
        msg = String()
        msg.data = 'Hello World: {0}'.format(self.count)
        self.helloworld_publisher.publish(msg)
        self.get_logger().info('Published message: {0}'.format(msg.data))
        self.count += 1
```

```
def main(args=None):
    rclpy.init(args=args)
    node = HelloworldPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

```
$ nano ~/robot_ws/src/my_first_ros_rclpy_pkg/my_first_ros_rclpy_pkg/helloworld_subscriber.py
```

```
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile
from std_msgs.msg import String
```

```
class HelloworldSubscriber(Node):
```

```
    def __init__(self):
        super().__init__('Helloworld_subscriber')
        qos_profile = QoSProfile(depth=10)
        self.helloworld_subscriber = self.create_subscription(
            String,
            'helloworld',
            self.subscribe_topic_message,
            qos_profile)

    def subscribe_topic_message(self, msg):
        self.get_logger().info('Received message: {0}'.format(msg.data))
```

```
def main(args=None):
    rclpy.init(args=args)
    node = HelloworldSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

254 빌드

ROS 2 특정 패키지 또는 전체 패키지를 빌드할 때에는 **colcon 빌드 툴**을 사용한다. 사용 방법은 매우 간단한 편인데 우선 소스 코드가 있는 **workspace**로 이동하고 **colcon build** 명령어로 전체를 빌드하게 된다. 여기서 빌드 옵션을 추가하여 사용하는게 일반적인데 특정 패키지만 선택하여 빌드하고자 할 때에는 `--packages-select` 옵션을 이용하고 **symlink**를 이용하려면 `--symlink-install` 옵션을 붙여주면 된다.

(워크스페이스내의 모든 패키지 빌드하는 방법)

```
$ cd ~/robot_ws && colcon build --symlink-install
```

(특정 패키지만 빌드하는 방법)

```
$ cd ~/robot_ws && colcon build --symlink-install --packages-select [패키지 이름1] [패키지 이름2] [패키지 이름N]
```

(특정 패키지 및 의존성 패키지를 함께 빌드하는 방법)

```
$ cd ~/robot_ws && colcon build --symlink-install --packages-up-to [패키지 이름]
```

```
$ cd ~/robot_ws
```

```
$ colcon build --symlink-install --packages-select my_first_ros_rclpy_pkg
```

```
Starting >>> my_first_ros_rclpy_pkg
```

```
Finished <<< my_first_ros_rclpy_pkg [0.66s]
```

```
Summary: 1 package finished [0.87s]
```

```
$ ros2 run my_first_ros_rclpy_pkg helloworld_subscriber  
[INFO]: Received message: Hello World: 0  
[INFO]: Received message: Hello World: 1  
[INFO]: Received message: Hello World: 2  
[INFO]: Received message: Hello World: 3  
[INFO]: Received message: Hello World: 4  
[INFO]: Received message: Hello World: 5
```

```
$ ros2 run my_first_ros_rclpy_pkg helloworld_publisher  
[INFO]: Published message: Hello World: 0  
[INFO]: Published message: Hello World: 1  
[INFO]: Published message: Hello World: 2  
[INFO]: Published message: Hello World: 3  
[INFO]: Published message: Hello World: 4  
[INFO]: Published message: Hello World: 5
```

토픽, 서비스, 액션 인터페이스

ROS 2 프로그래밍은 노드간의 메시지 통신을 위해 정수(integer), 부동 소수점(floating point), 불(boolean)같은 기본 인터페이스를 모아둔 `std_msgs`이나 병진속도 x, y, z , 회전속도 x, y, z 를 표현할 수 있는 `geometry_msgs`의 `Twist.msg` 인터페이스, 레이저 스캐닝 값을 담을 수 있는 `sensor_msgs`의 `LaserScan.msg`와 같은 이미 만들어 놓은 인터페이스를 사용하는 것이 일반적이다. 하지만 이러한 인터페이스들이 사용자가 원하는 모든 정보를 담을 수는 없고 토픽 인터페이스 이외에 서비스나 액션 인터페이스는 매우 기본적인 인터페이스만 제공하기에 사용자가 원하는 정보의 형태가 아니라면 새로 만들어 써야 한다.

이번 강좌에서는 ROS 2 기초 프로그래밍 예제에서 사용할 토픽(Topic), 서비스(Service), 액션(Action) 관련 ROS 2 인터페이스 (Interface)를 신규로 작성하려고 한다. 참고로 ROS 2 인터페이스는 이 인터페이스를 사용하려는 패키지에 포함시켜도 되지만 경험상 인터페이스으로만 구성된 패키지를 별도로 만들어 사용하는 것이 의존성면에서 관리하기 편하다. 예를 들어 A라는 패키지에서 a라는 인터페이스를 사용한다고 했을 때 a라는 인터페이스를 사용하는 B, C라는 패키지가 있다면 B와 C패키지는 a 인터페이스가 담긴 A라는 패키지를 통째로 의존하기 때문이다. 이러한 이유로 인터페이스는 별도의 독립된 패키지로 구성해야한다. (예: `geometry_msgs`, `turtlebot3_msgs`)

우리는 이 강좌에서 `msg_srv_action_interface_example` 이라는 이름의 패키지를 만들 것이고 이 인터페이스 전용 패키지에는 아래와 같이 msg 인터페이스 1개, srv 인터페이스 1개, action 인터페이스 1개를 포함시킬 것이다.

- `ArithmeticArgument.msg`
- `ArithmeticOperator.srv`
- `ArithmeticChecker.action`

258 인터페이스 패키지 만들기

```
$ cd ~/robot_ws/src  
$ ros2 pkg create --build-type ament_cmake msg_srv_action_interface_example  
$ cd msg_srv_action_interface_example  
$ mkdir msg srv action
```

```
|— action  
|   |— ArithmeticChecker.action  
|— msg  
|   |— ArithmeticArgument.msg  
|— srv  
    |— ArithmeticOperator.srv
```

259 msg, srv, action 생성

저장 위치:

`msg_srv_action_interface_example/msg/ArithmeticArgument.msg`

```
# Messages
builtin_interfaces/Time stamp
float32 argument_a
float32 argument_b
```

저장 위치:

`msg_srv_action_interface_example/srv/ArithmeticOperator.srv`

```
# Constants
int8 PLUS = 1
int8 MINUS = 2
int8 MULTIPLY = 3
int8 DIVISION = 4

# Request
int8 arithmetic_operator
---
# Response
float32 arithmetic_result
```

저장 위치:

`msg_srv_action_interface_example/action/ArithmeticChecker.action`

```
# Goal
float32 goal_sum
---
# Result
string[] all_formula
float32 total_sum
---
# Feedback
string[] formula
```

260 토픽, 서비스, 액션 비교

아래의 표는 토픽, 서비스, 액션에서 사용되는 `msg`, `srv`, `action` 인터페이스를 비교한 내용이다. 모든 인터페이스는 `msg` 인터페이스의 확장형이라고 생각하면 되고 `---` 구분자가 0, 1, 2개로 몇개를 넣어 데이터를 구분했냐의 차이이다.

	msg 인터페이스	srv 인터페이스	action 인터페이스
확장자	*.msg	*.srv	*.action
데이터	토픽 데이터 (data)	서비스 요청 (request) --- 서비스 응답 (response)	액션 목표 (goal) --- 액션 결과 (result) --- 액션 피드백 (feedback)
형식	fieldtype1 fieldname1 fieldtype2 fieldname2 fieldtype3 fieldname3	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4	fieldtype1 fieldname1 fieldtype2 fieldname2 --- fieldtype3 fieldname3 fieldtype4 fieldname4 --- fieldtype5 fieldname5 fieldtype6 fieldname6
사용 예	[ArithmeticArgument.msg] builtin_interfaces/Time stamp float32 argument_a float32 argument_b	[ArithmeticOperator.srv] int8 arithmetic_operator --- float32 arithmetic_result	[ArithmeticChecker.action] float32 goal_sum --- string[] all_formula float32 total_sum --- string[] formula

261 패키지 설정 파일 (package.xml)

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>msg_srv_action_interface_example</name>
  <version>0.6.0</version>
  <description>
    ROS 2 example for message, service and action interface
  </description>
  <maintainer email="passionvirus@gmail.com">Pyo</maintainer>
  <license>Apache 2.0</license>
  <author email="passionvirus@gmail.com">Pyo</author>
  <author email="routiful@gmail.com">Darby Lim</author>
  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>rosidl_default_generators</buildtool_depend>
  <exec_depend>builtin_interfaces</exec_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>
  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

```
#####
# Set minimum required version of cmake, project name and compile options
#####
cmake_minimum_required(VERSION 3.5)
project(msg_srv_action_interface_example)

if(NOT CMAKE_CXX_STANDARD)
  set(CMAKE_CXX_STANDARD 14)
endif()

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Wextra -Wpedantic")
endif()

#####
# Find and load build settings from external packages
#####
find_package(ament_cmake REQUIRED)
find_package(builtin_interfaces REQUIRED)
find_package(rosidl_default_generators REQUIRED)
```

```
#####
# Declare ROS messages, services and actions
#####
set(msg_files
  "msg/ArithmeticArgument.msg"
)

set(srv_files
  "srv/ArithmeticOperator.srv"
)

set(action_files
  "action/ArithmeticChecker.action"
)

rosidl_generate_interfaces(${PROJECT_NAME}
  ${msg_files}
  ${srv_files}
  ${action_files}
  DEPENDENCIES builtin_interfaces
)

#####
# Macro for ament package
#####
ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

```
$ cd ~/robot_ws  
$ colcon build --symlink-install
```

빌드한 후 빌드에 문제가 없다면 `~/robot\_ws/install/msg\_srv\_action\_interface\_example` 폴더 안에 우리가 작성한 ROS 인터페이스를 사용하기 위한 파일들이 저장되게 된다.

예를 들어 C는 위한 h, C++을 위한 hpp, 파이썬 모듈, IDL 파일들이다. 빌드 후에 참고삼아 해당 폴더의 파일들을 확인해보도록 하자.

예) msg_srv_action_interface_example의 변환된 인터페이스 파일 (*.h, *hpp, *.py, *.idl)

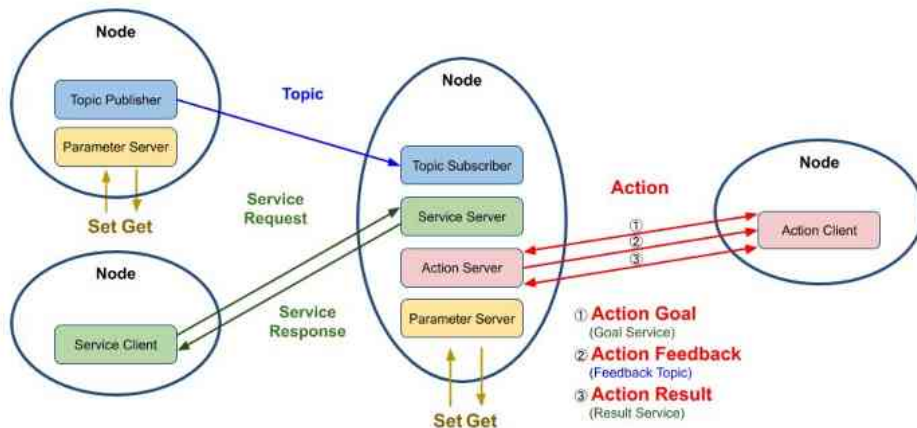
- *.h, *hpp : include/msg_srv_action_interface_example
- *.py : lib/python3.10/dist-packages/msg_srv_action_interface_example
- *.idl : share/msg_srv_action_interface_example

ROS 2 패키지 설계 (Python)

265 패키지 설계

ROS와 연동되는 로봇 프로그램과 일반적인 로봇 프로그램과의 차이는 프로세스를 목적별로 나누어 **노드 (node) 단위의 프로그램을 작성** 하고 노드와 노드간의 데이터 통신을 고려하여 설계해야 한다는 것이다. 이번 강좌에서는 ROS 2의 토픽, 서비스, 액션 프로그래밍 (Python)의 예제이면서 하나의 패키지로 토픽, 서비스, 액션이 따로 구동하는 것이 아닌 연동되어 구동하는 방식으로 설계하였다.

이를 위해 아래 그림과 같이 **4개의 노드에 토픽, 서비스, 액션 각각 1개씩을 사용** 하고 있다. 각 노드에서는 1개 이상의 **토픽 퍼블리셔, 토픽 서브스크라이브, 서비스 서버, 서비스 클라이언트, 액션 서버, 액션 클라이언트** 를 포함하고 있다. 특히 중앙에 있는 노드는 다른 노드들과의 연동에 있어서 가장 핵심적인 역할을 하도록 설계하였고 추후 강좌를 위해서 **파라미터 (parameter) 및 실행 인자(argument), 런치 (launch)** 파일이 추가되어 있다.



266 패키지 : topic_service_action_rclpy_example

1) argument node (Topic Publisher)

: arithmetic_argument 이라는 토픽 이름으로 현재 시간과 변수 a와 b를 퍼블리시한다.

2) calculator node (Topic Subscriber)

: 토픽이 생성된 시간과 변수 a와 b를 서브스크라이브한다.

3) operator node (Service Client)

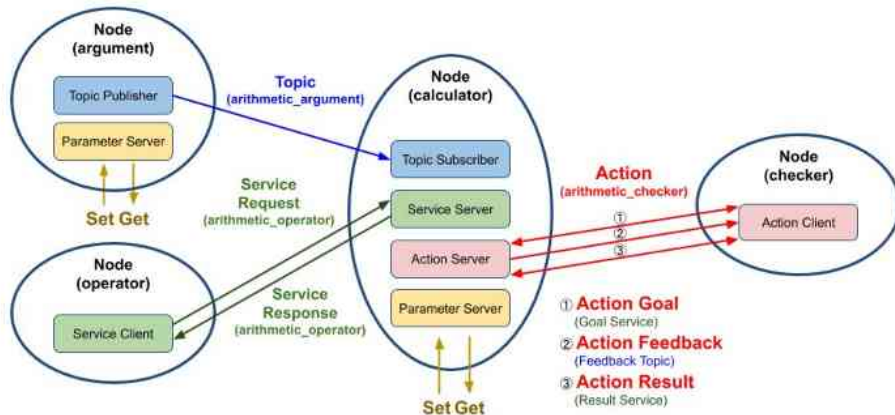
: arithmetic_operator 이라는 서비스 이름으로 calculator 노드에게 연산자(+, -, *, /)를 서비스 요청 값으로 보낸다.

4) calculator node (Service Server)

: 서브스크라이브하여 저장하고 있는 변수 a와 b를 operator 노드로부터 요청 값으로 받은 연산자를 이용하여 계산(a 연산자 b)하고 operator 노드에게 연산의 결과값을 서비스 응답값으로 보낸다.

5) checker node (Action)

: 연산값의 합계의 한계치를 액션 목표값으로 전달한 후, calculator 노드는 이를 받은 후 부더의 연산 값을 합하고 액션 피드백으로 각 연산 계산식을 보낸다. 지정한 연산값의 목표 합계를 넘기면 액션 결과값으로 최종 연산 합계를 보낸다.



위에서 설명한 `topic_service_action_rclpy_example` 패키지는 `argument` 노드, `operator` 노드, `calculator` 노드, `checker` 노드와 같이 4개의 노드로 구성되어 있다. 해당 노드의 [원본 코드](#)는 참고자료로 올린 리포지토리에서 미리 확인하실 수 있으며 자세한 내용은 아래와 같이 이어지는 강좌에서 자세히 설명할 예정이다. 이 강좌에서는 패키지 및 노드 설계 및 빌드, 실행에 대한 내용만을 다룰 예정이다.

- 토픽 프로그래밍 (Python)
- 서비스 프로그래밍 (Python)
- 액션 프로그래밍 (Python)
- 파라미터 프로그래밍 (Python)
- 실행 인자 프로그래밍 (Python)
- 런치 프로그래밍 (Python, C++)

268 패키지 설정 파일 (package.xml)

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>topic_service_action_rclpy_example</name>
  <version>0.2.0</version>
  <description>ROS 2 rclpy example package for the topic, service, action</description>
  <maintainer email="passionvirus@gmail.com">Pyo</maintainer>
  <license>Apache License 2.0</license>
  <author email="passionvirus@gmail.com">Pyo</author>
  <author email="routiful@gmail.com">Darby Lim</author>
  <depend>rclpy</depend>
  <depend>std_msgs</depend>
  <depend>msg_srv_action_interface_example</depend>
  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>
  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

269 파이썬 패키지 설정 파일 (setup.py)

```
#!/usr/bin/env python3

import glob
import os

from setuptools import find_packages
from setuptools import setup

package_name = 'topic_service_action_rclpy_example'
share_dir = 'share/' + package_name

setup(
    name=package_name,
    version='0.2.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        (share_dir, ['package.xml']),
        (share_dir + '/launch', glob.glob(os.path.join('launch', '*.launch.py'))),
        (share_dir + '/param', glob.glob(os.path.join('param', '*.yaml'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    author='Pyo, Darby Lim',
    author_email='passionvirus@gmail.com, routiful@gmail.com',
    maintainer='Pyo',
    maintainer_email='passionvirus@gmail.com',
```

```
keywords=['ROS'],
classifiers=[
    'Intended Audience :: Developers',
    'License :: OSI Approved :: Apache Software License',
    'Programming Language :: Python',
    'Topic :: Software Development',
],
description='ROS 2 rclpy example package for the topic, service, action',
license='Apache License, Version 2.0',
tests_require=['pytest'],
entry_points={
    'console_scripts': [
        'argument = topic_service_action_rclpy_example.arithmetic.argument:main',
        'operator = topic_service_action_rclpy_example.arithmetic.operator:main',
        'calculator = topic_service_action_rclpy_example.calculator.main:main',
        'checker = topic_service_action_rclpy_example.checker.main:main',
    ],
},
)
```

270 소스 코드 다운로드 및 빌드

소스 코드 다운로드 및 빌드는 하기와 같은 명령어로 진행하면 된다.

```
$ cd ~/robot_ws/src  
$ git clone https://github.com/robotpilot/ros2-seminar-examples.git  
$ cd ~/robot_ws && colcon build --symlink-install
```

또는 아래와 같이 미리 지정한 alias를 사용하면 매우 편하다.

```
$ cw  
$ cbp topic_service_action_rclpy_example
```

* `cw`는 `cd ~/robot\_ws`의 alias이다.

* `cbp`는 `colcon build --symlink-install --packages-select`의 alias이다.

271 소스 코드 다운로드 및 빌드

빌드한 후 빌드에 문제가 없다면 `~/robot\_ws/install/topic\_service\_action\_rclpy\_example` 폴더 안에 우리가 작성한 ROS 인터페이스를 사용하기 위한 파일들이 저장되게 된다.

예를 들어 하기 폴더에는 argument, operator, calculator, checker와 같은 실행 스크립트가 위치하게 된다. (lib)

- `~/robot_ws/install/topic_service_action_rclpy_example/lib/topic_service_action_rclpy_example`

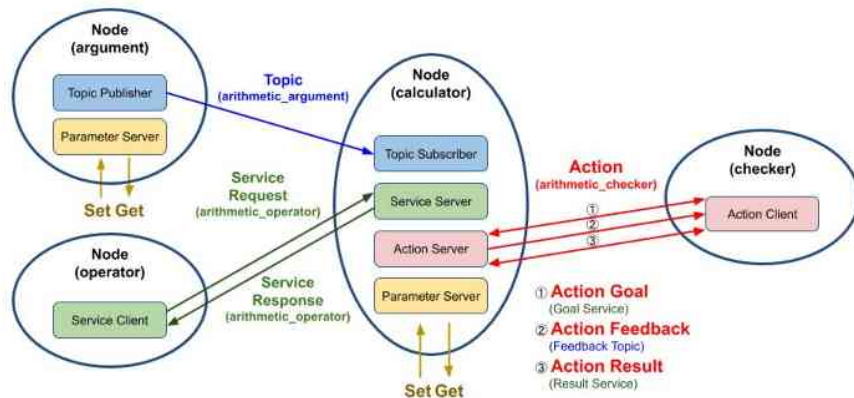
그리고 하기 폴더에는 launch 폴더와 param 폴더가 생성되고 각각 런치파일 arithmetic.launch.py 파일과 파라미터 파일인 arithmetic_config.yaml 파일이 위치하게 된다. (share)

- `~/robot_ws/install/topic_service_action_rclpy_example/share/topic_service_action_rclpy_example`

272 실행

제일 먼저 **calculator** 노드를 실행해보자. 이 노드는 이 패키지에서 토픽 서브스크라이버, 서비스 서버, 액션 서버 역할을 수행하는 노드로 실행 후 아무런 동작은 하고 있지 않은 대기 상태가 된다.

```
$ ros2 run topic_service_action_rclpy_example calculator
```



273 토픽 퍼블리셔 실행

다음으로 토픽 퍼블리셔 역할을 하는 `argument` 노드를 실행해보자. 그러면 터미널 창에 퍼블리시하고 있는 `argument a`, `argument b`가 표시된다. 더불어 `calculator` 노드를 실행 시킨 터미널 창에서는 수신 받은 시간 정보와 `argument a`, `argument b`가 표시된다.

```
$ ros2 run topic_service_action_rclpy_example argument
[INFO]: Published argument a: 5.0
[INFO]: Published argument b: 0.0
[INFO]: Published argument a: 1.0
[INFO]: Published argument b: 5.0
```

```
$ ros2 run topic_service_action_rclpy_example calculator
[INFO]: Subscribed at: builtin_interfaces.msg.Time(sec=1607301572, nanosec=687669256)
[INFO]: Subscribed argument a: 5.0
[INFO]: Subscribed argument b: 0.0
[INFO]: Subscribed at: builtin_interfaces.msg.Time(sec=1607301573, nanosec=687576780)
[INFO]: Subscribed argument a: 1.0
[INFO]: Subscribed argument b: 5.0
```

274 서비스 클라이언트 실행

다음으로 서비스 클라이언트 역할을 하는 `operator` 노드를 실행해보자. 그러면 `calculator` 노드에게 랜덤으로 선택한 연산자(+, -, *, /)를 서비스 요청값으로 보내고 연산된 결과값을 받아 터미널 창에 표시한다. 실제 계산식은 `calculator` 노드가 실행 중인 창에서 확인할 수 있다. 예를 들어 `calculator`는 변수 `a`, `b`로 9와 6을 받았고 연산자로 곱셈(`*`)을 받았을 때 $9.0 * 6.0 = 54.0$ 이므로 `operator`는 54.0를 표시하게 된다.

```
$ ros2 run topic_service_action_rclpy_example operator  
[INFO]: Result: 54.0  
[INFO]: Result: 0.625
```

```
[INFO]: Subscribed at: builtin_interfaces.msg.Time(sec=1607379440, nanosec=275268255)  
[INFO]: Subscribed argument a: 9.0  
[INFO]: Subscribed argument b: 6.0  
[INFO]: 9.0 * 6.0 = 54.0  
[INFO]: Subscribed at: builtin_interfaces.msg.Time(sec=1607379441, nanosec=275234062)  
[INFO]: Subscribed argument a: 5.0  
[INFO]: Subscribed argument b: 8.0  
[INFO]: 5.0 / 8.0 = 0.625
```

275 액션 클라이언트 실행

마지막으로 checker 노드는 연산값의 합계의 한계치를 액션 목표값으로 전달한 후, calculator 노드는 이를 받은 후 부터의 연산 값을 합하고 액션 피드백으로 각 연산 계산식을 보낸다. 지정한 연산값의 목표 합계를 넘기면 액션 결과값으로 최종 연산 합계를 보낸다.

```
$ ros2 run topic_service_action_rclpy_example checker
[INFO]: Action goal accepted.
[INFO]: Action feedback: ['7.0 + 3.0 = 10.0']
[INFO]: Action feedback: ['7.0 + 3.0 = 10.0', '8.0 + 1.0 = 9.0']
[INFO]: Action feedback: ['7.0 + 3.0 = 10.0', '8.0 + 1.0 = 9.0', '6.0 / 1.0 = 6.0']
[INFO]: Action feedback: ['7.0 + 3.0 = 10.0', '8.0 + 1.0 = 9.0', '6.0 / 1.0 = 6.0', '8.0 - 6.0 = 2.0']
[INFO]: Action feedback: ['7.0 + 3.0 = 10.0', '8.0 + 1.0 = 9.0', '6.0 / 1.0 = 6.0', '8.0 - 6.0 = 2.0', '6.0 * 6.0 = 36.0']
[INFO]: Action succeeded!
```

합계의 한계치는 기본으로 50으로 설정되어 있다. 이를 수정하여 실행하고 싶으면 아래와 같이 checker 노드를 실행시키면서 실행 인자로 `-g 100` 이라고 입력하면 `GOAL_TOTAL_SUM` 이라는 인자로 100을 할당할 수 있다. 여기서 실행인자에 대한 이해가 필요한데 이는 '실행 인자 프로그래밍 (Python)' 강좌에서 다룰 예정이다.

```
$ ros2 run topic_service_action_rclpy_example checker -g 100
```

276 런치 파일 실행

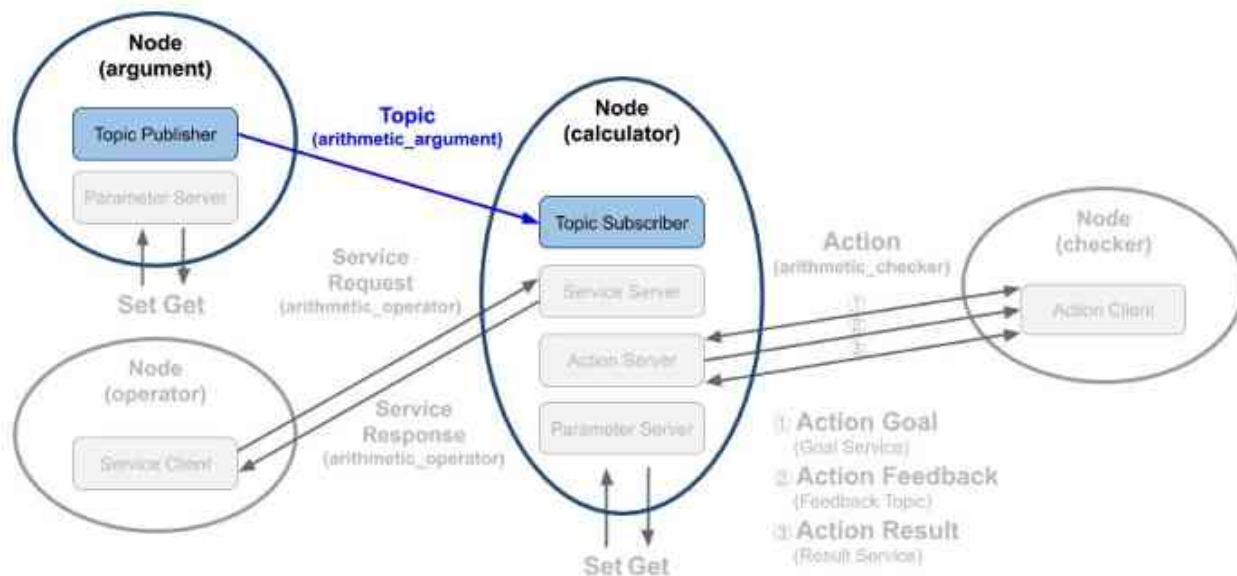
참고로 `argument` 노드와 `calculator` 노드를 한번에 실행시키고자 한다면 아래와 같이 `launch` 파일인 `arithmetic.launch.py`를 실행하는 방법으로 두개의 노드를 동시에 실행 시킬 수 있다. `launch` 파일에 대한 자세한 설명은 추후 '런치 프로그래밍 (Python, C++)' 에서 더 자세히 알아볼 예정이기에 여기서는 실행 방법만 알아보았다.

```
$ ros2 launch topic_service_action_rclpy_example arithmetic.launch.py
```

토픽 프로그래밍 (Python)

278 토픽 퍼블리셔 / 토픽 서브스크라이버

이 강좌에서 다음 그림과 같이 토픽을 생성한 시간과 연산에 사용할 변수 **a**와 변수 **b**를 퍼블리시하는 토픽 퍼블리셔 와 이를 서브스크라이브하는 토픽 서브스크라이버 를 작성해 볼 것이다.



```

import random

from msg_srv_action_interface_example.msg import ArithmeticArgument
from rcl_interfaces.msg import SetParametersResult
import rclpy
from rclpy.node import Node
from rclpy.parameter import Parameter
from rclpy.qos import QoSDurabilityPolicy
from rclpy.qos import QoSHistoryPolicy
from rclpy.qos import QoSProfile
from rclpy.qos import QoSReliabilityPolicy

class Argument(Node):

    def __init__(self):
        super().__init__('argument')
        self.declare_parameter('qos_depth', 10)
        qos_depth = self.get_parameter('qos_depth').value
        self.declare_parameter('min_random_num', 0)
        self.min_random_num = self.get_parameter('min_random_num').value
        self.declare_parameter('max_random_num', 9)
        self.max_random_num = self.get_parameter('max_random_num').value
        self.add_on_set_parameters_callback(self.update_parameter)

    QOS_RKL10V = QoSProfile(
        reliability=QoSReliabilityPolicy.RELIABLE,
        history=QoSHistoryPolicy.KEEP_LAST,
        depth=qos_depth,
        durability=QoSDurabilityPolicy.VOLATILE)

    self.arithmetic_argument_publisher = self.create_publisher(
        ArithmeticArgument,
        'arithmetic_argument',
        QOS_RKL10V)

```

```

self.timer = self.create_timer(1.0, self.publish_random_arithmetic_arguments)

def publish_random_arithmetic_arguments(self):
    msg = ArithmeticArgument()
    msg.stamp = self.get_clock().now().to_msg()
    msg.argument_a = float(random.randint(self.min_random_num, self.max_random_num))
    msg.argument_b = float(random.randint(self.min_random_num, self.max_random_num))
    self.arithmetic_argument_publisher.publish(msg)
    self.get_logger().info('Published argument a: {}'.format(msg.argument_a))
    self.get_logger().info('Published argument b: {}'.format(msg.argument_b))

def update_parameter(self, params):
    for param in params:
        if param.name == 'min_random_num' and param.type_ == Parameter.Type.INTEGER:
            self.min_random_num = param.value
        elif param.name == 'max_random_num' and param.type_ == Parameter.Type.INTEGER:
            self.max_random_num = param.value
    return SetParametersResult(successful=True)

def main(args=None):
    rclpy.init(args=args)
    try:
        argument = Argument()
        try:
            rclpy.spin(argument)
        except KeyboardInterrupt:
            argument.get_logger().info('Keyboard Interrupt (SIGINT)')
        finally:
            argument.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

```
class Argument(Node):  
  
    def __init__(self):  
        super().__init__('argument')  
  
    (일부 코드 생략)  
  
    QoS_RKL10V = QoSProfile(  
        reliability=QoSReliabilityPolicy.RELIABLE,  
        history=QoSHistoryPolicy.KEEP_LAST,  
        depth=qos_depth,  
        durability=QoSDurabilityPolicy.VOLATILE)
```

우선 **Argument** 클래스인데 rclpy.node 모듈의 Node 클래스를 상속하고 있으며 생성자에서 'argument' 이라는 노드 이름으로 초기화되었다.

그 뒤 rclpy.qos 모듈의 QoSProfile 클래스를 이용하여 토픽 퍼블리셔에서 사용할 **QoS 설정**을 해주었다.

여기서 QoS는 **RELIABLE, KEEP_LAST, DEPTH 10, VOLATILE**로 설정하였다.


```
self.arithmetic_argument_publisher = self.create_publisher(  
    ArithmeticArgument,  
    'arithmetic_argument',  
    QOS_RKL10V)
```

```
self.timer = self.create_timer(1.0, self.publish_random_arithmetic_arguments)
```

제일 중요한 설정은

arithmetic_argument_publisher 선언이다. 이는 Node 클래스의 **create_publisher** 함수를 이용하여 퍼블리셔를 선언하는 부분으로 토픽의 타입으로 **ArithmeticArgument**으로 선언하였고, 토픽 이름으로는 **'arithmetic_argument'**, QoS는 위에서 설정한 **QOS_RKL10V**를 사용하였다.

마지막 줄에서 **create_timer** 함수를 이용하여 1초마다

publish_random_arithmetic_arguments 이라는 함수를 실행시키도록 설정하였다.

이전 설정들은 퍼블리시를 위한 기본 설정이고 실제 토픽 발행이 이루어지는 부분은 **publish_random_arithmetic_arguments** 함수임을 알아두자.

```
def publish_random_arithmetic_arguments(self):
    msg = ArithmeticArgument()

    msg.stamp = self.get_clock().now().to_msg()

    msg.argument_a =
        float(random.randint(self.min_random_num, self.max_random_num))
    msg.argument_b =
        float(random.randint(self.min_random_num, self.max_random_num))

    self.arithmetic_argument_publisher.publish(msg)

    self.get_logger().info('Published argument a: {0}'.format(msg.argument_a))
    self.get_logger().info('Published argument b: {0}'.format(msg.argument_b))
```

publish_random_arithmetic_arguments 함수의 내용을 보도록 하자.

이 함수는 지정 타이머로 동작하는 콜백함수이며 1초마다 실행되며 매번 실행될 때 마다 **msg** 이라는 변수를 우리가 지난 `토픽, 서비스, 액션 인터페이스` 강좌에서 작성한 **msg** 인터페이스의 **ArithmeticArgument()** 클래스로 생성해주었고 **get_clock().now().to_msg()** 함수를 통하여 토픽이 생성된 시간을 **msg.stamp**에 기록해두었다. 그 뒤 랜덤 함수를 통해 0~9까지의 숫자를 float로 변환하여 **msg.argument_a**와 **msg.argument_b**에 저장하였다.

그 뒤 이어지는 **arithmetic_argument_publisher.publish(msg)** 함수가 실제로 토픽 발행이 이루어지는 함수로 우리가 **발행 시간 및 변수 a, b를 저장한 msg 메시지를 퍼블리쉬한다**는 의미이다.

마지막으로 **get_logger().info()**는 주로 디버깅에 사용되는 함수로 노드를 실행한 터미널 창에 특정 값을 표시하게 된다. 여기서는 토픽으로 보낸 변수 **a, b** 값을 화면에 표시하고 있다.

```
class Calculator(Node):

    def __init__(self):
        super().__init__('calculator')
        self.argument_a = 0.0
        self.argument_b = 0.0
        self.callback_group = ReentrantCallbackGroup()
```

(일부 코드 생략)

```
QOS_RKL10V = QoSProfile(
    reliability=QoSReliabilityPolicy.RELIABLE,
    history=QoSHistoryPolicy.KEEP_LAST,
    depth=qos_depth,
    durability=QoSDurabilityPolicy.VOLATILE)
```

이 소스 코드는 토픽 서브스크라이버, 서비스 서버, 액션 서버를 모두 포함하고 있어서 매우 길기 때문에 전체 코드를 강좌 글에 담는 것은 생략하도록 하고 전체 소스 코드 중 토픽 서브스크라이버와 관련한 코드는 다음과 같다.

우선 **Calculator** 클래스인데 토픽 퍼블리셔 노드와 마찬가지로 `rclpy.node` 모듈의 `Node` 클래스를 상속하고 있으며 생성자에서 '**calculator**' 이라는 노드 이름으로 초기화되었다.

그 뒤 위에서와 마찬가지로 `rclpy.qos` 모듈의 **QoSProfile** 클래스를 이용하여 토픽 서브스크라이버에서 사용할 QoS 설정을 해주었다. 여기서 QoS는 앞서 설명한 토픽 퍼블리셔 노드와 동일하게 **RELIABLE, KEEP_LAST, DEPTH 10, VOLATILE**로 설정하였다.

QoS에 대한 자세한 설명은 'DDS의 QoS(Quality of Service)' 강좌를 참고하자.

```
self.arithmetic_argument_subscriber = self.create_subscription(  
    ArithmeticArgument,  
    'arithmetic_argument',  
    self.get_arithmetic_argument,  
    QOS_RKL10V,  
    callback_group=self.callback_group)
```

제일 중요한 설정은 `arithmetic_argument_subscriber` 선언이다. 이는 Node 클래스의 `create_subscription` 함수를 이용하여 서브스크라이버로 선언하는 부분으로 **토픽의 타입**으로 퍼블리셔와 동일하게 `ArithmeticArgument`으로 선언하였고, **토픽 이름**으로는 `'arithmetic_argument'`, QoS는 위에서 설정한 `QOS_RKL10V`를 사용하였다. 여기까지는 퍼블리셔와 비슷한데 퍼블리셔와 다른점은

`get_arithmetic_argument` 라는 콜백함수를 두어 퍼블리셔로부터 메시지를 서브스크라이브할 때 마다 실행되는 함수를 지정한 것이다. 그리고 `callback_group`을 `ReentrantCallbackGroup`으로 지정했는데 콜백함수를 병렬로 실행할 수 있게 해주며 뒤에서 설명할 `MultiThreadedExecutor`와 함께 사용되곤 한다.

좀 더 자세히 `callback_group`을 설명하자면 원래 `callback_group`은 지정하지 않아도 되는데 지정하지 않게 되면 `MutuallyExclusiveCallbackGroup`이 기본 설정으로 사용된다. `MutuallyExclusiveCallbackGroup`은 한 번에 하나의 콜백함수만 실행하도록 허용하는 것이고, 다른 하나의 설정은 `ReentrantCallbackGroup`으로써 제한없이 콜백함수를 병렬로 실행할 수 있게 해준다.

```
def get_arithmetic_argument(self, msg):  
    self.argument_a = msg.argument_a  
    self.argument_b = msg.argument_b  
  
    self.get_logger().info('Subscribed at: {0}'.format(msg.stamp))  
    self.get_logger().info('Subscribed argument a: {0}'.format(self.argument_a))  
    self.get_logger().info('Subscribed argument b: {0}'.format(self.argument_b))
```

get_arithmetic_argument 함수는 앞서서 콜백함수라고 설명하였다. 이 함수는 `'arithmetic_argument'`이라는 토픽 이름에 `ArithmeticArgument` 타입의 메시지를 서브스크라이브하게 되면 실행되는 함수이다.

서브스크라이브한 msg의 `argument_a`와 `argument_b`를 멤버 변수에 저장하고, `get_logger().info()` 함수를 이용하여 토픽으로 받은 시간, 변수 a, b 값을 화면에 표시하고 있다.

286 토픽 퍼블리셔, 서브스크라이버 복습!

설명이 좀 길었으니 여기서 정리하고 넘어가자. 토픽은 아래와 같이 설정하여 사용하면 된다!

1) 토픽 퍼블리셔 (데이터를 송신하는 프로그램)

- 1) Node 설정
- 2) QoS 설정
- 3) create_publisher 설정
- 4) 퍼블리시 함수 작성

2) 토픽 서브스크라이버 (데이터를 수신하는 프로그램)

- 1) Node 설정
- 2) QoS 설정
- 3) create_subscription 설정
- 4) 서브스크라이브 함수 작성

지난 강좌에서 소스 코드를 집중적으로 보기전에 빌드하여 노드들을 실행해보는 실습시간을 가졌다. 이번에는 이 실행 노드를 어떻게 설정해야 사용 가능하게 되는지 알아보도록 하자. 우선 이전 강의에서 익힌 관련된 두 가지의 노드 실행 명령어는 다음과 같다. **calculator**가 이 강좌에서 다룬 토픽 서브스크라이버 노드이고, **argument**는 토픽 퍼블리셔 노드이다.

```
$ ros2 run topic_service_action_rclpy_example calculator
```

```
$ ros2 run topic_service_action_rclpy_example argument
```

```
entry_points={
    'console_scripts': [
        'argument = topic_service_action_rclpy_example.arithmetic.argument:main',
        'operator = topic_service_action_rclpy_example.arithmetic.operator:main',
        'calculator = topic_service_action_rclpy_example.calculator.main:main',
        'checker = topic_service_action_rclpy_example.checker.main:main',
    ],
},
```

이 두 개의 노드는 지난 강좌의 "파이썬 패키지 설정 파일 (setup.py)" 의 "entry_points" 부분에서 이미 설명하였던 부분이다. `entry\_points`는 설치하여 사용할 실행 가능한 콘솔 스크립트의 이름과 호출 함수를 기입하도록 되어 있는데 우리는 **4개의 노드**를 작성하고 `ros2 run` 과 같은 노드 실행 명령어를 통하여 각각의 노드를 실행할 예정이기에 하기와 같이 `entry\_points`를 추가했었다.

argument 노드는 topic_service_action_rclpy_example 패키지의 arithmetic 폴더에 argument.py의 main문에 실행 코드가 담겨져 있고, calculator 노드는 topic_service_action_rclpy_example 패키지의 calculator 폴더에 main.py의 main문에 실행 코드가 담겨져 있다. 약간 다름에 주의하도록 하자.


```
def main(args=None):
    rclpy.init(args=args)
    try:
        argument = Argument()
        try:
            rclpy.spin(argument)
        except KeyboardInterrupt:
            argument.get_logger().info('Keyboard Interrupt (SIGINT)')
        finally:
            argument.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

우선 `argument` 노드부터 살펴보자. **main** 함수로 `rclpy.init`를 이용하여 초기화하고 위에서 작성한 `Argument` 클래스를 `argument`라는 이름으로 생성한 다음 `rclpy.spin` 함수를 이용하여 생성한 노드를 `spin`시켜 지정된 콜백함수가 실행될 수 있도록 하고 있다.

그리고 종료 `Ctrl + c`와 같은 인터럽트 시그널 예외 상황에서는 `argument`를 소멸시키고 **`rclpy.shutdown`** 함수로 노드를 종료하게 된다.

```

import rclpy
from rclpy.executors import MultiThreadedExecutor

from topic_service_action_rclpy_example.calculator.calculator import Calculator

def main(args=None):
    rclpy.init(args=args)
    try:
        calculator = Calculator()
        executor = MultiThreadedExecutor(num_threads=4)
        executor.add_node(calculator)
    try:
        executor.spin()
    except KeyboardInterrupt:
        calculator.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        executor.shutdown()
        calculator.arithmetic_action_server.destroy()
        calculator.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

calculator 노드를 살펴보자. main 함수로 rclpy.init를 이용하여 초기화하고 위에서 작성한 Calculator 클래스를 calculator라는 이름으로 생성한 다음 MultiThreadedExecutor로 스레드 4개를 사용하는 executor를 생성하였다.

여기서 생소한 **MultiThreadedExecutor**이 나왔는데 MultiThreadedExecutor는 스레드 풀(**thread pool**)을 사용하여 콜백을 실행하는 것이다. num_threads로 스레드 수를 지정 가능하며 이를 지정되지 않은 경우 multiprocessing.cpu_count()를 통해 시스템에서 가용한 스레드 수를 지정받게 된다. 이 둘 모두 해당되지 않는다면 단일 스레드를 사용하게 된다. 이 **Executor**는 콜백이 병렬로 발생하도록 허용하여 앞서 설명한 **ReentrantCallbackGroup**와 함께 사용하면 콜백함수를 병렬로 실행할 수 있게 된다. 그 말인즉슨 create_subscription(), create_service(), ActionServer(), create_timer() 등의 함수를 이용하여 설정한 토픽 퍼블리셔의 콜백함수, 서비스 서버의 콜백함수, 액션 서버의 콜백함수, 특정 타이머의 콜백함수 등의 병렬 처리를 설정하는 방법이라고 보면 된다. 이어서 설명하면 MultiThreadedExecutor으로 선언된 executor에 calculator 노드를 추가하고 executor.spin() 이용하여 생성한 노드를 spin시켜 지정된 콜백함수가 실행될 수 있도록 하고 있다.

그리고 종료 `Ctrl + c`와 같은 인터럽트 시그널 예외 상황에서는 executor, calculator의 액션 서버(액션 서버는 별도로 소멸시켜야함), calculator를 소멸시키고 rclpy.shutdown 함수로 노드를 종료하게 된다.

서비스 프로그래밍 (Python)

292 서비스 클라이언트 / 서비스 서버

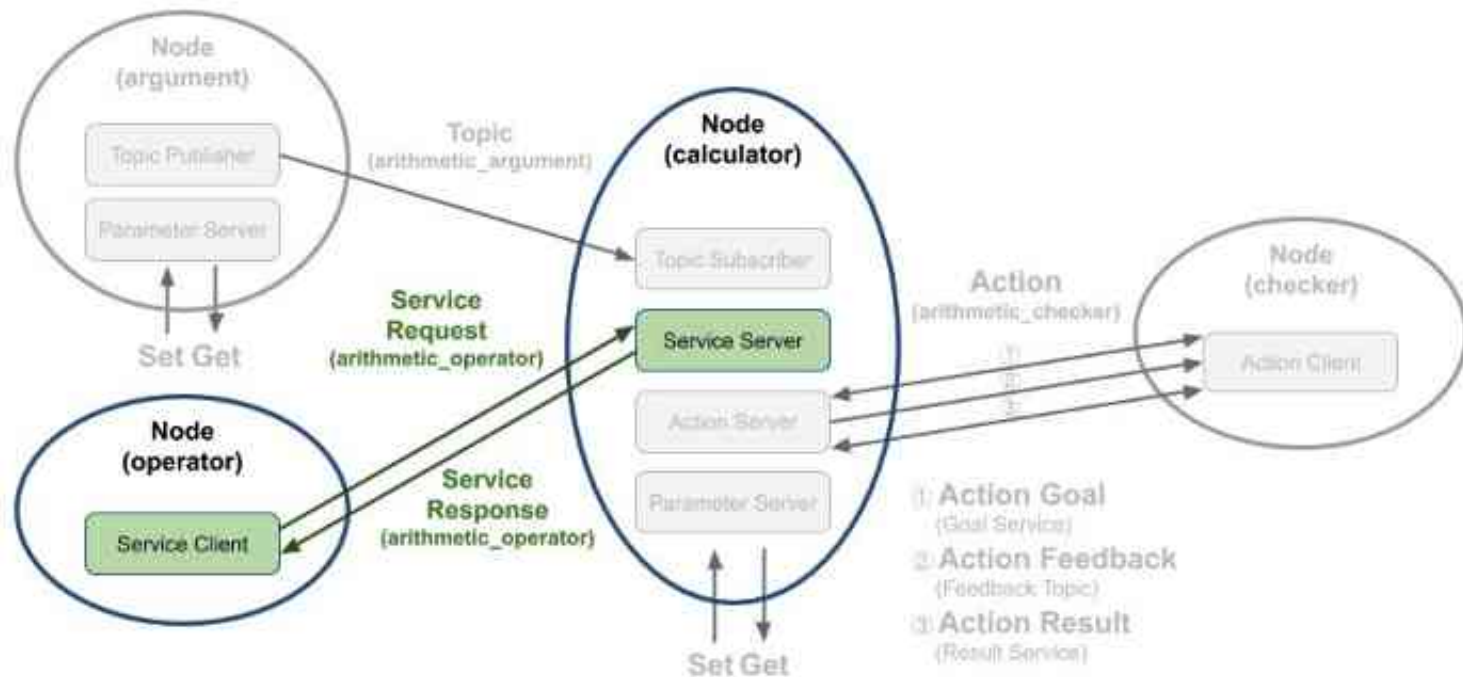
서비스 요청을 하는 서비스 클라이언트와 서비스 응답을 하는 서비스 서버를 작성해 볼 것이다. 여기서 서비스 요청 값으로는 연산자(+, -, *, /)를 임의로 선택 후에 보낼 것이고 기존에 저장한 변수 a, b를 요청 값으로 받은 연산자로 계산하여 결과값을 서비스 응답 값으로 보내는 프로그램을 짜볼 것이다. 강좌 진행에 앞서서 서비스에 대한 자세한 내용은 ROS 2 서비스 (service)' 강좌를 참고하도록 하자.

1) operator node

: arithmetic_operator 이라는 서비스 이름으로 calculator 노드에게 연산자(+, -, *, /)를 서비스 요청(Request) 값으로 보낸다.

2) calculator node

: 서브스크라이브하여 저장하고 있는 변수 a와 b와 operator 노드로부터 요청 값으로 받은 연산자를 이용하여 계산(a 연산자 b)하고 operator 노드에게 연산의 결과값을 서비스 응답(Response)값으로 보낸다.



```
self.arithmetic_service_server = self.create_service(  
    ArithmeticOperator,  
    'arithmetic_operator',  
    self.get_arithmetic_operator,  
    callback_group=self.callback_group)
```

calculator 노드는 서브스크라이브하여 저장하고 있는 변수 a와 b와 operator 노드로부터 요청 값으로 받은 연산자를 이용하여 계산(a 연산자 b)하고 operator 노드에게 연산의 결과값을 서비스 응답값으로 보낸다.

이 중 서비스 서버와 관련한 코드는 아래와 같다. 서버 관련 코드는 서비스 서버로 선언하는 부분과 콜백함수를 지정하는 것이다. arithmetic_service_server이 서비스 서버로 이는 Node 클래스의 create_service 함수를 이용하여 서비스 서버로 선언되었으며 서비스의 타입으로 ArithmeticOperator으로 선언하였고, 서비스 이름으로는 'arithmetic_operator', 서비스 클라이언트로부터 서비스 요청이 있으면 실행되는 콜백함수는 get_arithmetic_operator 으로 지정했으며 멀티 스레드 병렬 콜백함수 실행을 위해 지난번 강좌에서 설명한 callback_group 설정을 하였다.

이러한 설정들은 서비스 서버를 위한 기본 설정이고 **실제 서비스 요청에 해당되는 특정 수행 코드가 수행되는** 부분은 **get_arithmetic_operator** 이라는 콜백함수 임을 알아두자.

```
def get_arithmetic_operator(self, request, response):
    self.argument_operator = request.arithmetic_operator

    self.argument_result = self.calculate_given_formula(
        self.argument_a,
        self.argument_b,
        self.argument_operator)

    response.arithmetic_result = self.argument_result

    self.argument_formula = '{0} {1} {2} = {3}'.format(
        self.argument_a,
        self.operator[self.argument_operator-1],
        self.argument_b,
        self.argument_result)

    self.get_logger().info(self.argument_formula)

    return response
```

get_arithmetic_operator 함수의 내용을 보도록 하자. 우선 제일 먼저 **request**와 **response** 이라는 매개 변수가 보이는데 이는 **ArithmeticOperator()** 클래스로 생성된 인터페이스로 서비스 요청에 해당되는 **request** 부분과 응답에 해당되는 **response**으로 구분된다.

get_arithmetic_operator 함수는 서비스 요청이 있을 때 실행되는 콜백함수인데 여기서는 서비스 요청시 요청값으로 받은 연산자와 이전에 토픽 서브스크라이버가 토픽 값으로 전달받아 저장해둔 변수 **a, b**를 전달받은 연산자로 연산 후에 결과값을 서비스 응답값으로 반환한다.

이 함수의 첫 줄에서 **request.arithmetic_operator**를 받아와서 **calculate_given_formula** 함수에서 서비스 요청값으로 받은 연산자에 따라 연산하는 코드를 볼 수 있을 것이다. **calculate_given_formula** 함수로부터 받은 연산 결과값은 **response.arithmetic_result**에 저장하고 끝으로 관련 수식을 문자열로 표현하여 **get_logger().info()** 함수를 통해 화면에 표시하고 있다.

```
def calculate_given_formula(self, a, b, operator):
    if operator == ArithmeticOperator.Request.PLUS:
        self.argument_result = a + b
    elif operator == ArithmeticOperator.Request.MINUS:
        self.argument_result = a - b
    elif operator == ArithmeticOperator.Request.MULTIPLY:
        self.argument_result = a * b
    elif operator == ArithmeticOperator.Request.DIVISION:
        try:
            self.argument_result = a / b
        except ZeroDivisionError:
            self.get_logger().error('ZeroDivisionError!')
            self.argument_result = 0.0
        return self.argument_result
    else:
        self.get_logger().error(
            'Please make sure arithmetic operator(plus, minus, multiply, division).')
        self.argument_result = 0.0
    return self.argument_result
```

calculate_given_formula 함수로 앞서 설명한 것과 같이 인수 **a, b**를 가지고 주어진 연산자 **operator**에 따라 사칙연산을 하여 **결괏값**을 반환한다.


```
import rclpy
from rclpy.executors import MultiThreadedExecutor

from topic_service_action_rclpy_example.calculator.calculator import Calculator

def main(args=None):
    rclpy.init(args=args)
    try:
        calculator = Calculator()
        executor = MultiThreadedExecutor(num_threads=4)
        executor.add_node(calculator)
        try:
            executor.spin()
        except KeyboardInterrupt:
            calculator.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        executor.shutdown()
        calculator.arithmetic_action_server.destroy()
        calculator.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

서비스 서버인 `calculator` 노드는 토픽 서브스크라이버, 서비스 서버, 액션 서버를 역할을 하는 복합 기능의 노드로 실행 코드에 대한 설명은 '토픽 프로그래밍 (Python)'에서 이미 다루었기에 아래 코드에 대한 설명은 해당 강좌를 다시 보도록 하자.

```

import random

from msg_srv_action_interface_example.srv import ArithmeticOperator
import rclpy
from rclpy.node import Node

class Operator(Node):

    def __init__(self):
        super().__init__('operator')

        self.arithmetic_service_client = self.create_client(
            ArithmeticOperator,
            'arithmetic_operator')

        while not self.arithmetic_service_client.wait_for_service(timeout_sec=0.1):
            self.get_logger().warning('The arithmetic_operator service not available.')

    def send_request(self):
        service_request = ArithmeticOperator.Request()
        service_request.arithmetic_operator = random.randint(1, 4)
        futures = self.arithmetic_service_client.call_async(service_request)
        return futures

```

```

def main(args=None):
    rclpy.init(args=args)
    operator = Operator()
    future = operator.send_request()
    user_trigger = True
    try:
        while rclpy.ok():
            if user_trigger is True:
                rclpy.spin_once(operator)
                if future.done():
                    try:
                        service_response = future.result()
                    except Exception as e: # noqa: B902
                        operator.get_logger().warn('Service call failed: {}'.format(str(e)))
                    else:
                        operator.get_logger().info(
                            'Result: {}'.format(service_response.arithmetic_result))
                        user_trigger = False
            else:
                input('Press Enter for next service call.')
                future = operator.send_request()
                user_trigger = True

    except KeyboardInterrupt:
        operator.get_logger().info('Keyboard Interrupt (SIGINT)')

    operator.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

299 서비스 클라이언트 코드

```
class Operator(Node):

    def __init__(self):
        super().__init__('operator')

        self.arithmetic_service_client = self.create_client(
            ArithmeticOperator,
            'arithmetic_operator')

        while not self.arithmetic_service_client.wait_for_service(timeout_sec=0.1):
            self.get_logger().warning('The arithmetic_operator service not available.')
```

Operator 클래스이다. rclpy.node 모듈의 Node 클래스를 상속하고 있으며 생성자에서 'operator' 이라는 노드 이름으로 초기화되었다.

그 뒤 **arithmetic_service_client**이라는 이름으로 서비스 클라이언트를 선언해주는데 이는 Node 클래스의 **create_client** 함수를 이용하여 **서비스 클라이언트로 선언**하는 부분으로 **서비스의 타입**으로 서비스 서버와 동일하게 **ArithmeticOperator**으로 선언하였고, **서비스 이름**으로는 'arithmetic_operator'으로 선언하였다.

arithmetic_service_client 의 **wait_for_service** 함수는 서비스 요청을 할 수 있는 상태인지 알아보기 위해 서비스 서버가 실행되어 있는지 확인하는 함수로 **0.1 초 간격으로 서비스 서버가 실행되어 있는지 확인**하게 된다.

300 서비스 클라이언트 코드

```
def send_request(self):
    service_request = ArithmeticOperator.Request()
    service_request.arithmetic_operator = random.randint(1, 4)
    futures = self.arithmetic_service_client.call_async(service_request)
    return futures
```

우리가 작성하고 있는 서비스 클라이언트의 목적은 서비스 서버에게 연산에 필요한 연산자를 보내는 것이라고 했다.

이 **send_request** 함수가 실질적인 서비스 클라이언트의 실행 코드로 서비스 서버에게 서비스 요청값을 보내고 응답값을 받게 된다.

서비스 요청값을 보내기 위하여 제일 먼저 우리가 미리 작성해둔 서비스 인터페이스

ArithmeticOperator.Request() 클래스로 **service_request**를 선언하였고 서비스 요청값으로 **random.randint()** 함수를 이용하여 특정 연산자를 **self.request**의 **arithmetic_operator** 변수에 저장하였다.

그 뒤 '**call_async(self.request)**' 함수로 서비스 요청을 수행하게 설정하였다.

끝으로 서비스 상태 및 응답값을 담은 **futures**를 반환하게 된다.

```
entry_points={
    'console_scripts': [
        'argument = topic_service_action_rclpy_example.arithmetic.argument:main',
        'operator = topic_service_action_rclpy_example.arithmetic.operator:main',
        'calculator = topic_service_action_rclpy_example.calculator.main:main',
        'checker = topic_service_action_rclpy_example.checker.main:main',
    ],
},
```

서비스 클라이언트 노드인 operator 노드는 'topic_service_action_rclpy_example' 패키지의 일부로 패키지 설정 파일에 'entry_points'로 실행 가능한 콘솔 스크립트의 이름과 호출 함수를 기입하도록 되어 있는데 우리는 하기와 같이 4개의 노드를 작성하고 'ros2 run' 과 같은 노드 실행 명령어를 통하여 각각의 노드를 실행시키고 있다.

operator 노드는 topic_service_action_rclpy_example 패키지의 arithmetic 폴더에 operator.py의 main문에 실행 코드가 담겨져 있다.

```
def main(args=None):
    rclpy.init(args=args)
    operator = Operator()
    future = operator.send_request()
    user_trigger = True
    try:
        while rclpy.ok():
            if user_trigger is True:
                rclpy.spin_once(operator)
                if future.done():
                    try:
                        service_response = future.result()
                    except Exception as e: # noqa: B902
                        operator.get_logger().warn('Service call failed: {}'.format(str(e)))
                    else:
                        operator.get_logger().info(
                            'Result: {}'.format(service_response.arithmetic_result))
                        user_trigger = False
            else:
                input('Press Enter for next service call.')
                future = operator.send_request()
                user_trigger = True

    except KeyboardInterrupt:
        operator.get_logger().info('Keyboard Interrupt (SIGINT)')

    operator.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

즉, 다음이 main함수가 실행 코드인데 rclpy.init를 이용하여 초기화하고 위에서 작성한 **Operator** 클래스를 **operator**라는 이름으로 생성한 다음 **future = operator.send_request()** 와 같이 서비스 요청을 보내고 응답값을 받게된다.

그 뒤 **rclpy.spin_once** 함수를 이용하여 생성한 노드를 주기적으로 spin시켜 지정된 콜백함수가 실행될 수 있도록 하고 있다.

이때 매 spin마다 노드의 콜백함수가 실행되고 서비스 응답값을 받았을 때 **future**의 **done** 함수를 이용해 요청값을 제대로 받았는지 확인 후 결괏값은 **service_response = future.result()** 같이 **service_response**라는 변수에 저장하여 사용하게 된다.

서비스 응답값은 **get_logger().info()** 함수를 이용하여 화면에 서비스 응답값에 해당되는 연산 결괏값을 표시하는 것이다.

그리고 종료 **`Ctrl + c`**와 같은 인터럽트 시그널 예외 상황에서는 **operator**를 소멸시키고 **rclpy.shutdown** 함수로 노드를 종료하게 된다.

```
future = operator.send_request()
user_trigger = True

try:
    while rclpy.ok():
        if user_trigger is True:

            # 서비스 응답값 확인 코드

        else:
            input('Press Enter for next service call.')
            future = operator.send_request() # 서비스 요청값 전송
            user_trigger = True
```

참고로 서비스 클라이언트는 실행한 후 종료되는 컨셉으로 전 강좌에서 설명한 토픽과 같은 지속적인 수행은 하지는 않는다.

하지만 예제 작성을 위해 원하는 시점에 서비스 요청을 다시 전송하기 위하여 여기서는 `user_trigger` 변수와 `input('xxxxx')` 를 이용하여 처음 노드가 실행되면 최초 1회에 한해서는 바로 임의의 연산자를 서비스 요청값으로 서비스 서버에게 전달한 후 노드가 종료되기 전까지 사용자의 입력값을 받아 다시 임의의 연산자를 랜덤으로 골라 해당 연산자를 서비스 요청값으로 보내게 된다.

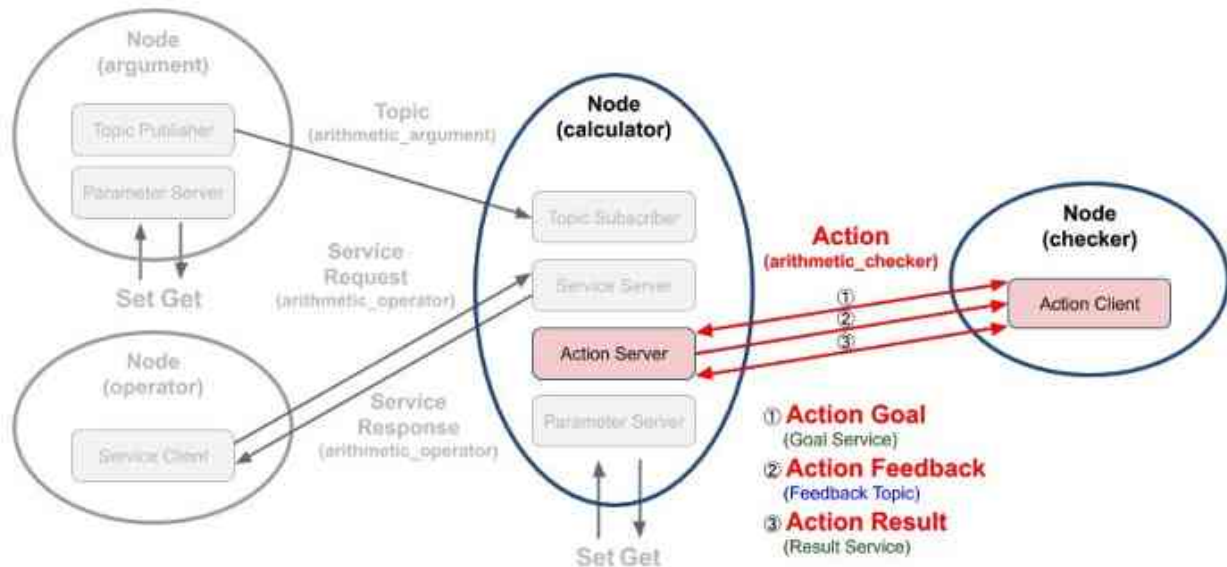
즉, 최초 서비스 요청값 전달 이후 서비스 요청을 다시하려면 `operator` 노드를 실행시킨 터미널 창에 엔터키를 누르면된다.

그러면 연산자 가 사칙 연산자(+, -, *, /) 중 랜덤으로 골라져 연산 결과값이 달라짐을 확인할 수 있다.

액션 프로그래밍 (Python)

305 액션 클라이언트 / 액션 서버

우리는 이 강좌에서 다음 그림과 같은 액션 목표(action goal)를 지정하는 **액션 클라이언트**와 액션 목표를 받아 특정 태스크를 수행하면서 중간 결괏값에 해당되는 액션 피드백(action feedback)과 최종 결괏값에 해당되는 액션 결과(action result)를 전송하는 **액션 서버**를 작성해 볼 것이다.



```
self.arithmetic_action_server = ActionServer(  
    self,  
    ArithmeticChecker,  
    'arithmetic_checker',  
    self.execute_checker,  
    callback_group=self.callback_group)
```

calculator 노드는 서브스크라이브하여 저장하고 있는 변수 a와 b와 operator 노드로부터 요청 값으로 받은 연산자를 이용하여 계산(a 연산자 b)하고 operator 노드에게 연산의 결과값을 서비스 응답값으로 보낸다는 것은 이전 강좌에서 설명하였다. 여기에 각 연산을 기록하고 액션 서버로 동작하는 부분에 대해 추가 설명을 하도록 하겠다.

액션 서버와 관련한 코드는 다음과 같다. 서버 관련 코드는 액션 서버로 선언하는 부분과 콜백함수를 지정하는 것이다. `arithmetic_action_server`이 액션 서버로 이는 `rclpy.action` 모듈의 `ActionServer` 클래스를 이용하여 액션 서버로 선언되었으며 액션의 타입으로 `ArithmeticChecker`으로 선언하였고, 액션 이름으로는 'arithmetic_checker', 액션 클라이언트로부터 액션 목표를 받으면 실행되는 콜백함수는 `execute_checker`으로 지정했으며 멀티 스레드 병렬 콜백함수 실행을 위해 지난 번 강좌에서 설명한 `callback_group` 설정을 하였다.

이러한 설정들은 액션 서버를 위한 기본 설정이고 실제 액션 목표를 받은 후에 실행되는 콜백함수는 `execute_checker` 함수임을 알아두자.

```
def execute_checker(self, goal_handle):
    self.get_logger().info('Execute arithmetic_checker action!')

    feedback_msg = ArithmeticChecker.Feedback()
    feedback_msg.formula = []
    total_sum = 0.0
    goal_sum = goal_handle.request.goal_sum

    while total_sum < goal_sum:
        total_sum += self.argument_result
        feedback_msg.formula.append(self.argument_formula)
        self.get_logger().info('Feedback: {0}'.format(feedback_msg.formula))
        goal_handle.publish_feedback(feedback_msg)
        time.sleep(1)

    goal_handle.succeed()

    result = ArithmeticChecker.Result()
    result.all_formula = feedback_msg.formula
    result.total_sum = total_sum

    return result
```

자 그러면 **execute_checker** 함수의 내용을 보도록 하자. 우선 제일 먼저 **goal_handle** 이라는 매개 변수가 보이는데 이는 rclpy.action 모듈의 **ServerGoalHandle** 클래스로 생성된 액션 상태 처리용 으로 execute, succeed, abort, canceled 등과 같이 액션 상태에 따른 관련 함수 호출이 가능하며 **publish_feedback**와 같이 피드백을 퍼블리시할 수도 있다.

이어서 함수 내용을 더 자세히 알아보도록 하자. 첫 줄에서는 **get_logger().info()** 함수를 이용해 터미널창에 액션 서버가 시작됨을 표시하고, 다음 줄에서 **ArithmeticChecker.Feedback()**으로 액션 피드백을 보낼 **feedback_msg** 변수를 선언하였다. 이어서 실제 피드백에 해당되는 **feedback_msg.formula**와 연산 합계값을 담을 **total_sum** 변수를 초기화하였다.

그 뒤 앞서 설명한 **goal_handle**를 이용하여 **goal_handle.request.goal_sum**에서 액션 목표값을 불러왔다.

```
while total_sum < goal_sum:
    total_sum += self.argument_result
    feedback_msg.formula.append(self.argument_formula)
    self.get_logger().info('Feedback: {0}'.format(feedback_msg.formula))
    goal_handle.publish_feedback(feedback_msg)
    time.sleep(1)
```

```
goal_handle.succeed()
result = ArithmeticChecker.Result()
result.all_formula = feedback_msg.formula
result.total_sum = total_sum
return result
```

다음 **while** 반복 구문은 액션 목표값(**goal_sum**)과 **get_arithmetic_operator**를 통해 매해 계산되는 연산 결과값인 **argument_result**를 누적한 합계(**total_sum**) 값을 비교하여 액션 목표값을 넘을 때까지의 연산식(**argument_formula**)을 액션 피드백(**feedback_msg.formula**)에 저장하는 구문을 볼 수 있다. 이 피드백 값은 디버깅을 위해 **get_logger().info()**을 통해 터미널창에 출력하고 **goal_handle.publish_feedback()** 함수를 통해 액션 클라이언트에서 전송하게 된다.

마지막으로 아래 구문들을 통해 액션 목표를 달성했다는 상태 전환 함수인 **goal_handle.succeed()**를 실행시켜 액션 클라이언트에게 현재의 액션 상태를 알린다. 그리고 액션 결과값인 **all_formula**에 계산식 전체를 저장하고 **total_sum**에 연산 합계를 저장하여 액션 결과값인 **result**은 리턴하는 것으로 액션 서버의 역할을 마치게 된다.

```
import rclpy
from rclpy.executors import MultiThreadedExecutor

from topic_service_action_rclpy_example.calculator.calculator import Calculator

def main(args=None):
    rclpy.init(args=args)
    try:
        calculator = Calculator()
        executor = MultiThreadedExecutor(num_threads=4)
        executor.add_node(calculator)
    try:
        executor.spin()
    except KeyboardInterrupt:
        calculator.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        executor.shutdown()
        calculator.arithmetic_action_server.destroy()
        calculator.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

액션 서버인 `calculator` 노드는 토픽 서브스크라이버, 서비스 서버, 액션 서버를 역할을 하는 복합 기능의 노드로 실행 코드에 대한 설명은 토픽 프로그래밍 (Python)' 에서 이미 다루었기에 아래 코드에 대한 설명은 해당 강좌를 다시 보도록 하자.

310 액션 클라이언트 코드

```

from action_msgs.msg import GoalStatus
from msg_srv_action_interface_example.action import ArithmeticChecker
from rclpy.action import ActionClient
from rclpy.node import Node

class Checker(Node):

    def __init__(self):
        super().__init__('checker')
        self.arithmetic_action_client = ActionClient(
            self,
            ArithmeticChecker,
            'arithmetic_checker')

    def send_goal_total_sum(self, goal_sum):
        wait_count = 1
        while not self.arithmetic_action_client.wait_for_server(timeout_sec=0.1):
            if wait_count > 3:
                self.get_logger().warning('Arithmetic action server is not available.')
                return False
            wait_count += 1
        goal_msg = ArithmeticChecker.Goal()
        goal_msg.goal_sum = (float)(goal_sum)
        self.send_goal_future = self.arithmetic_action_client.send_goal_async(
            goal_msg,
            feedback_callback=self.get_arithmetic_action_feedback)
        self.send_goal_future.add_done_callback(self.get_arithmetic_action_goal)
        return True

```

```

def get_arithmetic_action_goal(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().warning('Action goal rejected.')
        return
    self.get_logger().info('Action goal accepted.')
    self.action_result_future = goal_handle.get_result_async()
    self.action_result_future.add_done_callback(self.get_arithmetic_action_result)

def get_arithmetic_action_feedback(self, feedback_msg):
    action_feedback = feedback_msg.feedback.formula
    self.get_logger().info('Action feedback: {0}'.format(action_feedback))

def get_arithmetic_action_result(self, future):
    action_status = future.result().status
    action_result = future.result().result
    if action_status == GoalStatus.STATUS_SUCCEEDED:
        self.get_logger().info('Action succeeded!')
        self.get_logger().info(
            'Action result(all formula): {0}'.format(action_result.all_formula))
        self.get_logger().info(
            'Action result(total sum): {0}'.format(action_result.total_sum))
    else:
        self.get_logger().warning(
            'Action failed with status: {0}'.format(action_status))

```

311 액션 클라이언트 코드

```
class Checker(Node):  
  
    def __init__(self):  
        super().__init__('checker')  
        self.arithmetic_action_client = ActionClient(  
            self,  
            ArithmeticChecker,  
            'arithmetic_checker')
```

우선 **Checker** 클래스인데 `rclpy.node` 모듈의 **Node** 클래스를 상속하고 있으며 생성자에서 '**checker**'이라는 노드 이름으로 초기화되었다.

그 뒤 **arithmetic_action_client**이라는 이름으로 **액션 클라이언트**를 선언 해주는데 이는 `rclpy.action` 모듈의 **ActionClient** 클래스 이용하여 **액션 클라이언트**로 선언하는 부분으로 **액션의 타입**으로 액션 서버와 동일하게 **ArithmeticChecker**으로 선언하였고, **액션 이름**으로는 '**arithmetic_checker**'으로 선언하였다.

액션 클라이언트에서 수행하는 **액션 목표**는 항상 퍼블리시하고 서브스크라이브하는 토픽과는 달리 필요시 비정기적으로 수행하는 것이다.

여기서는 예시를 위해 액션 목표를 `main` 함수에서 한번 실행하게 된다. 이 부분은 이어지는 설명에서 다루겠다.

다음으로 **액션 목표 전송**, **액션 상태 파악**, **피드백 및 결과값 받기**를 위한 함수를 알아보자.

312 액션 클라이언트 코드

```
def send_goal_total_sum(self, goal_sum):
    wait_count = 1
    while not self.arithmetic_action_client.wait_for_server(timeout_sec=0.1):
        if wait_count > 3:
            self.get_logger().warning('Arithmetic action server is not available.')
            return False
        wait_count += 1
    goal_msg = ArithmeticChecker.Goal()
    goal_msg.goal_sum = (float)(goal_sum)
    self.send_goal_future = self.arithmetic_action_client.send_goal_async(
        goal_msg,
        feedback_callback=self.get_arithmetic_action_feedback)
    self.send_goal_future.add_done_callback(self.get_arithmetic_action_goal)
    return True
```

```
wait_count = 1
while not self.arithmetic_action_client.wait_for_server(timeout_sec=0.1):
    if wait_count > 3:
        self.get_logger().warning('Arithmetic action server is not available.')
        return False
    wait_count += 1
```

send_goal_total_sum 함수는 액션 목표를 액션 서버에게 전송하고, 액션 피드백 및 결과값을 받기 위한 콜백함수를 지정하는 함수로 사용하고 있다.

이부분은 액션 클라이언트가 액션 서버에 연결시도를 하는 것으로 연결에 문제가 있을 때에 **while**문을 반복하게 되고 문제 없이 연결되었을 때에는 다음 구문으로 넘어가게 된다.

313 액션 클라이언트 코드

```
goal_msg = ArithmeticChecker.Goal()
goal_msg.goal_sum = (float)(goal_sum)
self.send_goal_future = self.arithmetic_action_client.send_goal_async(
    goal_msg,
    feedback_callback=self.get_arithmetic_action_feedback)
self.send_goal_future.add_done_callback(self.get_arithmetic_action_goal)
```

그 뒤 `ArithmeticChecker.Goal()` 클래스로 액션 메시지 `goal_msg`를 선언하고, `goal_msg.goal_sum`과 같이 액션 목표값을 설정하게 된다.

그리고 `ActionClient` 클래스의 `send_goal_async` 함수를 이용하여 미리 설정해둔 액션 메시지를 매개변수로 넣고 액션 피드백을 전달 받기 위한 콜백함수로 `get_arithmetic_action_feedback` 함수를 지정하였다.

마지막으로 `send_goal_async`를 통해 선언된 비동기 `future task`인 액션 목표값 전달 함수 `send_goal_future`의 `add_done_callback` 함수를 통해 액션 결과값을 받을 때 사용할 콜백함수로 `get_arithmetic_action_goal`를 선언하였다.

- | | |
|----------------------|---|
| (1) 액션 클라이언트 선언 | : <code>arithmetic_action_client</code> |
| (2) 액션 목표값 전달 함수 선언 | : <code>send_goal_future</code> |
| (3) 액션 피드백값 콜백 함수 선언 | : <code>get_arithmetic_action_feedback</code> |
| (4) 액션 상태값 콜백 함수 선언 | : <code>get_arithmetic_action_goal</code> |
| (5) 액션 결과값 콜백 함수 선언 | : <code>get_arithmetic_action_result</code> |

314 액션 클라이언트 코드

```
def get_arithmetic_action_feedback(self, feedback_msg):
    action_feedback = feedback_msg.feedback.formula
    self.get_logger().info('Action feedback: {0}'.format(action_feedback))
```

```
def get_arithmetic_action_goal(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().warning('Action goal rejected.')
        return
    self.get_logger().info('Action goal accepted.')
    self.action_result_future = goal_handle.get_result_async()
    self.action_result_future.add_done_callback(self.get_arithmetic_action_result)
```

"액션 피드백값 콜백 함수"는 다음과 같다. 액션 피드백을 액션 서버로부터 전달 받으면 `get_arithmetic_action_feedback` 콜백함수가 실행되는데 피드백인 `feedback_msg.feedback.formula` 값을 받아 `get_logger().info()` 함수를 이용해 터미널창에 출력하는 것이다.

다음은 "액션 상태값 콜백 함수" 이다. 이 함수는 비동기 future task로 선언된 `send_goal_future`의 `add_done_callback` 함수를 통해 콜백함수로 선언된 함수로 액션 서버가 액션 목표값을 전달 받고 ROS 2 액션 (action)' 강좌에서 설명하였던 **Goal State Machine**의 **accepted** 상태일 때를 확인하여 처리하는 구문이다. 액션 목표값을 문제없이 전달하였다면 액션 결과값 콜백 함수를 선언하게 된다. 여기서 콜백 함수는 후술할 `get_arithmetic_action_result` 함수이다.

```
def get_arithmetic_action_result(self, future):
    action_status = future.result().status
    action_result = future.result().result
    if action_status == GoalStatus.STATUS_SUCCEEDED:
        self.get_logger().info('Action succeeded!')
        self.get_logger().info(
            'Action result(all formula): {0}'.format(action_result.all_formula))
        self.get_logger().info(
            'Action result(total sum): {0}'.format(action_result.total_sum))
    else:
        self.get_logger().warning(
            'Action failed with status: {0}'.format(action_status))
```

앞서 지정한 "액션 결과값 콜백 함수"는 다음과 같다. 비동기 future task로 현재의 상태값(status)과 결과값(result)을 받고, 상태 값이 STATUS_SUCCEEDED 일 때 액션 서버로부터 전달 받은 액션 결과값인 계산식(action_result.all_formula)과 연산 합계(action_result.total_sum)를 터미널 창에 출력하게 된다.

316 액션 클라이언트 노드 실행 코드

```
entry_points={
    'console_scripts': [
        'argument = topic_service_action_rclpy_example.arithmetic.argument:main',
        'operator = topic_service_action_rclpy_example.arithmetic.operator:main',
        'calculator = topic_service_action_rclpy_example.calculator.main:main',
        'checker = topic_service_action_rclpy_example.checker.main:main',
    ],
},
```

액션 클라이언트 노드인 checker 노드는 `topic\_service\_action\_rclpy\_example` 패키지의 일부로 패키지 설정 파일에 `entry\_points`로 실행 가능한 콘솔 스크립트의 이름과 호출 함수를 기입하도록 되어 있는데 우리는 하기와 같이 4개의 노드를 작성하고 `ros2 run` 과 같은 노드 실행 명령어를 통하여 각각의 노드를 실행시키고 있다.

checker 노드는 topic_service_action_rclpy_example 패키지의 checker 폴더에 main.py의 main문에 실행 코드가 담겨져 있다.

317 액션 클라이언트 노드 실행 코드

```
import argparse
import sys

import rclpy

from topic_service_action_rclpy_example.checker.checker import Checker

def main(argv=sys.argv[1:]):
    parser = argparse.ArgumentParser(formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument(
        '-g',
        '--goal_total_sum',
        type=int,
        default=50,
        help='Target goal value of total sum')
    parser.add_argument(
        'argv', nargs=argparse.REMAINDER,
        help='Pass arbitrary arguments to the executable')
    args = parser.parse_args()

    rclpy.init(args=args.argv)
    try:
        checker = Checker()
        checker.send_goal_total_sum(args.goal_total_sum)
        try:
            rclpy.spin(checker)
        except KeyboardInterrupt:
            checker.get_logger().info('Keyboard Interrupt (SIGINT)')
        finally:
            checker.arithmetic_action_client.destroy()
            checker.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

main 함수가 실행 코드인데 `rclpy.init`를 이용하여 초기화하고 위에서 작성한 **Checker** 클래스를 **checker**라는 이름으로 생성한 다음 **액션 목표값을 전달하는 `send_goal_total_sum` 함수**를 실행시키고 있다.

여기서 `args.goal_total_sum`와 같이 프로그램의 실행 인자를 사용하는 부분이 있는데 이는 추후 강좌에서 자세히 다루도록 하겠다. 우선 실행 인자로 사용자가 노드를 실행시킬 때 액션 목표값을 설정할 수 있고 지정하지 않았을 때에는 50이라는 초깃값이 입력된다는 것만 알아두자.

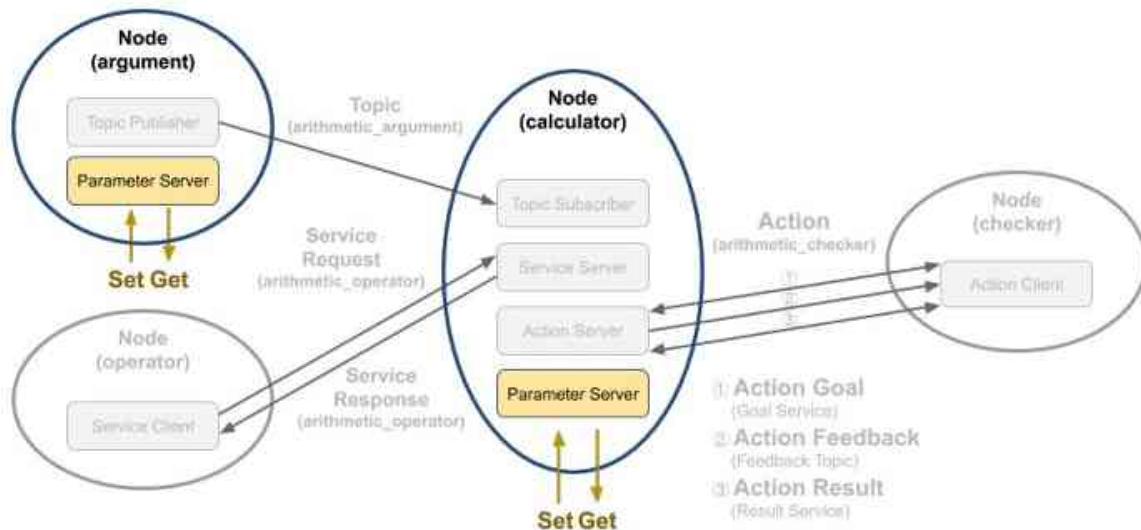
목표값 전달 이후에는 `rclpy.spin` 함수를 이용하여 `rclpy`의 콜백함수가 실행될 수 있도록 하고 있다.

그 이외에는 지금까지 다른 강좌에서 설명한 바와 같이 종료 ``Ctrl + c``와 같은 인터럽트 시그널 예외 상황에서는 `checker`를 소멸시키고 `rclpy.shutdown` 함수로 노드를 종료하게 된다. 약간의 차이점이 있다면 다른 토픽과 서비스와는 달리 액션은 별도로 ``checker.arithmetic_action_client.destroy()``와 같이 소멸시켜야 한다는 것이다.

파라미터 프로그래밍 (Python)

319 파라미터 (parameter)

우리는 이전 강좌에서 토픽, 서비스, 액션 관련 프로그래밍을 익히기 위하여 **argument**, **operator**, **calculator**, **checker** 노드를 작성해 보았다. 이들 노드 중에서 다음 그림과 같이 **argument** 노드와 **calculator** 노드는 파라미터를 사용하고 있다. **argument** 노드는 QoS 설정과 랜덤으로 생성되는 변수 **a**, **b**의 랜덤 생성 범위를 파라미터를 이용했었고 **calculator** 노드는 QoS 설정을 사용하였다. 우리는 여기서 **argument** 노드에서 사용되는 파라미터에 대해 자세히 알아볼 것이다.



320 파라미터 설정

argument 노드에서 파라미터를 선언하고 파라미터 값이 변경되는 함수에 대해 자세히 알아보도록 하자. **Argument** 클래스의 생성자 부분에서 다음과 같은 코드가 있다. ROS 2에서 파라미터를 사용하려면 하기와 같이 크게 3가지의 요소가 필요하다.

1) **declare_parameter** 함수

2) **get_parameter** 함수

3) **add_on_set_parameters_callback** 함수

1)번의 **declare_parameter** 함수는 노드에서 사용할 파라미터의 고유 이름을 지정하고 초깃값을 설정한다. 파라미터를 설명하는 **descriptor**를 추가하기도 하는데 여기서는 생략했다.

2)번의 **get_parameter** 함수는 노드에서 사용할 파라미터의 값을 불러오는 것으로 선언된 파라미터 고유 이름을 이용하여 지정된 파라미터 값을 불러온다. 이는 주로 **launch** 파일에서 선언된 *.yaml 형태의 파라미터 파일의 값을 불러오는데 사용된다.

3)번의 **add_on_set_parameters_callback** 함수는 서비스 형태로 파라미터 변경 요청이 있을 때 사용되는 함수로 지정된 콜백 함수를 호출하게 된다.


```
class Argument(Node):

    def __init__(self):
        super().__init__('argument')
        self.declare_parameter('qos_depth', 10)
        qos_depth = self.get_parameter('qos_depth').value
        self.declare_parameter('min_random_num', 0)
        self.min_random_num = self.get_parameter('min_random_num').value
        self.declare_parameter('max_random_num', 9)
        self.max_random_num = self.get_parameter('max_random_num').value
        self.add_on_set_parameters_callback(self.update_parameter)
```

```
def update_parameter(self, params):
    for param in params:
        if param.name == 'min_random_num' and param.type_ == Parameter.Type.INTEGER:
            self.min_random_num = param.value
        elif param.name == 'max_random_num' and param.type_ == Parameter.Type.INTEGER:
            self.max_random_num = param.value
    return SetParametersResult(successful=True)
```

322 파라미터 설정

```
self.declare_parameter('max_random_num', 9)
self.max_random_num = self.get_parameter('max_random_num').value
```

예를 들어, 하기와 같이 **declare_parameter** 함수로 'max_random_num'와 같은 파라미터를 선언하였을 때, 노드 실행시 **get_parameter** 함수로 지정된 파라미터 파일로부터 파라미터 초깃값을 가져와 설정하게 된다.

그 뒤 파라미터 변경 요청이 있을 때에 **add_on_set_parameters_callback**로부터 지정된 콜백 함수인 **update_parameter** 함수를 실행하게 된다. **update_parameter** 콜백 함수에서는 변경하려는 파라미터의 이름과 파라미터 타입이 같을 때에 해당 파라미터 값을 변경하게 된다.

```
def publish_random_arithmetic_arguments(self):
    msg = ArithmeticArgument()
    msg.stamp = self.get_clock().now().to_msg()
    msg.argument_a = float(random.randint(self.min_random_num,
    self.max_random_num))
    msg.argument_b = float(random.randint(self.min_random_num,
    self.max_random_num))
    self.arithmetic_argument_publisher.publish(msg)
    self.get_logger().info('Published argument a: {0}'.format(msg.argument_a))
    self.get_logger().info('Published argument b: {0}'.format(msg.argument_b))
```

이 argument 노드에서는 파라미터 값으로 QoS 설정과 랜덤으로 생성되는 변수 a, b의 랜덤 생성 범위를 파라미터를 이용했는데 하기와 같이 파라미터 min_random_num 값과 max_random_num 값을 이용하여 퍼블리시할 때 변수 a, b의 랜덤 생성 범위를 바꾸는 함수를 넣어 줬다.

323 파라미터 사용 방법 (CLI)

```
$ ros2 run topic_service_action_rclpy_example argument
[INFO]: Published argument a: 8.0
[INFO]: Published argument b: 6.0
[INFO]: Published argument a: 9.0
[INFO]: Published argument b: 4.0
...
```

```
$ ros2 param list
/argument:
  max_random_num
  min_random_num
  qos_depth
  use_sim_time
```

```
$ ros2 param get /argument max_random_num
Integer value is: 9
```

```
$ ros2 param set /argument max_random_num 100
Set parameter successful
```

```
[INFO]: Published argument a: 42.0
[INFO]: Published argument b: 52.0
[INFO]: Published argument a: 51.0
[INFO]: Published argument b: 65.0
...
```

324 파라미터 사용 방법 (서비스 클라이언트)

위 설명에서는 **CLI**를 이용하여 파라미터를 조회하고, 변경하고 읽어보는 실습을 해보았다. 파라미터는 CLI 이외에도 다른 노드의 소스 코드내에서도 파라미터를 읽고 변경할 수 있다. 예를 들어 다음의 코드는 **SetParameters** 이라는 인터페이스를 이용하여 서비스 클라이언트와 비슷한 설정으로 서비스 클라이언트로 서비스 요청을 통해 파라미터를 변경하게 된다.

여기서 클라이언트 선언 부분 및 서비스를 요청하는 부분은 서비스 클라이언트와 완전히 동일하다. 다른 부분을 꼽자면 **서비스 요청 값**으로 **파라미터의 이름**과 **형태, 값**을 지정하는게 다르다. 이 부분은 `set_max_random_num_parameter` 함수에 자세히 기술되어 있는데 **Parameter** 클래스를 선언하여 매개변수로 **name, type, integer_value** 등이 설정하게 되어 있다. 이를 통해 A 노드에서 B 노드의 파라미터를 변경할 수 있게 된다.

325 파라미터 사용 방법 (서비스 클라이언트)

```
from rcl_interfaces.msg import Parameter
from rcl_interfaces.msg import ParameterType
from rcl_interfaces.msg import ParameterValue
from rcl_interfaces.srv import SetParameters
```

```
self.random_num_parameter_client = self.create_client(
    SetParameters,
    'argument/set_parameters')
```

```
def set_max_random_num_parameter(self, max_value):
    request = SetParameters.Request()
    parameter = ParameterValue(type=ParameterType.PARAMETER_INTEGER, integer_value=max_value)
    request.parameters = [Parameter(name='max_random_num', value=parameter)]
    service_client = self.random_num_parameter_client
    return self.call_service(service_client, request, 'max_random_num parameter')
```

```
def call_service(self, service_client, request, service_name):
    wait_count = 1
    while not service_client.wait_for_service(timeout_sec=0.1):
        if wait_count > 3:
            self.get_logger().warn(service_name + ' service is not available.')
            return False
        wait_count += 1
    service_client.call_async(request)
    return True
```

326 기본 파라미터 설정 방법 (param 설정)

위와 같이 파라미터를 선언하고 파라미터를 읽어(get)오고, 파라미터를 변경(set)하는 방법으로 파라미터를 사용할 수 있는데 이는 한 두개의 파라미터의 경우에는 어려움 없이 사용 가능하다. 하지만 **하나의 노드에 수 십개의 파라미터, 그리고 수 십개의 노드가 실행되고 모두 파라미터를 이용한다면 관리 해야할 파라미터가 많아져서 매번 설정하기도 힘들다.** 이에 파라미터를 저장하고 노드 실행 시 지정된 파라미터로 초깃값을 주고 싶을 때에는 **파라미터 파일 (*.yaml)**을 미리 작성하고 이를 launch 파일에서 불러오는 방법이 있어서 소개한다.

우선 하기와 같은 파라미터 파일을 **`arithmetic\_config.yaml`** 이라는 이름으로 지정 후에 패키지의 **`param`** 폴더에 넣어준다. 위 설명해서 익히 본 3개의 파라미터 이름과 초깃값이 기재되어 있음을 확인할 수 있다.

```
/**: # namespace and node name
ros__parameters:
  qos_depth: 30
  min_random_num: 0
  max_random_num: 9
```

```
namespace:
node_name:
ros__parameters:
  qos_depth: 30
  min_random_num: 0
  max_random_num: 9
```

327 기본 파라미터 설정 방법 (launch 설정)

```
def generate_launch_description():
    param_dir = LaunchConfiguration(
        'param_dir',
        default=os.path.join(
            get_package_share_directory('topic_service_action_rclpy_example'),
            'param',
            'arithmetic_config.yaml'))

    return LaunchDescription([
        DeclareLaunchArgument(
            'param_dir',
            default_value=param_dir,
            description='Full path of parameter file'),

        Node(
            package='topic_service_action_rclpy_example',
            executable='argument',
            name='argument',
            parameters=[param_dir],
            output='screen'),

        Node(
            package='topic_service_action_rclpy_example',
            executable='calculator',
            name='calculator',
            parameters=[param_dir],
            output='screen'),

    ])
```

```
(share_dir + '/launch', glob.glob(os.path.join('launch', '*.launch.py'))),
(share_dir + '/param', glob.glob(os.path.join('param', '*.yaml'))),
```

* 참고로 새롭게 지정된 *.yaml 파일 및 *.launch.py 파일을 ROS 파일 시스템에 맞추어 설치하게 하려면 하기와 같이 파이썬 패키지 설정 파일 `setup.py`에 옵션을 추가해야 한다.

launch 파일에 특정 파라미터 파일을 추가 해주면 해당 launch 파일로 노드를 실행할 때에는 지정 파라미터 파일의 파라미터 이름과 파라미터 값을 확인하여 해당 노드의 파라미터를 초기화하여 사용할 수 있다.

실행 인자 프로그래밍 (Python)

329 실행 인자

프로그램의 실행 명령어와 함께 실행 시에 사용될 인수를 추가하여 프로그램을 실행하는 경우가 있는데 이때 사용되는 인자를 **실행 인자**라고 하고 `main` 함수에서 매개변수로 사용된다.

예를 들어 `ROS 2 패키지 설계 (Python)`에서 하기와 같은 명령어를 통해 `GOAL_TOTAL_SUM` 값을 100으로 설정할 수 있었다. 여기서 `ros2 run`가 명령어이고 `topic_service_action_rclpy_example` 패키지의 `checker` 노드를 실행하라는 의미이다. 여기에 추가로 `-g 100` 이 실행 인자로 사용되었다. 이 강좌에서는 이렇게 실행시에 사용되는 실행 인자를 이용한 프로그래밍에 대해 알아보자.

* 참고로 파라미터(parameter)는 매개 변수로 풀이하고 아규먼트(argument)는 실행 인자라 풀이한다. C++ 언어에서는 이들의 분류를 더 확실하게 하는 편인데 `Parameter`는 함수 선언시 사용되고 `Argument`는 함수 호출시의 인수라고 생각하면 된다.

- Parameter: 매개 변수
- Argument: 실행 인자

```
$ ros2 run topic_service_action_rclpy_example checker -g 100
```

330 ROS 2에서의 실행 인자 처리

```
int main(int argc, char * argv[])  
{  
    rclcpp::init(argc, argv);  
    (이하 생략)
```

ROS 2에서의 실행 인자 처리는 하기의 예제들과 같이 처리된다. C++ 언어의 경우 main 문에서 argc라고 argument의 수를 받고, argv로 인자들을 배열로 받은 후 rclcpp의 init 함수의 argument로 넘겨주게 된다.

```
def main(args=None):  
    rclpy.init(args=args)  
    (이하 생략)
```

Python의 경우도 비슷한데 인수들을 무시할 때에는 다음 예제와 같이 args를 None으로 설정 후에 rclpy 모듈의 init 함수에 바로 넘기게 된다.

```
def main(argv=sys.argv[1:]):  
    (argparse 구문 추가)  
    rclpy.init(args=argv)  
    (이하 생략)
```

만약 실행 인자를 사용하고자 한다면 첫 argv인 실행명 및 실행 경로인 첫번째 인자를 삭제한 것을 argv에 저장하고 이를 rclpy 모듈의 init 함수에 넘기게 된다. 이때에 C++과는 달리 argparse 모듈을 이용하여 실행 인자를 위한 구문 해석 프로그램을 작성해야한다.

* 참고로 argc, argv, args는 다음과 같은 의미로 사용된다.

- argc: argument count
- argv: argument vector or value

331 실행 인자의 구문 해석

```
import argparse
import sys

import rclpy

from topic_service_action_rclpy_example.checker.checker import Checker

def main(argv=sys.argv[1:]):
    parser =
    argparse.ArgumentParser(formatter_class=argparse.ArgumentDefaultsHelpFormatter)
    parser.add_argument(
        '-g',
        '--goal_total_sum',
        type=int,
        default=50,
        help='Target goal value of total sum')
    parser.add_argument(
        'argv', nargs=argparse.REMAINDER,
        help='Pass arbitrary arguments to the executable')
    args = parser.parse_args()

    rclpy.init(args=args.argv)
    try:
        checker = Checker()
        checker.send_goal_total_sum(args.goal_total_sum)
    try:
        rclpy.spin(checker)
    except KeyboardInterrupt:
        checker.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        checker.arithmetic_action_client.destroy()
        checker.destroy_node()
    finally:
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

실행 인자의 구문 해석과 관련한 부분은 Checker 노드의 main 함수 부분에서 구현되어 있기에 관련 코드를 살펴보며 실행 인자를 다루어 볼 것이다. 우선 전체 소스 코드는 다음과 같다.

실행 인자의 구문해석 프로그램은 파이썬의 argparse 모듈을 이용하여 파서를 선언 후 사용할 실행 인자 값을 지정하는 것이 주를 이루게 된다. 이를 순서대로 나열하면 다음과 같다. 몇 줄 되지 않는 구문이니 순서대로 알아보자.

- 1) 파서 만들기 (parser = argparse.ArgumentParser)
- 2) 인자 추가하기 (parser.add_argument)
- 3) 인자 파싱하기 (args = parser.parse_args())
- 4) 인자 사용하기 (args.xxx)

332 실행 인자의 구문 해석

```
def main(argv=sys.argv[1:]):
    parser = argparse.ArgumentParser(
        formatter_class=argparse.ArgumentDefaultsHelpFormatter)
```

```
parser.add_argument(
    '-g',
    '--goal_total_sum',
    type=int,
    default=50,
    help='Target goal value of total sum')
```

```
args = parser.parse_args()
```

```
checker.send_goal_total_sum(args.goal_total_sum)
```

1) 파서 만들기

우선 argparse 모듈의 ArgumentParser 객체를 parser라는 이름으로 선언하자. 여기서 formatter_class으로 argparse 모듈의 가장 기본적인 형식을 사용하도록 설정하였다.

2) 인자 추가하기

다음으로 인자 추가하기에 대해 알아보도록 하겠다. 실행 인자로 사용할 인자를 추가하려면 하기와 같이 add_argument() 메서드를 호출하고 인자의 내용을 채운다. 하기 예제에서는 인자 이름으로 줄여서는 '-g', 풀네이밍으로는 '--goal_total_sum'을 사용한다고 선언하였고, 형태로는 int형, 기본 값은 50을 지정하였으며 간단히 지정 인자의 설명을 넣어주었다. 이 설명은 프로그램을 실행할 때 '-h'와 같이 실행 인자에 대한 도움말을 실행하면 볼 수 있는 문구이다.

3) 인자 파싱하기

사용할 인자를 추가했으면 다음으로 parse_args() 메서드를 통해 인자를 파싱하면 된다. 사용 방법은 다음과 같이 매우 간단하다. 이것으로 간단한 사용을 위한 기본 설정은 모두 끝났다.

4) 인자 사용하기

인자를 사용하려면 하기와 같이 인자를 파싱하여 대입한 args의 변수처럼 인자를 사용하면 된다. 예를 들어 add_argument를 통해 인자로 추가했던 '--goal_total_sum'은 `args.goal\_total\_sum` 처럼 사용할 수 있게 된다.

런치 프로그래밍

334 ROS 2 Launch System

ROS 2에서 하나의 노드를 실행시키기 위해서는 `ros2 run`이라는 명령어를 사용한다. 이는 특정 패키지의 특정 하나의 노드를 실행하라는 명령어이다. 이 명령어만으로도 노드를 실행시키는 것에는 큰 문제가 없다. 하지만 ROS에서는 하나의 노드만을 실행시키는 일 보다 복수의 노드를 함께 실행시켜 노드 간의 메시지를 주고 받게 되는 경우가 더 많다. 그리고 직접 개발한 패키지의 노드만을 실행하기도 하지만 이미 개발되어 공개된 패키지의 노드들을 사용하는 경우도 많고 각종 옵션을 함께 입력하여 사용하게 된다.

ROS 2에서는 이를 위해 `launch`라는 개념이 있는데 이는 하나 이상의 정해진 노드를 실행시킬 수 있다. 더불어, 노드를 실행할 때 패키지의 매개변수나 노드 이름 변경, 노드 네임스페이스 설정, 환경 변수 변경 등의 옵션을 사용할 수 있다. ROS 1에서는 이를 `roslaunch`라 하여 `*.launch`라는 파일을 사용하여 실행 노드를 설정하는데 이는 XML 기반이었으며, 여러 태그별 옵션을 제공하여 사용자 편의성을 제공하였다. 하지만 더 다양한 환경에서 그리고 더 다양한 기능을 추가하기 위하여 기존 XML 방식 이외에도 Python 방식도 추가되었다.

이 강좌에서는 기존 XML 방식보다 더 활용도가 높아진 Python 방식을 설명할 것이다. ROS 2 Launch System에 대한 더 자세한 설명은 참고 자료 문서 및 코드를 참고하자.

<https://design.ros2.org/articles/roslaunch.html>

https://design.ros2.org/articles/roslaunch_xml.html

https://github.com/ros2/launch/tree/master/launch_yaml

<https://github.com/ros2/launch>

335 launch 작성

```
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    param_dir = LaunchConfiguration(
        'param_dir',
        default=os.path.join(
            get_package_share_directory('topic_service_action_rclpy_example'),
            'param',
            'arithmetic_config.yaml'))

    return LaunchDescription([
        DeclareLaunchArgument(
            'param_dir',
            default_value=param_dir,
            description='Full path of parameter file'),

        Node(
            package='topic_service_action_rclpy_example',
            executable='argument',
            name='argument',
            parameters=[param_dir],
            output='screen'),

        Node(
            package='topic_service_action_rclpy_example',
            executable='calculator',
            name='calculator',
            parameters=[param_dir],
            output='screen'),

    ])

```

우리가 지금까지 작업한 `topic\_service\_action\_rclpy\_example` 패키지에 새로운 launch 파일을 만들어 보자. 이 런치 파일은 기본적으로 argument 노드와 calculator 노드를 실행 시키는 역할을 하게되며 두 노드에서 사용할 파라미터를 설정할 파일을 지정하는 역할을 하게 된다.

* 참고로 RCLCPP 기반의 `topic\_service\_action\_rclcpp\_example`도 launch 사용에 있어서는 동일하다.

우선, 원하는 패키지에 **launch** 이라는 폴더가 있어야 하며 해당 폴더에는 **`.launch.py`** 라는 이름의 런치 파일을 만들어 사용할 것이다.

여기서는 **`.arithmetic.launch.py`** 이라는 파일명을 사용하였다. **`.arithmetic.launch.py`** 파일은 하기 위치에 위치해 있으며, 전체 소스 코드는 다음과 같다.

topic_service_action_rclpy_example/launch/arithmetic.launch.py

336 launch 작성

```
def generate_launch_description():
```

```
    xxx = LaunchConfiguration(yyy)
```

```
    return LaunchDescription([
        DeclareLaunchArgument(aaa),
        Node(bbb),
        Node(ccc),
    ])
```

launch 파일의 기본은 하기와 같이 `generate_launch_description` 메소드를 이용하는게 기본이다. 이 메소드 내용으로

'`LaunchConfiguration`' 클래스를 이용하여 필요시 실행 관련 설정을 선언하고 메소드의 리턴값으로 '`LaunchDescription`' 클래스로 반환해주면 된다.

```
def generate_launch_description():
```

```
    param_dir = LaunchConfiguration(
        'param_dir',
        default=os.path.join(
            get_package_share_directory('topic_service_action_rclpy_example'),
            'param',
            'arithmetic_config.yaml'))
```

'`arithmetic.launch.py`' 파일에서 `LaunchConfiguration`으로 설정한 내용을 우선 살펴보자. '`LaunchConfiguration`' 클래스의 생성자로 '`param_dir`'라는 파라미터 디렉토리를 설정하는 부분으로 '`topic_service_action_rclpy_example`' 패키지의 '`param`'폴더에 위치한 '`arithmetic_config.yaml`' 파라미터 설정 파일을 의미한다. 이는 앞서 다룬 '파라미터 프로그래밍 (Python)' 강좌에서 설명했던 내용이므로 해당 파일의 내용이 궁금하다면 해당 강좌를 참고하도록 하자.


```
Node(
    package='topic_service_action_rclpy_example',
    executable='argument',
    name='argument',
    remappings=[
        ('/arithmetic_argument', '/argument'),
    ]
)
```

위에서는 사용하지 않아 설명 못한 유용한 기능이 하나 더 있는데 'remappings' 기능이다. 이는 특정 이름을 변경할 수 있는 것으로 아래 예제와 같이 '/arithmetic_argument' 토픽 이름을 '/argument' 이라는 토픽 이름으로 변경할 수 있다. 내부 코드 변경없이 토픽, 서비스, 액션 등의 고유 이름을 변경할 수 있는 유용한 기능이니 알아두도록 하자.

```
def generate_launch_description():
    ros_namespace = LaunchConfiguration('ros_namespace')

    return LaunchDescription([
        DeclareLaunchArgument(
            'ros_namespace',
            default_value=os.environ['ROS_NAMESPACE'],
            description='Namespace for the robot'),

        Node(
            package='topic_service_action_rclpy_example',
            namespace=ros_namespace,
            executable='argument',
            name='argument',
            output='screen'),
    ])
```

launch의 유용한 기능을 하나 더 알아보자. 이번에는 'namespace' 이다. ['ROS 2 Tips'](#) 강의에서 설명했던 것과 같이 노드, 토픽, 서비스, 액션, 파라미터 등과 같은 고유의 이름을 Namespace를 통해 바꾸면 독립적으로 자신만의 네트워크 그룹핑을 할 수 있다고 했다. 이는 각 노드를 실행시킬 때 ROS 변수 중 하나인 ns(namespace)를 입력하여 변경하는 방법도 있고, launch 파일로 실행시킬 때 namespace 라는 항목을 변경하는 방법이 있다고 설명했는데 이번에는 launch 파일로 실행시킬 때 namespace를 변경하는 방법에 대해 알아보자.

우선 LaunchConfiguration와 DeclareLaunchArgument을 통해 namespace를 지정한다. 아래 예제에서는 환경 변수로 지정한 'ROS_NAMESPACE' 변수를 읽어오도록 했는데 실행할 때 'export ROS_NAMESPACE=robot_1' 과 같은 구문을 터미널 창에 입력해주거나 '~/bashrc'에 미리 등록시켜놓아도 좋다.

그 뒤 다음과 같이 Node 클래스를 사용할 때 namespace를 지정하면 실행시 모든 노드 이름과 노드에 포함된 토픽, 서비스, 액션, 파라미터 등 고유 이름 모두가 변경되게 된다. 이 namespace는 복수의 로봇을 사용할 때 동일 프로그램을 이용할 때 고유 이름을 사용함에 있어서 중복됨을 피할 수 있고 데이터를 구분지어 사용할 수 있게 된다.

338 launch 작성

```
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    param_dir = LaunchConfiguration(
        'param_dir',
        default=os.path.join(
            get_package_share_directory('topic_service_action_rclpy_example'),
            'param',
            'arithmetic_config.yaml'))

    launch_description = LaunchDescription()

    launch_description.add_action(launch.actions.DeclareLaunchArgument(
        'param_dir',
        default_value=param_dir,
        description='Full path of parameter file'))

    argument_node = Node(
        package='topic_service_action_rclpy_example',
        executable='argument',
        name='argument',
        parameters=[param_dir],
        output='screen')

    calculator_node = Node(
        package='topic_service_action_rclpy_example',
        executable='calculator',
        name='calculator',
        parameters=[param_dir],
        output='screen')

    launch_description.add_action(argument_node)
    launch_description.add_action(calculator_node)

    return launch_description
```

런치 파일의 `generate_launch_description` 함수의 `return` 값이 너무 많아진다고 여겨진다면 `LaunchDescription`의 `add_action` 함수를 이용할 수도 있다. 이를 사용하면 아래와 같이 좀 더 간결해지니 참고하도록 하자.

339 launch 작성

```

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.actions import LogInfo
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import ThisLaunchFileDir

def generate_launch_description():
    return LaunchDescription([
        LogInfo(msg=['Execute three launch files!']),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                [ThisLaunchFileDir(), '/xxxxx.launch.py']),
        ),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                [ThisLaunchFileDir(), '/yyyyy.launch.py']),
        ),

        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                [get_package_share_directory('bbbb'),
                 '/launch/zzzz.launch.py']),
        ),
    ])

```

마지막 팁으로 런치 파일에서 현재 패키지의 다른 런치 파일이나 다른 패키지의 런치 파일을 불러오는 방법에 대해 알아보도록 하겠다.

현재의 패키지가 aaaaa 패키지라면 aaaaa 패키지의 xxxxx.launch.py와 yyyyy.launch.py 런치 파일은 IncludeLaunchDescription를 이용하여 불러올 수 있다. 그리고 다른 패키지인 bbbbb 패키지의 zzzzz.launch.py 런치 파일은 IncludeLaunchDescription를 이용하는 것은 동일하지만 get_package_share_directory 함수를 이용하여 해당 패키지명을 함께 입력해주는 것으로 특정 다른 패키지의 런치 파일을 불러올 수 있다. 이 내용을 런치 파일로 작성하면 다음 예제와 같다.

런치 파일에서 다른 런치 파일을 불러오는 것은 런치 파일의 모듈화 성격을 띄고 있어서 하나의 런치 파일로 동일 패키지의 노드 실행뿐만이 아닌 다른 패키지의 런치 파일도 불러와서 실행시킬 수 있는 장점이 있다.

특히, 직접 작성한 패키지가 아닌 패키지를 바이너리로 설치하고 실행하고자 했을 때에 별도 수정없이 해당 패키지의 런치파일만 본인이 작성한 패키지에서 불러와 사용할 수 있기에 널리 사용되고 있다. 매우 유용한 기능이니 참고하도록 하자.

340 패키지 빌드

Launch 파일은 Python 파일 이기에 빌드 자체가 필요 없기는 하지만 해당 파일을 ROS 2 에코시스템 환경에서 사용하기 위해서는 패키지 빌드를 통해 정해진 위치에 설치를 해야만 한다. Launch 파일 관련해서는 C++ 언어를 사용하는 RCLCPP 패키지 계열이나 Python 언어를 사용하는 RCLPY 패키지 계열이나에 따라 좀 다르니 구분하여 설명하겠다.

RCLCPP 패키지 계열

C++ 언어를 사용하는 경우 하기와 같이 빌드 설정 파일(CMakeLists.txt)의 install 구문에 launch 라는 폴더명만 기재하면 된다.

```
install(DIRECTORY
  launch
  DESTINATION share/${PROJECT_NAME}/
)
```

341 패키지 빌드

RCLPY 패키지 계열

Python 언어를 사용하는 경우 하기와 같이 파이썬 패키지 설정 파일(`setup.py`)의 `data_files` 옵션 부분에 `launch` 옵션을 지정하면 된다. 이는 해당 패키지의 소스 코드 폴더의 `launch` 폴더의 `*.launch.py` 파일명을 가진 런치 파일들을 설치 폴더에 위치시키게 된다.

```
setup(  
    name=package_name,  
    version='0.1.0',  
    packages=find_packages(exclude=['test']),  
    data_files=[  
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),  
        (share_dir, ['package.xml']),  
        (share_dir + '/launch', glob.glob(os.path.join('launch', '*.launch.py'))),  
        (share_dir + '/param', glob.glob(os.path.join('param', '*.yaml'))),  
    ],  
)
```

빌드는 하기와 같이 특정 패키지만을 빌드하는 `cbp alias`를 사용하면 된다.

```
$ cw  
$ cbp topic_service_action_rclpy_example
```

342 launch 실행

런치 파일을 실행하려면 ROS 2의 CLI 명령어 중 '**ros2 launch**'를 사용한다. 기본 사용 방법은 하기와 같다.

```
$ ros2 launch <package_name> <launch_file_name>
```

위 강좌에서 설명한 예제 파일을 실행 시키려면 아래와 같이 사용하면 된다. 즉 `topic_service_action_rclpy_example` 패키지의 `arithmetic.launch.py` 런치 파일을 실행 시키라는 의미이다. 이를 실행시키면 위에서 설명한 것처럼 파라미터 파일을 공유하여 사용하게 되며 `argument` 노드와 `calculator` 노드를 한번에 실행시키게 된다.

```
$ ros2 launch topic_service_action_rclpy_example arithmetic.launch.py
```

SLAM

Navigation

Manipulation



Mobile Robot: TurtleBot3 (SLAM / Navigation)

- TurtleBot3 Software
 - TurtleBot3
 - TurtleBot3 Messages
 - TurtleBot3 Simulations
 - TurtleBot3 Applications
 - TurtleBot3 Applications Messages
 - TurtleBot3 Autorace
 - TurtleBot3 Deliver

- OpenCR Firmware
 - TurtleBot3 Burger, Waffle. Waffle Pi and Friends

- **turtlebot3 (metapackage)**
 - **turtlebot3_bringup** http://wiki.ros.org/turtlebot3_bringup
 - **turtlebot3_description** http://wiki.ros.org/turtlebot3_description
 - **turtlebot3_example** http://wiki.ros.org/turtlebot3_example
 - **turtlebot3_navigation** http://wiki.ros.org/turtlebot3_navigation
 - **turtlebot3_slam** http://wiki.ros.org/turtlebot3_slam
 - **turtlebot3_teleop** http://wiki.ros.org/turtlebot3_teleop

- **turtlebot3_msgs** https://github.com/ROBOTIS-GIT/turtlebot3_msgs

- **turtlebot3_simulations (metapackage)**
 - **turtlebot3_fake** http://wiki.ros.org/turtlebot3_fake
 - **turtlebot3_gazebo** http://wiki.ros.org/turtlebot3_gazebo

348 TurtleBot3 패키지 훑아보기

- **turtlebot3_applications (metapackage)**
 - **turtlebot3_automatic_parking** http://wiki.ros.org/turtlebot3_automatic_parking
 - **turtlebot3_automatic_parking_vision** http://wiki.ros.org/turtlebot3_automatic_parking_vision
 - **turtlebot3_follow_filter** http://wiki.ros.org/turtlebot3_follow_filter
 - **turtlebot3_follower** http://wiki.ros.org/turtlebot3_follower
 - **turtlebot3_panorama** http://wiki.ros.org/turtlebot3_panorama
- **turtlebot3_applications_msgs** http://wiki.ros.org/turtlebot3_applications_msgs

- turtlebot3_autorace (metapackage)
 - turtlebot3_autorace_camera http://wiki.ros.org/turtlebot3_autorace_camera
 - turtlebot3_autorace_control http://wiki.ros.org/turtlebot3_autorace_control
 - turtlebot3_autorace_core http://wiki.ros.org/turtlebot3_autorace_core
 - turtlebot3_autorace_detect http://wiki.ros.org/turtlebot3_autorace_detect

- turtlebot3_deliver (metapackage)
 - turtlebot3_deliver_service http://wiki.ros.org/turtlebot3_deliver_service

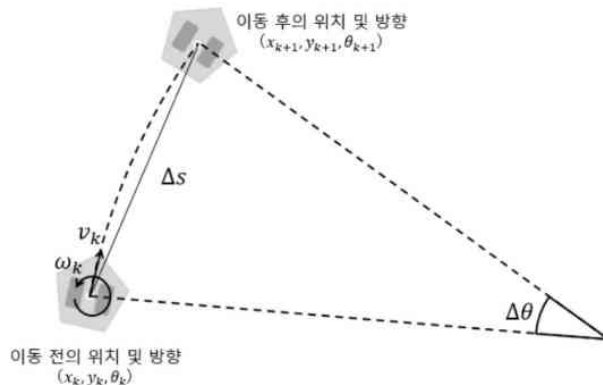
350 TurtleBot3 패키지 훑아보기

- dynamixel_sdk http://wiki.ros.org/dynamixel_sdk
- LDS
 - hls_lfcd_lds_driver http://wiki.ros.org/hls_lfcd_lds_driver
 - ld08_driver https://github.com/ROBOTIS-GIT/ld08_driver
- open_manipulator http://wiki.ros.org/open_manipulator
- open_manipulator_msgs http://wiki.ros.org/open_manipulator_msgs
- open_manipulator_simulations http://wiki.ros.org/open_manipulator_simulations
- open_manipulator_perceptions http://wiki.ros.org/open_manipulator_perceptions

- `turtlebot3_bringup` : 터틀봇 3를 시작할 때 필요한 노드와 설정을 포함하는 패키지
- `turtlebot3_description` : 터틀봇 3의 3D 모델 및 URDF을 포함하는 패키지
- `turtlebot3_node` : 터틀봇 3를 구동하게 하는 패키지
- `turtlebot3_teleop` : 터틀봇 3를 원격 제어하기 위한 패키지
- `turtlebot3_cartographer` : 터틀봇 3의 SLAM 기능을 제공하는 패키지
- `turtlebot3_navigation2` : 터틀봇 3의 위치 추정 및 경로 계획 기능을 제공하는 패키지
- `turtlebot3_example` : 터틀봇 3를 사용한 다양한 예제 코드를 포함하는 패키지
- `turtlebot3_msgs` : 터틀봇 3에서 사용되는 ROS 메시지 타입을 정의하는 패키지
- `turtlebot3_fake_node` : 터틀봇 3 로봇 없이 사용할 수 있는 패키지 (에뮬레이터 활용)
- `turtlebot3_gazebo` : 터틀봇 3 로봇 없이 사용할 수 있는 패키지 (Gazebo 활용)

352 TurtleBot3 패키지 훑아보기

- 데드레커닝 계산
 - 병진 속도 (linear velocity: v)
 - 회전 속도 (angular velocity: w)
- Runge-Kutta 공식 이용
 - 이동한 위치의 근사 값 x, y
 - 회전 각도 θ



$$v_l = \frac{(E_{lc} - E_{lp})}{T_e} \cdot \frac{\pi}{180} \text{ (radian/sec)}$$

$$v_r = \frac{(E_{rc} - E_{rp})}{T_e} \cdot \frac{\pi}{180} \text{ (radian/sec)}$$

$$V_l = v_l \cdot r \text{ (meter/sec)}$$

$$V_r = v_r \cdot r \text{ (meter/sec)}$$

$$v_k = \frac{(V_r + V_l)}{2} \text{ (meter/sec)}$$

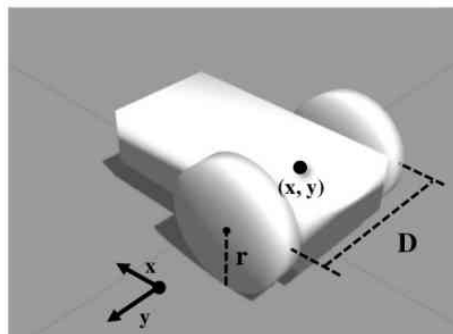
$$\omega_k = \frac{(V_r - V_l)}{D} \text{ (radian/sec)}$$

$$\Delta s = v_k T_e \quad \Delta \theta = \omega_k T_e$$

$$x_{(k+1)} = x_k + \Delta s \cos\left(\theta_k + \frac{\Delta \theta}{2}\right)$$

$$y_{(k+1)} = y_k + \Delta s \sin\left(\theta_k + \frac{\Delta \theta}{2}\right)$$

$$\theta_{(k+1)} = \theta_k + \Delta \theta$$




```

bool calcOdometry(double diff_time)
{
    float* orientation;
    double wheel_l, wheel_r;
    double delta_s, theta, delta_theta;
    static double last_theta = 0.0;
    double v, w;
    double step_time;

    wheel_l = wheel_r = 0.0;
    delta_s = delta_theta = theta = 0.0;
    v = w = 0.0;
    step_time = 0.0;

    step_time = diff_time;

    if (step_time == 0)
        return false;

    wheel_l = TICK2RAD * (double)last_diff_tick[LEFT];
    wheel_r = TICK2RAD * (double)last_diff_tick[RIGHT];

    if (isnan(wheel_l))
        wheel_l = 0.0;

    if (isnan(wheel_r))
        wheel_r = 0.0;

```

```

    delta_s = WHEEL_RADIUS * (wheel_r + wheel_l) / 2.0;
    orientation = sensors.getOrientation();
    theta = atan2f(
        orientation[1]*orientation[2] + orientation[0]*orientation[3],
        0.5f - orientation[2]*orientation[2] - orientation[3]*orientation[3]);

    delta_theta = theta - last_theta;

    // compute odometric pose
    odom_pose[0] += delta_s * cos(odom_pose[2] + (delta_theta / 2.0));
    odom_pose[1] += delta_s * sin(odom_pose[2] + (delta_theta / 2.0));
    odom_pose[2] += delta_theta;

    // compute odometric instantaneouse velocity
    v = delta_s / step_time;
    w = delta_theta / step_time;

    odom_vel[0] = v;
    odom_vel[1] = 0.0;
    odom_vel[2] = w;

    last_velocity[LEFT] = wheel_l / step_time;
    last_velocity[RIGHT] = wheel_r / step_time;
    last_theta = theta;

    return true;
}

```

turtlebot_core 예제
[calcOdometry 함수](#)

354 TurtleBot3 패키지 사용을 위한 준비 과정

```
$ nano ~/.bashrc
```

```
source /opt/ros/humble/setup.bash
source ~/robot_ws/install/local_setup.bash
source ~/turtlebot3_ws/install/local_setup.bash
source /usr/share/gazebo/setup.sh
source /usr/share/colcon_argcomplete/colcon-argcomplete.bash
source /usr/share/vcstool-completion/vcs.bash
source /usr/share/colcon_cd/function/colcon_cd.sh
```

```
export _colcon_cd_root=~/.robot_ws
export ROS_DOMAIN_ID=1
export ROS_LOCALHOST_ONLY=1
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
# export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{severity}] {time}
[{name}]: {message} ({function_name}() at {file_name}:{line_number})'
export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{name}] [{severity}]
[{time}]: {message}'
export RCUTILS_COLORIZED_OUTPUT=1
export RCUTILS_LOGGING_USE_STDOUT=0
export RCUTILS_LOGGING_BUFFERED_STREAM=1
export TURTLEBOT3_MODEL=waffle_pi
```

```
alias nr='nano ~/.bashrc'
alias sr='source ~/.bashrc'
alias cw='cd ~/robot_ws'
alias cs='cd ~/robot_ws/src'
alias dw='cd ~/robot_ws && rm -rf build/ install/ log/'
alias cb='cd ~/robot_ws && colcon build --symlink-install
--continue-on-error'
alias cbs='colcon build --symlink-install'
alias cbp='colcon build --symlink-install --packages-select'
alias cbu='colcon build --symlink-install --packages-up-to'
alias ct='cd ~/robot_ws && mkdir test_result && colcon test
--test-result-base ./test_result/ '
alias ctp='colcon test --packages-select'
alias ctr='colcon test-result'
alias rt='ros2 topic list'
alias re='ros2 topic echo'
alias rn='ros2 node list'
alias killgazebo='killall -9 gazebo & killall -9 gzserver & killall -9 gzclient'
alias roslint='ament_uncrustify && \ ament_cpplint && \ ament_cppcheck
&& \ ament_flake8 && \ ament_pep257 && \ ament_lint_cmake && \
ament_xmllint && \ ament_copyright'
```

```
$ source ~/.bashrc
```

355 TurtleBot3 패키지 사용을 위한 준비 과정

```
$ sudo apt update && sudo apt upgrade  
  
$ sudo apt install ros-humble-gazebo-*  
$ sudo apt install ros-humble-cartographer  
$ sudo apt install ros-humble-cartographer-ros  
$ sudo apt install ros-humble-navigation2  
$ sudo apt install ros-humble-nav2-bringup  
$ sudo apt install ros-humble-dynamixel-sdk  
  
$ sudo apt remove ros-humble-turtlebot3*  
  
$ mkdir -p ~/turtlebot3_ws/src/  
$ cd ~/turtlebot3_ws/src/  
  
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3.git  
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git  
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git  
$ cd ~/turtlebot3_ws && colcon build --symlink-install  
$ source ~/.bashrc
```

<http://turtlebot3.robotis.com/>

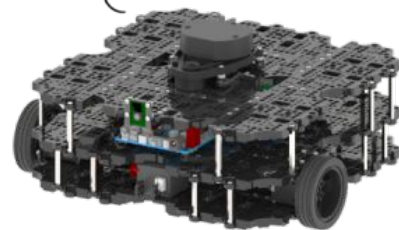


TURTLEBOT3

TurtleBot3
Burger



TurtleBot3
Waffle Pi



356 TurtleBot3 Simulation (Gazebo + Teleoperation)

```
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

```
$ ros2 launch turtlebot3_bringup rviz2.launch.py
```

```
$ ros2 run turtlebot3_teleop teleop_keyboard
```

(아래와 같이 키보드로 터틀봇3를 제어해 보세요.)

Moving around:

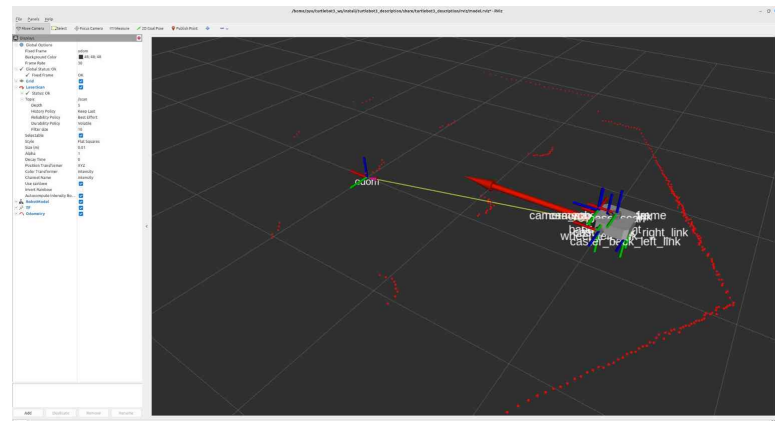
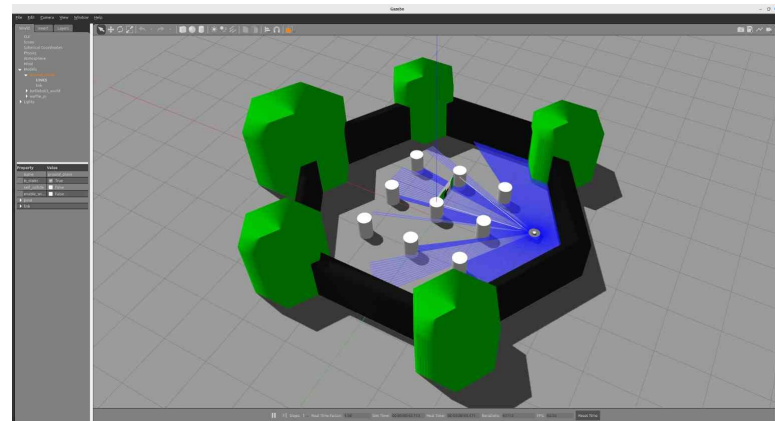
w
a s d
x

w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle Pi : ~ 0.26)

a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle Pi : ~ 1.82)

space key, s : force stop

CTRL-C to quit



357 TurtleBot3 Simulation (Gazebo + SLAM)

```
$ ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

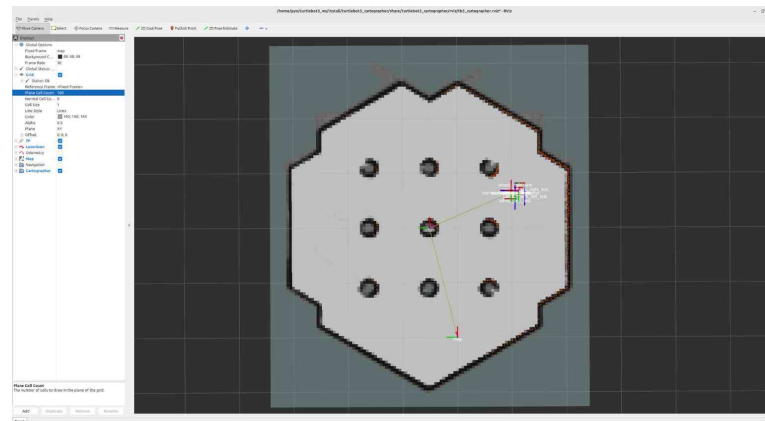
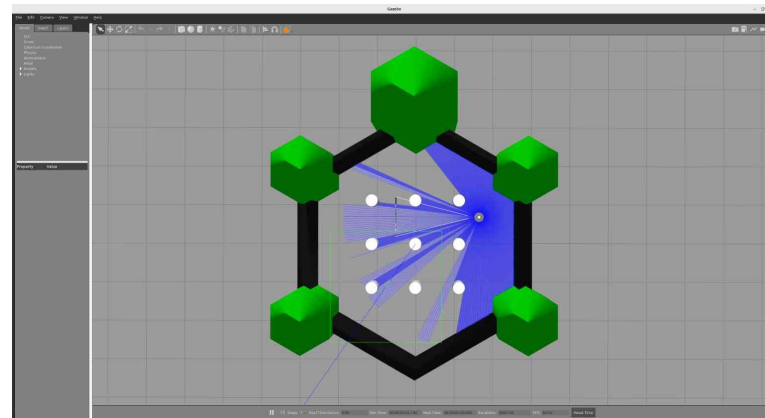
```
$ ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

```
$ ros2 run turtlebot3_teleop teleop_keyboard
```

(키보드 w, a, s, d, x 버튼으로 로봇을 움직이며 지도를 작성하세요.)

```
$ ros2 run nav2_map_server map_saver_cli -f ~/map
```

(작성한 지도를 파일 형태로 저장하세요.)



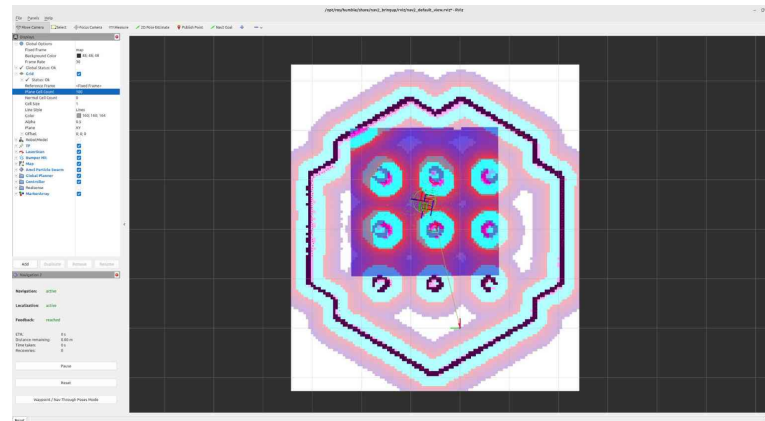
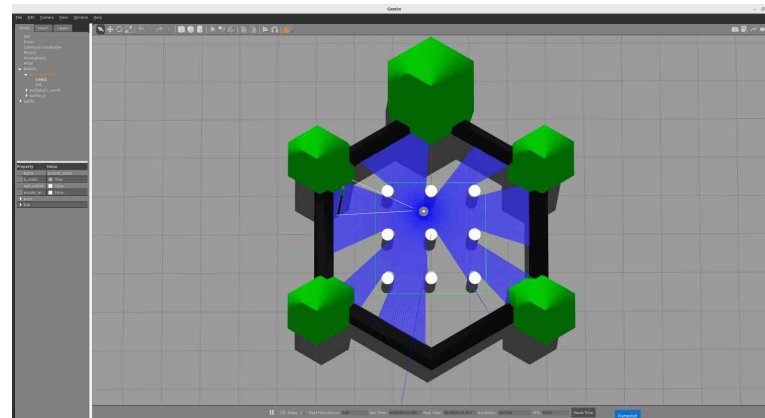
358 TurtleBot3 Simulation (Gazebo + Navigation)

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py use_sim_time:=True  
map:=$HOME/map.yaml
```

(RViz 화면 상단의 2D Pose Estimate 버튼을 눌러 초기 위치를 지정하세요.)

(RViz 화면 상단의 Nav2 Goal 버튼을 눌러 목적지를 지정하세요.)



Mobile Manipulator : TurtleBot3 + OpenManipulator-X (Manipulation)

360 TurtleBot3 Manipulation 패키지 사용을 위한 준비 과정

```
$ sudo apt install ros-humble-dynamixel-sdk ros-humble-ros2-control  
ros-humble-ros2-controllers ros-humble-gripper-controllers ros-humble-moveit  
ros-humble-moveit-servo
```

```
$ cd ~/turtlebot3_ws/src/  
$ git clone -b humble https://github.com/ROBOTIS-GIT/turtlebot3_manipulation.git  
$ cd ~/turtlebot3_ws && colcon build --symlink-install  
$ source ~/.bashrc
```

(의존성 이슈 발생 시)

```
$ cd ~/turtlebot3_ws  
$ sudo rosdep init  
$ rosdep update  
$ rosdep install --from-paths src --ignore-src -r -y
```

<http://turtlebot3.robotis.com/>



361 TurtleBot3 Manipulation (Gazebo + Teleoperation)

```
$ ros2 launch turtlebot3_manipulation_bringup gazebo.launch.py
```

```
$ ros2 launch turtlebot3_manipulation_moveit_config servo.launch.py
```

```
$ ros2 run turtlebot3_manipulation_teleop turtlebot3_manipulation_teleop
```

(아래와 같이 키보드로 터틀봇3와 매니플레이터를 제어해 보세요.)

Joint Control Keys:

1/q: Joint1 +/-

2/w: Joint2 +/-

3/e: Joint3 +/-

4/r: Joint4 +/-

Use o|p to open/close the gripper.

Command Control Keys:

i: Move up

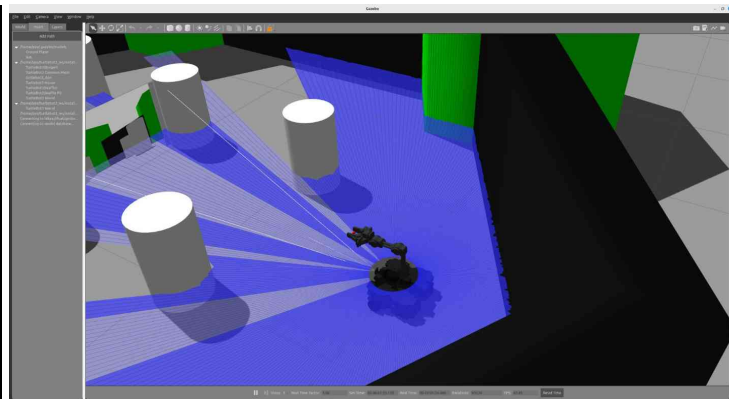
k: Move down

l: Move right

j: Move left

space bar: Move stop

'ESC' to quit.

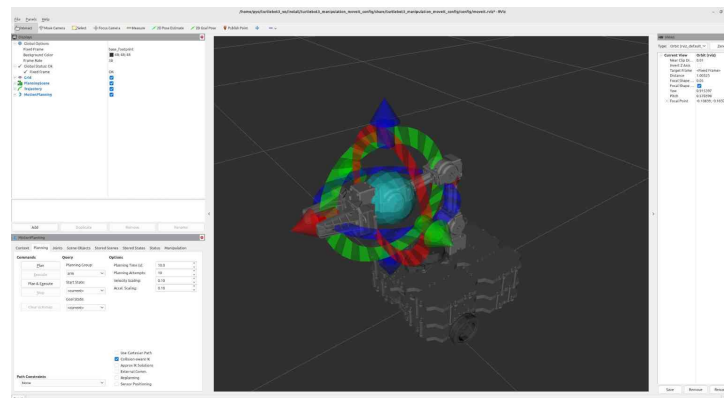
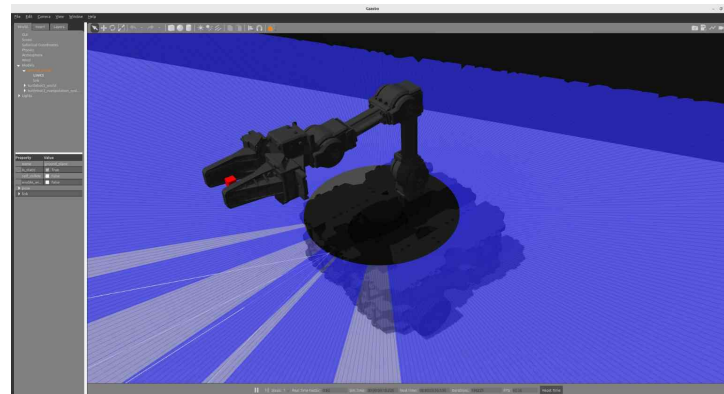


362 TurtleBot3 Manipulation (Gazebo + MoveIt)

```
$ ros2 launch turtlebot3_manipulation_bringup gazebo.launch.py
```

```
$ ros2 launch turtlebot3_manipulation_moveit_config moveit_core.launch.py
```

(RViz에 추가한 MoveIt의 MotionPlanning 툴을 활용하여 매니플레이터를 제어해보세요.)



ROS 2 Tips

364 설정 스크립트 (setup script)

처음 ROS 2를 사용함에 있어서 가장 놓칠 수 있는 실수는 ROS 2의 설정 스크립트를 사용하지 않고 ROS 2패키지를 찾는대거나 노드를 실행시키는 것이다.

새로운 패키지를 빌드하였다면 잊지말고 아래의 설정 스크립트를 터미널창에서 실행시키자. 또는 ROS 2 개발 환경 구축 강좌에서 언급한 것처럼 ~/.bashrc 에 저장한 후 사용하도록 하자.

```
$ source ~/robot_ws/install/local_setup.bash
```

365 설정 스크립트 (setup script)

- **underlay와 overlay**

우선 underlay와 overlay 개념을 알고 넘어가자. 우리가 ROS 2를 바이너리로 설치(binary installation)하게 된다면 /opt/ros/humble/ 라는 설치 폴더에 모든 구성 파일들이 위치하게 될 것이다. 혹은 소스로 빌드하여 설치(source installation)한다면 colcon 워크스페이스를 설정하기 나름이겠지만 ~/ros2_humble/ 으로 사용할 것이다. 이 개발 환경을 ROS에서는 **underlay**라고 한다.

그리고 개발자가 사용하는 워크스페이스가 있다. 예를 들어 ~/robot_ws/ 이라고 로봇 개발용 워크스페이스 하나 사용하고 ~/test_ws 이라고 테스트용 워크스페이스를 지정하여 사용하고 있다고 가정한다면 이 둘의 개발 환경을 ROS에서는 **overlay**이라고 한다.

즉, **overlay** 개발 환경은 설치된 ROS 패키지들에 의존하기에 **underlay** 개발 환경에 종속적이게 된다. 그렇기에 설정 스크립트(setup script)라고 하는 setup.bash의 호출 순서 및 사용 방법이 조금씩 달라지게 된다.

366 설정 스크립트 (setup script)

- **setup.bash** 및 **local_setup.bash** 사용 방법

setup.bash 및 local_setup.bash 파일은 **설정 스크립트 (setup script)**라고 부르며 underlay와 overlay 구분없이 모든 워크스페이스에 존재하며 각 설정 스크립트마다 사용 목적이 조금씩 다르다.

local_setup.bash 스크립트는 이 스크립트가 위치해 있는 접두사(prefix) 경로(예: ~/robot_ws/install/)의 모든 패키지에 대한 환경을 설정한다. 이는 **상위 작업 공간에 대한 설정은 포함되어 있지 않다**.

setup.bash 스크립트는 현재 작업 공간이 빌드될 때 환경에 제공된 다른 모든 작업 공간에 대한 local_setup.bash 스크립트를 포함하고 있다. 즉, **underlay 개발 환경의 설정 정보까지 포함한다**.

워크스페이스가 하나라면 local_setup.bash 이나 setup.bash 스크립트에는 차이가 없으나 둘 이상이라면 각 목적에 맞게 사용되어야 한다.

예를 들어 사용자가 지정한 워크스페이스(예: ~/robot_ws/install) 뿐만 아니라 ROS가 설치된 개발 환경 (예: /opt/ros/humble) 경로를 포함하도록 환경을 설정하려면 사용자가 지정한 워크스페이스의 setup.bash (예: ~/robot_ws/install/setup.bash)를 이용하자.

367 설정 스크립트 (setup script)

- **setup.bash** 및 **local_setup.bash** 사용 방법

이것은 `/opt/ros/humble/(local_)setup.bash` 을 먼저 소싱한 후, `~/robot_ws/install/local_setup.bash` 한 것과 동일한 효과입니다.

즉, 간단히 말하면 하기와 같이 `underlay` 개발 환경의 `setup.bash`를 우선 소싱한 후 자신의 워크스페이스인 `overlay` 개발 환경의 `local_setup.bash`를 소싱하는게 정석이다. 이 두 구문을 `./bashrc` 에 넣어두고 사용하면 편하다.

```
$ source /opt/ros/humble/setup.bash
$ source ~/robot_ws/install/local_setup.bash
```

368 ROS_DOMAIN_ID vs Namespace

ROS를 사용하면서 동일 네트워크를 다른 사람들과 함께 사용하고 있다면 다른 연구원들이 사용하고 있는 노드에 쉽게 접근 가능하여 데이터를 공유할 수 있게 된다. 이런 기능은 멀티 로봇 제어 및 협업 작업시 매우 편한데 독립적인 작업을 해야할 때에는 이 기능이 역으로 불편할 수 있다. 이를 방지하려면 아래 3가지 방법이 있을 수 있다.

- (1) **물리적으로 다른 네트워크**를 사용한다. (원천적인 해결방법으로 이게 참으로 속 편하다 ㅎㅎ)
- (2) **ROS_DOMAIN_ID**를 이용하여 DDS의 domain을 변경한다.
- (3) 각 노드 및 토픽/서비스/액션의 이름을 **Namespace**를 통해 변경한다.

(1) 물리적으로 다른 네트워크를 사용한다.

독립적인 스위치 허브 및 라우터를 사용하는 것으로 원천적인 해결방법이다. 때론 이 방법이 가장 속 편하다!

369 ROS_DOMAIN_ID vs Namespace

(2) ROS_DOMAIN_ID를 이용하여 DDS의 domain을 변경한다.

ROS 2에서는 DDS의 도입으로 멀티캐스트의 방식을 사용하고 있고 전역 공간이라 불리는 [DDS Global Space](#)이라는 공간에 있는 토픽들에 대해 구독 및 발행을 할 수 있게 된다. ROS 2에서는 이 DDS Global Space를 손쉽게 변경하는 방법으로 **ROS_DOMAIN_ID** 라는 환경 변수를 두고 있고 각 RMW에서는 이 환경 변수를 참조하여 도메인을 바꾸게 된다.

예를 들어 하기와 같이 각 터미널상에서 각 노드를 실행시켜 보자. `ROS\_DOMAIN\_ID`을 서로 동일하게 맞춘 **talker** 노드와 **listener** 노드만이 연결되어 서로 통신 됨을 확인할 수 있을 것이다. 참고로 `ROS\_DOMAIN\_ID`은 RMW마다 다르고 기본 운영체제에 따라 다르지만 기본적으로 [0에서 101까지의 정수](#)를 사용할 수 있다.

동일 네트워크를 다른 사람과 쓰고 있다면 이 `ROS\_DOMAIN\_ID`를 각 멤버들과 상의하여 지정하여 쓰면 된다. 단, 특정 번호 예를들어 행운의 7을 쓰기 위해서 싸우지는 말자! ROS_DOMAIN_ID=7 를 쓴다고 안 돌던 ROS 프로그램이 갑자기 동작하지는 않는다.

370 ROS_DOMAIN_ID vs Namespace

```
$ export ROS_DOMAIN_ID=11
$ ros2 run demo_nodes_cpp talker
[INFO]: Publishing: 'Hello World: 1'
[INFO]: Publishing: 'Hello World: 2'
[INFO]: Publishing: 'Hello World: 3'
(생략)
```

```
$ export ROS_DOMAIN_ID=12
$ ros2 run demo_nodes_cpp listener
(서로 다른 도메인을 사용하고 있기에 아무런 반응이 없을 것이다.)
```

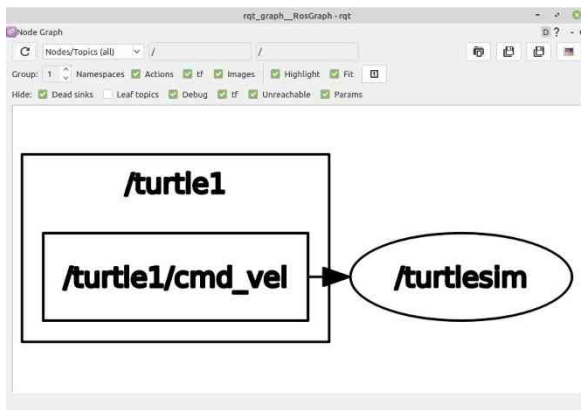
```
$ export ROS_DOMAIN_ID=11
$ ros2 run demo_nodes_cpp listener
[INFO]: I heard: [Hello World: 13]
[INFO]: I heard: [Hello World: 14]
[INFO]: I heard: [Hello World: 15]
(생략)
```

371 ROS_DOMAIN_ID vs Namespace

(3) 각 노드 및 토픽/서비스/액션의 이름을 **Namespace**를 통해 변경한다.

ROS 2의 노드는 각 노드별 고유의 이름이 존재한다. 그리고 각 노드에서 사용하는 토픽, 서비스, 액션, 파라미터 또한 고유의 이름이 존재한다. 이러한 고유의 이름을 **Namespace**를 통해 바꾸면 독립적으로 자신만의 네트워크 그룹핑을 할 수 있다. 이는 각 노드를 실행시킬 때 ROS 변수 중 하나인 **ns (namespace)**를 입력하여 변경하는 방법도 있고 launch 파일로 실행시킬 때 **node_namespace** 라는 항목을 변경하는 방법이 있다. 오늘은 노드를 실행시킬 때 ROS 변수 중 하나인 **ns(namespace)**를 입력하는 방법에 대해 알아보자. 하기와 같이 실행하고 rqt_graph를 통해 각 노드 및 토픽명을 확인해보면 다음 그림과 같다.

```
$ ros2 run turtlesim turtlesim_node
```

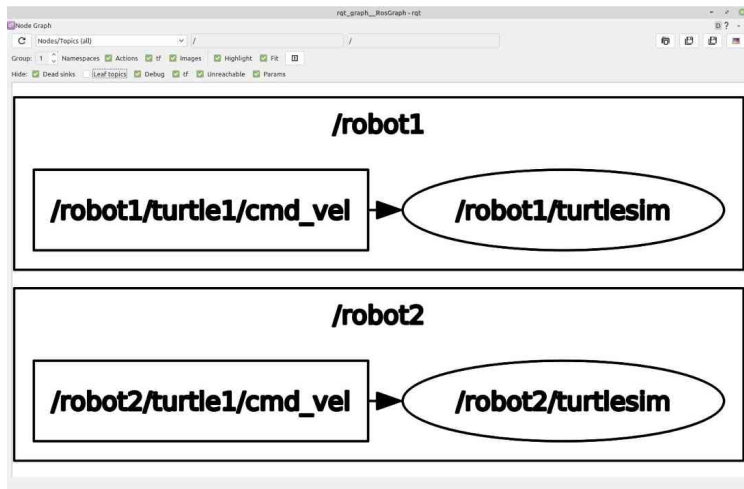


372 ROS_DOMAIN_ID vs Namespace

이번에는 ns를 변경하여 동일 노드를 두개 실행시켜보자. 하나는 robot1, 다른 하나는 robot2로 ns를 지정하였더니 rqt_graph에서 보이는 토픽과 노드명 앞에 ns가 붙여져 있는 것을 확인할 수 있다. 이런식으로 노드를 유저가 지정한 namespace로 묶는 방법도 유용할 때가 있다. 단, ns만 바꾼다면 그룹핑만 할뿐 rqt_graph나 node, topic, service, action, parameter 등을 볼때 다른 사람이 사용하고 있는 정보가 모두 보인다는 점을 잊지 말자.

```
$ ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/robot1
```

```
$ ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/robot2
```



373 빌드툴 설정 (colcon_cd)

터미널창에서 colcon_cd 명령을 사용하면 쉘의 현재 작업 디렉토리를 패키지 디렉토리로 빠르게 변경할 수 있다. ROS 1의 roscd와 비슷한 역할을 하는 ROS 2의 쉘 명령어라고 생각하면 된다.

```
$ colcon_cd 패키지이름
```

이를 사용하기 위해서는 다음과 같은 설정을 `./bashrc`에 추가하면 된다. 작업 폴더는 환경마다 다를 수 있는데 기본 작업 폴더를 `~/robot_ws`으로 했던 분들이라면 아래와 같이 `_colcon_cd_root`를 지정하는 것도 잊지말자.

```
source /usr/share/colcon_cd/function/colcon_cd.sh
export _colcon_cd_root=~/robot_ws
```

374 colcon과 vcstool 명령어 자동 완성 기능 추가

"ROS 2의 빌드 시스템과 빌드 툴" 강좌에서 소개하였던 빌드 툴(build tools)인 colcon과 버전 컨트롤 시스템 툴인 vcstool은 다양한 기능을 수행할 수 있는 [verbs]와 [sub-verbs]의 옵션이 있는데 기본 설정으로는 이러한 옵션이 터미널창에서 탭(tab)등의 키(key)로는 자동 완성(auto-completion)되지 않는다. 이 두개의 툴의 자동완성 기능을 사용하려면 홈폴더(~/)의 .bashrc 파일에 아래와 같은 설정을 미리 해두면 매우 편하다.

```
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
source /usr/share/vcstool-completion/vcs.bash
```

```
export ROS_LOCALHOST_ONLY=1
```

DDS 기반 RMW 구현의 기본 동작은 멀티캐스트를 통해 도달 가능한 모든 노드를 검색하도록 되어 있다. 이는 같은 네트워크를 사용할 때 의도치 않게 같은 네트워크내의 다른 개발자와 DDS로 연결됨에 따라 불편함이 있었다. 이에 위와 같이 **ROS_LOCALHOST_ONLY** 환경 변수를 **True** 설정하는 것으로 **단일 컴퓨터 안에서만 DDS 사용이 가능하게 된다**. 컴퓨터간의 DDS 통신을 방지하기 위해서는 위와 같이 설정하도록 하자.

* ROS 2 Humble의 다음 버전인 ROS 2 Iron 부터는 ROS_LOCALHOST_ONLY를 더 이상 사용하지 않도록 변경되었다. 대신에 보다 세분화된 옵션을 사용할 수 있는 **ROS_AUTOMATIC_DISCOVERY_RANGE** 로 변경되었다. SUBNET, LOCALHOST, OFF, SYSTEM_DEFAULT, ROS_STATIC_PEERS 값으로 세부적으로 설정이 가능합니다. 해당 기능은 추후 ROS 2 Jazzy로의 버전 이전 시 참고하도록 하자.

```
export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{name}] [{severity}] [{time}]: {message}'  
export RCUTILS_COLORIZED_OUTPUT=1  
export RCUTILS_LOGGING_USE_STDOUT=0  
export RCUTILS_LOGGING_BUFFERED_STREAM=1
```

- **RCUTILS_CONSOLE_OUTPUT_FORMAT**: 로그 메시지의 출력 양식 지정

'[{severity} {time}] [{name}]: {message} ({function_name}()) at {file_name}:{line_number}']

{severity}	심각도 수준 (DEBUG, INFO, WARN, ERROR, FATAL)
{name}	로거 이름
{message}	로그 메시지
{function_name}	호출된 함수 이름
{file_name}	호출된 파일 이름
{time}	로그 시간 (sec)
{time_as_nanoseconds}	로그 시간 (nanosec)
{date_time_with_ms}	ISO 형식의 로그 시간(예: 2024-06-11 09:29:19.304) (ROS 2 Jazzy부터 사용 가능)
{line_number}	호출된 파일의 코드 라인 번호


```
export RCUTILS_CONSOLE_OUTPUT_FORMAT='[{name}] [{severity}] [{time}]: {message}'  
export RCUTILS_COLORIZED_OUTPUT=1  
export RCUTILS_LOGGING_USE_STDOUT=0  
export RCUTILS_LOGGING_BUFFERED_STREAM=1
```

- **RCUTILS_COLORIZED_OUTPUT**
 - 메시지를 출력할 때 색상을 사용할지 여부를 설정한다.
 - 미설정 = 시스템 설정을 사용
 - **0** = 색상 미사용
 - **1** = 색상 사용
- **RCUTILS_LOGGING_USE_STDOUT**
 - 어떤 스트림 출력 메시지를 보낼지 설정한다.
 - 미설정 = **stderr**
 - **0** = **stderr**
 - **1** = **stdout**
- **RCUTILS_LOGGING_BUFFERED_STREAM**
 - 로깅 스트림을 버퍼링할지 아니면 버퍼링 해제할지 여부를 설정한다.
 - 미설정 = 스트림의 기본값 사용 (stdout: 버퍼링, stderr: 언버퍼링)
 - **0** = 언버퍼링
 - **1** = 버퍼링

378 rosdep

```
$ cd ~/turtlebot3_ws  
$ sudo rosdep init  
$ rosdep update  
$ rosdep install --from-paths src -y --ignore-src
```

의존성 관리 툴인 **rosdep** 명령어를 사용하면 손쉽게 패키지의 의존성 문제를 해결할 수 있다.

rosdep은 패키지 환경 설정 파일인 **package.xml** 의 **<depend>** 옵션과 같은 의존성 정보를 확인하여 의존성 패키지들을 설치해주기 때문에 의존성 패키지가 많은 패키지의 경우, 위 명령어를 사용하면 의존성 패키지 설치 및 관리에 있어서 매우 편하게 사용할 수 있게 된다.

ROS 2 관련 패키지들을 삭제하기 위한 방법은 다음과 같다.

```
$ sudo apt remove ~nros-humble-* && sudo apt autoremove
```

```
$ sudo rm /etc/apt/sources.list.d/ros2.list
```

```
$ sudo apt update
```

```
$ sudo apt autoremove
```

```
$ sudo apt upgrade
```

ROS 2 추천 패키지

ROS 2 추천 패키지를 소개하기 앞서 많이 사용하고 있는 ROS 패키지에는 어떤 것들이 있는지 먼저 알아보도록 하자. ROS 2 설치시에 아래와 같은 명령어로 ROS 2 사용에 필요한 패키지들을 한번에 설치할 수 있다. 여기서 설치한 'ros-humble-desktop'은 'ros humble 배포판의 'desktop' 패키지를 가리킨다. 우리가 설치한 이 'desktop' 패키지는 여러 패키지에 의존성을 갖는 메타패키지로 ROS 2 사용에 있어서 필수적인 패키지들을 포함하고 있다.

```
$ sudo apt install ros-humble-desktop
```

이러한 메타패키지는 [REP-2001](#)에 기술되어 있는데 기본적으로는 [ROS Core](#), [ROS Base](#), [Desktop](#)으로 나뉘어 있으며 아래와 같은 순서대로 하위 포함 관계를 가지고 있다.

ROS Core < ROS Base < Desktop

이 3가지 메타패키지의 설명과 포함된 패키지는 다음과 같다. 참고로 일반적인 경우는 Desktop을 기본으로 설치하여 사용하지만 로봇과 같은 제한된 리소스의 환경이라면 ROS Base를 설치하여 필요한 패키지만 추가로 설치하는 것을 추천한다.

382 ROS Core, Base, Desktop

ROS Core

ROS Core 패키지는 빌드, C/C++/Python 지원, 퍼블리시, 서브스크라이브, 서비스, 메시지 생성 및 기타 핵심 ROS 개념을 사용하는데 필요한 패키지를 모아둔 메타패키지이다. 현재 시점으로 [32개의 패키지](#)가 포함되어 있으며 `ament`, `rcl`, `rclcpp`, `rclpy`, `roscpp`, `roslaunch`, `roslint`, `roslz4`, `rostopic`, `rosversion`, `roswtf`, `roswtf2`, `roswtf3`, `roswtf4`, `roswtf5`, `roswtf6`, `roswtf7`, `roswtf8`, `roswtf9`, `roswtf10`, `roswtf11`, `roswtf12`, `roswtf13`, `roswtf14`, `roswtf15`, `roswtf16`, `roswtf17`, `roswtf18`, `roswtf19`, `roswtf20`, `roswtf21`, `roswtf22`, `roswtf23`, `roswtf24`, `roswtf25`, `roswtf26`, `roswtf27`, `roswtf28`, `roswtf29`, `roswtf30`, `roswtf31`, `roswtf32` 등으로 구성되어 있다.

ROS Base

ROS Base 패키지는 앞서 설명한 'ROS Core'를 포함하고 있으며 `tf2` 및 `urdf` 와 같은 ROS의 기본 기능을 포함하는 메타패키지이다. 현재 시점으로 `ros_core`를 포함하여 [37개의 패키지](#)가 포함되어 있다. 추가된 패키지는 `roscpp`, `roslaunch`, `roslint`, `roslz4`, `rostopic`, `rosversion`, `roswtf`, `roswtf2`, `roswtf3`, `roswtf4`, `roswtf5`, `roswtf6`, `roswtf7`, `roswtf8`, `roswtf9`, `roswtf10`, `roswtf11`, `roswtf12`, `roswtf13`, `roswtf14`, `roswtf15`, `roswtf16`, `roswtf17`, `roswtf18`, `roswtf19`, `roswtf20`, `roswtf21`, `roswtf22`, `roswtf23`, `roswtf24`, `roswtf25`, `roswtf26`, `roswtf27`, `roswtf28`, `roswtf29`, `roswtf30`, `roswtf31`, `roswtf32` 패키지이다.

Desktop

Desktop 패키지는 'ROS Base'에서 확장하고 있으며 시각화 도구 및 데모와 같은 응용 패키지를 포함하는 메타패키지이다. 현재 시점으로 `ros_base`를 포함하여 [85개의 패키지](#)로 구성되어 있다. 추가된 패키지는 `angles`, `depthimage_to_laserscan`, `joy`, `pcl_conversions`, `rviz2`, `rviz_default_plugins`, `teleop_twist_joy`, `teleop_twist_keyboard`, `tlsf`, `demos` 및 `examples` 계열의 패키지이다. `demos` 및 `examples` 계열의 패키지만 30개 넘는다.

ROS는 특정 회사나 개인이 만들어 가는 것이 아닌 ROS 커뮤니티에서 공동 개발하고 있는 오픈소스 로봇 소프트웨어 플랫폼 프로젝트이다. 이에 서로 다른 작성자, 관리자 및 기여자들이 포함되어 있으며 이러한 패키지는 여러 저장소에 분산되어 보관되어 개발된다. 이러한 이유로 ROS 사용자와 개발자가 프로젝트에 필수적인 ROS 2 패키지를 알기란 쉽지 않다.

그래서 ROS 2 TSC에서는 투표를 통해 오픈소스이면서 ROS 2 프로젝트와 직접적으로 연관되고 범용적으로 사용할 수 있는 패키지들을 선정하여 '[ROS 2 Common Packages](#)'라는 이름으로 관리하고 있다. 다른 패키지들과 다르게 'ROS 2 Common Packages'은 ROS 2 TSC 및 Working Groups에서 관리하고 있으며 [Quality Level 3](#) 이상의 QC를 거치고 있고 [보안 취약성](#)에 대응하고 있다. 이 패키지는 ROS 사용에 있어서 매우 중요한 패키지로 ROS 2의 새로운 버전이 릴리즈되었다면 이들의 패키지가 함께 포함되어 있다고 생각하면 된다.

ROS 2 Common Packages

Buildsystem	ament_cmake	ament_index_cpp	ament_lint	ament_package	ros_environment	...
RMW	rmw	rmw_implementation	rmw_vendor	rosidl_typesupport_vendor	Micro-XRCE-DDS	...
RCL	rcl	rcl_logging	rcl_c	rcl_cpp	rcl_py	...
Utility	tracetools	class_loader	pluginlib	rcutils	ros2test	...
ROS Interface pipeline	rosidl_cmake	rosidl_generator_cpp	rosidl_generator_py	rosidl_default_generators	rosidl_default_runtime	...
Interface definitions	std_msgs	std_srvs	sensor_msgs	geometry_msgs	builtin_interfaces	...
Launch	launch	launch_xml	launch_yaml	launch_ros	ros2launch	...
Features	navigation2	moveit2	kdl_parser	tf2	sros2	...
Robot	urdfdom_headers	urdf	urdfdom			
Tools	ignition	webots	rviz2	ros2cli	rqt	...
Third party packages	gmock_vendor	gtest_vendor	poco_vendor	spdlog_vendor	tinyclxml2_vendor	...
Documentation, Examples, Tutorials	demo	examples				

385 ROS 2 Common Packages의 소분류

- 1) Buildsystem, package index and tooling around (64개)
- 2) Third party packages (11개)
- 3) Utility functionality (18개)
- 4) ROS interface pipeline (17개)
- 5) Interface definitions (25개)
- 6) RMW (24개)
- 7) Client libraries (14개)
- 8) Launch (8개)
- 9) Features (128개)
- 10) Robot (3개)
- 11) Tools (94개)
- 12) Documentation, Examples, Tutorials (38개)

386 ROS 2 추천 패키지 (10가지)

아래의 패키지들은 패키지 리포지토리의 forks, stars, subscriptions를 기준으로 ROS 패키지를 랭킹한 자료의 상위 항목이다. 이 통계 자료를 보았을 때 ROS 커뮤니티에서는 센싱, 인식, SLAM, 내비게이션, 매니플레이션 분야쪽으로 많은 관심이 있고 시뮬레이터, 통신, 딥러닝 계열도 관심도가 높아지고 있는 것을 확인할 수 있다. 하드웨어와 연관된 패키지로는 센서분야로 realsense, rplidar, velodyne 등이 상위에 랭킹되어 있고 로봇 관련 패키지로는 UR, TurtleBot3, AR.Drone 등이 상위에 랭킹되어 있다.

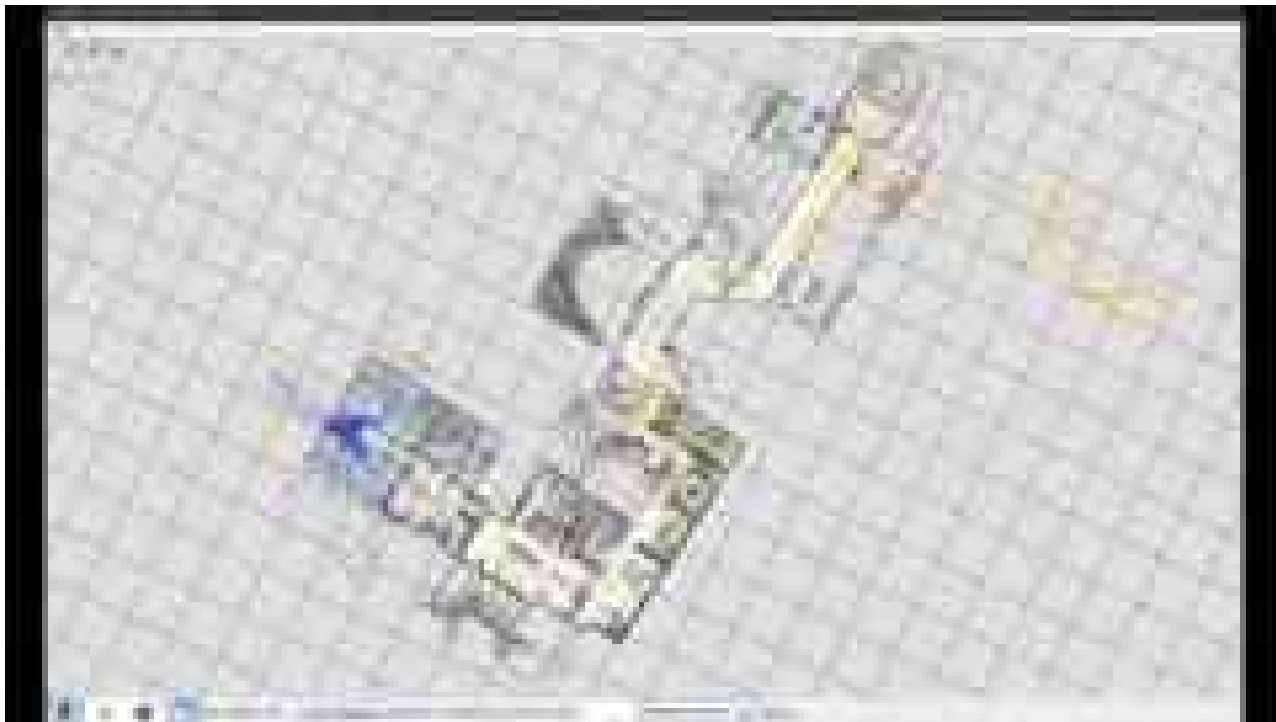
cartographer, librealsense, taskflow, lsd_slam, ros, navigation, serial, rtabmap, cartographer_ros, mrpt, grid_map, realsense-ros, rotors_simulator, plotjuggler, libpointmatcher, ros_comm, moveit, universal_robot, octomap, darknet_ros, mavros, fast-dds, turtlebot3, gazebo_ros_pkgs, gtsam ...

이 중 주목해야할 ROS 2 패키지를 추천한다면 아래와 같이 10가지를 추천하고 싶다.

ROS 2에서 가장 널리 쓰이는 SLAM계의 아이돌 Cartographer, 내비게이션 관련 패키지를 모아둔 NAV2, 자율주행차 소프트웨어 Autoware.Auto, 매니플레이션 종합 툴 MoveIt2, 드론 오픈소스 프로젝트 PX4 Autopilot, 산업계 컨소시엄인 ROS-Industrial, 차세대 3차원 시뮬레이터 Ignition, 오픈소스/하드웨어로 유명한 TurtleBot3, OpenMANIPULATOR-X, 태스크 플래너로 사용 가능한 Behavior Trees 그리고 끝으로 태스크 레벨의 멀티 로봇 운용 프레임워크인 RMF(Robotics Middleware Framework)이다. 이어지는 설명에서 이들 10가지에 대해서 간단히 알아보도록 하자.

1. Cartographer
2. NAV2
3. Autoware.Auto
4. MoveIt2
5. PX4 Autopilot
6. ROS-Industrial
7. GazeboSim
8. TurtleBot3, OpenMANIPULATOR-X
9. Behavior Trees
10. RMF(Robotics Middleware Framework)

- 웹사이트 : <https://opensource.google/projects/cartographer>
- 참고자료 : <https://google-cartographer-ros.readthedocs.io/>
- 리포지토리 : https://github.com/ros2/cartographer_ros
- 관련 영상 : <https://youtu.be/DM0dpHLtX0>



389 NAV2

- 웹사이트 : <https://navigation.ros.org/>
- 커뮤니티 : <https://discourse.ros.org/c/navigation/44>
- 리포지토리 : <https://github.com/ros-planning/navigation2>
- 관련 영상 : <https://youtu.be/QB7lOKp3ZDQ>



- 웹사이트 : <https://www.autoware.auto/>
- 커뮤니티 : <https://discourse.ros.org/c/autoware/46>
- 참고자료 : <https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/>
- 리포지토리 : <https://gitlab.com/autowarefoundation/autoware.auto/AutowareAuto>
- 관련 영상 : <https://youtu.be/XTmlhvlmcf8>

The image shows the front cover of a book. The cover has a dark blue background on the left and a white background on the right, separated by a diagonal line. The title 'Self-Driving Cars with ROS and Autoware Development Environment' is written in white text on the dark blue background. The book is shown at a slight angle, and a large number '1' is visible on the right side of the image.

Self-Driving Cars with
ROS and Autoware
Development
Environment

1

- 웹사이트: <https://moveit.ros.org/>
- 커뮤니티: <https://discourse.ros.org/c/moveit/13>
- 리포지토리: <https://github.com/ros-planning/moveit2>
- 관련 영상: <https://youtu.be/7KvF7Dj7bz0>



- 웹사이트 : <https://px4.io/>
- 커뮤니티 : <https://discuss.px4.io/>
- 참고자료 : https://docs.px4.io/master/en/ros/ros2_comm.html
- 리포지토리 : https://github.com/PX4/px4_ros_com
- 관련 영상 : <https://youtu.be/lZ8crGl16qA>



- 웹사이트 : <https://rosindustrial.org/>
- 커뮤니티 : <https://discourse.ros.org/c/ros-industrial/39>
- 리포지토리 : <https://github.com/ros-industrial>
- 리포지토리 : <https://github.com/ros-industrial-consortium>
- 관련 영상 : <https://youtu.be/lxTJ473MY3Y>



394 Gazebo Sim

- 웹사이트 : <https://ignitionrobotics.org/>
- 커뮤니티 : <https://community.gazebosim.org/>
- 리포지토리 : <https://github.com/gazebosim>
- 관련 영상 : <https://youtu.be/aKpUSXgZXYM>



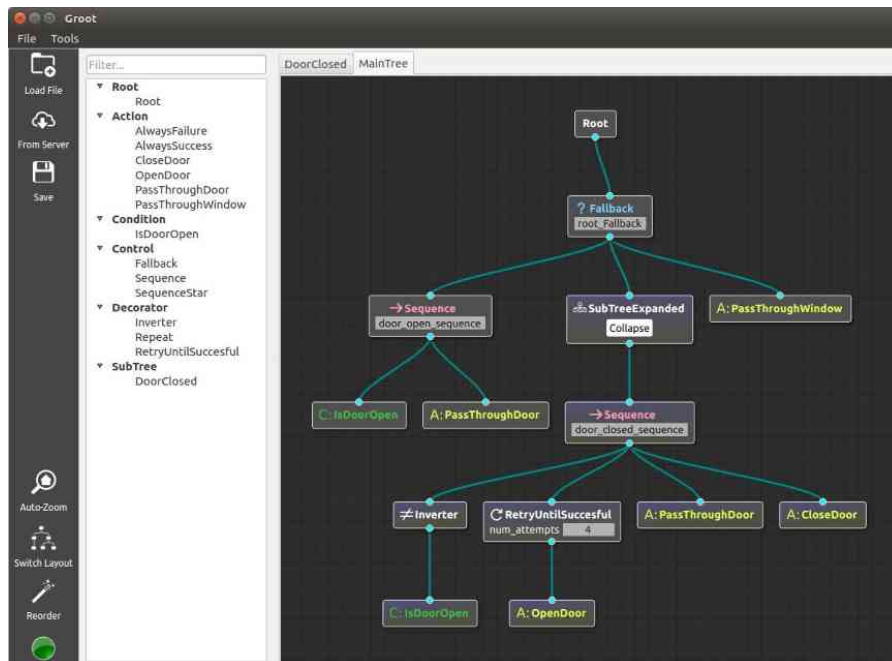
395 TurtleBot3, OpenMANIPULATOR

- 웹사이트 : <http://www.turtlebot.com/>
- 커뮤니티 : <https://discourse.ros.org/c/turtlebot/12>
- 커뮤니티 : <https://community.robotis.us/>
- 참고자료 : <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>
- 참고자료 : https://emanual.robotis.com/docs/en/platform/openmanipulator_x/overview/
- 리포지토리 : <https://github.com/ROBOTIS-GIT>
- 관련 영상 : <https://youtu.be/9OC3J53RUsk>
- 관련 영상 : https://www.youtube.com/watch?v=OdLsbAMy7m0&list=PLRG6WP3c31_WiSi5ZAMkaNy1_vbudTmmM&index=40



396 Behavior Trees

- 웹사이트 : <https://www.behaviortree.dev/>
- 리포지토리 : <https://github.com/BehaviorTree/BehaviorTree.CPP>
- 웹사이트 : <https://py-trees.readthedocs.io/>
- 리포지토리 : <https://github.com/splintered-reality/>



RMF(Robotics Middleware Framework)

- 웹사이트 : <https://osrf.github.io/ros2multirobotbook/>
- 리포지토리 : <https://github.com/open-rmf>
- 관련 영상 : <https://youtu.be/yticxB7tXFA>
- 관련 영상 : https://youtu.be/oSVQrjx_4w4



ROS 2 활용 사례



자율주행 서비스 로봇 개발 및 상용화 과정



**ROS 2 도입을 위한
다음 단계는?**

ROS 2 도입을 위한 다음 단계는? (프로그래밍에 왕도는 없어요.)

ROS 2 소개

- > [000 로봇 운영체제 ROS 강좌 목차](#)
- > [001 ROS 2 개발 환경 구축](#)
- > [002 ROS 2 기반 로봇 개발에 필요한 정보 링크 모음](#)
- > [003 왜? 'ROS 2'로 가야하는가?](#)
- > [004 ROS 2의 중요 컨셉과 특징](#)
- > [005 ROS 2의 특징](#)
- > [006 ROS 2와 DDS \(Data Distribution Service\)](#)
- > [007 패키지 설치와 노드 실행](#)
- > [008 ROS 2 노드와 메시지 통신](#)
- > [009 ROS 2 토픽 \(Topic\)](#)
- > [010 ROS 2 서비스 \(Service\)](#)
- > [011 ROS 2 액션 \(Action\)](#)
- > [012 ROS 2 토픽/서비스/액션 정리 및 비교](#)
- > [013 ROS 2 파라미터 \(Parameter\)](#)
- > [014 ROS 2 도구와 CLI 명령어](#)
- > [015 ROS의 종합 GUI 툴 RQt](#)
- > [016 ROS 2 인터페이스 \(interface\)](#)
- > [017 ROS 2의 물리량 표준 단위](#)
- > [018 ROS 2의 좌표 표현](#)
- > [019 DDS의 QoS \(Quality of Service\)](#)
- > [020 ROS 2의 파일 시스템](#)
- > [021 ROS 2의 빌드 시스템과 빌드 툴](#)
- > [022 패키지 파일 \(환경 설정, 빌드 설정\)](#)
- > [041 시간\(Time, Duration, Clock, Rate\)](#)

ROS 2 기본 프로그래밍

- > [023 ROS 프로그래밍 규칙 \(코드 스타일\)](#)
- > [024 ROS 프로그래밍 기초 \(Python\)](#)
- > [025 ROS 프로그래밍 기초 \(C++\)](#)
- > [027 토픽, 서비스, 액션 인터페이스](#)
- > [028 ROS 2 패키지 설계 \(Python\)](#)
- > [029 토픽 프로그래밍 \(Python\)](#)
- > [030 서비스 프로그래밍 \(Python\)](#)
- > [031 액션 프로그래밍 \(Python\)](#)
- > [032 파라미터 프로그래밍 \(Python\)](#)
- > [033 실행 인자 프로그래밍 \(Python\)](#)
- > [034 ROS 2 패키지 설계 \(C++\)](#)
- > [035 토픽 프로그래밍 \(C++\)](#)
- > [036 서비스 프로그래밍 \(C++\)](#)
- > [037 액션 프로그래밍 \(C++\)](#)
- > [038 파라미터 프로그래밍 \(C++\)](#)
- > [039 실행 인자 프로그래밍 \(C++\)](#)
- > [040 런치 프로그래밍 \(Python, C++\)](#)

ROS 2 심화 프로그래밍

- > [042 Logging](#)
- > [043 ROS2CLI](#)
- > [044 Intra-process communication](#)
- > [045 QoS](#)
- > [046 Component](#)
- > [047 RQt plugin](#)
- > [048 Life cycle](#)
- > [049 Security](#)
- > [050 Real-time](#)

기타

- > [026 ROS 2 Tips](#)
- > [051 TF](#)
- > [052 URDF](#)
- > [053 CI & Lint](#)
- > [054 Unit tests](#)
- > [055 Buildfarm, Binary Release](#)
- > [056 ROS 2 추천 패키지](#)

ROS 2 도입을 위한 다음 단계는?

- **프로그래밍에 왕도는 없어요.**
 - 다음 페이지의 1 ~ 56 강좌를 모두 봐 주세요.
- **프로그래밍은 귀로, 눈으로 공부할 수 없어요.**
 - 우선, 정해진 프로젝트가 없다면 아래 튜토리얼을 따라해 보세요.
 - 튜토리얼은 다 알려고 하기 보다는 뭐가 있는지 빠르게 훑고 마치는게 중요!
 - <https://docs.ros.org/en/jazzy/Tutorials.html>
 - ROS 2 추천 패키지도 한번 훑어 보세요. 완성도 높은 패키지는 코드 레벨로 익히는 것도 좋아요.
 - <https://cafe.naver.com/openrt/25422>
- **프로그래밍은 따라하면 안돼요. 직접 구현해 보세요.**
 - 튜토리얼이 끝나서 얼추 감을 익혔다면 자신만의 프로젝트를 정해 직접 구현해 보세요.
 - 백문이 불여일견, 백견이 불여일타! 일신우일신!
- **ROS 2 공개 패키지를 만들어 보세요.**
 - 대한민국에서 ROS 2 공개 패키지를 만들어 본 사람이 몇이나 있을까요?
 - 도전해 보세요! <https://cafe.naver.com/openrt/25343>

ROBOT

한가지 더!



- 오로카
- www.oroqa.org
- 오픈 로보틱스 지향
- 풀뿌리 로봇공학의 저변 활성화
- 공개 강좌, 세미나, 프로젝트 진행

4만 9,263명 회원



- 로봇공학을 위한 열린 모임 (KOS-ROBOT)
- www.facebook.com/groups/KoreanRobotics
- 로봇공학 통합 커뮤니티 지향
- 일반인과 전문가가 어울러지는 한마당
- 로봇공학 소식 공유
- 연구자 간의 협력

1만 2,525명 회원

ROS

로봇 운영체제

406

보다 더 많은 사람과 함께 나누는 로보틱스 저변화



407

보다 더 많은 사람과 함께 나누는 로보틱스 저변화







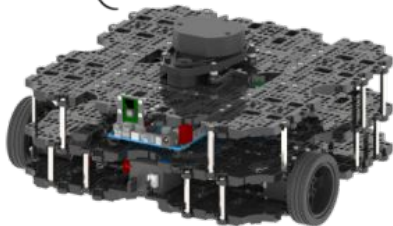
TURTLEBOT3

TurtleBot3

Burger

TurtleBot3

Waffle Pi



410 로봇공학 커뮤니티

여러분의 참여를 기다리고 있습니다.



- 오로카
- www.oroqa.org
- 오픈 로보틱스 지향
- 풀뿌리 로봇공학의 저변 활성화
- 공개 강좌, 세미나, 프로젝트 진행



- 로봇공학을 위한 열린 모임 (KOS-ROBOT)
- www.facebook.com/groups/KoreanRobotics
- 로봇공학 통합 커뮤니티 지향
- 일반인과 전문가가 어울러지는 한마당
- 로봇공학 소식 공유
- 연구자 간의 협력

ROS

그리고... 마지막 메시지!

로봇공학자

여러분을 열렬히 응원합니다.

Q&A

Q&A

ROS 2 특강 발표자료
(강의 자료)



<https://m.site.naver.com/1ptXy>

ROS 2 강좌 목차
(네이버 카페)



5만명

<https://m.site.naver.com/1ptNR>

오로카 ROS 유저모임
(카카오톡 단톡방)



1,400명

<https://open.kakao.com/o/gAeOb2nc>

ROBOTIS

본사 서울 강서구 마곡중앙5로1길 37 | Tel 070-8671-2600 | www.robotis.com